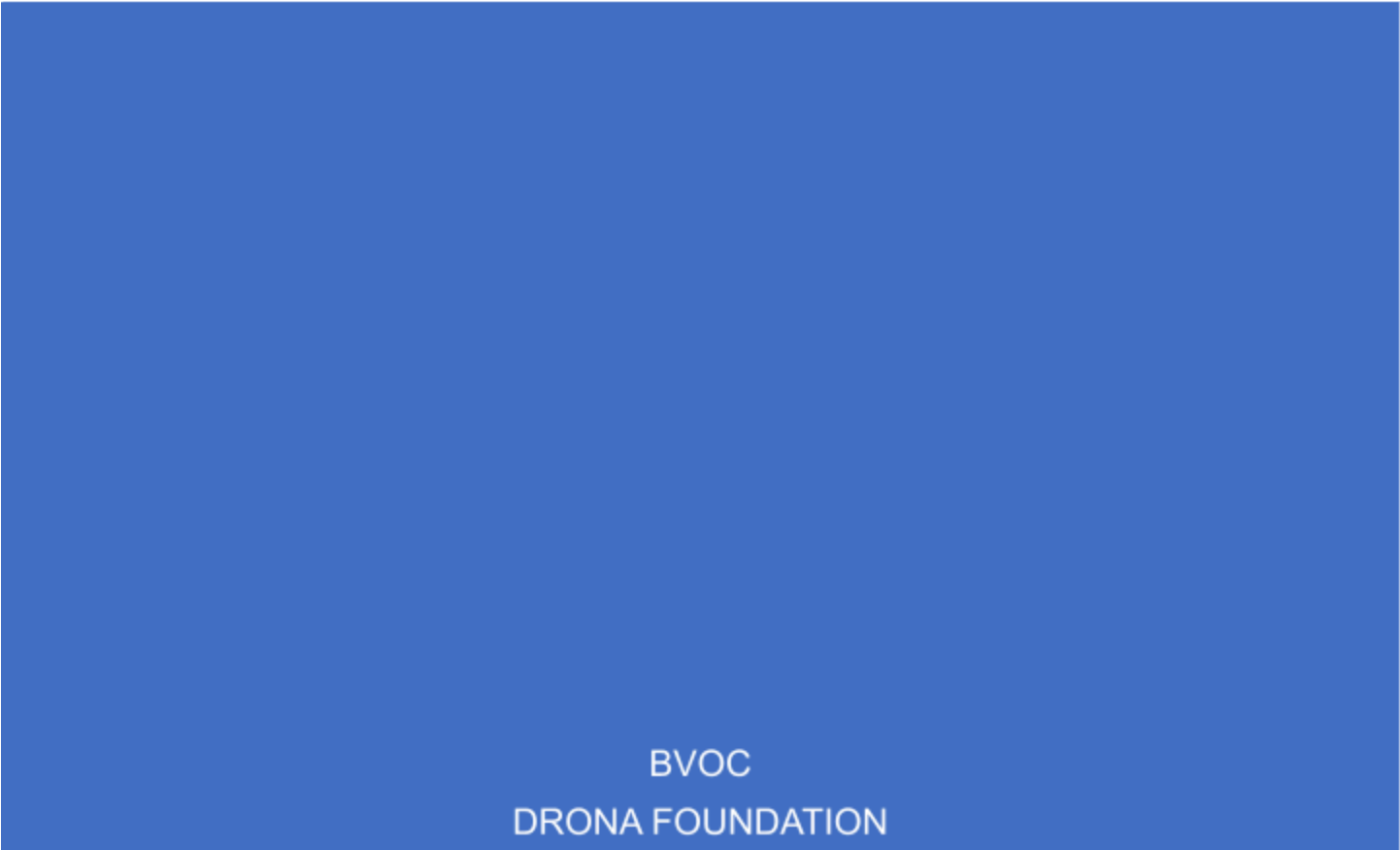




OBJECT ORIENTED PROGRAMMING USING JAVA (ADVANCE JAVA)



BVOC
DRONA FOUNDATION

Sr No	INDEX
4	<u>Classes and Objects</u>
4.1	Introduction to Classes
4.2	Classes: Syntax
4.3	Classes: Constructors
4.4	Classes: Instance Fields
4.5	Classes: Constructor Parameters
4.6	Classes: Assigning Values to Instance Fields
4.7	Classes: Multiple Fields
4.8	Classes: Review
4	Quiz
5	<u>METHODS</u>
5.1	Introduction
5.2	Defining Methods
5.3	Calling Methods
5.4	Scope
5.5	Adding Parameters
5.6	Reassigning Instance Fields
5.7	Returns
5.8	The toString() Method
5.9	Review
5	Quiz
4	Project : A Basic Calculator
5	Project : Build a Droid
	Concept Review
6	<u>CONDITIONALS & CONTROL FLOW</u>
6.1	Introduction to Control Flow
6.2	If-Then Statement
6.3	If-Then-Else
6.4	If-Then-Else-If
6.5	Nested Conditional Statements

6.6	Switch Statement
6.7	Review
7	<u>CONDITIONAL OPERATORS</u>
7.1	Introduction to Conditional Operators
7.2	Conditional-And: &&
7.3	Conditional-Or:
7.4	Logical NOT: !
7.5	Combining Conditional Operators
7.6	Review
7	Quiz
6	Project : A Simple Car Loan Payment Calculator
7	Project : Continents and Cities
	Concept Review
8	<u>ARRAYS</u>
8.1	Introduction
8.2	Creating an Array Explicitly
8.3	Importing Arrays
8.4	Get Element By Index
8.5	Creating an Empty Array
8.6	Length
8.7	String[] args
8.8	Review
8	Quiz
9	<u>ARRAYLISTS</u>
9.1	Introduction
9.2	Creating ArrayLists
9.3	Adding an Item
9.4	ArrayList Size
9.5	Accessing an Index

9.6	Changing a Value
9.7	Removing an Item
9.8	Getting an Item's Index
9.9	Review
9	Quiz
8	Project : Desert Island Playlist
	Concept Review
10	<u>LOOPS</u>
10.1	Introduction to Loops
10.2	While We're Here
10.3	Incrementing While Loops
10.4	For Loops
10.5	Using For Loops
10.6	Iterating Over Arrays and ArrayLists
10.7	break and continue
10.8	For-Each Loops
10.9	Removing Elements During Traversal
10.10	Review
10	Quiz
9	Project : Fizz Buzz
10	Project : The Prime Directive
	Concept Review
11	<u>STRING METHODS</u>
11.1	Introduction to String Methods
11.2	length()
11.3	concat()
11.4	equals()
11.5	indexOf()
11.6	charAt()
11.7	substring()

11.8	toUpperCase() / toLowerCase()
11.9	Review
11	Quiz
11	DNA Sequencing
	Concept Review
12	<u>Access, Encapsulation & Scope</u>
12.1	What are Access and Scope?
12.2	The public Keyword
12.3	The private Keyword and Encapsulation
12.4	Accessor and Mutator Methods
12.5	Scope: Local Variables
12.6	Scope: The this Keyword
12.7	Using this With Methods
12.8	Other Private Methods
12.9	Review
12	Quiz
	Concept Review
13	<u>STATIC VARIABLES AND METHODS</u>
13.1	Static Methods Refresher
13.2	Static Variables
13.3	Modifying Static Variables
13.4	Writing Your Own Static Methods
13.5	Review
13	Quiz
14	<u>Inheritance & Polymorphism</u>
14.1	Introducing Inheritance
14.2	Inheritance in Practice
14.3	Inheriting the Constructor
14.4	Parent Class Aspect Modifiers
14.5	Introducing Polymorphism

14.6	Method Overriding
14.7	Using a Child Class as its Parent Class
14.8	Child Classes in Arrays and ArrayLists
14.9	Child Classes in Method Parameters
14.10	Review of Inheritance and Polymorphism
14	Quiz
12	Project : Language Families
	Concept Review
15	<u>DEBUGGING</u>
15.1	Introduction to Bugs
15.2	Syntax Errors
15.3	Run-time Errors
15.4	Exceptions
15.5	Exception Handling
15.6	Logic Errors
15.7	Debugging Techniques
15.8	Review
15	Quiz
13	Project : Bug Detective
16	<u>2D ARRAYS</u>
16.1	Introduction to 2D Arrays
16.2	Declaration, Initialization, and Assignment
16.3	Accessing Elements in a 2D Array
16.4	Modifying Elements in a 2D Array
16.5	Review of Nested Loops
16.6	Traversing 2D Arrays: Introduction
16.7	Traversing 2D Arrays: Practice with Loops
16.8	Traversing 2D Arrays: Row-Major Order
16.9	Traversing 2D Arrays: Column-Major Order
16.10	Combining Traversal and Conditional Logic
16.11	2D Array Review

16	Quiz
14	Project : 2D Arrays: Image Manipulation Project
	Concept Review

4.Classes and Objects

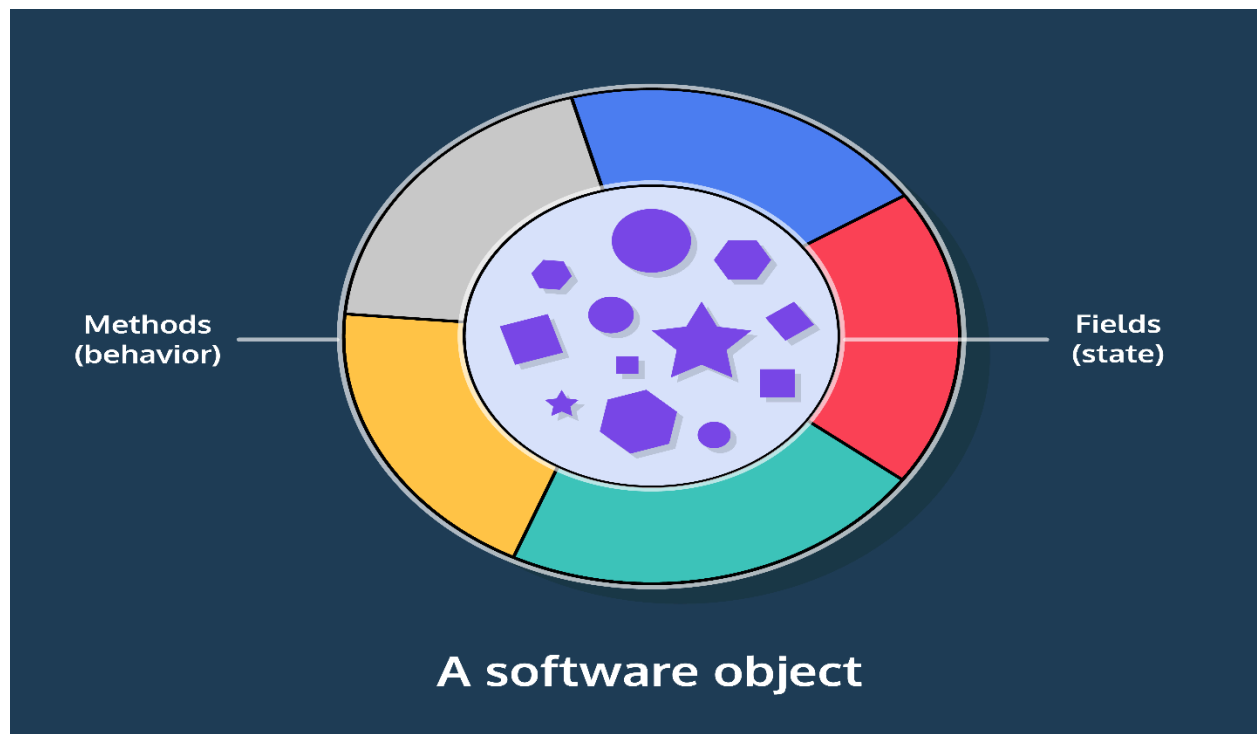
4.1 Introduction to Classes

All programs require one or more [classes](#) that act as a model for the world.

For example, a program to track student test scores might have Student, Course, and Grade classes. Our real-world concerns, students and their grades, are inside the program as classes.

We represent each student as an *instance*, or *object*, of the Student class.

This is *object-oriented programming* because programs are built around objects and their interactions. An object contains state and behavior.



Classes are a blueprint for objects. Blueprints detail the general structure. For example, all students have an ID, all courses can enroll a student, etc.

An instance is the thing itself. *This* student has an ID of 42, *this* course enrolled *that* student, etc.

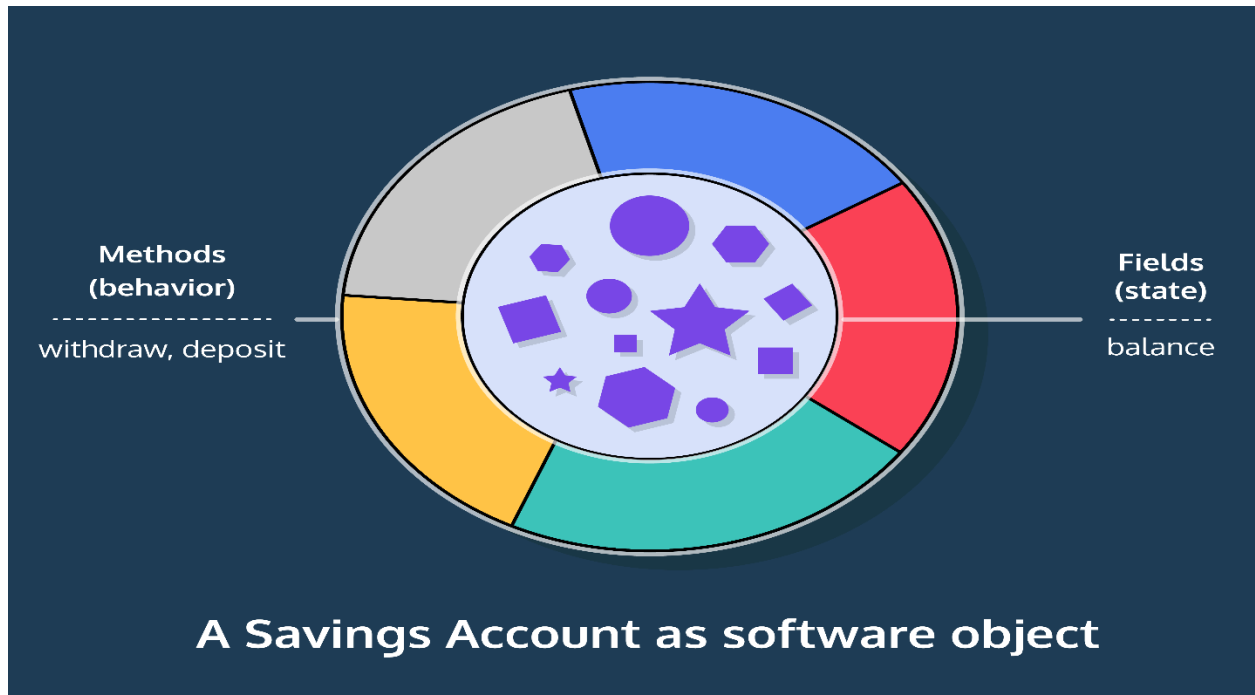
Let's review with another example, a savings account at a bank.

What should a savings account know?

- The balance of money available.

What should a savings account do?

- Deposit money.
- Withdraw money.



Imagine two people have accounts that are instances of the SavingsAccount class. They share behavior (how they deposit and withdraw) but have individual state (their balances), and even with the same balance amount these accounts are separate entities.

4.1 Introduction to Classes Instructions

Our text editor contains a complete class definition that we'll build up as we progress through the lesson.

Run the code to see it in action.

```
public class Store {  
    // instance fields  
    String productType;  
    int inventoryCount;  
    double inventoryPrice;  
  
    // constructor method
```

```

public Store(String product, int count, double price) {
    productType = product;
    inventoryCount = count;
    inventoryPrice = price;
}

// main method
public static void main(String[] args) {
    Store lemonadeStand = new Store("lemonade", 42, .99);
    Store cookieShop = new Store("cookies", 12, 3.75);

    System.out.println("Our first shop sells " + lemonadeStand.productType + " at " +
        lemonadeStand.inventoryPrice + " per unit.");

    System.out.println("Our second shop has " + cookieShop.inventoryCount + " units
        remaining.");
}
}

```

OUTPUT

Our first shop sells lemonade at 0.99 per unit.

Our second shop has 12 units remaining.

4.2 Classes: Syntax

The fundamental concept of object-oriented programming is the class.

A *class* is the set of instructions that describe how an instance can behave and what information it contains.

Java has pre-defined classes such as System, which we've used in logging text to our screen, but we also need to write our own classes for the custom needs of a program.

Here's a definition of a Java class:

```
public class Car {  
    // scope of Car class starts after curly brace  
  
    public static void main(String[] args) {  
        // scope of main() starts after curly brace  
  
        // program tasks  
  
    }  
    // scope of main() ends after curly brace  
  
}  
// scope of Car class ends after curly brace
```

This example defines a class named Car. public is an *access level modifier* that allows **other** classes to interact with this class. For now, all classes will be public.

This class has a main() method, which lists the tasks performed by the program. main() runs when we execute the compiled **Car.class** file.

4.2 Classes: Syntax Instructions

1.

In the code editor, create a public Store class.

2.

Your program will not compile without a main() method.

Define one within Store.

4.3 Classes: Constructors

In order to create an object (an instance of a class), we need a [constructor](#) method. The constructor is defined within the class.

Let's take a look at the Car class with a constructor. The constructor, Car(), shares the same name as the class:

```
public class Car {  
    // Constructor method  
    public Car() {  
        // instructions for creating a Car instance  
    }  
  
    public static void main(String[] args) {  
        // body of main method  
    }  
}
```

To create an instance, we need to *call* or *invoke* the constructor within main(). The following example assigns a Car instance to the variable ferrari:

```
public class Car {  
  
    public Car() {  
        // instructions for creating a Car instance  
    }  
  
    public static void main(String[] args) {  
        // Invoke the constructor  
        Car ferrari = new Car();  
    }  
}
```

In this example, instead of being declared with a primitive data type like `int` or `boolean`, our variable `ferrari` is declared as a *reference data type*. This means that the value of our variable is a reference to an instance's memory address. During its declaration, the class name is used as the variable's type. In this case, the type is `Car`.

After the assignment operator, `(=)`, we invoke the constructor method: `Car()`, and use the keyword `new` to indicate that we're creating an instance. **Omitting `new` causes an error.**

If we were to output the value of `ferrari` we would see its memory address:

```
Car@76ed5528
```

Keep Reading: AP Computer Science A Students

We can initialize a reference-type variable without assigning it a reference if we utilize the special value `null`. Something that is `null` has no value; if we were to assign `null` to an object, it would have a void reference.

For example, in the following code snippet, we'll create an instance of `Car`, assign it a reference, and then change its value to `null`:

```
Car thunderBird = new Car();  
System.out.println(thunderBird); // Prints: Car@76ed5528
```

```
thunderBird = null; // change value to null  
System.out.println(thunderBird); // Prints: null
```

It's important to note that if we use a `null` reference to call a method or access an instance variable, we will receive a `NullPointerException` error.

4.3 Classes: Constructors Instructions

1.

Let's explore how code execution moves around the file with two methods.

Add a print statement inside our `Store` constructor with the message: I am inside the constructor method.

We'll see this message whenever we create an instance of `Store` by calling the

constructor.

2.

We did not see our constructor message printed because **we haven't run the code inside the constructor**.

Inside main(), create an instance of Store called lemonadeStand. Don't forget the new keyword!

We should see the constructor message.

3.

Inside main(), print lemonadeStand to see how Java represents this instance.

Review the order of the printed messages:

1. Running the program invokes main()
2. We create an instance so we move from main() to Store()
3. The code inside Store() runs
4. When Store() finishes execution, we **return** to main()

```
public class Store {
```

```
    // new method: constructor!
```

```
    public Store() {
```

```
    }
```

```
    // main method is where we create instances!
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Start of the main method.");
```

```
// create the instance below
```

```
// print the instance below
```

```
}
```

```
}
```

OUTPUT

Start of the main method.

I am inside the constructor method.

Store@2aae9190

4.4 Classes: Instance Fields

Our last exercise ended with printing an instance of Store, which looked something like Store@6bc7c054. The first part, Store, refers to the class, and the second part @6bc7c054 refers to the instance's location in the computer's memory.

We don't care about memory location, but our instances have no other characteristics!

When an object is created, the constructor sets the initial state of the object.

The *state* is made up of associated data that represents the characteristics of an object.

We'll add data to an object by introducing *instance variables*, or *instance fields*.

We want Car instances of different colors, so we declare a String color instance field.

Often times, instance variables are described as a "has-a" relationship with the object.

For example, a Car "has-a" color. Another way to think of this is that instance variables are the nouns and adjectives associated with an object. What qualities other than color might a car *have*?

```
public class Car {
```

```
/*
```

```
declare fields inside the class
```

by specifying the type and name

```
*/
```

```
String color;
```

```
public Car() {
```

```
    /*
```

```
    instance fields available in
```

```
    scope of constructor method
```

```
    */
```

```
}
```

```
public static void main(String[] args) {
```

```
    // body of main method
```

```
}
```

```
}
```

The declaration is **within** the class and the instance variable will be available for assignment inside the constructor.

Fields are a type of state each instance will possess. One instance may have "red" as its color, another "blue", etc. It's the job of the constructor to give these instance fields initial value. We'll learn how to do this in the next exercise.

4.4 Classes: Instance Fields Instructions

1.

Add some state to our Store class.

Declare a String instance field for productType.

```
public class Store {
```

```
    // declare instance fields here!
```



```
// constructor method
public Store() {
    System.out.println("I am inside the constructor method.");
}

// main method
public static void main(String[] args) {
    System.out.println("This code is inside the main method.");

    Store lemonadeStand = new Store();

    System.out.println(lemonadeStand);
}
}
```

OUTPUT

This code is inside the main method.

I am inside the constructor method.

Store@2aae9190

4.5 Classes: Constructor Parameters

To create objects with dynamic, individual states, we'll use a combination of the constructor method and instance fields.

In order to assign a value to an instance variable, we need to alter our constructor method to include parameters so that it can access the data we want to assign to an instance.

We've already seen a parameter in the main() method: String[] args, but this is the first time we're using the parameter value within a method body.

The Car constructor now has a parameter: String carColor:

```
public class Car {  
    String color;  
    // constructor method with a parameter  
    public Car(String carColor) {  
        // parameter value assigned to the field  
        color = carColor;  
    }  
    public static void main(String[] args) {  
        // program tasks  
    }  
}
```

When a String value gets passed into Car(), it is assigned to the parameter carColor. Then, inside the constructor, carColor will be assigned as the value to the instance variable color.

Our method also has a *signature* which defines the name and parameters of the method. In the above example, the signature is Car(String carColor).

In the next exercise, we'll learn how to pass values into a method!

Keep Reading: AP Computer Science A Students

There are two types of parameters: formal and actual. A *formal parameter* specifies the type and name of data that can be passed into a method. In our example above, String carColor is a formal parameter; carColor will represent whatever String value is passed into the constructor. We'll learn about actual parameters in the next exercise.

In Java, because of *constructor overloading*, a class can have multiple constructors as long as they have different parameter values. The signature is useful in that it helps the compiler differentiate between the different methods. For example:

```
public class Car {
```

```

String color;
int mpg;
boolean isElectric;

// constructor 1
public Car(String carColor, int milesPerGallon) {
    color = carColor;
    mpg = milesPerGallon;
}
// constructor 2
public Car(boolean electricCar, int milesPerGallon) {
    isElectric = electricCar;
    mpg = milesPerGallon;
}
}

```

In the example above, there are two constructors. When we initialize an object, the compiler will know which constructor to use because of the values we pass into it. For example, `Car myCar = new Car(true, 40)` will be created by the second constructor because the arguments match the type and order of the second constructor's signature.

If we do not define a constructor, the Java compiler will generate a default constructor that contains no arguments and assigns the object default values. Default values can be created by assigning values to the instance fields during their declaration:

```

public class Car {
    String color = "red";
    boolean isElectric = false;
    int cupHolders = 4;

    public static void main(String[] args) {

```

```
    Car myCar = new Car();  
    System.out.println(myCar.color); // Prints: red  
}  
}
```

4.5 Classes: Constructor Parameters Instructions

1.

Add the String parameter product to the Store() constructor.

2.

Inside of the constructor method, set the instance variable productType equal to the product parameter.

```
public class Store {  
    // instance fields  
    String productType;  
  
    // constructor method  
    public Store() {  
  
    }  
  
    // main method  
    public static void main(String[] args) {  
  
    }  
}
```

4.6 Classes: Assigning Values to Instance Fields

Now that our constructor has a parameter, we must pass values into the method call. These values are referred to as *arguments*; once they are passed in, they will be used to give the instance fields initial value.

Here we create an instance, ferrari, in the main() method with "red" as its color field:

```
public class Car {  
    String color;  
  
    public Car(String carColor) {  
        // assign parameter value to instance field  
        color = carColor;  
    }  
  
    public static void main(String[] args) {  
        // parameter value supplied when calling constructor  
        Car ferrari = new Car("red");  
    }  
}
```

We pass the String value "red" to our constructor method call: new Car("red");.

The type of the value given to the invocation **must match** the type declared by the parameter.

Inside the constructor, the parameter carColor refers to the value passed in during the invocation: "red". This value is assigned to the instance field color.

color has already been declared, so we don't specify the type during assignment.

The object, ferrari, holds the state of color as an instance field referencing the value "red".

We access the value of this field with the *dot operator* (.):

```
/*
```

accessing a field:

```
objectName.fieldName
```

```
*/
```

```
ferrari.color;
```

```
// "red"
```

Keep Reading: AP Computer Science A Students

An *actual parameter*, or argument, refers to the value being passed during a method call.

Call by value is the process of calling a method with an argument value. When an argument is passed, the formal parameter is initialized with a copy of the argument value. For example, when we declared the ferrari object, the String value "red" is passed as an argument; then, the formal parameter carColor is assigned a copy of that value.

4.6 Classes: Assigning Values to Instance Fields Instructions

1.

Inside main(), create an instance of Store and assign it to the variable lemonadeStand. Use "lemonade" as the parameter value.

2.

Print the instance field productType from lemonadeStand.

```
public class Store {
```

```
    // instance fields
```

```
    String productType;
```

```
    // constructor method
```

```
    public Store(String product) {
```

```

    productType = product;
}

// main method
public static void main(String[] args) {

}
}

```

OUTPUT

lemonade

4.7 Classes: Multiple Fields

Objects are not limited to a single instance field. We can declare as many fields as are necessary for the requirements of our program.

Let's change Car instances so they have multiple fields.

We'll add a boolean `isRunning`, that indicates the car engine is on and an `int` `velocity`, that indicates the speed at which the car is traveling.

```

public class Car {
    String color;
    // new fields!
    boolean isRunning;
    int velocity;

    // new parameters that correspond to the new fields
    public Car(String carColor, boolean carRunning, int milesPerHour) {
        color = carColor;
        // assign new parameters to the new fields
    }
}

```

```
isRunning = carRunning;  
velocity = milesPerHour;  
}
```

```
public static void main(String[] args) {  
    // new values passed into the method call  
    Car ferrari = new Car("red", true, 27);  
    Car renault = new Car("blue", false, 70);  
  
    System.out.println(renault.isRunning);  
    // false  
    System.out.println(ferrari.velocity);  
    // 27  
}  
}
```

The constructor now has multiple parameters to receive values for the new fields. We still specify the type as well as the name for each parameter.

Ordering matters! We must pass values into the constructor invocation in the same order that they're listed in the parameters.

```
// values match types, no error  
Car honda = new Car("green", false, 0);
```

```
// values do not match types, error!  
Car junker = new Car(true, 42, "brown");
```

4.7 Classes: Multiple Fields Instructions

1.

Add two new instance fields for Store.

inventoryCount of type int. inventoryPrice of type double.

2.

Update the Store constructor method with the new parameters.

The parameters should be product, count, and price, **in that order**.

You must use that order and include the types for each parameter.

For example, product is of type String because that value is assigned to the instance field String productType.

3.

In the body of the Store constructor, assign the parameter values to the appropriate instance fields.

4.

Inside main(), create an instance of Store called cookieShop.

cookieShop has "cookies" as the product.

cookieShop has 12 cookies to sell and each cookie costs 3.75.

```
public class Store {  
    // instance fields  
    String productType;  
  
    // constructor method  
    public Store(String product) {  
        productType = product;  
    }  
}
```

```
// main method
public static void main(String[] args) {

}
}
```

4.8 Classes: Review

Java is an object-oriented programming language where every program has at least one class. Programs are often built from many classes and objects, which are the instances of a class.

Classes define the state and behavior of their instances. Behavior comes from methods defined in the class. State comes from instance fields declared inside the class.

Classes are modeled on the real-world things we want to represent in our program. Later we will explore how a program can be made from multiple classes. For now, our programs are a single class.

```
public class Dog {
    // instance field
    String breed;
    // constructor method
    public Dog(String dogBreed) {
        /*
         value of parameter dogBreed
         assigned to instance field breed
        */
        breed = dogBreed;
    }
    public static void main(String[] args) {
        /*
         create instance:

```

use 'new' operator and invoke constructor

```
*/
```

```
Dog fido = new Dog("poodle");
```

```
/*
```

fields are accessed using:

the instance name,`.` operator, and the field name.

```
*/
```

```
fido.breed;
```

```
// "poodle"
```

```
}
```

```
}
```

4.8 Classes: Review Instructions

The text editor contains a Dog class. Play around with the code!

Try to add and remove instance fields. Create instances with different values. Access and print different fields.

```
public class Dog {
```

```
    String breed;
```

```
    boolean hasOwner;
```

```
    int age;
```

```
    public Dog(String dogBreed, boolean dogOwned, int dogYears) {
```

```
        System.out.println("Constructor invoked!");
```

```
        breed = dogBreed;
```

```
        hasOwner = dogOwned;
```

```
        age = dogYears;
```

```
}
```

```
public static void main(String[] args) {  
    System.out.println("Main method started");  
    Dog fido = new Dog("poodle", false, 4);  
    Dog nunzio = new Dog("shiba inu", true, 12);  
    boolean isFidoOlder = fido.age > nunzio.age;  
    int totalDogYears = nunzio.age + fido.age;  
    System.out.println("Two dogs created: a " + fido.breed + " and a " + nunzio.breed);  
    System.out.println("The statement that fido is an older dog is: " + isFidoOlder);  
    System.out.println("The total age of the dogs is: " + totalDogYears);  
    System.out.println("Main method finished");  
}  
}
```

OUTPUT

Main method started

Constructor invoked!

Constructor invoked!

Two dogs created: a poodle and a shiba inu

The statement that fido is an older dog is: false

The total age of the dogs is: 16

Main method finished

4. Quiz

1.

The `Bank` class has one instance field: `branchLocationCount`.

Complete the constructor method for this class.

```
public class Bank {  
    int branchLocationCount;  
  
    public Bank(  ) {  
        branchLocationCount = numLocations;  
    }  
    public static void main(String[] args) {  
        // program tasks  
    }  
}
```

`double numLocations`

`boolean numLocations`

`String numLocations`

`int numLocations`

Click or drag and drop to fill in the blank

2.

An instance is declared in the `main()` function. Complete the method call.

```
public class Window {  
    boolean isOpen;  
    int height;  
    int width;  
  
    public Window(boolean open, int windowHeight, int windowWidth) {  
        isOpen = open;  
        height = windowHeight;  
        width = windowWidth;  
    }  
  
    public static void main(String[] args) {  
        Window bedroomWindow =   
    }  
}
```

`new Window(true, 72, 34);`

`new Window(true, 72);`

`new Window(72, false, 34);`

3. The following class definition requires a parameter to be passed in to the constructor in order to make an instance.

```
public class Dinosaur {  
    boolean isExtinct;
```

```
public Dinosaur(boolean extinct) {  
    isExtinct = extinct;  
}
```

```
public static void main(String[] args) {  
    // program tasks  
}
```

- a) False.
- b) True.

4. Why will the following program produce an error?

```
public class Shelf {
```

```
    String material;
```

```
    public Shelf() {
```

```
}
```

```
public static void main(String[] args) {
```

```
    Shelf bureau = new Shelf("pine");
```

```
}
```

```
}
```

- a) The constructor does not have a parameter listed.
- b) bureau has the incorrect type.
- c) No constructor method has been declared.
- d) The instance field has an incorrect declaration.

5. Instances of a Java class represent the real world with what two qualities?

- a) State and behavior.
- b) History and consequences.
- c) State and history.
- d) Behavior and history.

Complete the following class definition for a **Bank** class.

```
public  Bank {  
    public static void main(String[] args) {  
        // program tasks  
    }  
}
```

class void main field

Click or drag and drop to fill in the blank

6.

7. What is the signature of the following constructor?

```
public class Bug {
```

```
    String name;
```

```
    boolean ableToFly;
```

```
    int numberOfLegs;
```

```
    public Bug(String bugName, boolean canFly, int numLegs) {
```

```
        name = bugName;
```

```
        ableToFly = canFly;
```

```
        numberOfLegs = numLegs;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Bug ladybug = new Bug("Lady Bug", true, 6);
```

```
}  
}
```

- a) Bug(double, String, boolean)
- b) Bug(name, ableToFly, numberOfLegs)
- c) Bug(bugName String, canFly boolean, numLegs int)
- d) Bug(String bugName, boolean canFly, int numLegs)

8. What is the data type of the variable hedwig?

```
public class Owl {  
    String speak;  
    boolean nocturnal;  
    public Owl() {  
        speak = "Hoot";  
        nocturnal = true;  
    }  
  
    public static void main(String[] args) {  
        Owl hedwig = new Owl();  
    }  
}
```

- a) speak
- b) hedwig
- c) Owl
- d) nocturnal

9. The main() method is automatically run when the .class file is executed.

- a) True.
- b) False.

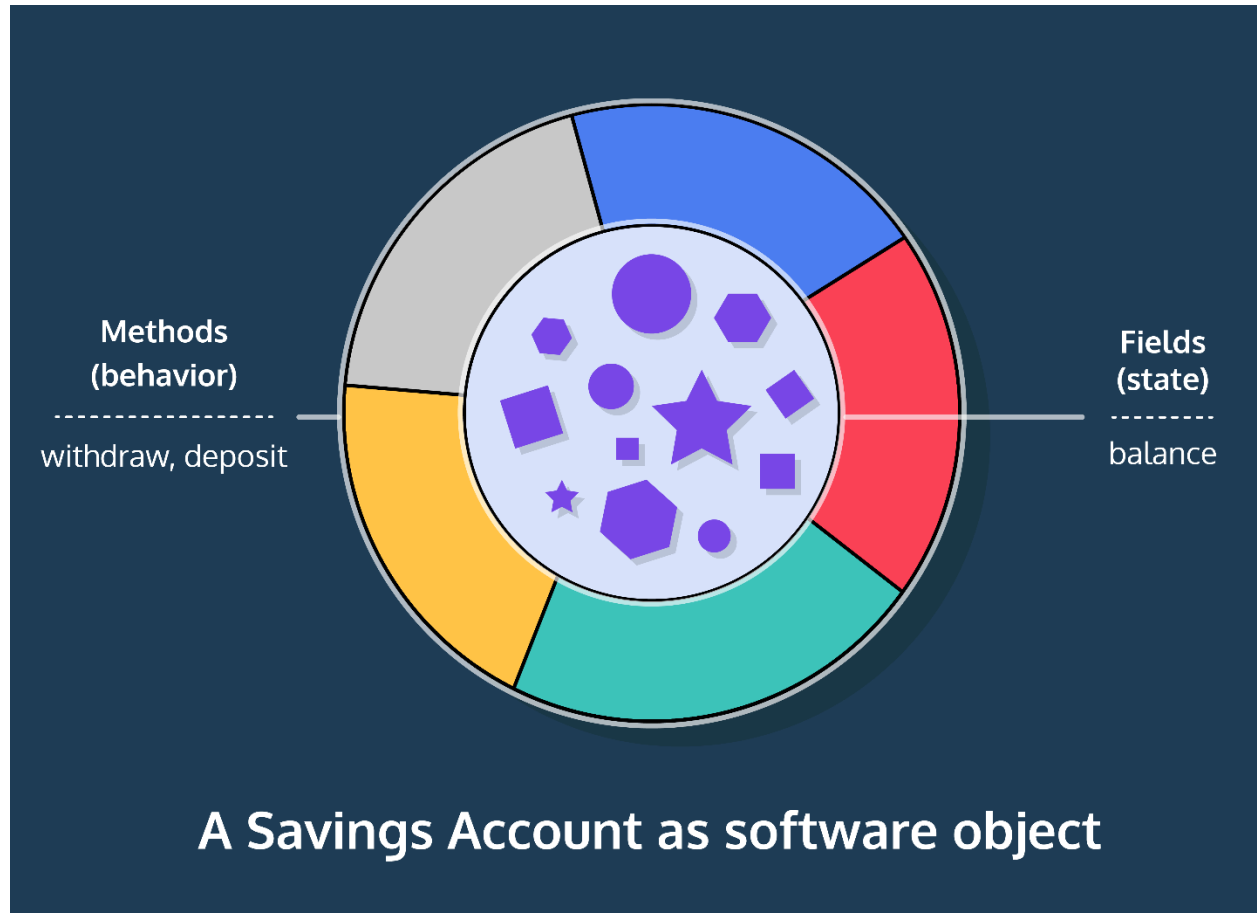
10. Every Java program contains at least one class.

- a) False.
- b) True.

5. METHODS

5.1 Introduction

In the last lesson, we learned that objects have state and behavior. We have seen how to give objects state through instance fields. Now, we're going to learn how to create object *behavior* using [methods](#). Remember our example of a Savings Account.



The state tells us what a savings account should know:

- The balance of money available

The behavior tells us what tasks a savings account should be able to perform:

- Depositing - increasing the amount available
- Withdrawing - decreasing the amount available
- Checking the balance - displaying the amount available.

Methods are repeatable, modular blocks of code used to accomplish specific tasks. We

have the ability to define our own methods that will take input, do something with it, and return the kind of output we want.

Looking at the example above, recreating a savings account is no easy task. How can one program tackle such a large problem? This is where methods with their ability to accomplish smaller, specific tasks come in handy. Through *method decomposition*, we can use methods to break down a large problem into smaller, more manageable problems.

Methods are also reusable. Imagine we wrote a sandwich-making program that used 20 lines of code to make a single sandwich. Our program would become very long very quickly if we were making multiple sandwiches. By creating a `makeSandwich()` method, we can make a sandwich anytime simply by calling it.

makeSandwich()

In this lesson, we'll learn how to create and call our very own methods inside of our programs.

Keep Reading: AP Computer Science A Students

If we were to share this sandwich-making program with another person, they wouldn't have to understand *how* `makeSandwich()` worked. If we wrote our program well, all they would need to know is that if they called `makeSandwich()`, they would receive a sandwich. This concept is known as *procedural abstraction*: knowing what a method does, but not how it accomplishes it.

5.1 Introduction Instructions

1.

We have made a SavingsAccount class without using any methods beyond main() and the constructor, SavingsAccount().

Run the code to see some account behavior happen.

Look at the main method! It's so long! There is so much repeated code! Can you imagine how messy it would look if you needed to make 10 deposits?

Throughout this lesson, we will learn how to make methods that would make checking the balance, depositing, and withdrawing all behavior that would take only one line of code.

```
public class SavingsAccount {  
  
    int balance;  
  
    public SavingsAccount(int initialBalance){  
        balance = initialBalance;  
    }  
  
    public static void main(String[] args){  
        SavingsAccount savings = new SavingsAccount(2000);  
  
        //Check balance:  
        System.out.println("Hello!");  
        System.out.println("Your balance is "+savings.balance);  
  
        //Withdrawing:  
        int afterWithdraw = savings.balance - 300;
```

```
savings.balance = afterWithdraw;
System.out.println("You just withdrew "+300);

//Check balance:
System.out.println("Hello!");
System.out.println("Your balance is "+savings.balance);

//Deposit:
int afterDeposit = savings.balance + 600;
savings.balance = afterDeposit;
System.out.println("You just deposited "+600);

//Check balance:
System.out.println("Hello!");
System.out.println("Your balance is "+savings.balance);

//Deposit:
int afterDeposit2 = savings.balance + 600;
savings.balance = afterDeposit2;
System.out.println("You just deposited "+600);

//Check balance:
System.out.println("Hello!");
System.out.println("Your balance is "+savings.balance);

}
```

```
}
```

OUTPUT

Hello!

Your balance is 2000

You just withdrew 300

Hello!

Your balance is 1700

You just deposited 600

Hello!

Your balance is 2300

You just deposited 600

Hello!

Your balance is 2900

5.2 Defining Methods

If we were to define a `checkBalance()` method for the Savings Account example we talked about earlier, it would look like the following:

```
public void checkBalance(){  
    System.out.println("Hello!");  
    System.out.println("Your balance is " + balance);  
}
```

The first line, `public void checkBalance()`, is the method declaration. It gives the program some information about the method:

- `public` means that other classes can access this method. We will learn more about that later in the course.
- The `void` keyword means that there is no specific output from the method. We will see methods that are not void later in this lesson, but for now, all of our methods will be void.

- `checkBalance()` is the name of the method.

Every method has its own unique *method signature* which is comprised of the method's name and its parameter type. In this example, the method signature is `checkBalance()`.

The two print statements are inside the *body* of the method, which is defined by the curly braces: { and }.

Anything we can do in our `main()` method, we can do in other methods! All of the Java tools you know, like the math and comparison operators, can be used to make interesting and useful methods.

Keep Reading: AP Computer Science A Students

`checkBalance()` is considered a non-static method because its signature does not include the keyword `static` like the `main()` method does. We'll learn more about non-static methods later in this course.

5.2 Defining Methods Instructions

1.

In between the constructor and the `main()` method, add a method called `advertise()` to the `Store` class. It should be accessible by other classes, and should have no output. You can leave the body of the method empty.

2.

Inside the `advertise()` method, type two print statements. They should result in the printouts:

```
"Come spend some money!"
```

```
"Selling productType!"
```

where `productType` is replaced with the value in the variable `productType`.

However, we're not going to see these Strings printed out yet! We'll see in the next exercise how we can make these printouts actually run.

```

public class Store {
    // instance fields
    String productType;

    // constructor method
    public Store(String product) {
        productType = product;
    }

    // advertise method

    // main method
    public static void main(String[] args) {

    }
}

```

5.3 Calling Methods

When we add a non-static method to a class, it becomes available to use on an object of that class. In order to have our methods get executed, we must *call* the method on the object we created.

Let's add a non-static `startEngine()` method to our `Car` class from the previous lesson. Inside the `main()` method, we'll call `startEngine()` on the `myFastCar` object:

```

class Car {

    String color;

```



```
public Car(String carColor) {  
    color = carColor;  
}
```

```
public void startEngine() {  
    System.out.println("Starting the car!");  
    System.out.println("Vroom!");  
}
```

```
public static void main(String[] args){  
    Car myFastCar = new Car("red");  
    // Call a method on an object  
    myFastCar.startEngine();  
    System.out.println("That was one fast car!");  
}
```

Let's take a closer look at the method call:

```
myFastCar.startEngine();
```

First, we reference our object `myFastCar`. Then, we use the *dot operator* (`.`) to call the method `startEngine()`. Note that we must include parentheses (`()`) after our method name in order to call it.

If we run the above program, we will get the following output.

Starting the car!

Vroom!

That was one fast car!

Code generally runs in a top-down order where code execution starts at the top of a program and ends at the bottom of a program; however, methods are ignored by the compiler unless they are being called.

When a method is called, the compiler executes every statement contained within the method. Once all method instructions are executed, the top-down order of execution continues. This is why Starting the car! and Vroom! are outputted before That was one fast car!.

5.3 Calling Methods Instructions

1.

Last exercise, we defined a new method, `advertise()`, but we didn't actually see it run. We now have a `Store` class with `advertise()` defined.

Call the `advertise()` method on the `lemonadeStand` object in the `main()` method and see what the output is!

2.

Now, call the `advertise()` method on the `lemonadeStand` object two more times. It should be called in the `main()` method three times total.

```
public class Store {  
    // instance fields  
    String productType;  
  
    // constructor method  
    public Store(String product) {  
        productType = product;  
    }  
  
    // advertise method  
    public void advertise() {  
        System.out.println("Selling " + productType + "!");  
        System.out.println("Come spend some money!");  
    }  
}
```

```
// main method
public static void main(String[] args) {
    Store lemonadeStand = new Store("Lemonade");

}
}
```

OUTPUT

```
Selling Lemonade!
Come spend some money!
Selling Lemonade!
Come spend some money!
Selling Lemonade!
Come spend some money!
```

5.4 Scope

A method is a task that an object of a class performs.

We mark the domain of this task using curly braces: {, and }. Everything inside the curly braces is part of the task. This domain is called the *scope* of a method.

We can't access variables that are declared inside a method in code that is outside the scope of that method.

Looking at the Car class again:

```
class Car {
    String color;
    int milesDriven;

    public Car(String carColor) {
        color = carColor;
    }
}
```

```

    milesDriven = 0;
}

public void drive() {
    String message = "Miles driven: " + milesDriven;
    System.out.println(message);
}

public static void main(String[] args){
    Car myFastCar = new Car("red");
    myFastCar.drive();
}
}

```

The variable `message`, which is declared and initialized inside of `drive`, cannot be used inside of `main()`! It only exists within the *scope* of the `drive()` method.

However, `milesDriven`, which is declared at the top of the class, can be used inside all methods in the class, since it is in the scope of the whole class.

5.4 Scope Instructions

1.

Inside of the `advertise()` method, change the `productType` variable to the `cookie` variable, which is declared in the `main()` method. This should also result in the printout:

Selling cookies!

Right?

2.

No! We got an error! The `cookie` variable cannot be accessed inside of the `advertise` method. The scope is wrong! Change it back to `productType`:

```
String message = "Selling " + productType + "!";
```

3.

Inside of the main() method, print the String message, which is declared in the advertise() method. This should print:

Selling Cookies!

Right?

4.

Foiled again! The message variable only exists inside the scope of the advertise() method!

Delete the faulty print statement from the main() method.

```
public class Store {  
    // instance fields  
    String productType;  
  
    // constructor method  
    public Store(String product) {  
        productType = product;  
    }  
  
    // advertise method  
    public void advertise() {  
        String message = "Selling " + productType + "!";  
        System.out.println(message);  
    }  
}
```

```
// main method
public static void main(String[] args) {
    String cookie = "Cookies";
    Store cookieShop = new Store(cookie);

    cookieShop.advertise();
}
```

OUTPUT

Selling Cookies!

5.5 Adding Parameters

We saw how a method's scope prevents us from using variables declared in one method in another method. What if we had some information in one method that we needed to pass into another method?

Similar to how we added parameters to *constructors*, we can customize all other methods to accept parameters. For example, in the following code, we create a `startRadio()` method that accepts a `Double` parameter, `stationNum`, and a `String` parameter called `stationName`:

```
class Car {

    String color;

    public Car(String carColor) {
        color = carColor;
    }
}
```

```

public void startRadio(double stationNum, String stationName) {
    System.out.println("Turning on the radio to " + stationNum + ", " + stationName
+ "!");
    System.out.println("Enjoy!");
}

public static void main(String[] args){
    Car myCar = new Car("red");
    myCar.startRadio(103.7, "Meditation Station");
}
}

```

Adding parameter values impacts our method's signature. Like constructor signatures, the method signature includes the method name as well as the parameter types of the method. The signature of the above method is `startRadio(double, String)`.

In the `main()` method, we call the `startRadio()` method on the `myCar` object and provide a double argument of 103.7 and String argument of "Meditation Station", resulting in the following output:

```

Turning on the radio to 103.7, Meditation Station!
Enjoy!

```

Note that when we call on a method with multiple parameters, the arguments given in the call must be placed in the same order as the parameters appear in the signature. If the argument types do not match the parameter types, we'll receive an error.

Keep Reading: AP Computer Science A Students

Through method overloading, our Java programs can contain multiple methods with the same name as long as each method's parameter list is unique. For example, we can recreate our above program to contain two `startRadio()` methods:

```

// Method 1
public void startRadio(double stationNum, String stationName) {

```

```

    System.out.println("Turning on the radio to " + stationNum + ", " + stationName + "!");
    System.out.println("Enjoy!");
}

// Method 2
public void startRadio(double stationNum) {
    System.out.println("Turning on the radio to " + stationNum + "!");
}

public static void main(String[] args){
    Car myCar = new Car("red");
    // Calls the first startRadio() method
    myCar.startRadio(103.7, "Meditation Station");

    // Calls the second startRadio() method
    myCar.startRadio(98.2);
}

```

5.5 Adding Parameters Instructions

1.

Add a method to the Store class called `greetCustomer()`. It should be accessible by other classes, and return no output. For now, have it take no parameters and leave the body of the method empty.

2.

Modify the `greetCustomer()` method so that it accepts a `String` parameter called `customer`.

3.

Inside of the greetCustomer() method, add a print statement to print:

"Welcome to the store, " + customer + "!"

4.

Inside the main() method, call the greetCustomer() method on the lemonadeStand object. Pass in a String argument of your choice!

```
public class Store {  
    // instance fields  
    String productType;  
  
    // constructor method  
    public Store(String product) {  
        productType = product;  
    }  
  
    // advertise method  
    public void advertise() {  
        String message = "Selling " + productType + "!";  
        System.out.println(message);  
    }  
  
    // main method  
    public static void main(String[] args) {  
        Store lemonadeStand = new Store("Lemonade");  
  
    }  
}
```

```
}
```

OUTPUT

Welcome to the store, Laura!

5.6 Reassigning Instance Fields

Earlier, we thought about a Savings Account as a type of object we could represent in Java.

Two of the methods we need are depositing and withdrawing:

```
public class SavingsAccount{
    double balance;
    public SavingsAccount(double startingBalance){
        balance = startingBalance;
    }

    public void deposit(double amountToDeposit){
        //Add amountToDeposit to the balance
    }

    public void withdraw(double amountToWithdraw){
        //Subtract amountToWithdraw from the balance
    }

    public static void main(String[] args){

    }
}
```

These methods would change the value of the variable balance. We can *reassign* balance to be a new value by using our assignment operator, =, again.

```
public void deposit(double amountToDeposit){
```

```
double updatedBalance = balance + amountToDeposit;  
balance = updatedBalance;  
}
```

Now, when we call deposit(), it should change the value of the instance field balance:

```
public static void main(String[] args){  
    SavingsAccount myAccount = new SavingsAccount(2000);  
    System.out.println(myAccount.balance);  
    myAccount.deposit(100);  
    System.out.println(myAccount.balance);  
}
```

This code first prints 2000, the initial value of myAccount.balance, and then prints 2100, which is the value of myAccount.balance after the deposit() method has run.

Changing instance fields is how we change the state of an object and make our objects more flexible and realistic.

5.6 Reassigning Instance Fields Instructions

1.

We have added a price instance field to the Store class.

However, to combat inflation costs, we've found ourselves increasing the price of our product over and over. We've added an empty increasePrice() method to the Store class. It takes a double parameter priceToAdd.

Inside of the increasePrice() method, create a variable called newPrice. Declare it to be a double, and set it equal to the price plus the priceToAdd.

2.

Inside of increasePrice(), set the instance field price to be newPrice!

3.

In the main() method, increase the price at the lemonade stand by 1.5. Then, print the lemonadeStand.price to see how it has changed!

```

public class Store {
    // instance fields
    String productType;
    double price;

    // constructor method
    public Store(String product, double initialPrice) {
        productType = product;
        price = initialPrice;
    }

    // increase price method
    public void increasePrice(double priceToAdd){

    }

    // main method
    public static void main(String[] args) {
        Store lemonadeStand = new Store("Lemonade", 3.75);

    }
}

```

OUTPUT : 5.25

5.7 Returns

Remember, variables can only exist in the *scope* that they were declared in. We can use a value outside of the method it was created in if we *return* it from the method.

We return a value by using the keyword `return`:

```
public int numberOfTires() {  
    int tires = 4;  
    // return statement  
    return tires;  
}
```

This method, called `numberOfTires()`, returns 4. Once the `return` statement is executed, the compiler exits the function. Any code that exists after the `return` statement in a function is ignored.

In past exercises, when creating new methods, we used the keyword `void`. Here, we are replacing `void` with `int`, to signify that the *return type* is an `int`.

The `void` keyword (which means “completely empty”) indicates that no value is returned after calling that method.

A non-void method, like `numberOfTires()` returns a value when it is called. We can use datatype keywords (such as `int`, `char`, etc.) to specify the type of value the method should return. The return value’s type must match the return type of the method. If the return expression is compatible with the return type, a copy of that value gets returned in a process known as *return by value*.

Unlike `void` methods, non-void methods can be used as either a variable value or as part of an expression like so:

```
public static void main(String[] args){  
    Car myCar = new Car("red");  
    int numTires = myCar.numberOfTires();  
}
```

Within `main()`, we called the `numberOfTires()` method on `myCar`. Since the method returns an `int` value of 4, we store the value in an integer variable called `numTires`. If we printed `numTires`, we would see 4.

Keep Reading: AP Computer Science A Students

We learned how to return primitive values from a method, but what if we wanted our method to return an object? Returning an object works a little differently than returning a primitive value. When we return a primitive value, a copy of the value is returned; however, when we return an object, we return a reference to the object instead of a copy of it.

Let's create a second class, `carLot`, that takes in a `Car` as a parameter and contains a method which returns a `Car` object.

```
class CarLot {
    Car carInLot;
    public CarLot(Car givenCar) {
        carInLot = givenCar;
    }

    public Car returnACar() {
        // return Car object
        return carInLot;
    }

    public static void main(String[] args) {
        Car myCar = new Car("red", 70);
        System.out.println(myCar);
        CarLot myCarLot = new CarLot(myCar);
        System.out.println(myCarLot.returnACar());
    }
}
```

This code outputs the same memory address because `myCar` and `carInLot` have the same reference value:

```
Car@2f333739
Car@2f333739
```

5.7 Returns Instructions

1.

We want to have a method that returns the price plus tax.

Define a method called `getPriceWithTax()` that is intended to return the price plus the tax. It should take in no parameters and return a double.

You can leave the body of the method empty for now. **Note:** the code will have an error until we return the correct type from the method, which we will do in the next step.

2.

Inside the `getPriceWithTax()` method, create a double variable `totalPrice` that is equal to `price + price * 0.08`. 0.08 is the tax applied to the price.

Then, return `totalPrice`.

3.

Inside of `main()`, set a double variable `lemonadePrice` to the value returned by `lemonadeStand.getPriceWithTax()`.

4.

Now, print out `lemonadePrice`.

```
public class Store {  
    // instance fields  
    String productType;  
    double price;  
  
    // constructor method  
    public Store(String product, double initialPrice) {  
        productType = product;  
    }  
}
```

```

    price = initialPrice;
}

// increase price method
public void increasePrice(double priceToAdd){
    double newPrice = price + priceToAdd;
    price = newPrice;
}

// get price with tax method

// main method
public static void main(String[] args) {
    Store lemonadeStand = new Store("Lemonade", 3.75);

}
}

```

OUTPUT

4.05

5.8 The toString() Method

When we print out Objects, we often see a String that is not very helpful in determining what the Object represents. In the last lesson, we saw that when we printed our Store objects, we would see output like:

Store@6bc7c054

where Store is the name of the object and 6bc7c054 is its position in memory.

This doesn't tell us anything about what the Store sells, the price, or the other instance

fields we've defined. We can add a method to our classes that makes this printout more descriptive.

When we define a *toString()* method for a class, we can return a String that will print when we print the object:

```
class Car {  
  
    String color;  
  
    public Car(String carColor) {  
        color = carColor;  
    }  
  
    public static void main(String[] args){  
        Car myCar = new Car("red");  
        System.out.println(myCar);  
    }  
  
    public String toString(){  
        return "This is a " + color + " car!";  
    }  
}
```

When this runs, the command `System.out.println(myCar)` will print `This is a red car!`, which tells us about the Object `myCar`.

5.8 The *toString()* Method Instructions

1.

In the `main()` method, print the Objects `lemonadeStand` and `cookieShop`. Are these printouts helpful in understanding these Objects?

2.

Create a toString() method for the Store class. The method signature should say that it is public, and that it returns a String. It shouldn't take in any parameters. For now, have it return the String "Store".

3.

"Store" isn't very helpful! What kind of Store is it?

Change the toString() to return a String that describes this Store object.

Your String should look like:

This store sells productType at a price of price.

where productType and price are the values in those instance fields. For example, if it was a hat store where hats cost 8, the String would say:

This store sells hats at a price of 8.

4.

Look at the printouts again. Are they more helpful now?

```
public class Store {  
    // instance fields  
    String productType;  
    double price;  
  
    // constructor method  
    public Store(String product, double initialPrice) {  
        productType = product;  
        price = initialPrice;  
    }  
  
    // increase price method  
    public void increasePrice(double priceToAdd){
```

```
double newPrice = price + priceToAdd;
price = newPrice;
}

// get price with tax method
public double getPriceWithTax(){
    double tax = 0.08;
    double totalPrice = price + price*tax;
    return totalPrice;
}

// main method
public static void main(String[] args) {
    Store lemonadeStand = new Store("Lemonade", 3.75);
    Store cookieShop = new Store("Cookies", 5);

}
}
```

OUTPUT

This store sells Lemonade at a price of 3.75.

This store sells Cookies at a price of 5.0.

5.9 Review

Great work! Methods are a powerful way to abstract tasks away and make them repeatable. They allow us to define behavior for classes, so that the Objects we create can do the things we expect them to. Let's review everything we have learned about methods so far.

- *Defining a method* : Method declarations will declare a method's return type, name, and parameters
- *Calling a method* : Methods are invoked with a . and ()
- *Parameters* : Inputs to the method and their types are declared in parentheses in the method signature
- *Changing Instance Fields* : Methods can be used to change the value of an instance field
- *Scope* : Variables only exist within the domain that they are created in
- *Return* : The type of the variables that will be output are declared in the method declaration

As you move through more Java material, it will be helpful to frame the tasks you create in terms of methods. This will help you think about what inputs you might need and what output you expect.

5.9 Review Instructions

1.

Now that we've learned about behavior, we can apply behavior to our SavingsAccount class using methods!

We've added the functionality for each method inside main() now, but you will be rebuilding each above main(). Note that your methods can directly access the balance field.

First, write a method called checkBalance() that prints:

Hello!

Your balance is

with the balance of the account displayed.

It should take in no parameters and return nothing.

2.

Now, write a method called `deposit()` that takes in an int parameter `amountToDeposit` and adds it to the balance. It should return nothing.

If you want, you can also have the method print:

You just deposited `amountToDeposit`
with the value of `amountToDeposit` displayed.

3.

Now, write a method called `withdraw()` that takes in an int parameter `amountToWithdraw` and subtracts it from the balance. It should return the `amountToWithdraw`.

If you want, you can also have the method print:

You just withdrew `amountToWithdraw`
with the value of `amountToWithdraw` displayed.

4.

Test out your methods by trying to replace some of the code in the `main()` method with the equivalent methods!

Make sure to use `checkBalance()`, `deposit()`, and `withdraw()` at least once each.

5.

Congratulations! You've made a basic `SavingsAccount`.

If you want, you can add more functionality to this! What other instance fields might you want to keep track of? What might a `toString()` look like for this class?

```
public class SavingsAccount {
```

```
int balance;
```

```
public SavingsAccount(int initialBalance){  
    balance = initialBalance;  
}
```

```
public static void main(String[] args){  
    SavingsAccount savings = new SavingsAccount(2000);  
  
    //Check balance:  
    System.out.println("Hello!");  
    System.out.println("Your balance is "+savings.balance);
```

```
  
    //Withdrawing:  
    int afterWithdraw = savings.balance - 300;  
    savings.balance = afterWithdraw;  
    System.out.println("You just withdrew "+300);
```

```
  
    //Check balance:  
    System.out.println("Hello!");  
    System.out.println("Your balance is "+savings.balance);
```

```
  
    //Deposit:  
    int afterDeposit = savings.balance + 600;  
    savings.balance = afterDeposit;  
    System.out.println("You just deposited "+600);
```

```
//Check balance:
System.out.println("Hello!");
System.out.println("Your balance is "+savings.balance);

//Deposit:
int afterDeposit2 = savings.balance + 600;
savings.balance = afterDeposit2;
System.out.println("You just deposited "+600);

//Check balance:
System.out.println("Hello!");
System.out.println("Your balance is "+savings.balance);
}
}
```

OUTPUT

```
Hello!
Your balance is 2000
You just withdrew 300
Hello!
Your balance is 1700
You just deposited 600
Hello!
Your balance is 2300
You just deposited 600
Hello!
```

Your balance is 2900

This is a savings account with 2900 saved.

5. Quiz

1. An object named dalmatian belongs to the Spot class. Which of the following correctly calls the run method on dalmatian?

- a) Spot.dalmatian()
- b) dalmatian.run();
- c) Spot.run();
- d) Spot[run]

2. When defining a method, the void keyword specifies which of the following?

- a) The method does not return a value
- b) The method can run without being called
- c) The method returns default Java errors
- d) The method does not take any parameters

3. Is there an error that will prevent this code from running?

```
class Coffee{

    public Coffee(){
    }

    public void stir(){
        String stirring = "Stirring the coffee!";
    }

    public static void main(String[] args){
        Coffee myCup = new Coffee();
        myCup.stir();
        System.out.println(stirring);
    }
}
```


}

- a) Yes! The constructor cannot be empty.
- b) No.
- c) Yes! The stir() method should be returning a value.
- d) Yes! The variable stirring cannot be accessed in the main() method.

4. Which method signature would make a method that returns an int and takes in a String?

- a) public intReturn()
- b) public int countLetters(String message)
- c) public void countLetters(String message, int numLetters)
- d) public String countLetters(int numLetters)

5. In this method definition, what does miles represent?

```
public void run(int miles) {  
}
```

- a) An object belonging to the Miles class
- b) A parameter of type int
- c) A Java specific keyword
- d) A constructor

6. Which line of code is NOT an example of calling a method?

- a) System.out.println("More sugar please!");
- b) int sugarCubes = 7;
- c) "hello".equals("goodbye");
- d) myCoffeeCup.emptyCup();

7.

Fill in the code that will call the `addSugar()` method with 7 as an input.

```
public class Coffee {  
  
    public Coffee(){  
  
    }  
  
    public void addSugar(int sugarCubes){  
        System.out.println("Added " + sugarCubes + " sugar cubes!");  
    }  
  
    public static void main(String[] args){  
        Coffee myCup = new Coffee()  
          
    }  
}
```

`int sugar = myCup.addSugar();`

`addSugar(7);`

`myCup.addSugar(7);`

`myCup.addSugar;`

Project-4

A Basic Calculator

In this project, you will use classes, methods, and objects to create a simple arithmetic calculator. The calculator will be able to:

- Add two integers
- Subtract two integers
- Multiply two integers
- Divide two integers
- Apply the modulo operator on two integers

The instructions provided are general guidelines. Upon completion of the project, feel free to explore more in the learning environment.

Important: Moving forward, all projects will require that you define the main method manually. When you click the “Save” button below, an attempt to run your Java code is made. However, Java will return an error if you attempt to run a Java program without first defining a main method. If you encounter such an error, do not worry, you can define the main method ahead of time in order to avoid seeing the error.

Tasks

Basic Calculator

1.

Create a public class called Calculator.

2.

Inside of the class, create a Calculator() constructor. You can leave the contents of the constructor empty.

3.

Next, create a public method that returns an int and call it add().

4.

The add() method should accept two int parameters. For example: int a, int b.

5.

The add method should add the two integer parameters that a user will specify. Inside of the add method, return the sum of a and b.

6.

Next, create another similar method called subtract(). The subtract() method should accept two int parameters, just like the add() method does.

7.

Inside of the `subtract()` method, return the difference of a and b.

8.

Create another method called `multiply()`. The `multiply()` method should accept two int parameters.

9.

Inside of the `multiply` method, return the product of a and b.

10.

Create another method called `divide()`. It should accept two int parameters.

11.

Inside of the `divide()` function, return a divided by b.

12.

Create another method called `modulo`. It should accept two int parameters.

13.

Inside of the `modulo()` function, return a modulo b.

14.

Next, create a `main()` method. Can you remember all the keywords necessary for a `main()` method?

15.

Inside of `main()`, create a `Calculator` object called `myCalculator`.

16.

Print out the value of calling the `add()` method on `myCalculator`. Pass in 5 and 7 as parameters.

17.

On the next line, print out the value of calling the `subtract()` method on `myCalculator`. Pass in 45 and 11 as parameters.

18.

If you completed this project correctly, the output should be 12 and 34. Feel free to explore more with the program. What are some ways in which the program could be improved?

19.

Add comments near the top of the program that describe what the program does.

OUTPUT

12

34

Project-5

Build A Droid

We've set up a robot workshop to build some droids. All that's missing are the instructions on how to create the robots and what they'll do.

Can we write a Java class to help?

We'll need to define the state and behavior of the droids using instance fields and methods. Let's get to work!

Tasks

1.

The **Droid.java** file is empty.

Start by defining the class `Droid`.

Don't forget to include a `main()` method! You can leave it empty for now.

We want a Droid object that has the following state:

- name
- battery level

and the following behavior:

- performing a task
- stating its battery level

2.

Declare an instance field called `batteryLevel`. We want to store whole number values in this field.

3.

Declare another instance field called `name` which will store our droid's name.

What type should this be?

4.

Create a constructor method for the Droid class.

The method should have one parameter of `String droidName`.

5.

Inside the constructor, assign the parameter value of `droidName` to the appropriate instance field.

Set the value of `batteryLevel` to 100. Every new instance of Droid will have a `batteryLevel` of 100.

Declaring Instances of Drone

6.

Inside `main()`, create a new instance of `Droid` named "Codey".

7.

Print out the variable using `System.out.println()`.

8.

That output isn't very informative!

Define a `toString()` method within `Droid`.

The return type is `String`.

Inside `toString()`, return a string that introduces the `Droid` using their name.

Something like "Hello, I'm the droid: `droidNameHere`"

Doing Droid Things

9.

Define a new method: `performTask()`. This method should have a single parameter: `String task`.

This method does not return any value.

10.

Inside `performTask()`, print a statement like "name is performing task: `task`".

For example, `codey.performTask("dancing");` will print:

Codey is performing task: dancing

11.

Performing tasks is hard work. After the print statement, set `batteryLevel` to be 10 less

than it was before.

We'll need to reassign the instance field to be the current value minus 10.

12.

Have your Droid instance perform some tasks inside of main().

Next steps

13.

Create new instances and play around with methods. Here are some ideas to get you started:

- Create a `energyReport()` method that prints the instance's `batteryLevel`.
- Create another instance.
- Create a method `energyTransfer()` that exchanges `batteryLevel` between instances.

Concept Review

Java objects' state and behavior

In Java, instances of a class are known as objects. Every object has state and behavior in the form of instance fields and methods respectively.

```
public class Person {  
    // state of an object  
    int age;  
    String name;  
  
    // behavior of an object
```



```

public void set_value() {
    age = 20;
    name = "Robin";
}

public void get_value() {
    System.out.println("Age is " + age);
    System.out.println("Name is " + name);
}

// main method
public static void main(String [] args) {
    // creates a new Person object
    Person p = new Person();

    // changes state through behavior
    p.set_value();
}
}

```

Java instance

Java instances are objects that are based on classes. For example, Bob may be an instance of the class Person.

Every instance has access to its own set of variables which are known as *instance fields*, which are variables declared within the scope of the instance. Values for instance fields are assigned within the constructor method.

```

public class Person {
    int age;

```

```
String name;
```

```
// Constructor method
```

```
public Person(int age, String name) {  
    this.age = age;  
    this.name = name;  
}
```

```
public static void main(String[] args) {  
    Person Bob = new Person(31, "Bob");  
    Person Alice = new Person(27, "Alice");  
}  
}
```

Java dot notation

In Java programming language, we use . to access the variables and methods of an object or a Class.

This is known as *dot notation* and the structure looks like this-

instanceOrClassName.fieldOrMethodName

```
public class Person {
```

```
    int age;
```

```
    public static void main(String [] args) {
```

```
        Person p = new Person();
```

```
        // here we use dot notation to set age
```

```
        p.age = 20;
```

```
// here we use dot notation to access age and print
System.out.println("Age is " + p.age);
// Output: Age is 20
}
}
```

Constructor Method in Java

Java classes contain a *constructor* method which is used to create instances of the class.

The constructor is named after the class. If no constructor is defined, a default empty constructor is used.

```
public class Maths {
    public Maths() {
        System.out.println("I am constructor");
    }
    public static void main(String [] args) {
        System.out.println("I am main");
        Maths obj1 = new Maths();
    }
}
```

Creating a new Class instance in Java

In Java, we use the new keyword followed by a call to the class constructor in order to create a new *instance* of a class.

The constructor can be used to provide initial values to instance fields.

```
public class Person {
    int age;
```

```
// Constructor:
public Person(int a) {
    age = a;
}

public static void main(String [] args) {
    // Here, we create a new instance of the Person class:
    Person p = new Person(20);
    System.out.println("Age is " + p.age); // Prints: Age is 20
}
}
```

Reference Data Types

A variable with a reference data type has a value that references the memory address of an instance. During variable declaration, the class name is used as the variable's type.

```
public class Cat {
    public Cat() {
        // instructions for creating a Cat instance
    }

    public static void main(String[] args) {
        // garfield is declared with reference data type `Cat`
        Cat garfield = new Cat();
        System.out.println(garfield); // Prints: Cat@76ed5528
    }
}
```

Constructor Signatures

A class can contain multiple constructors as long as they have different parameter values. A signature helps the compiler differentiate between the different constructors.

A signature is made up of the constructor's name and a list of its parameters.

// The signature is `Cat(String furLength, boolean hasClaws)`.

```
public class Cat {  
    String furType;  
    boolean containsClaws;  
  
    public Cat(String furLength, boolean hasClaws) {  
        furType = furLength;  
        containsClaws = hasClaws;  
    }  
    public static void main(String[] args) {  
        Cat garfield = new Cat("Long-hair", true);  
    }  
}
```

null Values

null is a special value that denotes that an object has a void reference.

```
public class Bear {  
    String species;  
    public Bear(String speciesOfBear;) {  
        species = speciesOfBear;  
    }  
  
    public static void main(String[] args) {
```

```

    Bear baloo = new Bear("Sloth bear");
    System.out.println(baloo); // Prints: Bear@4517d9a3
    // set object to null
    baloo = null;
    System.out.println(baloo); // Prints: null
}
}

```

The body of a Java method

In Java, we use curly brackets {} to enclose the body of a method.

The statements written inside the {} are executed when a method is called.

```

public class Maths {
    public static void sum(int a, int b) { // Start of sum
        int result = a + b;
        System.out.println("Sum is " + result);
    } // End of sum
}

```

```

    public static void main(String [] args) {
        // Here, we call the sum method
        sum(10, 20);
        // Output: Sum is 30
    }
}

```

Method parameters in Java

In java, parameters are declared in a method definition. The parameters act as variables inside the method and hold the value that was passed in. They can be used

inside a method for printing or calculation purposes.

In the example, a and b are two parameters which, when the method is called, hold the value 10 and 20 respectively.

```
public class Maths {  
    public int sum(int a, int b) {  
        int k = a + b;  
        return k;  
    }  
  
    public static void main(String [] args) {  
        Maths m = new Maths();  
        int result = m.sum(10, 20);  
        System.out.println("sum is " + result);  
        // prints - sum is 30  
    }  
}
```

Java Variables Inside a Method

Java variables defined inside a method cannot be used outside the scope of that method.

//For example, `i` and `j` variables are available in the `main` method only:

```
public class Maths {  
    public static void main(String [] args) {  
        int i, j;  
        System.out.println("These two variables are available in main method only");  
    }  
}
```

```
}
```

Returning info from a Java method

A Java method can return any value that can be saved in a variable. The value returned must match with the return type specified in the method signature.

The value is returned using the return keyword.

```
public class Maths {  
  
    // return type is int  
    public int sum(int a, int b) {  
        int k;  
        k = a + b;  
  
        // sum is returned using the return keyword  
        return k;  
    }  
  
    public static void main(String [] args) {  
        Maths m = new Maths();  
        int result;  
        result = m.sum(10, 20);  
        System.out.println("Sum is " + result);  
        // Output: Sum is 30  
    }  
}
```

Declaring a Method

Method declarations should define the following method information: scope (private or public), return type, method name, and any parameters it receives.

// Here is a public method named sum whose return type is int and has two int parameters a and b

```
public int sum(int a, int b) {  
    return(a + b);  
}
```

6.CONDITIONALS & CONTROL FLOW

6.1 Introduction to Control Flow

Imagine we're writing a program that enrolls students in courses.

- *If* a student has completed the prerequisites, *then* they can enroll in a course.
- *Else*, they need to take the prerequisite courses.

They can't take Physics II without finishing Physics I.

We represent this kind of decision-making in our program using [*conditional*](#) or *control flow* statements. Before this point, our code runs line-by-line from the top down, but conditional statements allow us to be selective in which portions will run.

Conditional statements check a boolean condition and run a *block* of code depending on the condition. Curly braces mark the scope of a conditional block similar to a method or class.

Here's a complete conditional statement:

```
if (true) {  
  
    System.out.println("Hello World!");  
  
}
```

If the condition is true, then the block is run. So Hello World! is printed.

But suppose the condition is different:

```
if (false) {  
  
    System.out.println("Hello World!");  
  
}
```

If the condition is false, then the block does not run.

This code is also called *if-then* statements: "If (condition) is true, then do something".

6.1 Introduction to Control Flow Instructions

Let's get started!

Click Next to continue.

```
public class ConditionalPlayground {  
    public static void main(String[] args) {  
  
  
  
  
  
  
    }  
}
```

6.2 If-Then Statement

The *if-then* statement is the most simple control flow we can write. It tests an expression for truth and executes some code based on it.

```
if (flip == 1) {  
  
    System.out.println("Heads!");  
  
}
```

- The `if` keyword marks the beginning of the conditional statement, followed by parentheses `()`.
- The parentheses hold a boolean datatype.

For the condition in parentheses we can also use variables that reference a boolean, or comparisons that evaluate to a boolean.

The boolean condition is followed by opening and closing curly braces that mark a block of code. This block runs if, and only if, the boolean is true.

```
boolean isValidPassword = true;
```

```

if (isValidPassword) {

    System.out.println("Password accepted!");

}

// Prints "Password accepted!"

int numberOfItemsInCart = 9;

if (numberOfItemsInCart > 12) {

    System.out.println("Express checkout not available");

}

// Nothing is printed.

```

If a conditional is brief we can omit the curly braces entirely:

```
if (true) System.out.println("Brevity is the soul of wit");
```

Note: Conditional statements do not end in a semicolon.

6.2 If-Then Statement Instructions

1.

The code editor contains an Order class to track retail shipments.

Write an if-then statement that prints High value item! when itemCost is greater than 24.00.

```

public class Order {

    public static void main(String[] args) {

        double itemCost = 30.99;
    }
}

```

```
// Write an if-then statement:
```

```
}
```

```
}
```

OUTPUT

High value item!

6.3 If-Then-Else

We've seen how to conditionally execute one block of code, but what if there are two possible blocks of code we'd like to execute?

Let's say *if* a student has the required prerequisite, *then* they enroll in the selected course, *else* they're enrolled in the prerequisite course instead.

We create an alternate conditional branch with the `else` keyword:

```
if (hasPrerequisite) {
```

```
    // Enroll in course
```

```
} else {
```

```
    // Enroll in prerequisite
```

```
}
```

This conditional statement ensures that exactly one code block will be run. If the condition, `hasPrerequisite`, is false, the block after `else` runs.

There are now two separate code blocks in our conditional statement. The first block runs if the condition evaluates to true, the second block runs if the condition evaluates to false.

This code is also called an *if-then-else* statement:

- If *condition* is true, then do something.
- Else, do a different thing.

6.3 If-Then-Else Instructions

1.

In the code editor, there is an `isFilled` value, that represents whether the order is ready to ship.

Write an if-then-else statement that:

- When `isFilled` is true, print Shipping.
- When `isFilled` is false, print Order not ready.

```
public class Order {  
  
    public static void main(String[] args) {  
  
        boolean isFilled = false;  
  
        // Write an if-then-else statement:  
  
    }
```

```
}
```

OUTPUT

Order not ready

6.4 If-Then-Else-If

The conditional structure we've learned can be chained together to check as many conditions as are required by our program.

Imagine our program is now selecting the appropriate course for a student. We'll check their submission to find the correct course enrollment.

The conditional statement now has multiple conditions that are evaluated from the top down:

```
String course = "Theatre";

if (course.equals("Biology")) {

    // Enroll in Biology course

} else if (course.equals("Algebra")) {

    // Enroll in Algebra course

} else if (course.equals("Theatre")) {

    // Enroll in Theatre course

} else {

    System.out.println("Course not found!");

}
```

The first condition to evaluate to true will have that code block run. Here's an example demonstrating the order:

```
int testScore = 72;

if (testScore >= 90) {

    System.out.println("A");

} else if (testScore >= 80) {

    System.out.println("B");

} else if (testScore >= 70) {

    System.out.println("C");

} else if (testScore >= 60) {

    System.out.println("D");

} else {

    System.out.println("F");

}

// prints: C
```

This chained conditional statement has two conditions that evaluate true. Because `testScore >= 70` comes before `testScore >= 60`, only the earlier code block is run.

Note: Only one of the code blocks will run.

6.4 If-Then-Else-If Instructions

1.

We need to calculate the shipping costs for our orders.

There's a new instance field, `String shipping`, that we use to calculate the cost.

Use a chained if-then-else to check for different values within the calculateShipping() method.

When the shipping instance field equals "Regular", the method should return 0.

When the shipping instance field equals "Express", the method should return 1.75.

Else the method should return .50.

```
public class Order {  
    boolean isFilled;  
    double billAmount;  
    String shipping;  
  
    public Order(boolean filled, double cost, String shippingMethod) {  
        if (cost > 24.00) {  
            System.out.println("High value item!");  
        }  
        isFilled = filled;  
        billAmount = cost;  
        shipping = shippingMethod;  
    }  
  
    public void ship() {  
        if (isFilled) {  
            System.out.println("Shipping");  
            System.out.println("Shipping cost: " + calculateShipping());  
        } else {
```

```

        System.out.println("Order not ready");
    }
}

public double calculateShipping() {
    // declare conditional statement here

}

```

```

public static void main(String[] args) {
    // do not alter the main method!

    Order book = new Order(true, 9.99, "Express");
    Order chemistrySet = new Order(false, 72.50, "Regular");

    book.ship();
    chemistrySet.ship();
}
}

```

OUTPUT

High value item!

Shipping

Shipping cost: 1.75

Order not ready

6.5 Nested Conditional Statements

We can create more complex conditional structures by creating *nested conditional statements*, which is created by placing conditional statements inside other conditional statements:

```
if (outer condition) {  
    if (nested condition) {  
        Instruction to execute if both conditions are true  
    }  
}
```

When we implement nested conditional statements, the outer statement is evaluated first. If the outer condition is true, then the inner, nested statement is evaluated.

Let's create a program that helps us decide what to wear based on the weather:

```
int temp = 45;  
boolean raining = true;  
  
if (temp < 60) {  
    System.out.println("Wear a jacket!");  
    if (raining == true) {  
        System.out.println("Bring your umbrella.");  
    } else {  
        System.out.println("Leave your umbrella home.");  
    }  
}
```

In the code snippet above, our compiler will check the condition in the first if-then statement: `temp < 60`. Since `temp` has a value of 45, this condition is true; therefore, our program will print `Wear a jacket!`.

Then, we'll evaluate the condition of the nested if-then statement: `raining == true`. This condition is also true, so `Bring your umbrella` is also printed to the screen.

Note that, if the first condition was false, the nested condition would not be evaluated.

6.5 Nested Conditional Statements Instructions

1.

The company offers a temporary deal that, if the consumer uses the coupon "ship50", the company will reduce the express shipping price.

Let's rewrite the body of else-if statement from the last exercise. Inside the else-if statement, create a nested if-then statement that checks if couponCode equals "ship50".

If the nested condition is true, return the value .85.

If the condition is false, use a nested else statement to return the value 1.75.

```
public class Order {  
    boolean isFilled;  
    double billAmount;  
    String shipping;  
    String couponCode;  
  
    public Order(boolean filled, double cost, String shippingMethod, String coupon) {  
        if (cost > 24.00) {  
            System.out.println("High value item!");  
        }  
        isFilled = filled;  
        billAmount = cost;  
        shipping = shippingMethod;  
        couponCode = coupon;  
    }  
}
```

```
public void ship() {  
    if (isFilled) {  
        System.out.println("Shipping");  
        System.out.println("Shipping cost: " + calculateShipping());  
    } else {  
        System.out.println("Order not ready");  
    }  
}
```

```
public double calculateShipping() {  
    if (shipping.equals("Regular")) {  
        return 0;  
    } else if (shipping.equals("Express")) {  
        // Add your code here  
  
    } else {  
        return .50;  
    }  
}
```

```
public static void main(String[] args) {  
    // do not alter the main method!  
    Order book = new Order(true, 9.99, "Express", "ship50");
```

```
Order chemistrySet = new Order(false, 72.50, "Regular", "freeShipping");
```

```
book.ship();
```

```
chemistrySet.ship();
```

```
}
```

```
}
```

OUTPUT

High value item!

Shipping

Shipping cost: 0.85

Order not ready

6.6 Switch Statement

An alternative to chaining if-then-else conditions together is to use the [switch](#) statement. This conditional will check a given value against any number of conditions and run the code block where there is a match.

Here's an example of our course selection conditional as a switch statement instead:

```
String course = "History";
```

```
switch (course) {
```

```
case "Algebra":
```

```
    // Enroll in Algebra
```

```
    break;
```

```
case "Biology":
```

```
    // Enroll in Biology
```

```
    break;
```

```
case "History":
```

```
    // Enroll in History
```

```
    break;
```

```

case "Theatre":
    // Enroll in Theatre
    break;
default:
    System.out.println("Course not found");
}

```

This example enrolls the student in History class by checking the value contained in the parentheses, `course`, against each of the case labels. If the value after the case label matches the value within the parentheses, the *switch block* is run.

In the above example, `course` references the string "History", which matches case "History":.

When no value matches, the default block runs. Think of this as the else equivalent.

Switch blocks are different than other code blocks because they are not marked by curly braces and we use the `break` keyword to exit the switch statement.

Without `break`, code below the matching case label is run, *including code under other case labels*, which is rarely the desired behavior.

```
String course = "Biology";
```

```

switch (course) {
    case "Algebra":
        // Enroll in Algebra
    case "Biology":
        // Enroll in Biology
    case "History":
        // Enroll in History
    case "Theatre":
        // Enroll in Theatre
    default:
        System.out.println("Course not found");
}

```

```
// enrolls student in Biology... AND History and Theatre!
```

6.6 Switch Statement Instructions

1.

We'll rewrite the `calculateShipping()` method so it uses a switch statement instead.

There's an uninitialized variable `shippingCost` in `calculateShipping()`. Assign the correct value to `shippingCost` using a switch statement:

We'll check the value of the instance field `shipping`.

- When `shipping` matches "Regular", `shippingCost` should be 0.
- When `shipping` matches "Express", `shippingCost` should be 1.75.
- The default should assign .50 to `shippingCost`.

Make sure the method returns `shippingCost` after the switch statement.

```
public class Order {  
    boolean isFilled;  
    double billAmount;  
    String shipping;  
  
    public Order(boolean filled, double cost, String shippingMethod) {  
        if (cost > 24.00) {  
            System.out.println("High value item!");  
        }  
        isFilled = filled;  
        billAmount = cost;  
        shipping = shippingMethod;  
    }  
}
```



```
}
```

```
public void ship() {  
    if (isFilled) {  
        System.out.println("Shipping");  
        System.out.println("Shipping cost: " + calculateShipping());  
    } else {  
        System.out.println("Order not ready");  
    }  
}
```

```
public double calculateShipping() {  
    double shippingCost;  
        // declare switch statement here  
  
    return shippingCost;  
}
```

```
public static void main(String[] args) {  
    // do not alter the main method!  
    Order book = new Order(true, 9.99, "Express");  
    Order chemistrySet = new Order(false, 72.50, "Regular");
```

```
    book.ship();  
    chemistrySet.ship();  
}  
}
```

OUTPUT

High value item!

Shipping

Shipping cost: 1.75

Order not ready

6.7 Review

Before this lesson, our code executed from top to bottom, line by line.

Conditional statements add branching paths to our programs. We use conditionals to make decisions in the program so that different inputs will produce different results.

Conditionals have the general structure:

```
if (condition) {  
    // consequent path  
} else {  
    // alternative path  
}
```

Specific conditional statements have the following behavior:

- if-then:
 - code block runs if condition is true
- if-then-else:
 - one block runs if conditions is true
 - another block runs if condition is false

- if-then-else chained:
 - same as if-then but an arbitrary number of conditions
- switch:
 - switch block runs if condition value matches case value

6.7 Review Instructions

Our complete Order program is in the text editor but the main() method is empty. Create different Order instances and see if you can run the code in all the different conditional blocks!

```
public class Order {  
    boolean isFilled;  
    double billAmount;  
    String shipping;  
  
    public Order(boolean filled, double cost, String shippingMethod) {  
        if (cost > 24.00) {  
            System.out.println("High value item!");  
        } else {  
            System.out.println("Low value item!");  
        }  
        isFilled = filled;  
        billAmount = cost;  
        shipping = shippingMethod;  
    }  
}
```

```
public void ship() {  
    if (isFilled) {  
        System.out.println("Shipping");  
    } else {  
        System.out.println("Order not ready");  
    }  
  
    double shippingCost = calculateShipping();  
  
    System.out.println("Shipping cost: ");  
    System.out.println(shippingCost);  
}
```

```
public double calculateShipping() {  
    double shippingCost;  
    switch (shipping) {  
        case "Regular":  
            shippingCost = 0;  
            break;  
        case "Express":  
            shippingCost = 1.75;  
            break;  
        default:  
            shippingCost = .50;
```

```
}  
    return shippingCost;  
}
```

```
public static void main(String[] args) {  
    // create instances and call methods here!
```

```
}
```

```
}
```

OUTPUT

Low value item!

Shipping

Shipping cost:

0.0

High value item!

Order not ready

Shipping cost:

1.75

Low value item!

Shipping

Shipping cost:

0.5

High value item!

Order not ready

Shipping cost:

0.0

7. CONDITIONAL OPERATORS

7.1 Introduction to Conditional Operators

Java includes operators that only use boolean values. These *conditional operators* help simplify expressions containing complex boolean relationships by reducing multiple boolean values to a single value: true or false.

For example, what if we want to run a code block only if *multiple* conditions are true. We could use the *AND* operator: `&&`.

Or, we want to run a code block if *at least one* of two conditions are true. We could use the *OR* operator: `||`.

Finally, we can produce the opposite value, where true becomes false and false becomes true, with the *NOT* operator: `!`.

Understanding these complex relationships can feel overwhelming at first. Luckily, *truth tables*, like the ones seen to the right, can assist us in determining the relationship between two boolean-based conditions.

In this lesson, we'll explore each of these conditional operators to see how they can be implemented into our conditional statements.

AND		
A	B	A && B
True	True	True
True	False	False
False	True	False
False	False	False

OR		
A	B	A B
True	True	True
True	False	True
False	True	True
False	False	False

NOT	
A	!A
True	False
False	True

7.1 Introduction to Conditional Operators Instructions

The text editor contains a Reservation class we'll build in this lesson.

Note the different conditional statements and operators that we're using to control the execution of the program.

Move on when you're ready!

```
public class Reservation {
    int guestCount;
    int restaurantCapacity;
    boolean isRestaurantOpen;
    boolean isConfirmed;
```



```
public Reservation(int count, int capacity, boolean open) {  
    if (count < 1 || count > 8) {  
        System.out.println("Invalid reservation!");  
    }  
    guestCount = count;  
        restaurantCapacity = capacity;  
        isRestaurantOpen = open;  
}
```

```
public void confirmReservation() {  
    if (restaurantCapacity >= guestCount && isRestaurantOpen) {  
        System.out.println("Reservation confirmed");  
        isConfirmed = true;  
    } else {  
        System.out.println("Reservation denied");  
        isConfirmed = false;  
    }  
}
```

```
public void informUser() {  
    if (!isConfirmed) {  
        System.out.println("Unable to confirm reservation, please contact  
restaurant.");  
    } else {  
        System.out.println("Please enjoy your meal!");  
    }  
}
```

```
}  
}
```

```
public static void main(String[] args) {  
    Reservation partyOfThree = new Reservation(3, 12, true);  
    Reservation partyOfFour = new Reservation(4, 3, true);  
    partyOfThree.confirmReservation();  
    partyOfThree.informUser();  
    partyOfFour.confirmReservation();  
    partyOfFour.informUser();  
}  
}
```

OUTPUT

Reservation confirmed

Please enjoy your meal!

Reservation denied

Unable to confirm reservation, please contact restaurant.

7.2 Conditional-And: &&

Let's return to our student enrollment program. We've added an additional requirement: not only must students have the prerequisite, but their tuition must be paid up as well. We have *two* conditions that must be true before we enroll the student.

Here's one way we could write the code:

```
if (tuitionPaid) {  
    if (hasPrerequisite) {  
        // enroll student
```

```
}  
}
```

We've nested two if-then statements. This does the job but we can be more concise with the *AND* operator:

```
if (tuitionPaid && hasPrerequisite) {  
    // enroll student  
}
```

The AND operator, `&&`, is used between two boolean values and evaluates to a single boolean value. If the values **on both sides** are true, then the resulting value is true, otherwise the resulting value is false.

This code illustrates every combination:

```
true && true  
// true  
false && true  
// false  
true && false  
// false  
false && false  
// false
```

7.2 Conditional-And: `&&` Instructions

1.

Our Reservation class has the method `confirmReservation()` which validates if a restaurant can accomodate a given reservation.

We need to build the conditional logic into `confirmReservation()` using three instance variables:

- `restaurantCapacity`
- `guestCount`
- `isRestaurantOpen`

Use an if-then-else statement:

If restaurantCapacity is greater than or equal to guestCount **and** the restaurant is open, print "Reservation confirmed" and set isConfirmed to true.

else print "Reservation denied" and set isConfirmed to false.

```
public class Reservation {  
    int guestCount;  
    int restaurantCapacity;  
    boolean isRestaurantOpen;  
    boolean isConfirmed;  
  
    public Reservation(int count, int capacity, boolean open) {  
        guestCount = count;  
        restaurantCapacity = capacity;  
        isRestaurantOpen = open;  
    }  
  
    public void confirmReservation() {  
        /*  
        Write conditional  
        ~~~~~~  
        if restaurantCapacity is greater  
        or equal to guestCount  
        AND  
        the restaurant is open:  
        print "Reservation confirmed"  
        set isConfirmed to true
```

```

        else:
            print "Reservation denied"
            set isConfirmed to false
        */
    }

    public void informUser() {
        System.out.println("Please enjoy your meal!");
    }

    public static void main(String[] args) {
        Reservation partyOfThree = new Reservation(3, 12, true);
        Reservation partyOfFour = new Reservation(4, 3, true);
        partyOfThree.confirmReservation();
        partyOfThree.informUser();
        partyOfFour.confirmReservation();
        partyOfFour.informUser();
    }
}

```

OUTPUT

Reservation confirmed.

Please enjoy your meal!

Reservation denied.

Please enjoy your meal!

7.3 Conditional-Or: ||

The requirements of our enrollment program have changed again. Certain courses have prerequisites that are satisfied by multiple courses. As long as students have taken **at least one** prerequisite, they should be allowed to enroll.

Here's one way we could write the code:

```
if (hasAlgebraPrerequisite) {  
    // Enroll in course  
}
```

```
if (hasGeometryPrerequisite) {  
    // Enroll in course  
}
```

We're using two different if-then statements with **the same code block**. We can be more concise with the *OR* operator:

```
if (hasAlgebraPrerequisite || hasGeometryPrerequisite) {  
    // Enroll in course  
}
```

The OR operator, `||`, is used between two boolean values and evaluates to a single boolean value. If **either of the two values** is true, then the resulting value is true, otherwise the resulting value is false.

This code illustrates every combination:

```
true || true  
// true  
false || true  
// true  
true || false  
// true  
false || false  
// false
```

Keep Reading: AP Computer Science A Students

On some occasions, the compiler can determine the truth value of a logical expression by only evaluating the first boolean operand; this is known as *short-circuited evaluation*. Short-circuited evaluation only works with expressions that use && or ||.

In an expression that uses ||, the resulting value will be true as long as one of the operands has a true value. If the first operand of an expression is true, we don't need to see what the value of the other operand is to know that the final value will also be true.

For example, we can run the following code without error despite dividing a number by 0 in the second operand because the first operand had a true value:

```
if (1 > 0 || 2 / 0 == 7) {  
    System.out.println("No errors here!");  
}
```

An expression that uses && will only result in true if both operands are true. If the first operand in the expression is false, the entire value will be false.

7.3 Conditional-Or: || Instructions

1.

Let's write a message inside the Reservation() constructor that warns against bad input.

Our restaurants can't seat parties of more than 8 people, and we don't want reservations for 0 or less because that would be silly.

Inside Reservation(), write a conditional that uses ||.

If count is less than 1 **OR** greater than 8 we want to write the following message: Invalid reservation!.

```
public class Reservation {  
    int guestCount;  
    int restaurantCapacity;  
    boolean isRestaurantOpen;
```

```
boolean isConfirmed;
```

```
public Reservation(int count, int capacity, boolean open) {  
    // Write conditional statement below
```

```
    guestCount = count;  
        restaurantCapacity = capacity;  
        isRestaurantOpen = open;  
}
```

```
public void confirmReservation() {  
    if (restaurantCapacity >= guestCount && isRestaurantOpen) {  
        System.out.println("Reservation confirmed");  
        isConfirmed = true;  
    } else {  
        System.out.println("Reservation denied");  
        isConfirmed = false;  
    }  
}
```

```
public void informUser() {  
    System.out.println("Please enjoy your meal!");  
}
```



```

public static void main(String[] args) {
    Reservation partyOfThree = new Reservation(3, 12, true);
    Reservation partyOfFour = new Reservation(4, 3, true);
    partyOfThree.confirmReservation();
    partyOfThree.informUser();
    partyOfFour.confirmReservation();
    partyOfFour.informUser();
}
}

```

OUTPUT

```

Reservation confirmed
Please enjoy your meal!
Reservation denied
Please enjoy your meal!

```

7.4 Logical NOT: !

The *unary* operator NOT, `!`, works on a **single** value. This operator evaluates to the opposite boolean to which it's applied:

```

!false
// true
!true
// false

```

NOT is useful for expressing our intent clearly in programs. For example, sometimes we need the opposite behavior of an if-then: run a code block **only if** the condition is false.

```

boolean hasPrerequisite = false;

```

```

if (hasPrerequisite) {
    // do nothing
} else {
    System.out.println("Must complete prerequisite course!");
}

```

This code does what we want but it's strange to have a code block that does nothing!

The logical NOT operator cleans up our example:

```
boolean hasPrerequisite = false;
```

```

if (!hasPrerequisite) {
    System.out.println("Must complete prerequisite course!");
}

```

We can write a succinct conditional statement without an empty code block.

7.4 Logical NOT: ! Instructions

1.

Let's make `informUser()` more informative. If their reservation is not confirmed, they should know!

Write an if-then-else statement and use `!` with `isConfirmed` as the condition.

If their reservation is **not** confirmed, write Unable to confirm reservation, please contact restaurant.

Else write: Please enjoy your meal!

```

public class Reservation {
    int guestCount;
    int restaurantCapacity;
    boolean isRestaurantOpen;
    boolean isConfirmed;
}

```

```

public Reservation(int count, int capacity, boolean open) {
    if (count < 1 || count > 8) {
        System.out.println("Invalid reservation!");
    }
    guestCount = count;
        restaurantCapacity = capacity;
        isRestaurantOpen = open;
}

public void confirmReservation() {
    if (restaurantCapacity >= guestCount && isRestaurantOpen) {
        System.out.println("Reservation confirmed");
        isConfirmed = true;
    } else {
        System.out.println("Reservation denied");
        isConfirmed = false;
    }
}

public void informUser() {
    // Write conditional here

}

```

```
public static void main(String[] args) {  
    Reservation partyOfThree = new Reservation(3, 12, true);  
    Reservation partyOfFour = new Reservation(4, 3, true);  
    partyOfThree.confirmReservation();  
    partyOfThree.informUser();  
    partyOfFour.confirmReservation();  
    partyOfFour.informUser();  
}  
}
```

OUTPUT

Reservation confirmed

Please enjoy your meal!

Reservation denied

Unable to confirm reservation, please contact restaurant.

7.5 Combining Conditional Operators

We have the ability to expand our boolean expressions by using multiple conditional operators in a single expression.

For example:

```
boolean foo = true && !(false || !true)
```

How does an expression like this get evaluated by the compiler? The order of evaluation when it comes to conditional operators is as follows:

1. Conditions placed in parentheses - ()
2. NOT - !
3. AND - &&
4. OR - ||

Using this information, let's dissect the expression above to find the value of foo:

```
true && !(false || !true)
```

First, we'll evaluate (false || !true) because it is enclosed within parentheses.

Following the order of evaluation, we will evaluate !true, which equals false:

```
true && !(false || false)
```

Then, we'll evaluate (false || false) which equals false. Now our expression looks like this:

```
true && !false
```

Next, we'll evaluate !false because it uses the NOT operator. This expression equals true making our expression the following:

```
true && true
```

true && true evaluates to true; therefore, the value of foo is true.

7.5 Combining Conditional Operators Instructions

Take a look at the three expressions in **Operators.java**.

Using your understanding of the order of execution, find out whether the value of each expression is true or false.

When you're ready, uncomment the print statements to find out if you are right.

```
public class Operators {  
    public static void main(String[] args) {  
        int a = 6;  
        int b = 3;  
  
        boolean ex1 = !(a == 7 && (b >= a || a != a));  
        // System.out.println(ex1);  
  
        boolean ex2 = a == b || !(b > 3);
```

```
// System.out.println(ex2);

boolean ex3 = !(b <= a && b != a + b);

// System.out.println(ex3);

}

}
```

OUTPUT

true

true

false

7.6 Review

Conditional operators work on boolean values to simplify our code. They're often combined with conditional statements to consolidate the branching logic.

Conditional-AND, &&, evaluates to true if the booleans on *both sides* are true.

```
if (true && false) {
    System.out.println("You won't see me print!");
} else if (true && true) {
    System.out.println("You will see me print!");
}
```

Conditional-OR, ||, evaluates to true if one or both of the booleans on either side is true.

```
if (false || false) {
    System.out.println("You won't see me print!");
} else if (false || true) {
    System.out.println("You will see me print!");
}
```

```
if (!false) {
    System.out.println("You will see me print!");
}
```

The complete Reservation class is in the code editor.

```
public class Reservation {  
    int guestCount;  
    int restaurantCapacity;  
    boolean isRestaurantOpen;  
    boolean isConfirmed;  
  
    public Reservation(int count, int capacity, boolean open) {  
        if (count < 1 || count > 8) {  
            System.out.println("Invalid reservation!");  
        }  
        guestCount = count;  
        restaurantCapacity = capacity;  
        isRestaurantOpen = open;  
    }  
  
    public void confirmReservation() {  
        if (restaurantCapacity >= guestCount && isRestaurantOpen) {
```

```

        System.out.println("Reservation confirmed");
        isConfirmed = true;
    } else {
        System.out.println("Reservation denied");
        isConfirmed = false;
    }
}

public void informUser() {
    if (!isConfirmed) {
        System.out.println("Unable to confirm reservation, please contact
restaurant.");
    } else {
        System.out.println("Please enjoy your meal!");
    }
}

public static void main(String[] args) {
    // Create instances here

}
}

```

OUTPUT

Reservation confirmed

Please enjoy your meal!

Invalid reservation!

Reservation confirmed

Please enjoy your meal!

Reservation denied

Unable to confirm reservation, please contact restaurant.

6. Quiz

1.What will the following code print?

```
if (5 > 6) {  
    System.out.println("C. Senor");  
} else if (6 < 5) {  
    System.out.println("Sue Yu");  
} else {  
    System.out.println("Jim Sox");  
}
```

- a) C. Senor
- b) Sue Yu
- c) A blank line.
- d) Jim Sox

2. The break keyword is important in a switch statement due to which of the following?

- a) Java will return an error if it is not specified.
- b) It is necessary whenever the switch statement contains more than 4 cases.
- c) Without break, Java will continue to check other cases in the switch statement.
- d) Without it, the switch statement will not execute.

3. If multiple conditional operators are placed in a single expression, which operator gets evaluated first?

- a) The NOT operator - !

- b) An expression cannot contain more than one conditional operator.
- c) The AND operator - &&
- d) The OR operator - ||

4. The AND logical operator is represented in Java by which operator?

- a) &&
- b) !
- c) ||
- d) and

5. The following Boolean expression will be evaluated to what value?

!(false) && (5 > 4)

- a) unknown
- b) true
- c) (5 > 4)
- d) false

6. The OR logical operator is represented in Java by which operator?

- a) ||
- b) !
- c) OR
- d) &&

7. What will the following code print?

```
int choice = 5;
```

```
switch (choice) {  
    case 4:  
        System.out.println("Jo Fraizer");  
        break;  
    case 5:  
        System.out.println("Laila A. Lee");  
        break;  
    case 6:  
        System.out.println("Hall E. Home");  
}
```

```
break;  
default:  
    System.out.println(200);  
}
```

- a) Laila A. Lee
- b) Jo Fraizer
- c) 200
- d) Hall E. Home

Project-6

A Simple Car Loan Payment Calculator

Let's combine a few of the concepts that you have learned so far: conditionals, Boolean expressions, and arithmetic expressions.

In this project, you will write a program that will calculate the *monthly* car payment a user should expect to make after taking out a car loan. The program will include the following:

- Car loan amount
- Interest rate of the loan
- Length of the loan (in years)
- Down payment

The instructions provided are general guidelines. Upon completion of the project, feel free to add functionality of your own.

If you get stuck during this project or would like to see an experienced developer work through it, click "**Get Unstuck**" to see a **project walkthrough video**.

Tasks

Car Loan Payment Calculator

1.

Create an int variable called carLoan and set it equal to 10000.

2.

Next, create an int variable called loanLength and set it equal to 3. This will represent a loan length of 3 years.

3.

Now create an int variable called interestRate and set it equal to 5. This will represent an interest rate of 5% on the loan.

4.

Next, create an int variable called downPayment and set it equal 2000. This will represent the down payment provided by a user for this car purchase.

5.

Let's build in some requirements that would prevent a buyer from taking out an invalid car loan. Write an if statement that checks whether the loan length is less than or equal to zero or whether the interest rate is less than or equal to zero.

6.

Next, inside of the if statement, print out a helpful error message to the user. For example, you can print out something like: Error! You must take out a valid car loan.

7.

What if the down payment is more than the price of the car? Add to the if statement and use else if to check whether the down payment is greater than or equal to the car loan.

8.

Inside of the else if block, print out a helpful message to the user about the down payment and car loan amounts. For example, you can print out something like: The car can be paid in full.

9.

Finally, if none of the previous Boolean expressions evaluate to true, calculate the monthly payment in an else block.

10.

Inside of the else block, create an int variable called remainingBalance and set it equal to carLoan minus downPayment.

11.

Since we need the *monthly* payment, we must convert the loan length from years to months. On the next line, create an int variable called months and set it equal to loanLength times 12.

12.

Create an int variable called monthlyBalance and set it equal to remainingBalance divided by months. This represents the monthly payment *without* interest included.

13.

The user needs to pay interest on the loan borrowed. Create an int variable called interest and set it equal to the monthly balance times the interest rate, divided all by 100.

14.

Calculate the final monthly payment. Create an int variable called monthlyPayment and set it equal to the monthly balance plus the interest.

15.

On the next line, print out the monthly payment. If you correctly completed this project, the console should print out 233 as the monthly payment.

16.

Add comments near the top of the program that describe what the program does.

```
public class CarLoan {  
    public static void main(String[] args) {
```

```
}  
  
}
```

Project-7

Continents and Cities

Planet Earth is a magical place. Let's practice the Java switch statement that you learned about.

Write a **Continents.java** program that will print out a continent and the largest city in that continent, based on the value of an integer.

The instructions provided are general guidelines. Upon completion of the project, feel free to add functionality of your own.

If you get stuck during this project or would like to see an experienced developer work through it, click **Get Unstuck** to see a project walkthrough.

Tasks

Setting up:

1.

Let's create a skeleton for the program. Add:

```
public class Continents {  
  
    public static void main(String[] args) {  
  
  
  
  
  
  
  
  
}
```

}

2.

Write a comment near the top of the program that describe what the program does.

3.

Create an int variable called continent and set it equal to 4.

Write a switch statement:

4.

Create a switch statement that will switch based on the value of continent.

5.

Inside of the switch statement, add a case that will run when the value of continent is 1.

6.

When the value of continent is 1, print out North America: Mexico City, Mexico.

7.

Make sure the next line exits out of the case.

8.

Add another case that will run when the value of continent is 2. When this value is met, print out South America: Sao Paulo, Brazil.

9.

Make sure the next line exits out of the case.

10.

Add another case that will run when the value of continent is 3. When this value is met, print out Europe: Moscow, Russia. Make sure the next line exits out of the case.

11.

Add another case that will run when the value of continent is 4. When this value is met, print out Africa: Lagos, Nigeria. Make sure the next line exits out of the case.

12.

Add another case that will run when the value of continent is 5. When this value is met, print out Asia: Shanghai, China. Make sure the next line exits out of the case.

13.

Add another case that will run when the value of continent is 6. When this value is met, print out Australia: Sydney, Australia. Make sure the next line exits out of the case.

14.

Add another case that will run when the value of continent is 7. When this value is met, print out Antarctica: McMurdo Station, US. Make sure the next line exits out of the case.

15.

Finally, add the default case. The default case should print out Undefined continent! Make sure the next line exits out of the case.

16.

If the program is written correctly, your output should be Africa: Lagos, Nigeria. Great work!

Concept Review

else Statement

The else statement executes a block of code when the condition inside the if statement is false. The else statement is always the last condition.

```
boolean condition1 = false;
```

```
if (condition1){  
    System.out.println("condition1 is true");  
}  
else{  
    System.out.println("condition1 is not true");  
}  
// Prints: condition1 is not true
```

else if Statements

else-if statements can be chained together to check multiple conditions. Once a condition is true, a code block will be executed and the conditional statement will be exited.

There can be multiple else-if statements in a single conditional statement.

```
int testScore = 76;  
char grade;  
  
if (testScore >= 90) {  
    grade = 'A';  
} else if (testScore >= 80) {  
    grade = 'B';  
} else if (testScore >= 70) {  
    grade = 'C';  
}
```

```
} else if (testScore >= 60) {  
    grade = 'D';  
} else {  
    grade = 'F';  
}
```

```
System.out.println("Grade: " + grade); // Prints: C
```

if Statement

An if statement executes a block of code when a specified boolean expression is evaluated as true.

```
if (true) {  
    System.out.println("This code executes");  
}
```

```
// Prints: This code executes
```

```
if (false) {  
    System.out.println("This code does not execute");  
}
```

```
// There is no output for the above statement
```

Nested Conditional Statements

A nested conditional statement is a conditional statement nested inside of another conditional statement. The outer conditional statement is evaluated first; if the condition is true, then the nested conditional statement will be evaluated.

```
boolean studied = true;
```

```
boolean wellRested = true;
```

```
if (wellRested) {  
    System.out.println("Best of luck today!");  
    if (studied) {  
        System.out.println("You are prepared for your exam!");  
    } else {  
        System.out.println("Study before your exam!");  
    }  
}
```

```
// Prints: Best of luck today!
```

```
// Prints: You are prepared for your exam!
```

AND Operator

The AND logical operator is represented by `&&`. This operator returns true if the boolean expressions on both sides of the operator are true; otherwise, it returns false.

```
System.out.println(true && true); // Prints: true
```

```
System.out.println(true && false); // Prints: false
```

```
System.out.println(false && true); // Prints: false
```

```
System.out.println(false && false); // Prints: false
```

NOT Operator

The NOT logical operator is represented by `!`. This operator negates the value of a boolean expression.

```
boolean a = true;
```

```
System.out.println(!a); // Prints: false
```

```
System.out.println(!false) // Prints: true
```

The OR Operator

The logical OR operator is represented by `||`. This operator will return true if at least one of the boolean expressions being compared has a true value; otherwise, it will return false.

```
System.out.println(true || true); // Prints: true
```

```
System.out.println(true || false); // Prints: true
```

```
System.out.println(false || true); // Prints: true
```

```
System.out.println(false || false); // Prints: false
```

Conditional Operators - Order of Evaluation

If an expression contains multiple conditional operators, the order of evaluation is as follows: Expressions in parentheses -> NOT -> AND -> OR.

```
boolean foo = true && (!false || true); // true
```

```
/*
```

`(!false || true)` is evaluated first because it is contained within parentheses.

Then `!false` is evaluated as true because it uses the NOT operator.

Next, `(true || true)` is evaluation as true.

Finally, `true && true` is evaluated as true meaning `foo` is true. `*/`

8. ARRAYS

8.1 Introduction

We have seen how to store single pieces of data in variables. What happens when we need to store a group of data? What if we have a list of students in a classroom? Or a ranking of the top 10 horses finishing a horse race?

If we were storing 5 lottery ticket numbers, for example, we could create a different variable for each value:

```
int firstNumber = 4;  
int secondNumber = 8;  
int thirdNumber = 15;  
int fourthNumber = 16;  
int fifthNumber = 23;
```

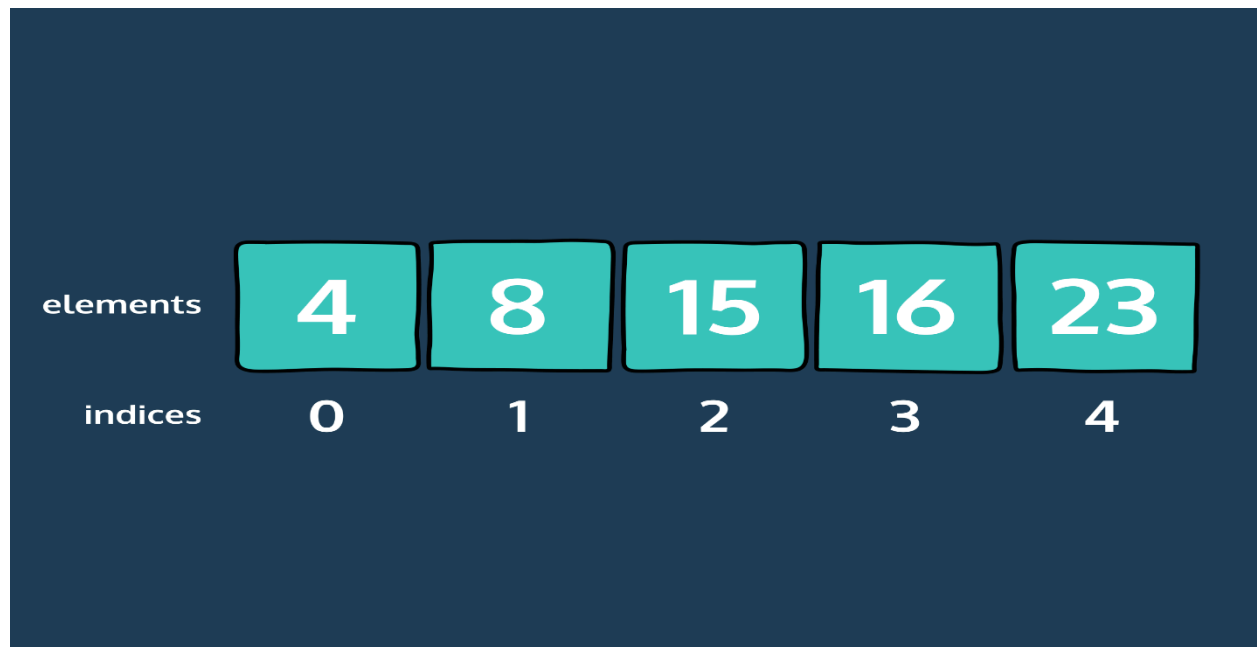
That is a lot of ungainly repeated code. What if we had a hundred lottery numbers? It is more clean and convenient to use a Java [array](#) to store the data as a list.

An array holds a fixed number of values of one type. Arrays hold doubles, ints, booleans, or any other primitives. Arrays can also contain Strings as well as object references!

Each index of an array corresponds with a different value. Here is a diagram of an array filled with integer values:

Notice that the indexes start at 0! The element at index 0 is 4, while the element at index 1 is 8. This array has a length of 5, since it holds five elements, but the highest index of the array is 4.

Let's explore how to create and use arrays in Java, so that we can store all of our Java data types.



8.1 Introduction Instructions

1.

In **Newsfeed.java**, we have created a Newsfeed class to keep track of trending articles and their associated views and ratings. We did this using Java arrays, which you'll learn about throughout the lesson!

For now, run the code to see some of our method calls.

```
public class Newsfeed {  
  
    String[] trendingArticles;  
    int[] views;  
    double[] ratings;  
  
    public Newsfeed(String[] initialArticles, int[] initialViews, double[] initialRatings){  
        trendingArticles = initialArticles;  
        views = initialViews;
```

```

    ratings = initialRatings;
}

public String getTopArticle(){
    return trendingArticles[0];
}

public void viewArticle(int articleNumber){
    views[articleNumber] = views[articleNumber] + 1;

    System.out.println("The article " + trendingArticles[articleNumber] + " has now
been viewed " + views[articleNumber] + " times!");
}

public void changeRating(int articleNumber, double newRating){
    if (newRating > 5 || newRating < 0) {
        System.out.println("The rating must be between 0 and 5 stars!");
    } else {
        ratings[articleNumber] = newRating;

        System.out.println("The article " + trendingArticles[articleNumber] + " is now
rated " + ratings[articleNumber] + " stars!");
    }
}

public static void main(String[] args){

```

```
String[] robotArticles = {"Oil News", "Innovative Motors", "Humans: Exterminate Or Not?", "Organic Eye Implants", "Path Finding in an Unknown World"};
```

```
int[] robotViewers = {87, 32, 13, 11, 7};
```

```
double[] robotRatings = {2.5, 3.2, 5.0, 1.7, 4.3};
```

```
Newsfeed robotTimes = new Newsfeed(robotArticles, robotViewers, robotRatings);
```

```
robotTimes.viewArticle(2);
```

```
robotTimes.viewArticle(2);
```

```
System.out.println("The top article is " + robotTimes.getTopArticle());
```

```
robotTimes.changeRating(3, 5);
```

```
}
```

```
}
```

OUTPUT

The article 'Humans: Exterminate Or Not?' has now been viewed 14 times!

The article 'Humans: Exterminate Or Not?' has now been viewed 15 times!

The top article is Oil News

The article 'Organic Eye Implants' is now rated 5.0 stars!

8.2 Creating an Array Explicitly

Imagine that we're using a program to keep track of the prices of different clothing items we want to buy. We would want a list of the prices and a list of the items they correspond to. To create an array, we provide a name and declare the type of data it holds:

```
double[] prices;
```

Just like with variables, we can declare and initialize in the same line. This allows us to explicitly initialize the array to contain the data we want to store :


```
double[] prices = {13.15, 15.87, 14.22, 16.66};
```

We can use arrays to hold Strings and other objects as well as primitives:

```
String[] clothingItems = {"Tank Top", "Beanie", "Funny Socks", "Corduroys"};
```

8.2 Creating an Array Explicitly Instructions

1.

We have an empty Newsfeed class that does not store anything yet.

First, make a method called `getTopics()`, which:

- is public
- returns a String array
- does not take any parameters

For now, leave the method empty.

2.

Inside the `getTopics()` method, create a String array called `topics` and set it equal to an array containing these elements, in order:

Opinion

Tech

Science

Health

Then, return `topics` from the method!

3.

Uncomment the lines in the main method to see how the `getTopics()` method works on a Newsfeed instance.

```
public class Newsfeed {
```

```
public Newsfeed(){
```

```
}
```

```
// Create getTopics() below:
```

```
public static void main(String[] args){
```

```
    Newsfeed sampleFeed = new Newsfeed();
```

```
    /*
```

```
    String[] topics = sampleFeed.getTopics();
```

```
    System.out.println(topics);
```

```
    */
```

```
}
```

```
}
```

OUTPUT

```
[Ljava.lang.String;@2aae9190
```

8.3 Importing Arrays

When we printed out the array we created in the last exercise, we saw a memory address that did not help us understand what was contained in the array.

If we want to have a more descriptive printout of the array itself, we need a `toString()` method that is provided by the *Arrays package* in Java.

```
import java.util.Arrays;
```

We put this at the top of the file, before we even define the class!

When we import a package in Java, we are making all of the methods of that package available in our code.

The Arrays package has many useful methods, including `Arrays.toString()`. When we pass an array into `Arrays.toString()`, we can see the contents of the array printed out:

```
import java.util.Arrays;

public class Lottery(){

    public static void main(String[] args){
        int[] lotteryNumbers = {4, 8, 15, 16, 23, 42};
        String betterPrintout = Arrays.toString(lotteryNumbers);
        System.out.println(betterPrintout);
    }

}
```

This code will print:

```
[4, 8, 15, 16, 23, 42]
```

8.3 Importing Arrays Instructions

1.

In order to make our printout of the topics array more helpful, we are going to want to use the `toString()` method from the Arrays package.

To use it, we first must import the Arrays package from `java.util`. Import the package at the top of the **Newsfeed.java** file.

2.

Now that you have the Arrays package, use its `toString()` method to print out the topics array in the `main()` method.

```
// import the Arrays package here:
```

```
public class Newsfeed {  
  
    public Newsfeed(){  
  
    }  
  
    public String[] getTopics(){  
        String[] topics = {"Opinion", "Tech", "Science", "Health"};  
        return topics;  
    }  
  
    public static void main(String[] args){  
        Newsfeed sampleFeed = new Newsfeed();  
        String[] topics = sampleFeed.getTopics();  
        System.out.println(topics);  
    }  
}
```

OUTPUT

[Opinion, Tech, Science, Health]

8.4 Get Element By Index

Now that we have an array declared and initialized, we want to be able to get values out of it.

We use square brackets, [and], to access data at a certain index:

```
double[] prices = {13.1, 15.87, 14.22, 16.66};
```

```
System.out.println(prices[1]);
```

This command would print:

```
15.87
```

This happens because 15.87 is the item at the 1 index of the array. Remember, the index of an array starts at 0 and ends at an index of one less than the number of elements in the array.

If we try to access an element outside of its appropriate index range, we will receive an `ArrayIndexOutOfBoundsException` error.

For example, if we were to run the command `System.out.println(prices[5])`, we'd get the following output:

```
java.lang.ArrayIndexOutOfBoundsException: 5
```

8.4 Get Element By Index Instructions

1.

We have augmented the `Newsfeed` class to start with the `topics` array as an instance field.

Fill in the `getTopTopic()` method to return the 0th index of the `topics` array.

2.

But wait — we added more to the `Newsfeed` class. Now, each instance starts with an array of zeros called `views` as an instance field.

Every time someone views a topic, we want to increment the value of the appropriate field in `views`.

For example, if someone views an "Opinion" piece, we want to increase the value of the 0th index of `views` from 0 to 1. If they view it again, we want to increase the value to 2.

We have written a method signature for `viewTopic()`, which takes in an `int` called `topicIndex`.

Inside the method, set the value of the `views` array at the index `topicIndex` to the current value plus 1.

```
import java.util.Arrays;
```

```
public class Newsfeed {
```

```
    String[] topics = {"Opinion", "Tech", "Science", "Health"};
```

```
    int[] views = {0, 0, 0, 0};
```

```
    public Newsfeed(){
```

```
}
```

```
    public String[] getTopics(){
```

```
        return topics;
```

```
}
```

```
    public String getTopTopic(){
```

```
}
```

```
    public void viewTopic(int topicIndex){
```

```
}
```

```
public static void main(String[] args){
```

```
    Newsfeed sampleFeed = new Newsfeed();
```

```
    System.out.println("The top topic is "+ sampleFeed.getTopTopic());
```

```
    sampleFeed.viewTopic(1);
```

```
    sampleFeed.viewTopic(1);
```

```
    sampleFeed.viewTopic(3);
```

```
    sampleFeed.viewTopic(2);
```

```
    sampleFeed.viewTopic(2);
```

```
    sampleFeed.viewTopic(1);
```

```
    System.out.println("The " + sampleFeed.topics[1] + " topic has been viewed " +  
sampleFeed.views[1] + " times!");
```

```
}
```

```
}
```

OUTPUT

The top topic is Opinion

The Tech topic has been viewed 3 times!

8.5 Creating an Empty Array

We can also create empty arrays and then fill the items one by one. Empty arrays have to be initialized with a fixed size:

```
String[] menuItems = new String[5];
```

Once you declare this size, it cannot be changed! This array will always be of size 5.

After declaring and initializing, we can set each index of the array to be a different item:

```
menuItems[0] = "Veggie hot dog";  
menuItems[1] = "Potato salad";  
menuItems[2] = "Cornbread";  
menuItems[3] = "Roasted broccoli";  
menuItems[4] = "Coffee ice cream";
```

This group of commands has the same effect as assigning the entire array at once:

```
String[] menuItems = {"Veggie hot dog", "Potato salad", "Cornbread", "Roasted  
broccoli", "Coffee ice cream"};
```

We can also change an item after it has been assigned! Let's say this restaurant is changing its broccoli dish to a cauliflower one:

```
menuItems[3] = "Baked cauliflower";
```

Now, the array looks like:

```
["Veggie hot dog", "Potato salad", "Cornbread", "Baked cauliflower", "Coffee ice  
cream"]
```

Keep Reading: AP Computer Science A Students

When we use new to create an empty array, each element of the array is initialized with a specific value depending on what type the element is:

Data Type	Initialized Value
int	0
double	0.0
boolean	False
Reference	Null

For example, consider the following arrays:

```
String[] my_names = new String[5];  
int[] my_ages = new int[5];
```


Because a String is a reference to an Object, my_names will contain five nulls. my_ages will contain five 0s to begin with.

8.5 Creating an Empty Array Instructions

1.

We now want a Newsfeed to be able to keep track of a user's top 10 articles.

We've declared a String array called favoriteArticles, but we haven't initialized it yet.

In the constructor for Newsfeed, set favoriteArticles to be a new empty String array of size 10.

2.

Inside the method setFavoriteArticle(), set the value of the favoriteArticles array at index favoriteIndex to be the value of newArticle.

For example, if I called setFavoriteArticle(2, "Celebrity Hands Throughout the Decades"), the value of favoriteArticles at index 2 would be set to "Celebrity Hands Throughout the Decades".

```
import java.util.Arrays;
```

```
public class Newsfeed {
```

```
    String[] topics = {"Opinion", "Tech", "Science", "Health"};
```

```
    int[] views = {0, 0, 0, 0};
```

```
    String[] favoriteArticles;
```

```
    public Newsfeed(){
```

```
        // Initialize favoriteArticles here:
```

```
}
```

```
public void setFavoriteArticle(int favoriteIndex, String newArticle){  
    // Add newArticle to favoriteArticles:
```

```
}
```

```
public static void main(String[] args){  
    Newsfeed sampleFeed = new Newsfeed();  
  
    sampleFeed.setFavoriteArticle(2, "Humans: Exterminate Or Not?");  
    sampleFeed.setFavoriteArticle(3, "Organic Eye Implants");  
    sampleFeed.setFavoriteArticle(0, "Oil News");  
  
    System.out.println(Arrays.toString(sampleFeed.favoriteArticles));  
}  
}
```

OUTPUT

[Oil News, null, Humans: Exterminate Or Not?, Organic Eye Implants, null, null, null, null, null, null]

8.6 Length

What if we have an array storing all the usernames for our program, and we want to quickly see how many users we have? To get the length of an array, we can access the length field of the array object:

```
String[] menuItems = new String[5];  
System.out.println(menuItems.length);
```

This command would print 5, since the menuItems array has 5 slots, even though they are all empty.

If we print out the length of the prices array:

```
double[] prices = {13.1, 15.87, 14.22, 16.66};
```

```
System.out.println(prices.length);
```

We would see 4, since there are four items in the prices array!

8.6 Length Instructions

1.

In the method `getNumTopics()`, return the length of the topics array.

```
import java.util.Arrays;
```

```
public class Newsfeed {
```

```
    String[] topics = {"Opinion", "Tech", "Science", "Health"};
```

```
    int[] views = {0, 0, 0, 0};
```

```
    public Newsfeed(){
```

```
    }
```

```
    public String[] getTopics(){
```

```
        return topics;
```

```
    }
```

```
public int getNumTopics(){
```

```
}
```

```
public static void main(String[] args){
```

```
    Newsfeed sampleFeed = new Newsfeed();
```

```
    System.out.println("The number of topics is "+ sampleFeed.getNumTopics());
```

```
}
```

```
}
```

OUTPUT

The number of topics is 4

8.7 String[] args

When we write main() methods for our programs, we use the parameter String[] args. Now that we know about array syntax, we can start to parse what this means.

A String[] is an array made up of Strings. Examples of String arrays:

```
String[] humans = {"Talesha", "Gareth", "Cassie", "Alex"};
```

```
String[] robots = {"R2D2", "Marvin", "Bender", "Ava"};
```

The args parameter is another example of a String array. In this case, the array args contains the arguments that we pass in from the terminal when we run the class file. (So far, args has been empty.)

So how can you pass arguments to main()? Let's say we have this class HelloYou:

```
public class HelloYou {  
    public static void main(String[] args) {  
        System.out.println("Hello " + args[0]);  
    }  
}
```

When we run the file HelloYou in the terminal with an argument of "Laura":

```
java HelloYou Laura
```

We get the output:

```
Hello Laura
```

The String[] args would be interpreted as an array with one element, "Laura".

When we use args[0] in the main method, we can access that element like we did in HelloYou.

8.7 String[] args Instructions

1.

We want to give the user an option to make a Newsfeed object for robots or for humans. We will take either "Robot" or "Human" as an argument to the main() method when the **Newsfeed.java** file is run.

- If the args array holds "Human", we will initialize the feed with human topics.
- If the args array holds "Robot", we will initialize the feed with robot topics.

Replace the blank in the conditional statement to reflect this expected control flow.

2.

In the terminal, run the **Newsfeed.java** file with the argument "Robot".

Now, run the **Newsfeed.java** file with the argument "Human".

```
import java.util.Arrays;
```

```

public class Newsfeed {

    String[] topics;

    public Newsfeed(String[] initialTopics) {
        topics = initialTopics;
    }

    public static void main(String[] args) {
        Newsfeed feed;
        if (args[0].equals("Human")) {

            //topics for a Human feed:
            String[] humanTopics = {"Politics", "Science", "Sports", "Love"};
            feed = new Newsfeed(humanTopics);

        } else if(_____) {

            //topics for a Robot feed:
            String[] robotTopics = {"Oil", "Parts", "Algorithms", "Love"};
            feed = new Newsfeed(robotTopics);

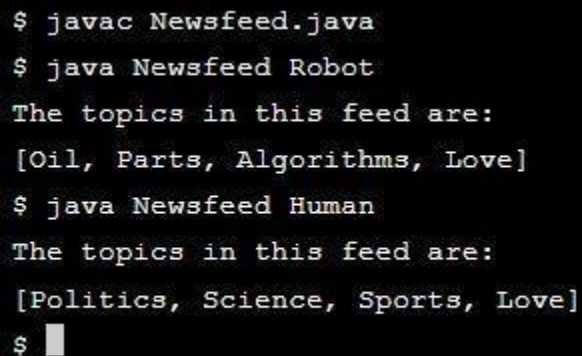
        } else {

```

```
String[] genericTopics = {"Opinion", "Tech", "Science", "Health"};
feed = new Newsfeed(genericTopics);
}
```

```
System.out.println("The topics in this feed are:");
System.out.println(Arrays.toString(feed.topics));
}
}
```

OUTPUT



```
$ javac Newsfeed.java
$ java Newsfeed Robot
The topics in this feed are:
[Oil, Parts, Algorithms, Love]
$ java Newsfeed Human
The topics in this feed are:
[Politics, Science, Sports, Love]
$ █
```

8.8 Review

We have now seen how to store a list of values in arrays. We can use this knowledge to make organized programs with more complex variables.

Throughout the lesson, we have learned about:

- Creating arrays explicitly, using { and }.
- Accessing an index of an array using [and].
- Creating empty arrays of a certain size, and filling the indices one by one.
- Getting the length of an array using length.
- Using the argument array args that is passed into the main() method of a class.

8.8 Review Instructions

1.

In order to practice the skills you've learned throughout the lesson, we're going to make and use some arrays in the `main()` method of a new `Classroom` class.

First, inside `main()`, make a `String` array called `students`, and set it equal to the following students, in order:

Sade

Alexus

Sam

Koma

2.

Now, we're going to store the averages on the most recent math test. Create an array called `mathScores` and set it to an empty array of size 4. It should hold doubles.

3.

Sade got a 94.5 on the test. Store this value in the 0th index of the `mathScores` array.

4.

Sam got a 76.8 on the test. Store this value in the appropriate spot in the `mathScores` array.

5.

Write a print statement to say:

The number of students in the class is `<numStudents>`.

with `<numStudents>` replaced by the length of the `students` array.

For example, if the array has 7 elements, the printout should read:

The number of students in the class is 7.

```
import java.util.Arrays;
```

```
public class Classroom {
```

```
    public static void main(String[] args){
```

```
    }
```

```
}
```

OUTPUT

The number of students in the class is 4.

8. Quiz

1. Fill in the code to set taxRates to be an empty array of size 10, holding doubles.

```
class Tax {
```

```
    public static void main(String[] args){
```

```
        double[] taxRates = _____;
```

```
    }
```

```
}
```

- double.length(10)
- double[10]
- new double[10]
- {1.0, 2.5, 3.5}
- Arrays.new(10)

Click or drag and drop to fill in the blank

2. How would you access the element "Gorgeous" of the following array?

```
String[] tsSongs = {"Delicate", "Gorgeous", "All Too Well", "Hey Stephen"};
```

- a) tsSongs[1]
- b) tsSongs[2]
- c) tsSongs["Gorgeous"]
- d) String[1]

3. What property holds the number of elements in an array?

- a) .size
- b) .toString
- c) .numElements
- d) .length

4. Fill in the code to print the array so that its actual values display, instead of the memory address.

```
import java.util.Arrays;
```

```
class Restaurant{
```

```
    public void printSpecials(){  
        String[] specials = {"Chicken Mountain", "Lake of Broth", "Wishbone Tower"};  
        System.out.println(_____);  
    }
```

```
    public static void main(String[] args){  
        Restaurant chickenKitchen = new Restaurant();  
        chickenKitchen.printSpecials();  
    }  
}
```

- specials.length
- specials.toString()

- Arrays.toString(specials)
- specials
- specials[0]

Click or drag and drop to fill in the blank

5. What would be printed out by the following code if a user ran the command:

java Restaurant 6 vegetarians

```
class Restaurant{

    public static void main(String[] args){
        System.out.println("Table for a party of " + args[0]);
        if(args[1].equals("vegetarians")){
            // Do something for vegetarian eaters
        }
        else {
            // Do something for meat eaters
        }
    }
}
```

- a) Table for a party of 6
- b) Table for a party of vegetarians
- c) Error – args is not defined
- d) Table for a party of Restaurant

6. What type is the following object?

{4, 5, 2, 1, 7}

- a) java.util.Arrays
- b) String[]
- c) int[]
- d) int

9. ARRAYLISTS

9.1 Introduction

When we work with arrays in Java, we've been limited by the fact that once an array is created, it has a fixed size. We can't add or remove elements.

But what if we needed to add to the book lists, newsfeeds, and other structures we were using arrays to represent?

To create mutable and dynamic lists, we can use Java's [ArrayList](#) class. ArrayList allows us to:

- Store object references as elements
- Store elements of the same type (just like arrays)
- Access elements by index (just like arrays)
- Add elements
- Remove elements

Remember how we had to import `java.util.Arrays` in order to use additional array methods? To use an ArrayList at all, we need to import them from Java's `util` package as well:

```
import java.util.ArrayList;
```

Let's learn how to make use of this powerful object...

9.1 Introduction Instructions

1.

In **Shopping.java** we've defined two arrays:

- `groceryItems`, a `String` array
- `prices`, a `double` array

We've tried to add a new item to the end of each. Run the code — does it work?

```
import java.util.Arrays;
```

```
class Shopping {
```

```
    public static void main(String[] args) {
```

```
        String[] groceryItems = {"steak", "milk", "jelly beans"};
```

```
double[] prices = {25.00, 2.95, 2.50};

// Adding ham to the groceries
groceryItems[3] = "ham";
prices[3] = 4.99;

}

}
```

OUTPUT

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3
    at Shopping.main(Shopping.java:11)
```

9.2 Creating ArrayLists

To create an ArrayList, we need to declare the type of objects it will hold, just as we do with arrays:

```
ArrayList<String> babyNames;
```

We use angle brackets < and > to declare the type of the ArrayList. These symbols are used for *generics*. Generics are a Java construct that allows us to define classes and objects as parameters of an ArrayList. For this reason, we can't use primitive types in an ArrayList:

```
// This code won't compile:
ArrayList<int> ages;
```

```
// This code will compile:
ArrayList<Integer> ages;
```

The <Integer> generic has to be used in an ArrayList instead. You can also use <Double> and <Character> for types you would normally declare as doubles or chars.

We can initialize to an empty ArrayList using the new keyword:

```
// Declaring:
```

```
ArrayList<Integer> ages;
```

```
// Initializing:
```

```
ages = new ArrayList<Integer>();
```

```
// Declaring and initializing in one line:
```

```
ArrayList<String> babyNames = new ArrayList<String>();
```

9.2 Creating ArrayLists Instructions

1.

Import the ArrayList package from java.util.

2.

Create a new ArrayList that will contain String elements and call it toDoList.

```
// import the ArrayList package here:
```

```
class ToDos {
```

```
    public static void main(String[] args) {
```

```
        // Create toDoList below:
```

```
}
```

```
}
```

9.3 Adding an Item

Now we have an empty ArrayList, but how do we get it to store values?

ArrayList comes with an [add\(\)](#) method which inserts an element into the structure. There are two ways we can use add().

If we want to add an element to the end of the ArrayList, we'll call add() using only one argument that represents the value we are inserting. In this example, we'll add objects from the Car class to an ArrayList called carShow:

```
ArrayList<Car> carShow = new ArrayList<Car>();
```

```
carShow.add(ferrari);  
// carShow now holds [ferrari]  
carShow.add(thunderbird);  
// carShow now holds [ferrari, thunderbird]  
carShow.add(volkswagen);  
// carShow now holds [ferrari, thunderbird, volkswagen]
```

If we want to add an element at a specific index of our ArrayList, we'll need two arguments in our method call: the first argument will define the index of the new element while the second argument defines the value of the new element:

```
// Insert object corvette at index 1  
carShow.add(1, corvette);  
// carShow now holds [ferrari, corvette, thunderbird, volkswagen]
```

```
// Insert object porsche at index 2  
carShow.add(2, porsche);  
// carShow now holds [ferrari, corvette, porsche, thunderbird, volkswagen]
```

By inserting a value at a specified index, any elements that appear after this new element will have their index value shift over by 1.

Also, note that an error will occur if we try to insert a value at an index that does not exist.

Keep Reading: AP Computer Science A Students

When using ArrayList methods (like add()), the reference parameters and return type of a method must match the declared element type of the ArrayList. For example, we cannot add an Integer type value to an ArrayList of String elements.

We've discussed how to specify the element type of an ArrayList; however, it is possible to create an ArrayList that holds values of different types.

In the following snippet, assortment is an ArrayList that can store different values because we do not specify its type during initialization.

```
ArrayList assortment = new ArrayList<>();  
assortment.add("Hello"); // String  
assortment.add(12); // Integer  
assortment.add(ferrari); // reference to Car  
// assortment holds ["Hello", 12, ferrari]
```

In this case, the items stored in this ArrayList will be considered Objects. As a result, they won't have access to some of their methods without doing some fancy casting. Although this type of ArrayList is allowed, using an ArrayList that specifies its type is preferred.

9.3 Adding an Item Instructions

1.

We've created an empty ArrayList called toDoList. Time to add some to-dos!

Below where we've initialized toDo1, initialize two new String variables: toDo2 and toDo3.

Set their values to any tasks you like.

2.

Use .add() to add toDo1, toDo2, and toDo3 to toDoList.

```
import java.util.ArrayList;

class ToDos {

    public static void main(String[] args) {

        ArrayList<String> toDoList = new ArrayList<String>();
        String toDo1 = "Water plants";
        // Add more to-dos here:

        // Add to-dos to toDoList

        System.out.println(toDoList);

    }

}
```

OUTPUT

```
[]
```

9.4 ArrayList Size

Let's say we have an ArrayList that stores items in a user's online shopping cart. As the user navigates through the site and adds items, their cart grows bigger and bigger.

If we wanted to display the number of items in the cart, we could find the size of it using the `size()` method:

```
ArrayList<String> shoppingCart = new ArrayList<String>();
```

```
shoppingCart.add("Trench Coat");  
System.out.println(shoppingCart.size());  
// 1 is printed  
shoppingCart.add("Tweed Houndstooth Hat");  
System.out.println(shoppingCart.size());  
// 2 is printed  
shoppingCart.add("Magnifying Glass");  
System.out.println(shoppingCart.size());  
// 3 is printed
```

In dynamic objects like ArrayLists, it's important to know how to access the amount of objects we have stored.

9.4 ArrayList Size Instructions

1.

Detectives do a lot to solve a case. But who has more to do?

Print out the size of each detective's to-do ArrayList:

- sherlocksToDos for Sherlock Holmes
- poirotsToDos for Hercule Poirot

2.

So who has more to do? Print the name of the detective whose to-do list is longer. Was it Sherlock or Poirot?

```
import java.util.ArrayList;
```

```
class ToDos {
```

```
    public static void main(String[] args) {
```

```
// Sherlock
ArrayList<String> sherlocksToDos = new ArrayList<String>();

sherlocksToDos.add("visit the crime scene");
sherlocksToDos.add("play violin");
sherlocksToDos.add("interview suspects");
sherlocksToDos.add("solve the case");
sherlocksToDos.add("apprehend the criminal");

// Poirot
ArrayList<String> poirotsToDos = new ArrayList<String>();

poirotsToDos.add("visit the crime scene");
poirotsToDos.add("interview suspects");
poirotsToDos.add("let the little grey cells do their work");
poirotsToDos.add("trim mustache");
poirotsToDos.add("call all suspects together");
poirotsToDos.add("reveal the truth of the crime");

// Print the size of each ArrayList below:

// Print the name of the detective with the larger to-do list:
```

```
}
```

```
}
```

OUTPUT

5

6

Poirot

9.5 Accessing an Index

With arrays, we can use bracket notation to access a value at a particular index:

```
double[] ratings = {3.2, 2.5, 1.7};
```

```
System.out.println(ratings[1]);
```

This code prints 2.5, the value at index 1 of the array.

For ArrayLists, bracket notation won't work. Instead, we use the method `get()` to access an index:

```
ArrayList<String> shoppingCart = new ArrayList<String>();
```

```
shoppingCart.add("Trench Coat");
```

```
shoppingCart.add("Tweed Houndstooth Hat");
```

```
shoppingCart.add("Magnifying Glass");
```

```
System.out.println(shoppingCart.get(2));
```

This code prints "Magnifying Glass", which is the value at index 2 of the ArrayList.

9.5 Accessing an Index Instructions

1.

Use `get()` to access the third to-do element of `sherlocksToDos` and print the result.

2.

Use `get()` to access the second to-do element of `poirotsToDos` and print the result.

```
import java.util.ArrayList;
```

```
class ToDos {
```

```
    public static void main(String[] args) {
```

```
        // Sherlock
```

```
        ArrayList<String> sherlocksToDos = new ArrayList<String>();
```

```
        sherlocksToDos.add("visit the crime scene");
```

```
        sherlocksToDos.add("play violin");
```

```
        sherlocksToDos.add("interview suspects");
```

```
        sherlocksToDos.add("solve the case");
```

```
        sherlocksToDos.add("apprehend the criminal");
```

```
        // Poirot
```

```
        ArrayList<String> poirotsToDos = new ArrayList<String>();
```

```
        poirotsToDos.add("visit the crime scene");
```

```
        poirotsToDos.add("interview suspects");
```

```
poirotsToDos.add("let the little grey cells do their work");  
poirotsToDos.add("trim mustache");  
poirotsToDos.add("call all suspects together");  
poirotsToDos.add("reveal the truth of the crime");
```

```
System.out.println("Sherlock's third to-do:");  
// Print Sherlock's third to-do:
```

```
System.out.println("\nPoirot's second to-do:");  
// Print Poirot's second to-do:
```

```
}
```

```
}
```

OUTPUT

Sherlock's third to-do:

interview suspects

Poirot's second to-do:

interview suspects

9.6 Changing a Value

When we were using arrays, we could rewrite entries by using bracket notation to reassign values:

```
String[] shoppingCart = {"Trench Coat", "Tweed Houndstooth Hat", "Magnifying Glass"};
```

```
shoppingCart[0] = "Tweed Cape";
```

```
// shoppingCart now holds ["Tweed Cape", "Tweed Houndstooth Hat",  
"Magnifying Glass"]
```

ArrayList has a slightly different way of doing this, using the set() method:

```
ArrayList<String> shoppingCart = new ArrayList<String>();
```

```
shoppingCart.add("Trench Coat");  
shoppingCart.add("Tweed Houndstooth Hat");  
shoppingCart.add("Magnifying Glass");
```

```
shoppingCart.set(0, "Tweed Cape");
```

```
// shoppingCart now holds ["Tweed Cape", "Tweed Houndstooth Hat",  
"Magnifying Glass"]
```

9.6 Changing a Value Instructions

1.

Modify sherlocksToDos so that the value at "play violin" becomes "listen to Dr. Watson for amusement".

2.

Modify poirotsToDos so that the value at "trim mustache" becomes "listen to Captain Hastings for amusement".

```
import java.util.ArrayList;
```

```
class ToDos {
```



```
public static void main(String[] args) {

    // Sherlock
    ArrayList<String> sherlocksToDos = new ArrayList<String>();

    sherlocksToDos.add("visit the crime scene");
    sherlocksToDos.add("play violin");
    sherlocksToDos.add("interview suspects");
    sherlocksToDos.add("solve the case");
    sherlocksToDos.add("apprehend the criminal");

    // Poirot
    ArrayList<String> poirotsToDos = new ArrayList<String>();

    poirotsToDos.add("visit the crime scene");
    poirotsToDos.add("interview suspects");
    poirotsToDos.add("let the little grey cells do their work");
    poirotsToDos.add("trim mustache");
    poirotsToDos.add("call all suspects together");
    poirotsToDos.add("reveal the truth of the crime");

    // Set each to-do below:
```

```

        System.out.println("Sherlock's to-do list:");
        System.out.println(sherlocksToDos.toString() + "\n");
        System.out.println("Poirot's to-do list:");
        System.out.println(poirotsToDos.toString());
    }

}

```

OUTPUT

Sherlock's to-do list:

[visit the crime scene, listen to Dr. Watson for amusement, interview suspects, solve the case, apprehend the criminal]

Poirot's to-do list:

[visit the crime scene, interview suspects, let the little grey cells do their work, listen to Captain Hastings for amusement, call all suspects together, reveal the truth of the crime]

9.7 Removing an Item

What if we wanted to get rid of an entry altogether? For arrays, we would have to make a completely new array without the value.

Luckily, ArrayLists allow us to remove an item by specifying the index to [remove](#):

```
ArrayList<String> shoppingCart = new ArrayList<String>();
```

```

shoppingCart.add("Trench Coat");
shoppingCart.add("Tweed Houndstooth Hat");
shoppingCart.add("Magnifying Glass");

```

```

shoppingCart.remove(1);
// shoppingCart now holds ["Trench Coat", "Magnifying Glass"]

```

We can also remove an item by specifying the value to remove:

```
ArrayList<String> shoppingCart = new ArrayList<String>();
```

```
shoppingCart.add("Trench Coat");  
shoppingCart.add("Tweed Houndstooth Hat");  
shoppingCart.add("Magnifying Glass");
```

```
shoppingCart.remove("Trench Coat");  
// shoppingCart now holds ["Tweed Houndstooth Hat", "Magnifying Glass"]
```

Note: This command removes the FIRST instance of the value "Trench Coat".

9.7 Removing an Item Instructions

1.

Sherlock Holmes and Hercule Poirot have each already visited their respective crime scenes.

Remove "visit the crime scene" from sherlocksToDos and poirotsToDos using remove().

Moreover, Sherlock Holmes has also gotten some violin playing done.

So you can remove "play violin" from sherlocksToDos as well.

```
import java.util.ArrayList;
```

```
class ToDos {
```

```
    public static void main(String[] args) {
```

```
        // Sherlock
```

```
        ArrayList<String> sherlocksToDos = new ArrayList<String>();
```

```

sherlocksToDos.add("visit the crime scene");
sherlocksToDos.add("play violin");
sherlocksToDos.add("interview suspects");
sherlocksToDos.add("solve the case");
sherlocksToDos.add("apprehend the criminal");

// Poirot
ArrayList<String> poirotsToDos = new ArrayList<String>();

poirotsToDos.add("visit the crime scene");
poirotsToDos.add("interview suspects");
poirotsToDos.add("let the little grey cells do their work");
poirotsToDos.add("trim mustache");
poirotsToDos.add("call all suspects together");
poirotsToDos.add("reveal the truth of the crime");

// Remove each to-do below:

System.out.println(sherlocksToDos.toString() + "\n");
System.out.println(poirotsToDos.toString());
}

```

```
}
```

OUTPUT

[interview suspects, solve the case, apprehend the criminal]

[interview suspects, let the little grey cells do their work, trim mustache, call all suspects together, reveal the truth of the crime]

9.8 Getting an Item's Index

What if we had a really large list and wanted to know the position of a certain element in it? For instance, what if we had an ArrayList detectives with the names of fictional detectives in chronological order, and we wanted to know what position "Fletcher" was.

```
// detectives holds ["Holmes", "Poirot", "Marple", "Spade", "Fletcher", "Conan",  
"Ramotswe"];  
System.out.println(detectives.indexOf("Fletcher"));
```

This code would print 4, since "Fletcher" is at index 4 of the detectives ArrayList.

9.8 Getting an Item's Index Instructions

1.

After visiting the crime scene, the ever-impatient Dr. Watson wants to know how many to-dos are left until Sherlock solves the case.

To help Dr. Watson figure this out, use `indexOf()` to determine where in the to-do list "solve the case" is.

Print this information out. That's the number of to-dos remaining before Sherlock reaches "solve the case".

```
import java.util.ArrayList;
```

```
class ToDos {
```

```

public static void main(String[] args) {

    // Sherlock
    ArrayList<String> sherlocksToDos = new ArrayList<String>();

    sherlocksToDos.add("visit the crime scene");
    sherlocksToDos.add("play violin");
    sherlocksToDos.add("interview suspects");
    sherlocksToDos.add("listen to Dr. Watson for amusement");
    sherlocksToDos.add("solve the case");
    sherlocksToDos.add("apprehend the criminal");

    sherlocksToDos.remove("visit the crime scene");

    // Calculate to-dos until case is solved:

    // Change the value printed:
    System.out.println("PRINT THE ANSWER HERE");

}

}

```

OUTPUT

3

9.9 Review

Nice work! You now know the basics of ArrayLists including:

- Creating an ArrayList.
- Adding a new ArrayList item using add().
- Accessing the size of an ArrayList using size().
- Finding an item by index using get().
- Changing the value of an ArrayList item using set().
- Removing an item with a specific value using remove().
- Retrieving the index of an item with a specific value using indexOf().

Now if only there were some way to move through an array or ArrayList, item by item...

9.9 Review Instructions

We've included a workspace for you to test out your newfound knowledge of arrays and ArrayLists.

Remember: To run your code, enter the following commands in the terminal:

```
javac List.java
```

and then:

```
java List
```

```
import java.util.ArrayList;
```

```
class List {
```

```
    public static void main(String[] args) {
```

```
}
```

}

9. Quiz

1. Fill in the blank to declare the ArrayList to hold the correct type.

```
import java.util.ArrayList;
```

```
public class Gradebook{
```

```
    public static void main(String[] args){
```

```
        ArrayList _____ quizGrades;
```

```
        // we will add the values 96, 87, and 46 later in the method
```

```
    }
```

```
}
```

- [int]
- <Char>
- <char>
- <int>
- [Integer]
- <Integer>

Click or drag and drop to fill in the blank

2. What method can you use to find the position of a specific element in an ArrayList?

- a) size()

b) indexOf()

c) get()

d) remove()

3. What will the ArrayList kidsMovies hold after the code is run?

```
import java.util.ArrayList;
```

```
public class Rankings {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> kidsMovies = new ArrayList<String>();
```

```
        kidsMovies.add("Moana");
```

```
        kidsMovies.add("Up");
```

```
        kidsMovies.add("WALL-E");
```

```
        kidsMovies.set(0, "Babe");
```

```
    }
```

```
}
```

a) ["Babe", "Moana", "Up", "WALL-E"]

b) ["Babe", "Up", "WALL-E"]

c) ["Moana", "Up", "WALL-E", "Babe"]

d) ["Babe", "Moana", "Up"]

4. What will the ArrayList usernames hold after this code runs?

```
import java.util.ArrayList;
```

```
public class Login {
```

```
    public static void main(String[] args) {
```

```

ArrayList<String> usernames = new ArrayList<String>();

usernames.add("teraCoder");
usernames.add("javaKnight");

}
}

```

- a) ["teraCoder", "javaKnight"]
- b) This code has an error.
- c) It will be empty.
- d) ["javaKnight", "teraCoder"]

5. Fill in the code to print out how many elements are stored in the grades ArrayList.

```

import java.util.ArrayList;

public class TemperaturesC {

    public static void main(String[] args) {

        ArrayList<Char> grades = new ArrayList<Char>();
        grades.add('B');
        grades.add('B');
        grades.add('D');

        System.out.println(_____);

    }
}

```

- grades.size
- grades.length

- `grades.size()`
- `grades.length()`

Click or drag and drop to fill in the blank

6. What will the following code print?

```
import java.util.ArrayList;
```

```
public class Temperatures {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Integer> temperatures = new ArrayList<Integer>();
```

```
        temperatures.add(78);
```

```
        temperatures.add(67);
```

```
        temperatures.add(89);
```

```
        temperatures.add(94);
```

```
        System.out.println(temperatures.get(2));
```

```
    }
```

```
}
```

a) 67

b) 2

c) 94

d) 89

7. What does the following code print?

```
import java.util.ArrayList;
```

```
public class RatingMachine {
```

```
public static void main(String[] args) {  
  
    ArrayList<Double> ratings = new ArrayList<Double>();  
  
    ratings.add(1.7);  
    ratings.add(2.5);  
    ratings.add(3.5);  
    ratings.remove(1);  
  
    System.out.println(ratings.get(1));  
  
}  
}
```

- a) 3.5
- b) 2.5
- c) 1.7
- d) Error

Project-8

Desert Island Playlist

You're heading to a desert island, cut off from the world, for the next six months. Luckily, you can bring a playlist of your favorite music 🎵

Also, this is a chance to put your hard-earned Java skills to the test. Your mission, should you choose to accept it:

Build **Playlist.java** with the best possible playlist of music using a Java ArrayList. Choose wisely.

Tasks

Creating Your Playlist

1.

We've provided you with a file **Playlist.java**.

- Since you're working with ArrayLists, import ArrayList from java.util.
- Create a Playlist class with a main() method.

2.

Inside main(), create a new ArrayList called desertIslandPlaylist that can hold String values.

3.

Time to add songs!

Use add() to add several (more than five) of your favorite songs to desertIslandPlaylist.

4.

Check out your personally curated selection of music by printing desertIslandPlaylist to the terminal. Then run your code.

Cut It Down

5.

Yikes! It turns out you can only bring five songs now... You'll need to make some cuts.

First, check the number of songs on your playlist using size(). Print the number to the terminal.

6.

Now, you'll need to make some tough decisions. Remove songs from your playlist using `remove()` until you only have five songs left.

You can check the size of your playlist along the way using `.size()` and print out the playlist to see the current songs remaining.

Swap Songs

7.

You've grown a little bored of the playlist order. Fortunately, you can change up the `ArrayList`. Pick out two songs whose order you'd like to swap.

This part is a bit trickier, so start by reading the steps you'll take to make the switch:

1. Get the indices of the songs you want to swap.
2. Create a temporary variable to store the value of song a. (We'll call the songs a and b here.)
3. Rewrite the value at the index of a to the value of b.
4. Rewrite the value at the index of b to the value of the temporary variable.

8.

Use `indexOf()` to get the indices of the two songs you want to swap in `desertIslandPlaylist`. (You can store these as variables or print them out.)

9.

Create a new `String` variable called `tempA` and initialize it with the value of song a.

10.

Call `set()` on `desertIslandPlaylist` to rewrite the song at the index of `a` with song `b`. (You're adding song `b` where `a` was in the playlist.)

You can print the playlist and run the code. Song `b` should appear twice in the playlist now.

11.

Ready to complete the swap?

Use `set()` again on `desertIslandPlaylist`. This time, give the original index of song `b` the value of your temporary variable (which is really song `a`). In other words, you're putting song `a` where song `b` used to be.

Print the final playlist to see the songs swapped from their original positions.

Remix!

12.

Nice work using `ArrayList` to build your desert island playlist!

```
import java.util.ArrayList;
```

```
class Playlist {
```

```
    public static void main(String[] args) {
```

```
    }
```

```
}
```

```
$ javac Playlist.java
$ java Playlist
[Sneaker Pimps - Six Underground, Buena Vista Social Club - Murmullo, A Tribe Called Quest - Electric Relaxation, Little Dragon - Blinking Pigs, Radiohead - Idioteque]
$ █
```

Concept Review

Index

An index refers to an element's position within an array.

The index of an array starts from 0 and goes up to one less than the total length of the array.

```
int[] marks = {50, 55, 60, 70, 80};
```

```
System.out.println(marks[0]);
```

```
// Output: 50
```

```
System.out.println(marks[4]);
```

```
// Output: 80
```

Arrays

In Java, an array is used to store a list of elements of the same datatype.

Arrays are fixed in size and their elements are ordered.

```
// Create an array of 5 int elements
```

```
int[] marks = {10, 20, 30, 40, 50};
```

Array creation in Java

In Java, an array can be created in the following ways:

- Using the {} notation, by adding each element all at once.
- Using the new keyword, and assigning each position of the array individually.

```
int[] age = {20, 21, 30};
```

```
int[] marks = new int[3];
```

```
marks[0] = 50;
```

```
marks[1] = 70;
```

```
marks[2] = 93;
```

Changing an Element Value

To change an element value, select the element via its index and use the assignment operator to set a new value.

```
int[] nums = {1, 2, 0, 4};
```

```
// Change value at index 2
```

```
nums[2] = 3;
```

Java ArrayList

In Java, an ArrayList is used to represent a dynamic list.

While Java arrays are fixed in size (the size cannot be modified), an ArrayList allows flexibility by being able to both add and remove elements.

```
// import the ArrayList package
```

```
import java.util.ArrayList;
```

```
// create an ArrayList called students
```

```
ArrayList<String> students = new ArrayList<String>();
```

Modifying ArrayLists in Java

An ArrayList can easily be modified using built in methods.

To add elements to an ArrayList, you use the add() method. The element that you want to add goes inside of the ().

To remove elements from an ArrayList, you use the remove() method. Inside the () you can specify the index of the element that you want to remove. Alternatively, you can specify directly the element that you want to remove.

```
import java.util.ArrayList;
```

```
public class Students {
```

```
    public static void main(String[] args) {
```

```
        // create an ArrayList called studentList, which initially holds []
```

```
        ArrayList<String> studentList = new ArrayList<String>();
```

```
        // add students to the ArrayList
```

```
        studentList.add("John");
```

```
        studentList.add("Lily");
```

```
        studentList.add("Samantha");
```

```
        studentList.add("Tony");
```

```
        // remove John from the ArrayList, then Lily
```

```
        studentList.remove(0);
```

```
        studentList.remove("Lily");
```

```
// studentList now holds [Samantha, Tony]

System.out.println(studentList);
}
}
```

10. LOOPS

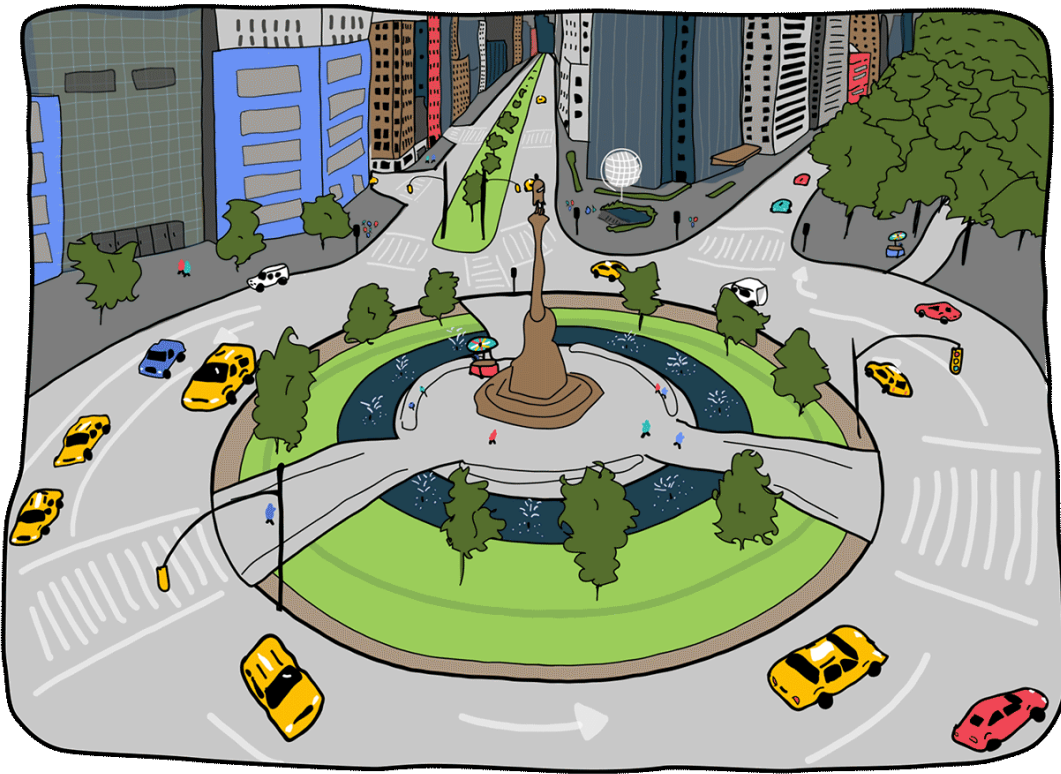
10.1 Introduction to Loops

In the programming world, we hate repeating ourselves. There are two reasons for this:

- Writing the same code over and over is time-consuming.
- Having less code means having less to debug.

But we often need to do the same task more than once. Fortunately, computers are really good (and fast) at doing repetitive tasks. And in Java, we can use loops.

A [loop](#) is a programming tool that allows developers to repeat the same block of code until some condition is met.



First, a starting condition is evaluated. If the starting condition is true, then the loop body is executed. When the last line of the loop body is executed, the condition is *re-evaluated*. This process continues until the condition is false (if the condition never becomes false, we can actually end up with an **infinite loop!**). If the starting condition is false, the loop never gets executed.

We employ loops to easily scale programs - saving time and minimizing mistakes.

We'll go over three types of loops that we'll see everywhere:

- while loops

- for loops
- for-each loops

10.2 While We're Here

A while loop looks a bit like an if statement:

```
while (silliness > 10) {
```

```
    // code to run
```

```
}
```

Like an if statement, the code inside a while loop will only run if the condition is true. However, a while loop will continue running the code over and over until the condition evaluates to false. So the code block will repeat until silliness is less than or equal to 10.

```
// set attempts to 0
```

```
int attempts = 0;
```

```
// enter loop if condition is true
```

```
while (passcode != 0524 && attempts < 4) {
```

```
    System.out.println("Try again.");
```

```
    passcode = getNewPasscode();
```

```
    attempts += 1;
```

```
    // is condition still true?
```

```
    // if so, repeat code block
```

```
}
```

```
// exit when condition is not true
```

while loops are extremely useful when you want to run some code until a specific change happens. However, if you aren't certain that change will occur, beware the infinite loop!

Infinite loops occur when the condition will never evaluate to false. This can cause your entire program to crash.

```
int hedgehogs = 5;

// This will cause an infinite loop:
while (hedgehogs < 6) {

    System.out.println("Not enough hedgehogs!");

}
```

In the example above, hedgehogs remains equal to 5, which is less than 6. So we would get an infinite loop.

10.2 While We're Here Instructions

1.

Take a look at **LuckyFive.java**. We've set up a random number generator that allows you to simulate the roll of a die.

Create a while loop that will continue to loop while dieRoll is NOT 5.

Do NOT run your code yet — you will get an infinite loop here because the value of dieRoll is never changed.

Inside the loop, reset dieRoll with a new random value between 1 and 6.

Now you can run the code.

2.

Inside the while loop, above the line where you reset dieRoll, print out dieRoll to the terminal.

```
// Importing the Random library
import java.util.Random;
```

```
class LuckyFive {  
  
    public static void main(String[] args) {  
  
        // Creating a random number generator  
        Random randomGenerator = new Random();  
  
        // Generate a number between 1 and 6  
        int dieRoll = randomGenerator.nextInt(6) + 1;  
  
        // Repeat while roll isn't 5  
  
    }  
  
}
```

OUTPUT

2
2
3
6
4

10.3 Incrementing While Loops

When looping through code, it's common to use a counter variable.

A *counter* (also known as an *iterator*) is a variable used in the conditional logic of the loop and (usually) incremented in value during each iteration through the code. For example:

```
// counter is initialized
int wishes = 0;

// conditional logic uses counter
while (wishes < 3) {

    System.out.println("Wish granted.");
    // counter is incremented
    wishes++;

}
```

In the above example, the counter wishes gets initialized before the loop with a value of 0, then the program will keep printing "Wish granted." and adding 1 to wishes as long as wishes has a value of less than 3.

Once wishes reaches a value of 3 or more, the program will exit the loop.

So the output would look like:

```
Wish granted.
Wish granted.
Wish granted.
```

We can also decrement counters like this:

```
int pushupsToDo = 10;

while (pushupsToDo > 0) {

    doPushup();
    pushupsToDo--;

}
```


In the code above, the counter, pushupsToDo, starts at 10, and increments down one at a time. When it hits 0, the loop exits.

10.3 Incrementing While Loops Instructions

1.

In **Coffee.java**, initialize an int variable called cupsOfCoffee with a value of 1.

2.

Create a while loop that runs as long as cupsOfCoffee is less than or equal to 100.

Important: Inside the while loop, increment cupsOfCoffee by 1 to prevent an infinite loop.

3.

Inside the while loop above where you incremented cupsOfCoffee print the following:

Fry drinks cup of coffee #1

The 1 in this statement should correspond with the current value of cupsOfCoffee. When cupsOfCoffee is 100, this should be printed:

Fry drinks cup of coffee #100

```
class Coffee {
```

```
    public static void main(String[] args) {
```

```
        // initialize cupsOfCoffee
```

```
        // add while loop with counter
```

}

}

OUTPUT

Fry drinks cup of coffee #1

Fry drinks cup of coffee #2

Fry drinks cup of coffee #3

Fry drinks cup of coffee #4

Fry drinks cup of coffee #5

Fry drinks cup of coffee #6

Fry drinks cup of coffee #7

Fry drinks cup of coffee #8

Fry drinks cup of coffee #9

Fry drinks cup of coffee #10

Fry drinks cup of coffee #11

Fry drinks cup of coffee #12

Fry drinks cup of coffee #13

Fry drinks cup of coffee #14

Fry drinks cup of coffee #15

Fry drinks cup of coffee #16

Fry drinks cup of coffee #17

Fry drinks cup of coffee #18

Fry drinks cup of coffee #19

Fry drinks cup of coffee #20
Fry drinks cup of coffee #21
Fry drinks cup of coffee #22
Fry drinks cup of coffee #23
Fry drinks cup of coffee #24
Fry drinks cup of coffee #25
Fry drinks cup of coffee #26
Fry drinks cup of coffee #27
Fry drinks cup of coffee #28
Fry drinks cup of coffee #29
Fry drinks cup of coffee #30
Fry drinks cup of coffee #31
Fry drinks cup of coffee #32
Fry drinks cup of coffee #33
Fry drinks cup of coffee #34
Fry drinks cup of coffee #35
Fry drinks cup of coffee #36
Fry drinks cup of coffee #37
Fry drinks cup of coffee #38
Fry drinks cup of coffee #39
Fry drinks cup of coffee #40
Fry drinks cup of coffee #41
Fry drinks cup of coffee #42
Fry drinks cup of coffee #43

Fry drinks cup of coffee #44
Fry drinks cup of coffee #45
Fry drinks cup of coffee #46
Fry drinks cup of coffee #47
Fry drinks cup of coffee #48
Fry drinks cup of coffee #49
Fry drinks cup of coffee #50
Fry drinks cup of coffee #51
Fry drinks cup of coffee #52
Fry drinks cup of coffee #53
Fry drinks cup of coffee #54
Fry drinks cup of coffee #55
Fry drinks cup of coffee #56
Fry drinks cup of coffee #57
Fry drinks cup of coffee #58
Fry drinks cup of coffee #59
Fry drinks cup of coffee #60
Fry drinks cup of coffee #61
Fry drinks cup of coffee #62
Fry drinks cup of coffee #63
Fry drinks cup of coffee #64
Fry drinks cup of coffee #65
Fry drinks cup of coffee #66
Fry drinks cup of coffee #67

Fry drinks cup of coffee #68
Fry drinks cup of coffee #69
Fry drinks cup of coffee #70
Fry drinks cup of coffee #71
Fry drinks cup of coffee #72
Fry drinks cup of coffee #73
Fry drinks cup of coffee #74
Fry drinks cup of coffee #75
Fry drinks cup of coffee #76
Fry drinks cup of coffee #77
Fry drinks cup of coffee #78
Fry drinks cup of coffee #79
Fry drinks cup of coffee #80
Fry drinks cup of coffee #81
Fry drinks cup of coffee #82
Fry drinks cup of coffee #83
Fry drinks cup of coffee #84
Fry drinks cup of coffee #85
Fry drinks cup of coffee #86
Fry drinks cup of coffee #87
Fry drinks cup of coffee #88
Fry drinks cup of coffee #89
Fry drinks cup of coffee #90
Fry drinks cup of coffee #91

Fry drinks cup of coffee #92
Fry drinks cup of coffee #93
Fry drinks cup of coffee #94
Fry drinks cup of coffee #95
Fry drinks cup of coffee #96
Fry drinks cup of coffee #97
Fry drinks cup of coffee #98
Fry drinks cup of coffee #99
Fry drinks cup of coffee #100

10.4 For Loops

Incrementing with loops is actually so common in programming that Java (like many other programming languages) includes syntax specifically to address this pattern: for loops.

A *for loop* header is made up of the following three parts, each separated by a semicolon:

1. The initialization of the loop control variable.
2. A [boolean](#) expression.
3. An increment or decrement statement.

The opening line might look like this:

```
for (int i = 0; i < 5; i++) {  
  
    // code that will run  
  
}
```

In a for loop, an initialization statement is run once in order to initialize the loop control variable. This variable is modified in every iteration, can be referenced in

the loop body, and used to test the boolean condition. In the example above, *i* is the loop control variable.

Let's breakdown the above example:

1. *i* = 0: *i* is initialized to 0
2. *i* < 5: the loop is given a boolean condition that relies on the value of *i*. The loop will continue to execute until *i* < 5 is false.
3. *i*++: *i* will increment at the end of each loop and before the condition is re-evaluated.

So the code will run through the loop a total of five times.

We'll also hear the term "iteration" in reference to loops. When we *iterate*, it just means that we are repeating the same block of code.

```
for (int i = 0; i < 3; i++) {  
    System.out.println("i is now " + i);  
}
```

10.5 Using For Loops

Even though we can write while loops that accomplish the same task, for loops are useful because they help us remember to increment our counter — something that is easy to forget when we increment with a while loop.

Leaving out that line of code would cause an infinite loop — yikes!

Fortunately, equipped with our new understanding of for loops, we can help prevent infinite loops in our own code.

It's important to be aware that, if we don't create the correct for loop header, we can cause the iteration to loop one too many or one too few times; this occurrence is known as an “off by one” error.

For example, imagine we wanted to find the sum of the first ten numbers and wrote the following code:

```
int sum = 0;
for (int i = 0; i < 10; i++) {
    sum += i
}
```

This code would produce an incorrect value of 45. We skipped adding 10 to sum because our loop control variable started with a value of 0 and stopped the iteration after it had a value of 9. We were off by one! We could fix this by changing the condition of our loop to be `i <= 10`; or `i < 11`;

These errors can be tricky because, while they do not always produce an error in the terminal, they can cause some miscalculations in our code. These are called logical errors — the code runs fine, but it didn't do what you expected it to do.

10.5 Using For Loops Instructions

1.

We've provided a while loop that loops from 1 to 100 outputting a string on each iteration. Refactor (rewrite) this code as a for loop.

Remember, the syntax of a for loop looks like:

```
for (int i = 0; i < 5; i++) {
```

```
// code that will run
```



```
}  
class Coffee {  
  
    public static void main(String[] args) {  
  
        int cupsOfCoffee = 1;  
  
        while (cupsOfCoffee <= 100) {  
  
            System.out.println("Fry drinks cup of coffee #" + cupsOfCoffee);  
            cupsOfCoffee++;  
  
        }  
  
    }  
  
}
```

OUTPUT

Fry drinks cup of coffee #1
Fry drinks cup of coffee #2
Fry drinks cup of coffee #3
Fry drinks cup of coffee #4
Fry drinks cup of coffee #5
Fry drinks cup of coffee #6

Fry drinks cup of coffee #7
Fry drinks cup of coffee #8
Fry drinks cup of coffee #9
Fry drinks cup of coffee #10
Fry drinks cup of coffee #11
Fry drinks cup of coffee #12
Fry drinks cup of coffee #13
Fry drinks cup of coffee #14
Fry drinks cup of coffee #15
Fry drinks cup of coffee #16
Fry drinks cup of coffee #17
Fry drinks cup of coffee #18
Fry drinks cup of coffee #19
Fry drinks cup of coffee #20
Fry drinks cup of coffee #21
Fry drinks cup of coffee #22
Fry drinks cup of coffee #23
Fry drinks cup of coffee #24
Fry drinks cup of coffee #25
Fry drinks cup of coffee #26
Fry drinks cup of coffee #27
Fry drinks cup of coffee #28
Fry drinks cup of coffee #29
Fry drinks cup of coffee #30

Fry drinks cup of coffee #31
Fry drinks cup of coffee #32
Fry drinks cup of coffee #33
Fry drinks cup of coffee #34
Fry drinks cup of coffee #35
Fry drinks cup of coffee #36
Fry drinks cup of coffee #37
Fry drinks cup of coffee #38
Fry drinks cup of coffee #39
Fry drinks cup of coffee #40
Fry drinks cup of coffee #41
Fry drinks cup of coffee #42
Fry drinks cup of coffee #43
Fry drinks cup of coffee #44
Fry drinks cup of coffee #45
Fry drinks cup of coffee #46
Fry drinks cup of coffee #47
Fry drinks cup of coffee #48
Fry drinks cup of coffee #49
Fry drinks cup of coffee #50
Fry drinks cup of coffee #51
Fry drinks cup of coffee #52
Fry drinks cup of coffee #53
Fry drinks cup of coffee #54

Fry drinks cup of coffee #55
Fry drinks cup of coffee #56
Fry drinks cup of coffee #57
Fry drinks cup of coffee #58
Fry drinks cup of coffee #59
Fry drinks cup of coffee #60
Fry drinks cup of coffee #61
Fry drinks cup of coffee #62
Fry drinks cup of coffee #63
Fry drinks cup of coffee #64
Fry drinks cup of coffee #65
Fry drinks cup of coffee #66
Fry drinks cup of coffee #67
Fry drinks cup of coffee #68
Fry drinks cup of coffee #69
Fry drinks cup of coffee #70
Fry drinks cup of coffee #71
Fry drinks cup of coffee #72
Fry drinks cup of coffee #73
Fry drinks cup of coffee #74
Fry drinks cup of coffee #75
Fry drinks cup of coffee #76
Fry drinks cup of coffee #77
Fry drinks cup of coffee #78

Fry drinks cup of coffee #79
Fry drinks cup of coffee #80
Fry drinks cup of coffee #81
Fry drinks cup of coffee #82
Fry drinks cup of coffee #83
Fry drinks cup of coffee #84
Fry drinks cup of coffee #85
Fry drinks cup of coffee #86
Fry drinks cup of coffee #87
Fry drinks cup of coffee #88
Fry drinks cup of coffee #89
Fry drinks cup of coffee #90
Fry drinks cup of coffee #91
Fry drinks cup of coffee #92
Fry drinks cup of coffee #93
Fry drinks cup of coffee #94
Fry drinks cup of coffee #95
Fry drinks cup of coffee #96
Fry drinks cup of coffee #97
Fry drinks cup of coffee #98
Fry drinks cup of coffee #99
Fry drinks cup of coffee #100

10.6 Iterating Over Arrays and ArrayLists

One common pattern we'll encounter as a programmer is *traversing*, or looping, through a list of data and doing something with each item. In Java, that list would be an [array](#) or [ArrayList](#) and the loop could be a for loop. But wait, how does this work?

In order to traverse an array or ArrayList using a loop, we must find a way to access each element via its index. We may recall that for loops are created with a counter variable. We can use that counter to track the index of the current element as we iterate over the list of data.

Because the first index in an array or ArrayList is 0, the counter would begin with a value of 0 and increment until the end of the list. So we can increment through the array or ArrayList using its indices.

For example, if we wanted to add 1 to every int item in an array `secretCode`, we could do this:

```
for (int i = 0; i < secretCode.length; i++) {  
    // Increase value of element value by 1  
    secretCode[i] += 1;  
}
```

Notice that our condition in this example is `i < secretCode.length`. Because array indices start at 0, the length of `secretCode` is 1 larger than its final index. A loop should stop its traversal before its counter variable is equal to the length of the list.

To give a concrete example, if the length of an array is 5, the last index we want to access is 4. If we were to try to access index 5, we would get an [ArrayIndexOutOfBoundsException](#) error! This is a very common mistake when first starting to traverse arrays.

Traversing an ArrayList looks very similar:

```
for (int i = 0; i < secretCode.size(); i++) {  
    // Increase value of element value by 1  
    int num = secretCode.get(i);  
    secretCode.set(i, num + 1);  
}
```

We can also use while loops to traverse through arrays and ArrayLists. If we use a while loop, we need to create our own counter variable to access individual elements. We'll also set our condition to continue looping until our counter variable equals the list length.

For example, let's use a while loop to traverse through an array:

```
int i = 0; // initialize counter
```

```
while (i < secretCode.length) {  
    secretCode[i] += 1;  
    i++; // increment the while loop  
}
```

Traversing through an ArrayList with a while loop would look like this:

```
int i = 0; // initialize counter
```

```
while (i < secretCode.size()) {  
    int num = secretCode.get(i);  
    secretCode.set(i, num + 1);  
    i++; // increment the while loop  
}
```

10.6 Iterating Over Arrays and ArrayLists Instructions

1.

Let's use a for loop to iterate over expenses and sum up the total of all items.

Start with the skeleton of a for loop:

- Initialize a counter *i* with a value of 0.
- The loop should run while *i* is less than the `size()` of expenses.
- Increment *i*.

You can leave the body empty for now.

2.

Inside the for loop, add the item's value to total.

```
import java.util.ArrayList;
```

```
class CalculateTotal {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Double> expenses = new ArrayList<Double>();
```

```
        expenses.add(74.46);
```

```
        expenses.add(63.99);
```

```
        expenses.add(10.57);
```

```
        expenses.add(81.37);
```

```
        double total = 0;
```

```
        // Iterate over expenses
```

```
        System.out.println(total);
```

```
    }
```

```
}
```

OUTPUT

230.39

10.7 break and continue

If we ever want to exit a loop before it finishes all its iterations or want to skip one of the iterations, we can use the break and continue keywords.

The break keyword is used to exit, or break, a loop. Once break is executed, the loop will stop iterating. For example:

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
    if (i == 4) {  
        break;  
    }  
}
```

Even though the loop was set to iterate until the condition `i < 10` is false, the above code will output the following because we used break:

```
0  
1  
2  
3  
4
```

The continue keyword can be placed inside of a loop if we want to skip an iteration. If continue is executed, the current loop iteration will immediately end, and the next iteration will begin. We can use the continue keyword to skip any even valued iteration:

```
int[] numbers = {1, 2, 3, 4, 5};
```

```
for (int i = 0; i < numbers.length; i++) {  
    if (numbers[i] % 2 == 0) {  
        continue;  
    }  
    System.out.println(numbers[i]);  
}
```

This program would output the following:

1
3
5

In this case, if a number is even, we hit a continue statement, which skips the rest of that iteration, so the print statement is skipped. As a result, we only see odd numbers print.

Keep Reading: AP Computer Science A Students

Loops can exist all throughout our code - including inside a method. If the return keyword was executed inside a loop contained in a method, then the loop iteration would be stopped and the method/constructor would be exited.

For example, we have a method called checkForJacket() that takes in an array of Strings. If any of the elements are equivalent to the String value "jacket", the method will return true:

```
public static boolean checkForJacket(String[] lst) {  
    for (int i = 0; i < lst.length; i++) {  
        System.out.println(lst[i]);  
        if (lst[i] == "jacket") {  
            return true;  
        }  
    }  
    return false;  
}  
  
public static void main(String[] args) {  
    String[] suitcase = {"shirt", "jacket", "pants", "socks"};  
    System.out.println(checkForJacket(suitcase));  
}
```

As soon as an element equals "jacket", return true; is executed. This causes the loop to stop and the compiler to exit checkForJacket(). Running this code would output the following:

shirt
jacket
true

10.7 break and continue Instructions

1.

Take a look at the for loop in the code editor. It starts its iteration at 0 and continues to iterate until $i < 100$ is false.

Inside the loop, create a condition that checks if i is **not** divisible by 5. If the condition is true, skip the iteration. Outside the condition statement, print i . The final solution **should not** contain an else statement.

The only numbers that should be printed are those that are divisible by 5!

```
class Numbers {  
  
    public static void main(String[] args) {  
  
        for (int i = 0; i < 100; i++) {  
  
            // Add your code below
```

```
        }  
    }  
}
```

OUTPUT

0

5

10

15

20
25
30
35
40
45
50
55
60
65
70
75
80
85
90
95

10.8 For-Each Loops

Sometimes we need access to the elements' indices or we only want to iterate through a portion of a list. If that's the case, a regular for loop or while loop is a great choice.

For example, we can use a for loop to print out each element in an array called `inventoryItems`:

```
for (int inventoryItem = 0; inventoryItem < inventoryItems.length;  
inventoryItem++) {  
    // Print element at current index
```

```
System.out.println(inventoryItems[inventoryItem]);  
}
```

But sometimes we couldn't care less about the indices; we only care about the element itself.

At times like this, for-each loops come in handy.

For-each loops, which are also referred to as *enhanced loops*, allow us to directly loop through each item in a list of items (like an array or ArrayList) and perform some action with each item.

If we want to use a for-each loop to rewrite our program above, the syntax looks like this:

```
for (String inventoryItem : inventoryItems) {  
    // Print element value  
    System.out.println(inventoryItem);  
  
}
```

Our enhanced loop contains two items: an enhanced for loop variable (inventoryItem) and a list to traverse through (inventoryItems).

We can read the : as "in" like this: for each inventoryItem (which should be a String) in inventoryItems, print inventoryItem.

If we try to assign a new value to the enhanced for loop variable, the value stored in the array or ArrayList will not change. This is because, for every iteration in the enhanced loop, the loop variable is assigned a copy of the list element.

Note: We can name the enhanced for loop variable whatever we want; using the singular of a plural is just a convention. We may also encounter conventions like String word : sentence.

10.8 For-Each Loops Instructions

1.

Let's use a for-each loop to find the priciest item in expenses.

Build a for-each loop that iterates through each expense in expenses. For now, leave the body of the loop empty.

2.

Inside the for-each loop, check if expense is greater than mostExpensive.

If it is, set mostExpensive equal to expense.

```
import java.util.ArrayList;
```

```
class MostExpensive {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Double> expenses = new ArrayList<Double>();
```

```
        expenses.add(74.46);
```

```
        expenses.add(63.99);
```

```
        expenses.add(10.57);
```

```
        expenses.add(81.37);
```

```
        double mostExpensive = 0;
```

```
        // Iterate over expenses
```

```
        System.out.println(mostExpensive);
```

```
}
```

```
}
```

OUTPUT

81.37

10.9 Removing Elements During Traversal

If we want to remove elements from an ArrayList while traversing through one, we can easily run into an error if we aren't careful.

When an element is removed from an ArrayList, all the items that appear after the removed element will have their index value shift by negative one — it's like all elements shifted to the left! We'll have to be very careful with how we use our counter variable to avoid skipping elements.

Removing An Element Using while

When using a while loop and removing elements from an ArrayList, we should **not** increment the while loop's counter whenever we remove an element. We don't need to increase the counter because all of the other elements have now shifted to the left.

For example, if we removed the element at index 3, then the element that was at index 4 will be moved to index 3. If we increase our counter to 4, we'll skip that element!

Take a look at this block of code that will remove all odd numbers from an ArrayList. Think about what the value of *i* is, when we're increasing the value of *i*, and when *i* < *lst.size()* becomes False.

```
int i = 0; // initialize counter
```

```
while (i < lst.size()) {  
    // if value is odd, remove value  
    if (lst.get(i) % 2 != 0){  
        lst.remove(i);  
    } else {
```

```

    // if value is even, increment counter
    i++;
}
}

```

Removing An Element Using for

We can use a similar strategy when removing elements using a for loop. When using a while loop, we decided to not increase our loop control variable whenever we removed an element. This ensured that we would not skip an element when all of the other elements shifted to the left.

When using a for loop, we, unfortunately, *must* increase our loop control variable — the loop control variable will always change when we reach the end of the loop (and it will usually change by 1 because we often use something like `i++`.) Since we can't avoid increasing our loop control variable, we can take matters into our own hands and decrease the loop control variable whenever we remove an item.

For example:

```

for (int i = 0; i < lst.size(); i++) {
    if (lst.get(i) == "value to remove"){
        // remove value from ArrayList
        lst.remove(lst.get(i));
        // Decrease loop control variable by 1
        i--;
    }
}

```

Now whenever we remove an item, we'll decrease `i` by 1. Then when we reach the end of the loop, `i` will increase by 1. It will be like `i` never changed!

Note: Avoid manipulating the size of an `ArrayList` when using an enhanced for loop. Actions like adding or removing elements from an `ArrayList` when using a for each loop can cause a `ConcurrentModificationException` error.

10.9 Removing Elements During Traversal Instructions

1.

Take a look at the code placed in the main() method of the Lunch class.

Inside the method removeAnts(), use a for loop or a while loop to iterate through lunchBox and remove any element that has the value "ant".

Outside the loop, return the value of lunchBox.

```
import java.util.ArrayList;
```

```
class Lunch {
```

```
    public static ArrayList<String> removeAnts(ArrayList<String> lunchBox) {
```

```
        // Add your code below
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> lunchContainer = new ArrayList<String>();
```

```
        lunchContainer.add("apple");
```

```
        lunchContainer.add("ant");
```

```
        lunchContainer.add("ant");
```

```
        lunchContainer.add("sandwich");
```

```
        lunchContainer.add("ant");
```

```
        lunchContainer = removeAnts(lunchContainer);
```

```
        System.out.println(lunchContainer);
```

```
}  
}
```

OUTPUT

[apple, sandwich]

10.10 Review

Nice work! Let's iterate over what you've just learned about loops:

- while loops: These are useful to repeat a code block an unknown number of times until some condition is met. For example:

```
int wishes = 0;
```

```
while (wishes < 3) {
```

```
    // code that will run  
    wishes++;
```

```
}
```

- for loops: These are ideal for when you are incrementing or decrementing with a counter variable. For example:

```
for (int i = 0; i < 5; i++) {
```

```
    // code that will run
```

```
}
```

- For-each loops: These make it simple to do something with each item in a list. For example:

```
for (String inventoryItem : inventoryItems) {
```

```
    // do something with each inventoryItem
```

```
}
```

10.10 Review Instructions

Feel free to play around with loops, arrays, and ArrayLists in the code editor.

```
import java.util.ArrayList;
```

```
import java.util.Arrays;
```

```
class Playground {
```

```
    public static void main(String[] args) {
```

```
        for (int i = 0; i < 28; i++) {
```

```
            System.out.println("Congrats on finishing Java loops!");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!

Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!
Congrats on finishing Java loops!

10. Quiz

1. Complete the for-each loop.

```
for ( _____ _____ llamas) {  
  
    sayHello(llama);  
}
```

```
}
```

- String llama
- :
- in
- llama

Click or drag and drop to fill in the blank

2. How can you modify this code so that it won't cause an infinite while loop?

```
int i = 7;
```

```
while (i > 0) {
```

```
    System.out.println(i);
```

```
}
```

- a) Add `i--` after the while loop.
- b) Add `i++` after the while loop.
- c) Add `i--` inside the while loop.
- d) Add `i++` inside the while loop.

3. When would it be helpful to use a for loop instead of a for-each loop when iterating over an ArrayList of items?

- a) When you're worried about causing an infinite loop.
- b) When you don't know how long the ArrayList will be.
- c) When you aren't starting from the beginning of the ArrayList.
- d) When each item in the ArrayList has a different value.

4. How many times will **"Hello, I.T. Have you tried turning it off and on again?"** be printed to the console?

```
int i = 1;
```

```
while (i < 100) {
```

```
System.out.println("Hello, I.T. Have you tried turning it off and on again?");
```

```
i++;
```

```
}
```

- a) 99
- b) An unknown number of times.
- c) 100
- d) It will continue printing until the program crashes because of an infinite loop.

5. Complete the **for** loop so that it won't cause an infinite loop.

```
for (____;____;____) {
```

```
System.out.println(i);
```

```
}
```

- i < 0
- i = 0
- int i = 0
- i--
- i < 5
- i++
- int i

Click or drag and drop to fill in the blank

6. Rebuild the **while** loop as a **for** loop.

```
int i = 99;
```

```
while (i > 0) {
```

```
System.out.println(i + " bottles of soda on the wall.");  
i--;  
  
}
```

// Rebuild here:

```
for (____;____;____) {
```

```
System.out.println(j + " bottles of soda on the wall.");  
  
}
```

- j < 0
- j++
- int j = 99
- j > 0
- j--

Click or drag and drop to fill in the blank

Project-9

Fizz Buzz

[Fizz Buzz](#) is a common developer interview question; so common it's almost cliché!

The challenge is designed to weed out 99.5% programming job candidates who cannot creatively use their coding knowledge to solve coding problems.

Want to give it a shot?

1! 2! Fizz! 4! Buzz!

Write a **FizzBuzz.java** program that outputs numbers from 1 to 100... with a catch:

- For multiples of 3, print Fizz instead of the number.
- For the multiples of 5, print Buzz.
- For numbers which are multiples of both 3 and 5, print FizzBuzz.

1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
...
97
98

Fizz

Buzz

Happy coding!

Tasks

What You'll Be Building

1.

As the inventor of this challenge, Imran Ghory, states:

“After a fair bit of trial and error I’ve come to discover that people who struggle to code don’t just struggle on big problems, or even smallish problems (i.e. write an implementation of a linked list). They struggle with tiny problems.

So I set out to develop questions that can identify this kind of developer and came up with a class of questions I call “FizzBuzz Questions” named after a game children often play (or are made to play) in schools in the UK.”

Setting Up

2.

Start by creating a skeleton for the program:

- Create a FizzBuzz class.
- Inside FizzBuzz, build an empty main() method.

Outputting Numbers from 1 to 100

3.

How do we output all the numbers from 1 to 100?

Well, we could write `System.out.println()` 100 times back to back, but that’s no fun. What else can we do?

That’s right! A for loop!

Write a for loop that starts at 1 and iterates to 100.

4.

Inside the for loop, output the counter i.

5.

Run your code — does it print every number between 1 and 100?

```
java FizzBuzz
```

6.

Now that we have all the numbers, let's solve the hard part.

Before coding further, take out a piece of paper and brainstorm.

How do we check whether or not a number is divisible by 3? What about for the other two conditions?

When you are ready, try to implement a solution.

Project-10

The Prime Directive

Finding prime numbers is a common technical challenge in programming interviews.

As it happens, Java loops are a great tool to help you do this!

A prime number is an integer greater than 1 that is only divisible by 1 and itself.

For example, 2, 3, 5, and 7 are all prime numbers, but 4, 6, 8, and 9 are not.

Your prime directive: Build a **PrimeDirective.java** program that creates an ArrayList of all prime numbers in an array.

Tasks

Setting Up

1.

Take a look at **PrimeDirective.java**:

There's a class PrimeDirective where you'll be creating your program.

Inside main(), an instance of PrimeDirective (pd) has been instantiated. Below that, you'll see an int array called numbers that has a series of integers.

To Do: Import ArrayList from java.util at the very top of your program, above the PrimeDirective class. This will allow you to use ArrayLists.

Optimus Prime

2.

First, we need a way to determine whether a number is prime or isn't prime.

Create an empty public method isPrime() that:

- has one parameter: an int called number
- will return true if number is prime
- will return false if number is not prime

3.

Take a moment to consider what makes a prime number prime:

- greater than 1
- only divisible by 1 and itself

In fact, every number is divisible by 1, so we don't really care about being able to divide by 1.

Imagine we have a number n. If n is prime, then n should not be divisible by any integers between 2 and n-1.

But how can we check this?

Before you move on, take out paper and a pencil and write down your ideas about how to check if a number is prime.

4.

Inside `isPrime()`, create a for loop:

- set a counter `i` equal to 2
- run the loop while `i` is less than `number`
- increment `i`

5.

As you loop through each `i` value, you want to check if `number` is divisible by it.

Inside the loop:

- Check if `number` is divisible by `i`.
- If it is, then `number` is not prime, so you can return `false` from the method.

6.

Below the for loop, return `true` because `number` isn't divisible by any two smaller integers.

7.

Wait a second... what about 2 or numbers less than 2? Well, those are our edge cases.

In `isPrime()` above the for loop, build an if/else if statement to handle the following edge cases:

- If `number` is 2, it is the smallest prime number.
- If `number` is less than 2, it is not prime.

8.

Test out your `isPrime()` method on `pd` in `main()` and see if it works!

Try it out with a few numbers:

- 7 should return true
- 28 should return false
- 2 should return true
- 0 should return false

Don't forget to run your code with the following:

```
java PrimeDirective
```

Only Primes

9.

Nice work! Now, all that's left is building an `ArrayList` of the prime numbers in the numbers array.

You can create another method to handle this. Build an empty method called `onlyPrimes()` that:

- returns an `ArrayList` of integers
- has a parameter `numbers`, which is an array of ints

10.

Inside the `onlyPrimes()` body, create a new empty `ArrayList` called `primes` to store all the prime numbers that are found.

11.

So how do you find all of the primes in an array? By using a for-each loop!

Set up a for-each loop that checks each number in numbers.

12.

Now, if number is prime, you can add it to primes.

13.

At the end of the method below the for-each loop, return primes from onlyPrimes().

14.

Time to put it all together!

In main(), test out pd.onlyPrimes() on the numbers array.

Don't forget to run your code with

java PrimeDirective

Primed For More

15.

Congrats on completing the Prime Directive!

Want to do even more? Check the hint for some ideas to expand the project.

// Import statement:

class PrimeDirective {

// Add your methods here:

```
public static void main(String[] args) {  
    PrimeDirective pd = new PrimeDirective();  
    int[] numbers = {6, 29, 28, 33, 11, 100, 101, 43, 89};  
}  
}
```

```
$ javac PrimeDirective.java  
$ java PrimeDirective  
true  
false  
true  
false  
[29, 11, 101, 43, 89]  
$
```

Concept Review

For-each statement in Java

In Java, the for-each statement allows you to directly loop through each item in an array or ArrayList and perform some action with each item.

When creating a for-each statement, you must include the for keyword and two expressions inside of parentheses, separated by a colon. These include:

1. The handle for an element we're currently iterating over.
2. The source array or ArrayList we're iterating over.

```
// array of numbers
```

```
int[] numbers = {1, 2, 3, 4, 5};
```

```
// for-each loop that prints each number in numbers
// int num is the handle while numbers is the source array
for (int num : numbers) {
    System.out.println(num);
}
```

11. STRING METHODS

11.1 Introduction to String Methods

As you may recall, a [String](#), which is widely used in Java, is an object that represents a sequence of characters. It is a great way to store information.

Because character strings are so vital to programming, Java dedicated an entire class to them. This is great news for us because the String class has a lot of useful methods to help us perform operations on Strings and data manipulation. We don't have to import anything to use the String class because it's part of the java.lang package which is available by default.

In this lesson, we will go over several String methods:

- length()
- concat()
- equals()
- indexOf()
- charAt()
- substring()
- toUpperCase() / toLowerCase()

Let's get started!

11.1 Introduction to String Methods Instructions

1.

Run the code in the code editor.

There are three string methods used in this program. We'll learn about them later, but can you guess what some of them do?

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        String str = "Hello, World!";  
  
        // Examples:  
  
        System.out.println(str.length());  
  
        System.out.println(str.substring(4));  
  
        System.out.println(str.toUpperCase());  
  
    }  
  
}
```

OUTPUT

13

o, World!

HELLO, WORLD!

11.2 length()

In Java, the [length\(\)](#) string method returns the length — total number of characters — of a String.

Suppose we have a String called `str`, `str.length()` would return its length.

Take a look at this code for example:

```
String str = "Hello World!";
```

```
System.out.println(str.length());
```

12 would be printed because `str` has 12 characters:

```
H e l l o _ W o r l d !
```

In theory, the length of a String is the same as the [Unicode](#) units of the String. For example, escape sequences such as `\n` count as only one character.

11.2 length() Instructions

1.

Twitter allows users to post tweets with a character limit. They capture the message the user enters as a String, count the length of the message, and use its length to decide whether or not to let the user post the message. The limit is 280 characters.

Let's check if tweet follows Twitter's policy. Print out the number of characters in tweet using the `length()` method.

```
public class TweetCounter {
```

```
    public static void main(String[] args) {
```

```
        String tweet = "Liz Lemon, ninjas are kind of cool... I just dont know any  
        personally. Get on that.";
```

```
// What's the character count?
```

```
}
```

```
}
```

OUTPUT

83

11.3 concat()

The [concat\(\)](#) method concatenates one string to the end of another string. Concatenation is the operation of joining two strings together.

Suppose we have a String called str1 and another String called str2, using str1.concat(str2) would return str1 with str2 appended to the end of it.

For example:

```
String name = new String("Code");
```

```
name = name.concat("cademy");
```

```
System.out.println(name);
```

Codecademy would be printed.

Strings are immutable objects which means that String methods, like concat() do not actually change a String object.

Our variable, name holds a reference to the String object, "Code". When we use concat() on name, we changed its value so that it references a new object — "Code", combined with the String literal, "cademy".

Suppose we do something slightly different. We'll use `concat()` on `name` without reassigning its value:

```
String name = "Code";
```

```
name.concat("cademy");
```

```
System.out.println(name);
```

Code would be printed instead. The value of the `String` can't change! Instead, we create a new object and need to assign that new object to some variable.

Keep Reading: AP Computer Science A Students

When we first discussed Objects we learned that if we tried printing an Object, we'd get an output of the class name and the Object's memory address. If we wanted to get a more useful printout, we'd have to call the Object's `toString()` method.

This `toString()` method comes into play with `concat()`. If we concatenate a `String` with another Object, we're really adding the result of that Object's `toString()` method to our original `String`. We can even see this when we concatenate two `Strings` together (remember a `String` is an Object). When we use `concat()` on another `String`, we don't concatenate its memory address to the original `String`. Instead, we combine the result of its `toString()` method to the original `String`.

You can refresh yourself on the `toString()` method in [this exercise](#).

11.3 `concat()` Instructions

1.

Store your first name in `firstName` and your last name in `lastName`.

Print out your full name using `concat()`.

```
public class CombineNames {
```

```
    public static void main(String[] args) {
```

```
// Add your first names:  
  
String firstName = "";  
  
String lastName = "";  
  
  
// What's the full name?  
  
  
}  
  
}
```

OUTPUT

Ziggy Stardust

11.4 equals()

With objects, such as Strings, we can't use the primitive equality operator `==` to check for equality between two strings. To test equality with strings, we use a built-in method called [equals\(\)](#).

For example:

```
String flavor1 = "Mango";  
String flavor2 = "Peach";  
  
System.out.println(flavor1.equals("Mango"));  
// prints true  
  
System.out.println(flavor2.equals("Mango"));  
// prints false
```

Side note, there's also an [equalsIgnoreCase\(\)](#) method that compares two strings without considering upper/lower cases.

Keep Reading: AP Computer Science A Students

We can also compare String values lexicographically (think dictionary order) using the [.compareTo\(\)](#) method. When we call the `.compareTo()` method, each character in the String is converted to Unicode; then the Unicode character from each String is compared.

The method will return an int that represents the difference between the two Strings.

For example:

```
String flavor1 = "Mango";  
String flavor2 = "Peach";
```

```
System.out.println(flavor1.compareTo(flavor2));
```

Our program above will output -3.

When we use `.compareTo()`, we must pay attention to the return value:

- If the method returns 0, the two Strings are equal.
- If the value is less than 0, then the String object is lexicographically less than the String object argument.
- If the value is greater than 0, then the String object is lexicographically greater than the String object argument.

In the example above, "Mango" comes before "Peach", so we get a negative number (we specifically get -3 because the Unicode values of "M" and "P" differ by 3). If we did `flavor2.compareTo(flavor1)`, we would get 3, signifying that "Peach" is greater than "Mango".

Note: Make sure to pay attention to capitalization when using `.compareTo()`. Upper case and lower case letters have different Unicode values. For example, when comparing "Mango" and "Peach", we got -3, meaning that "Mango" was smaller. But if we compare "mango" and "Peach" we get 29. The Unicode value for lower case "m" is actually larger than upper case "P".

Using [.compareToIgnoreCase\(\)](#) will perform the same task, but will not consider upper/lower case.

11.4 equals() Instructions

1.

In **Password.java**, there's a password with a value of "correcthorsebatterystaple".

Use equals() to print out whether it's the same as another string.

```
public class Password {  
  
    public static void main(String[] args) {  
  
        String password = "correcthorsebatterystaple";  
  
        // Write the code:  
  
    }  
  
}
```

OUTPUT

False

11.5 indexOf()

If we want to know the index of the first occurrence of a character in a string, we can use the [indexOf\(\)](#) method on a string.

Remember that the indices in Java start with 0:

```
String letters = "ABCDEFGHJKLMNOP";
```

```
System.out.println(letters.indexOf("C"));
```

This would output 2.

Although C is the third letter in the English alphabet, it is located in the second index of the string.

Suppose we want to know the index of the first occurrence of an entire substring. The `indexOf()` instance method can also return where the substring begins (the index of the first character in the substring):

```
String letters = "ABCDEFGHJKLMN";
```

```
System.out.println(letters.indexOf("EFG"));
```

This would output 4, because EFG starts at index 4.

If the `indexOf()` doesn't find what it's looking for, it'll return a -1.

11.5 `indexOf()` Instructions

1.

In **MorseCode.java**, there's a string named `code` that has some funny characters.

Find the index where the substring `. - -` (which means "eat") begins!

```
public class MorseCode {
```

```
    public static void main(String[] args) {
```

```
        String code = ".... .. / .. .----. -- / ... --- -. .-.- / .- -. -- / .. / .-.. .. -.- . / - --- / . - - / .-  
.- -- . -.";
```

```
        // Write the code:
```

```
    }
```



```
}
```

OUTPUT

87

11.6 charAt()

The [charAt\(\)](#) method returns the character located at a String's specified index.

For example:

```
String str = "qwer";
```

```
System.out.println(str.charAt(2));
```

It would output e because that's what's at index 2. (Once again, indices start with 0.)

Suppose we try to return the character located at index 4. It would produce an `IndexOutOfBoundsException` error because index 4 is out of str's range:

```
java.lang.StringIndexOutOfBoundsException: String index out of range: 4
```

11.6 charAt() Instructions

1.

In **Initials.java**, add your first name and last name.

2.

Print out the initials of the `firstName` and `lastName`.

```
public class Initials {
```

```
    public static void main(String[] args) {
```

```
        // Add a first name and a last name:
```

```
        String firstName = "";
```

```
String lastName = "";

// What are the initials?

}

}
```

OUTPUT

S
L

11.7 substring()

There may be times when we only want a part of a string. In such cases, we may want to extract a *substring* from a string.

The `substring()` method does exactly that. For example:

```
String line = "The Heav'ns and all the Constellations rung";
```

```
System.out.println(line.substring(24));
```

It would output `Constellations rung` because that's what begins at index 24 and ends at the end of line. The substring begins with the character at the specified index and extends to the end of the string.

Suppose we want a substring from the middle of the string. We can instead include two arguments in this method. For example:

```
String line = "The Heav'ns and all the Constellations rung";
```

```
System.out.println(line.substring(27, 33));
```

When `substring()` is called with two arguments, the first argument is *inclusive* and the second is *exclusive*. This means the resulting substring will begin at index 27 and extend up to, but *not* including, index 33. Therefore, the example above

would output stella because that's the substring that begins at index 27 and ends at index 32, one index before 33.

We can use this method to return a single-element substring at a specific index. We do this by calling `substring()` with the desired index value as the first argument and then the index value plus one as the second argument.

For example, we can use this method to output just C:

```
String line = "The Heav'ns and all the Constellations rung";
```

```
System.out.println(line.substring(24, 25));  
// Prints: C
```

11.7 substring() Instructions

1.

Change the arguments of the `line.substring()` method so that it prints out Heav'ns instead.

```
public class Poetry {  
  
    public static void main(String[] args) {  
  
        String line = "The Heav'ns and all the Constellations rung";  
        // Change the arguments:  
        System.out.println(line.substring(23, 38));  
  
    }  
  
}
```

OUTPUT

Constellations

11.8 toUpperCase() / toLowerCase()

There will be times when we have a word in a case other than what we need it in. Luckily, Java has a couple String methods to help us out:

- toUpperCase(): returns the string value converted to uppercase
- toLowerCase(): returns the string value converted to lowercase

For example:

```
String input = "Cricket!";
```

```
String upper = input.toUpperCase();  
// stores "CRICKET!"
```

```
String lower = input.toLowerCase();  
// stores "cricket!"
```

A good use of this functionality is to ensure consistency of the data you store in a database. Making sure all of the data you get from a user is lowercase before you store it in your database will make your database easier to search through later.

11.8 toUpperCase() / toLowerCase() Instructions

1.

In **Hashtag.java**, use toLowerCase() to print out the lower case version of hashtag.

```
public class Hashtag {
```

```
    public static void main(String[] args) {
```

```
        String hashtag = "#100DaysOfCode";
```

```
        // Make the hashtag lowercase:
```

```
}
```

```
}
```

OUTPUT

#100daysofcode

11.9 Review

Congratulations! 🙌

We have learned some of the string methods that come with the String class:

- length()
- concat()
- indexOf()
- charAt()
- equals() / equalsIgnoreCase()
- substring()
- toUpperCase() / toLowerCase()

11. Quiz

1. Suppose we have a String like so. How would we use substring() to print out a value of 940?

String address = "940 Lorimer";

- a) address.substring(4)
- b) address.substring(1, 3)
- c) address.substring(0, 3)

d) `address.substring(0, 2)`

2. Which of the following string methods returns the string value converted to uppercase?

- a) `indexOf()`
- b) `charAt()`
- c) `toLowerCase()`
- d) `toUpperCase()`

3. True/False: `concat()` doesn't actually change the value of the original string.

- a) True.
- b) False.

4. Complete the code below to find the character located at index 2:

```
String str = "qwer";
```

```
System.out.println(____.____(____));
```

- `charAt`
- `indexOf`
- `2`
- `str`

Click or drag and drop to fill in the blank

5. What will the following code print out?

```
String message = "how r u?";
```

```
System.out.println(message.length());
```

- a) 7
- b) 8
- c) 6
- d) 5

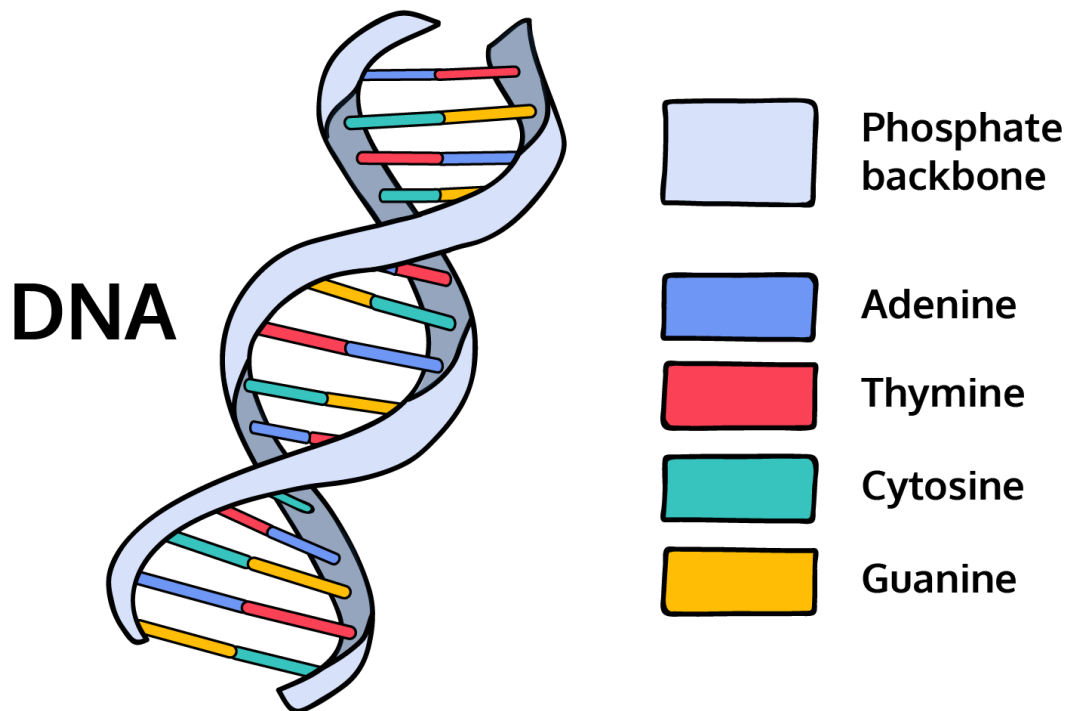
Project-11

DNA Sequencing

The genome of an organism stores all the genetic information necessary to build and maintain that organism. It is an organism's complete set of DNA.

DNA is composed of a series of nucleotides abbreviated as:

- A: Adenine
- C: Cytosine
- G: Guanine
- T: Thymine



So a strand of DNA could look something like ACGAATTCCG.

Write a **DNA.java** program that determines whether there is a protein in a strand of DNA.

A protein has the following qualities:

1. It begins with a “start codon”: ATG.
2. It ends with a “stop codon”: TGA.
3. In between, each additional codon is a sequence of three nucleotides.

So for example:

- ATGCGATACTGA is a protein because it has the start codon ATG, the stop codon TGA, and the length is divisible by 3.
- ATGCGATAGA is not a protein because the sequence length is not divisible by 3, so the third condition is not satisfied.

Tasks

String methods:

1.

string methods	value
length()	returns the length
concat()	concatenates two strings
equals()	checks for equality between two strings
indexOf()	returns the index of a substring
charAt()	returns a character
substring()	returns a substring
toUpperCase()	returns the upper case version
toLowerCase()	returns the lower case version

Note: Scroll through the table to see each method.

Setting up:

2.

Let's create a skeleton for the program. Add the following into **DNA.java**:

```
public class DNA {  
  
    public static void main(String[] args) {  
  
        // -. -. -. -. .  
        // \ \ / \ \ /  
        // /\ \ /\ \  
        // ~ `~ `~ `~`  
  
    }  
  
}
```

3.

Write a comment near the top of the program that describes what the program does.

4.

Here are the three DNA strands that you are going to use to test your program:

- "ATGCGATACGCTTGA"
- "ATGCGATACGTGA"
- "ATTAATATGTACTGA"

Store them in different strings: dna1, dna2, and dna3.

Find the length:

5.

Create a generic String variable called dna that can be set to any DNA sequence (dna1, dna2, dna3).

6.

To warm up, find the length of the dna string.

Find the start codon and stop codon:

7.

Remember that a protein has the following qualities:

1. It begins with a start codon ATG.
2. It ends with a stop codon TGA.
3. In between, the number of nucleotides is divisible by 3.

First, let's start with the first condition. Does the DNA strand have the start codon ATG within it?

Find the index where ATG begins using indexOf().

8.

Next, does the DNA strand have the stop codon TGA?

Find the index where TGA begins.

Find the protein:

9.

Lastly, you'll find out whether or not there is a protein!

Let's start with an if statement that checks for a start codon and a stop codon using the `&&` operator.

Remember that the `indexOf()` string method will return -1 if the substring doesn't exist within a String.

10.

Add a third condition that checks whether or not that the number of nucleotides in between the start codon and the stop condon is a multiple of 3.

Remember that the modulo operator `%` returns the remainder of a division.

11.

Inside the if statement, create a String variable named `protein`.

And find this protein in the dna by using the `substring()` string method. Think about where you want the substring to begin and where you want the substring to end.

Remember that a codon is 3 nucleotides long.

12.

Add an else clause that print out No protein..

13.

You are all done!

Let's test your code with each DNA strand. These should be the results:

- dna1: Contains a protein.
- dna2: Does not contain a protein.

- dna3: Contains a protein.

OUTPUT

- Protein: ATGTACTGA

Concept Review

length() String Method in Java

In Java, the length() string method returns the total number of characters – the length – of a String.

```
String str = "Codecademy";  
System.out.println(str.length());  
  
// prints 10
```

concat() String Method in Java

In Java, the concat() string method is used to append one String to the end of another String. This method returns a String representing the text of the combined strings.

```
String s1 = "Hello";  
String s2 = " World!";  
String s3 = s1.concat(s2);  
  
// concatenates strings s1 and s2  
System.out.println(s3);  
  
// prints "Hello World!"
```

String Method equals() in Java

In Java, the equals() string method tests for equality between two Strings.

`equals()` compares the contents of each `String`. If all of the characters between the two match, the method returns `true`. If any of the characters do not match, it returns `false`.

Additionally, if you want to compare two strings without considering upper/lower cases, you can use `.equalsIgnoreCase()`.

```
String s1 = "Hello";
```

```
String s2 = "World";
```

```
System.out.println(s1.equals("Hello"));
```

```
// prints true
```

```
System.out.println(s2.equals("Hello"));
```

```
// prints false
```

```
System.out.println(s2.equalsIgnoreCase("world"));
```

```
// prints true
```

indexOf() String Method in Java

In Java, the `indexOf()` string method returns the first occurrence of a character or a substring in a `String`. The character/substring that you want to find the index of goes inside of the `()`.

If `indexOf()` cannot find the character or substring, it will return `-1`.

```
String str = "Hello World!";
```

```
System.out.println(str.indexOf("l"));
```

```
// prints 2
```

```
System.out.println(str.indexOf("Wor"));  
// prints 6
```

```
System.out.println(str.indexOf("z"));  
// prints -1
```

charAt() String Method in Java

In Java, the charAt() string method returns the character of a String at a specified index. The index value is passed inside of the (), and should lie between 0 and length()-1.

```
String str = "This is a string";
```

```
System.out.println(str.charAt(0));  
// prints 'T'
```

```
System.out.println(str.charAt(15));  
// prints 'g'
```

toUpperCase() and toLowerCase() String Methods

In Java, we can easily convert a String to upper and lower case with the help of a few string methods:

- toUpperCase() returns the string value converted to uppercase.
- toLowerCase() returns the string value converted to lowercase.

```
String str = "Hello World!";  
String uppercase = str.toUpperCase();  
// uppercase = "HELLO WORLD!"
```

```
String lowercase = str.toLowerCase();
```

```
// lower
```

12. Access, Encapsulation & Scope

12.1 What are Access and Scope?

As our Java programs begin to get bigger and we begin to have multiple Objects and Classes that interact with each other, the concepts of *access* and *scope* come into play. To oversimplify things, the concepts of access and scope both center around what parts of your programs can interact with specific variables or methods from other parts of your program. Let's take a brief look at some of the concepts we'll cover:

Access

- The public and private keywords and how they relate to Classes, variables, constructors, and methods
- The concept of encapsulation
- Accessor methods, sometimes known as "getters"
- Mutator methods, sometimes known as "setters"

Scope

- Local variables vs. instance variables
- The this keyword

Let's get started!

12.1 What are Access and Scope? Instructions

For now, let's get a preview of how access works in Java. In the code editor, you should see two files - **Bank.java** and **CheckingAccount.java**. Take a look at each file and notice the instance variables are declared using the private keyword. When you're ready, select the **Bank.java** file, and run your code.

You should get two errors! Take a look at the main() method in **Bank.java**. Our Bank is trying to access some instance variables from a CheckingAccount that is marked as private! Take a look at the error message to see which variables are giving us trouble.

```
public class Bank{

    private CheckingAccount accountOne;

    private CheckingAccount accountTwo;


    public Bank(){

        accountOne = new CheckingAccount("Zeus", 100);
        accountTwo = new CheckingAccount("Hades", 200);
    }


    public static void main(String[] args){

        Bank bankOfGods = new Bank();

        System.out.println(bankOfGods.accountOne.name);

        System.out.println(bankOfGods.accountOne.balance);

    }

}
```

OUTPUT

Bank.java:12: error: name has private access in CheckingAccount

```
    System.out.println(bankOfGods.accountOne.name);
```

^

Bank.java:13: error: balance has private access in CheckingAccount

```
    System.out.println(bankOfGods.accountOne.balance);
```


^

2 errors

12.2 The public Keyword

After running the code in the last exercise, you should be developing an intuition on what the public and private keywords are doing. These keywords are defining what parts of your code have *access* to other parts of your code.

We can define the access of many different parts of our code including instance variables, methods, constructors, and even a class itself. If we choose to declare these as public this means that any part of our code can interact with them - even if that code is in a different class!

The way we declare something to be public is to use the public keyword in the declaration statement. In the code block below, we have a public class, constructor, instance variables, and method. Notice the five different uses of public.

```
public class Dog{
    public String name;
    public int age;

    public Dog(String input_name, int input_age){
        name = input_name;
        age = input_age;
    }

    public void speak() {
        System.out.println("Arf Arf! My name is " + name + " and I am a good dog!");
    }
}
```

Since everything about a Dog is public, any other class can access anything about a Dog. For example, let's say there was a DogSchool class. Any method of the DogSchool class could make a new Dog using the public Dog constructor, directly access that Dog's instance variables, and directly use that Dog's methods:

```

public class DogSchool{
    public void makeADog(){
        Dog cujo = new Dog("Cujo", 7);
        System.out.println(cujo.age);
        cujo.speak();
    }
}

```

Notice that the DogSchool class and the makeADog() method are also public. This means that if some other class created a DogSchool, they would have access to these methods as well! We have public methods calling public methods!

One final thing to note is that for the purposes of this lesson, we'll almost always make our classes and constructors public. While you can set them to private, it's fairly uncommon to do so. Instead, we'll focus on why you might make your instance variables and methods private. We'll start looking into the private keyword in the next exercise.

12.2 The public Keyword Instructions

1.

Once again we've given you a Bank class and a CheckingAccount class. First, take a look at **CheckingAccount.java** to see what methods and variables have been set to private or public. Then, switch to **Bank.java**, look at the main method, and hit the "Run" button. You should get an error. Why?

2.

In **Bank.java**'s main() method, we're trying to access three different private fields from the CheckingAccount class. Edit the checking account class to make those fields public. Make sure to edit only the fields you need in order to make Bank's main() method run — don't change any of the other fields in CheckingAccount.

After you're done editing **CheckingAccount.java**, go back to **Bank.java** and run your code. A Bank should now have access to these fields from a CheckingAccount.

```
public class Bank{  
    private CheckingAccount accountOne;  
    private CheckingAccount accountTwo;  
  
    public Bank(){  
        accountOne = new CheckingAccount("Zeus", 100, "1");  
        accountTwo = new CheckingAccount("Hades", 200, "2");  
    }  
  
    public static void main(String[] args){  
        Bank bankOfGods = new Bank();  
        System.out.println(bankOfGods.accountOne.name);  
        bankOfGods.accountOne.addFunds(5);  
        bankOfGods.accountOne.getInfo();  
  
    }  
  
}
```

OUTPUT

Output:

Zeus

This checking account belongs to Zeus. It has 105 dollars in it.

12.3 The private Keyword and Encapsulation

By now you're probably catching onto what the private keyword does. When a Class' instance variable or method is marked as private, that means that you can only access those structures from elsewhere inside that same class. Let's look back at our DogSchool example:

```
public class DogSchool{  
  
    public void makeADog(){  
        Dog cujo = new Dog("Cujo", 7);  
        System.out.println(cujo.age);  
        cujo.speak();  
    }  
}
```

makeADog is trying to directly access Dog's .age variable. It's also trying to use the .speak() method. If those are marked as private in the Dog class, the DogSchool class won't be able to do that. Other methods within the Dog class would be able to use .age or .speak() (for example, we could use cujo.age within the Dog class), but other classes won't have access.

Keep Reading: AP Computer Science A Students

At this point, you might be thinking to yourself "Why even bother with any of this? In the last exercise, my code was broken until I flipped some variables and methods to public. Why don't I just make everything public?"

While those are valid points, sometimes restricting our code is actually useful from a design perspective. This is one of the core ideas behind *encapsulation*. By making our instance variables (and some methods) private, we encapsulate our code into nice little bundles of logic.

For example, a Bank object doesn't necessarily need to know the inner workings of a CheckingAccount object. It doesn't need to know that the money is stored in a field named money, or that interest is added to an account by using a method named .addInterest(). In fact, if it had access to those fields or methods, it's possible that someone using a Bank object could change things in a CheckingAccount without realizing it. By limiting access by using

the `private` keyword, we are able to segment, or encapsulate, our code into individual units.

Note that we don't necessarily want to completely block everything from other classes. In the next exercise, we'll get into when you might want to make methods public — we'll take a look at getter and setter methods.

12.3 The `private` Keyword and Encapsulation Instructions

1.

We've changed the variables and methods in **CheckingAccount.java** back to private. To begin, go to **Bank.java** and run your code. Let's confirm that we get errors when a Bank tries to access private data from a CheckingAccount

2.

A Bank couldn't access a CheckingAccount's private structures. Let's prove to ourselves that CheckingAccount can access its own private fields.

Go to **CheckingAccount.java**. In the `main()` method, create a new CheckingAccount named `myAccount`. You'll have to use the constructor that takes a String and an int.

Run **CheckingAccount.java** to create this object.

3.

Let's confirm that we can access a CheckingAccount's private fields since we're writing in CheckingAccount's `main()` method. Still in **CheckingAccount.java**, print out `myAccount`'s balance.

4.

Let's also confirm that we can use private methods. Call `addFunds()` to add 5 dollars to `myAccount`'s balance. Then print the balance again to confirm the money was added. Continue to do this work in the `main()` method of **CheckingAccount.java**.

Bank.JAVA

```
public class Bank{  
    private CheckingAccount accountOne;  
    private CheckingAccount accountTwo;  
  
    public Bank(){  
        accountOne = new CheckingAccount("Zeus", 100);  
        accountTwo = new CheckingAccount("Hades", 200);  
    }  
  
    public static void main(String[] args){  
        Bank bankOfGods = new Bank();  
        System.out.println(bankOfGods.accountOne.name);  
        bankOfGods.accountOne.addFunds(5);  
        bankOfGods.accountOne.getInfo();  
    }  
}
```

CheckingAccount.JAVA

```
public class CheckingAccount{  
    private String name;  
    private int balance;
```

```

public CheckingAccount(String inputName, int inputBalance){
    name = inputName;
    balance = inputBalance;
}

private void addFunds(int fundsToAdd){
    balance += fundsToAdd;
}

private void getInfo(){
    System.out.println("This checking account belongs to " + name + ". It has " +
balance + " dollars in it.");
}

public static void main(String[] args){

}

}

```

OUTPUT

2000

2005

12.4 Accessor and Mutator Methods

When writing classes, we often make all of our instance variables private. However, we still might want some other classes to have access to them, we just

don't want those classes to know the *exact* variable name. To give other classes access to a private instance variable, we would write an *accessor method* (sometimes also known as a "getter" method).

```
public class Dog{
    private String name;

    //Other methods and constructors

    public String getName() {
        return name;
    }
}
```

Even though the instance variable name is private, other classes could call the public method `getName()` which returns the value of that instance variable. Accessor methods will always be public, and will have a return type that matches the type of the instance variable they're accessing.

Similarly, private instance variables often have mutator methods (sometimes known as "setters"). These methods allow other classes to reset the value stored in private instance variables.

```
public class Dog{
    private String name;

    //Other methods and constructors

    public void setName(String newName) {
        name = newName;
    }

    public static void main(String[] args){
        Dog myDog = new Dog("Cujo");
        myDog.setName("Lassie");
    }
}
```


Mutator methods, or “setters”, often are void methods — they don’t return anything, they just reset the value of an existing variable. Similarly, they often have one parameter that is the same type as the variable they’re trying to change.

12.4 Accessor and Mutator Methods Instructions

1.

Take a look at the `main()` method of **Bank.java**. Right now it’s trying to directly access and change the balance in a checking account. Unfortunately, that instance variable is private. Run your code with **Bank.java** open. You should expect an error.

2.

Let’s go to **CheckingAccount.java** and write an accessor method for the balance instance field. Your method should be named `getBalance()`. This method should be public.

When you’re done writing that method, go back to the `main()` method in **Bank.java** and change the code to use that new method.

3.

Let’s do the same for a mutator method. In **CheckingAccount.java**, write a mutator method named `setBalance()` that has an `int` parameter named `newBalance`. The method should set the balance instance variable to the value passed into the method.

After writing this method, go back to **Bank.java** and add a line of code to your `main()` method where you use `setBalance()` to set `accountOne`’s balance to 5000.

After setting the balance to 5000, use your `.getBalance()` method again to print out the new balance.

BANK.java

```
public class Bank{  
    private CheckingAccount accountOne;
```

```
private CheckingAccount accountTwo;
```

```
public Bank(){
```

```
    accountOne = new CheckingAccount("Zeus", 100, "1");
```

```
    accountTwo = new CheckingAccount("Hades", 200, "2");
```

```
}
```

```
public static void main(String[] args){
```

```
    Bank bankOfGods = new Bank();
```

```
    System.out.println(bankOfGods.accountOne.balance);
```

```
}
```

```
}
```

CheckingAccount.java

```
public class CheckingAccount{
```

```
    private String name;
```

```
    private int balance;
```

```
    private String id;
```

```
public CheckingAccount(String inputName, int inputBalance, String inputId){
```

```
    name = inputName;
```

```
    balance = inputBalance;
```

```
    id = inputId;
```

```
}
```

```
//Write new methods here:
```

```
}
```

OUTPUT

100

5000

12.5 Scope: Local Variables

In addition to access modifiers like `public` and `private`, the *scope* of the variable also determines what parts of your code can access that variable.

The *scope* of a variable is determined by where the variable is declared. For example, because instance variables are declared inside a class but outside any methods or constructors, all methods and constructors are within the scope of that variable. For example, in the code block below, constructors and methods of the `Dog` class are using the `Dog` instance variables like `name` and `age`:

```
class Dog{
    public String name;
    public int age;
    public int weight;

    public Dog(){
        name = "Winston";
        age = 8;
        weight = 30;
    }

    public void speak(){
        System.out.println("My name is " + name);
    }
}
```

However, if we have a variable declared inside a method, that variable can only be used inside that method. The same is true for parameters. The scope of those parameters is only the method they're associated with. If you try to use a parameter outside the function it's defined in, you'll get an error. These variables are often called local variables. Note that we don't use public or private when declaring local variables.

This idea of scope extends to conditionals and loops as well. If you declare a variable inside the body of a conditional or in a loop, that variable can only be used inside that structure. This also includes the variable you're using as your looping variable. For example, consider the following block of code:

```
for(int i = 0; i < 10; i++){  
    // You can use i here  
}  
// i is out of scope here
```

You can only use `i` between the curly braces of the for loop. In general, whenever you see curly braces, be aware of scope. If a variable is defined inside curly braces, and you try to use that variable outside of those curly braces, you will likely see an error!

12.5 Scope: Local Variables Instructions

1.

Take a look at the `main()` method which tries to sum the values from a list. Right now, we're looping through the values and trying to store the sum in a variable called `sum`. However, that variable is declared inside the for loop.

There are two problems with this. The first is a logical error. Since we're setting `int sum = 0;` at the start of the loop, every time the loop runs, `sum` gets reset back to 0.

However, there is also a syntactical error. Right now we try to print `sum` after the loop. But that's outside of `sum`'s scope since the variable was declared inside the loop.

Run your code as-is. You should expect to get an error related to the scope of `sum`.

2.

Move the line of code `int sum = 0;` to be above the for loop. Now the for loop has access to `sum` and we're able to return `sum` after the loop finishes.

After moving the line of code, run your code to test the `main()` method.

```
public class ScopeExample{  
    public static void main(String[] args){  
  
        int[] myArray = {1, 2, 3, 4};  
        for(int i = 0; i < myArray.length; i++){  
            int sum = 0;  
            sum += myArray[i];  
        }  
  
        System.out.println(sum);  
    }  
}
```

OUTPUT

10

12.6 Scope: The `this` Keyword

Often times when creating classes, programmers will create local variables with the same name as instance variables. For example, consider the code block below:

```
public class Dog{  
    public String name;  
  
    public Dog(String inputName){  
        name = inputName;  
    }  
}
```

```
public void speakNewName(String name){
    System.out.println("Hello, my new name is" + name);
}
```

```
public static void main(String[] args){
    Dog myDog = new Dog("Winston");
    myDog.speakNewName("Darla"); // Prints "Darla" - "Winston" ignored

}
}
```

We have an instance variable named name, but the method speakNewName has a parameter named name. So when the method tries to print name, which variable will be printed? By default, Java refers to the local variable name. So in this case, the value passed to the parameter will be printed and not the instance variable.

If we wanted to access the instance variable and not the local variable, we could use the this keyword.

```
public class Dog{
    public String name;

    public Dog(String inputName){
        name = inputName;
    }

    public void speakNewName(String name){
        System.out.println("Hello, my new name is" + this.name);
    }

    public static void main(String[] args){
        Dog a = new Dog("Fido");
        Dog b = new Dog("Odie");

        a.speakNewName("Winston");
    }
}
```

```

    // "Fido", the instance variable of Dog a is printed. "Winston" is ignored

    b.speakNewName("Darla");
    // "Odie", the instance variable of Dog b is printed. "Darla" is ignored.
}
}

```

The `this` keyword is a reference to the current object. We used `this.name` in our `speakNewName()` method. This caused the method to print out the value stored in the instance variable `name` of whatever Dog Object called `speakNewName()`. (Note that in this somewhat contrived example, the local variable `name` used as a parameter gets completely ignored).

Oftentimes, you'll see constructors have parameters with the same name as the instance variable. For example, you might see something like:

```

public Dog(String name){
    this.name = name;
}

```

You can read this as “set this Dog’s instance variable `name` equal to the variable passed into the constructor”. While this naming is a common convention, it can also be confusing. There’s nothing wrong with naming your parameters something else to be more clear. Sometimes you will see something like:

```

public Dog(String inputName){
    this.name = inputName;
}

```

This is now a little clearer — we’re setting the Dog’s instance variable `name` equal to the name we give the constructor.

Finally, mutator methods also usually follow this pattern:

```

public void setName(String name){
    this.name = name;
}

```

We reset the instance variable to the value passed into the parameter.

Throughout the rest of this lesson, we'll use this. when referring to an instance variable. This isn't always explicitly necessary — if there's no local variable with the same name, Java will know to use the instance variable with that name. That being said, it is a good habit to use this. when working with your instance variables to avoid potential confusion.

12.6 Scope: The this Keyword Instructions

1.

Take a look at the instance variables for our SavingsAccount class. There are three of them. owner, balanceDollar, balanceEuro. Next, take a look at the constructor we've started for you. The job of the constructor is to initialize these instance variables.

Complete the constructor to give value to all three instance variables. The instance variable owner should be set to the local variable owner. The instance variable balanceDollar should be set equal to the local variable balanceDollar. Finally, the instance variable balanceEuro should be set equal to the local variable balanceDollar multiplied by 0.85. (Right now, every dollar is worth 0.85 euros.)

2.

Next, look at the addMoney() method. This method should add the value stored in the parameter balanceDollar to the instance variable balanceDollar. Before adding the money to the account, print out the statement "Adding ____ dollars to the account.". After adding the money, print the statement "The new balance is ____ dollars.". Be careful of spacing, capitalization, and punctuation with your print statements.

Call the addMoney() method once in the main() method to test your code. Add 2000 dollars to Zeus's account.

(Note that this function should probably be updating balanceEuro as well based on the parameter. We won't test your code for that, but you can try implementing it yourself. Also, note that balanceDollar is a pretty bad name for the parameter of this function. Something like moneyToAdd would make more sense. But for the

sake of practicing local variables vs instance variables, keep the parameter named balanceDollar.)

```
public class SavingsAccount{

    public String owner;
    public int balanceDollar;
    public double balanceEuro;

    public SavingsAccount(String owner, int balanceDollar){
        // Complete the constructor
    }

    public void addMoney(int balanceDollar){
        // Complete this method
    }

    public static void main(String[] args){
        SavingsAccount zeusSavingsAccount = new SavingsAccount("Zeus", 1000);

        // Make a call to addMoney() to test your method
    }

}
```

OUTPUT

Adding 2000 dollars to the account.

The new balance is 3000 dollars.

12.7 Using this With Methods

We've seen how the `this` works with variables, but we can also use the `this` with methods.

Recall how we've been calling methods up to this point:

```
public static void main(String[] args){
    Dog myDog = new Dog("Odie");
    myDog.speak();
}
```

Here we're creating an instance of a `Dog` and using that `Dog` to call the `speak()` method. However, when defining methods, we can also use the `this` keyword to call other methods. Consider the code block below:

```
public class Computer{
    public int brightness;
    public int volume;

    public void setBrightness(int inputBrightness){
        this.brightness = inputBrightness;
    }

    public void setVolume(int inputVolume){
        this.volume = inputVolume;
    }

    public void resetSettings(){
        this.setBrightness(0);
        this.setVolume(0);
    }
}
```

Take a look at the `resetSettings()` method in particular. This method calls other methods from the class. But it needs an object to call those methods! Rather than create a new object (like we did with the Dog named `myDog` earlier), we use `this` as the object. What this means is that the object that calls `resetSettings()` will be used to call `setBrightness(0)` and `setVolume(0)`.

```
public static void main(String[] args){
    Computer myComputer = new Computer();
    myComputer.resetSettings();
}
```

In this example, calling `myComputer.resetSettings()` is as if we called `myComputer.setBrightness(0)` and `myComputer.setVolume(0)`. `this` serves as a placeholder for whatever object was used to call the original method.

Keep Reading: AP Computer Science A Students

Finally, this can be used as a value for a parameter. Let's say a method exists that takes a `Computer` as a parameter (that method's signature might be something like `public void pairWithOtherComputer(Computer other)`). If you're writing another method of the `Computer`, and want to call the `pairWithOtherComputer()` method, you could use `this` as the parameter. That call might look something like `this.pairWithOtherComputer(this)`. You're using the current object to call the method *and* are passing that object as that method's parameter.

```
public void pairWithOtherComputer(Computer other){
    // Code for method that uses the parameter other
}
```

```
public void setUpConnection(){
    // We use "this" to call the method and also pass "this" to the method so it can
    // be used in that method
    this.pairWithOtherComputer(this);
}
```

12.7 Using this With Methods Instructions

1.

We've given you a Person class with three instance variables — age, wisdom, and fitness. Those values all get initialized to different values when the constructor is called. We've also provided you with mutator methods for all three variables.

Complete the hasBirthday() method. This method should call the mutator methods to increase age by 1, increase wisdom by 5, and decrease fitness by 3.

When you're done writing hasBirthday() look at the main() method to see how we're calling it.

```
public class Person{

    public int age;

    public int wisdom;

    public int fitness;


    public Person(int inputAge){

        this.age = inputAge;

        this.wisdom = inputAge * 5;

        this.fitness = 100 - inputAge;

    }


    public void setAge(int newAge){

        this.age = newAge;

    }


    public void setWisdom(int newWisdom){

        this.wisdom = newWisdom;

    }

}
```

```
public void setFitness(int newFitness){  
    this.fitness = newFitness;  
}
```

```
public void hasBirthday(){  
    //Complete this method  
}
```

```
public static void main(String[] args){  
    Person emily = new Person(20);  
    emily.hasBirthday();  
    System.out.println("New age is: " + emily.age);  
    System.out.println("New wisdom is: " + emily.wisdom);  
    System.out.println("New fitness is: " + emily.fitness);  
}  
}
```

OUTPUT

New age is: 20

New wisdom is: 100

New fitness is: 80

12.8 Other Private Methods

Now that we've seen how methods can call other methods using this., let's look at a situation where you might want to use private methods.

Oftentimes, private methods are helper methods — that is to say that they're methods that other, bigger methods use.

For example, for our `CheckingAccount` example, we might want a public method like `getAccountInformation()` that prints information like the name of the account owner, the amount of money in the account, and the amount of interest the account will make in a month. That way, another class, like a `Bank`, could call that public method to get all of that information quickly.

Well, in order to get that information, we might want to break that larger method into several helper methods. For example, inside `getAccountInformation()`, we might want to call a function called `calculateNextMonthInterest()`. That helper method should probably be private. There's no need for a `Bank` to call these smaller helper methods — instead, a `Bank` can call the one public method, and rely on that method to do all of the complicated work by calling smaller private methods.

12.8 Other Private Methods Instructions

1.

Let's implement what we described in the narrative. We've written our `getAccountInformation()` method in the **`CheckingAccount.java`** class. But we haven't yet implemented the `calculateNextMonthInterest()` method. This should be a private method and return a double — the balance of the account multiplied by the `interestRate`. Write that function.

Then flip back to **`Bank.java`** and take a look at the `main()` method. We're calling `bankOfGods.accountOne.getAccountInformation();`. Will a `Bank` be able to do this? Run your code to find out.

2.

Nice work! We were able to get all of the account information, including the next month's interest, even though `calculateNextMonthInterest()` is private.

Let's prove to ourselves that we can't call this private method from a Bank.

In **Bank.java**'s main() method, try

calling bankOfGods.accountOne.calculateNextMonthInterest();.

You should see an error since you're trying to use a Bank to call a private method from a CheckingAccount.

Bank.java

```
public class Bank{

    private CheckingAccount accountOne;

    private CheckingAccount accountTwo;


    public Bank(){

        this.accountOne = new CheckingAccount("Zeus", 100, "1");
        this.accountTwo = new CheckingAccount("Hades", 200, "2");
    }


    public static void main(String[] args){

        Bank bankOfGods = new Bank();

        bankOfGods.accountOne.getAccountInformation();

    }

}
```

CheckingAccount.java

```
public class CheckingAccount{

    private String name;

    private int balance;
```

```

private String id;

private double interestRate;


public CheckingAccount(String inputName, int inputBalance, String inputId){
    this.name = inputName;
    this.balance = inputBalance;
    this.id = inputId;
    this.interestRate = 0.02;
}


public void getAccountInformation(){
    System.out.println("Money in account: " + this.getBalance());
    System.out.println("Next Month's Interest: " +
this.calculateNextMonthInterest());
}

private int getBalance(){
    return this.balance;
}

// Write the calculateNextMonthInterest() here
}

```

OUTPUT

Bank.java:13: error: calculateNextMonthInterest() has private access in CheckingAccount

```
    bankOfGods.accountOne.calculateNextMonthInterest();
```

1 error

12.9 Review

Nice work! In this lesson, we dove into some of the more subtle features of classes with a focus on access, encapsulation, and scope. Here are some of the main takeaways from this lesson:

- The `public` and `private` keywords are used to define what parts of code have access to other classes, methods, constructors, and instance variables.
- Encapsulation is a technique used to keep implementation details hidden from other classes. Its aim is to create small bundles of logic.
- The `this` keyword can be used to designate the difference between instance variables and local variables.
- Local variables can only be used within the scope that they were defined in.
- The `this` keyword can be used to call methods when writing classes.

12. Quiz

Fill in the code so the `String` instance variable named `name` is accessible from other classes, but the `int` variable named `age` isn't.

```
public class Student{  
    [ ] String [ ] ;  
    [ ] int [ ] ;  
}
```

"Winston"

public

age

int

20

name

private

String

Click or drag and drop to fill in the blank

1.

2. Let's say you're writing a method named `increaseAgeByFive()` that is trying to increase the instance variable of the Dog class named `age` by 5. However, there is also a local variable named `age`. How would you increase only the instance variable by 5.

- a) `self.age += 5;`
- b) `age += 5;`
- c) `Dog.age += 5;`
- d) `this.age += 5;`

3. Given the following instance variables for a Dog class, which of the following methods is an accessor method for the `age` variable?

```
public class Dog{  
    private String name;  
    private int age;
```

```
}
```

- a)

```
private String getAge(){  
    return this.age;  
}
```
- b)

```
public String getAge(){  
    return this.age;  
}
```
- c)

```
private int getAge(){  
    return this.age;  
}
```
- d)

```
public int getAge(){  
    return this.age;  
}
```

4. What does encapsulation mean?

- a) The concept of creating different classes to represent objects in the real world.
- b) The concept of separating methods and variables into distinct bundles of logic.

- c) The concept of returning values from your methods.
- d) The concept that some variables only exist in a certain scope.

5. What value will be printed given the following code snippet?

```
public class Human(){  
    public String name;  
  
    public Human(String inputName){  
        this.name = inputName;  
    }  
  
    public void sayName(String name){  
        System.out.println("My name is " + name);  
    }  
  
    public static void main(String[] args){  
        Human hero = new Human("Clark Kent");  
        hero.sayName("Superman");  
    }  
}
```

- a) My name is Clark Kent
- b) My name is Superman
- c) Nothing will be printed since sayName() is a void method.
- d) My name is hero

6. Which of the following is NOT true about mutator methods.

- a) Mutator methods should be marked as public.
- b) Mutator methods often have one parameter of the same type as the instance variable they're interacting with.
- c) Mutator methods are often void methods.
- d) Mutator methods return the value of an instance variable.

7. Explain the body of the methodOne() method in simple terms.

```
class myDemoClass(){
```

```

int demoInstanceVariable;

public void methodOne(){
    this.methodTwo(this);
}

public void methodTwo(myDemoClass a){
    System.out.println(a.demoInstanceVariable);
}
}

```

- a) Whatever object (this) that called methodOne() proceeds to call methodTwo(). methodTwo() has a default parameter of this which will automatically be whatever object called methodOne.
- b) Look, Codecademy, we're using that weird this keyword twice, and that's really confusing. I can't explain what that's doing.
- c) Whatever object (this) that called methodOne() proceeds to call methodTwo(). methodTwo() takes a myDemoClass object as a parameter, so in addition to calling the method, this passes itself as a parameter to methodTwo().
- d) The method will print the instance variable demoInstanceVariable of whatever myDemoClass object that it gets as a parameter.

8. What will the following statement output?

```

int x = Math.abs(-15) + 5;
System.out.println(x)

```

- a) -20
- b) 20
- c) 10
- d) -10

9. What will the following program output?

```
double x = Math.pow(2, 4);  
System.out.println(x);
```

- a) 8.0
- b) 16.0
- c) 16
- d) 8

10. Which option will produce a random int value in a range starting from 3 up to (and including) 8?

- a) `int x = (int)(Math.random() * 3) + 8;`
- b) `int x = (int)(Math.random() * 8) + 3;`
- c) `int x = (int)(Math.random() * 6) + 3;`
- d) `int x = (int)(Math.random() * 11) + 3;`

11. Which of the following options is not a method in the Math class?

Hint: You might need to check the Math class [documentation](#).

- a) `negateExact(int a)`
- b) `roundDown(float x)`
- c) `hypot(double x, double y)`
- d) `tanh(double x)`

Static Methods of the Math Class

Learn how to use static methods from the Math class to implement numerical calculations in Java.

The Math Class

Math can play a major role in our programs. It can become tedious to write out every equation we need in our code. Luckily, a lot of this work can be truncated using the **Math** class. The Java **Math** class is part of the **java.lang** package; it contains a variety of methods that can be used to perform numerical calculations in our

programs. In this article, we'll discuss the different types of **Math** methods available to us and how to implement them in our Java programs.

The **Math** class also offers a good opportunity for us to learn about Static methods. We'll take a look at what static methods are and how they're different from non-static methods.

Calling Static Methods

Every method in the **Math** class is static. This means that we can call and use these methods without creating an object of the class. There are two main options for calling a static method.

Our first option is to append the dot operator to the class name followed by the method we want to execute. If we wanted to reference a method of the **Math** class, we would use **Math.NameOfMethodHere**. Let's see this in action with the **Math** class method **min()** which returns the smaller value of two given numbers:

```
class Numbers {  
    public static void main(String[] args) {  
        // Call method using the Class name, the dot operator, the method name, and  
arguments  
        int smallerNumber = Math.min(3, 10);  
        System.out.println(smallerNumber); // Prints: 3  
    }  
}
```

How is this any different from calling a non-static method? We don't need to create an object of the class in order to use the methods it contains. Let's see an example of a non-static method:

```
class Numbers {  
    int firstNumber;  
    int secondNumber;
```

```

public Numbers (int num1, int num2) {
    firstNumber = num1;
    secondNumber = num2;
}

// non-static method
public int returnSum() {
    return firstNumber + secondNumber;
}

public static void main(String[] args) {
    // Create an object
    Numbers myNumbers = new Numbers(2, 5);
    // Call a non-static method on object
    int sum = myNumbers.returnSum();
    System.out.println(sum); // Prints: 7
}
}

```

In our code above, we had to create an object of type **Numbers** in order to use the non-static method **returnSum()**. With non-static methods, if we don't create an object of this class (or one of its subclasses), we do not have access to its methods. This isn't the case for static methods.

Our second option for calling a static method from the **Math** class is to import the class by adding **import static java.lang.Math.*;** to the top of our program. If we import the **Math** class, we can reference the method using only the method name like so:

```
import static java.lang.Math.*; // import Math class
```

```

class Numbers {
    public static void main(String[] args) {

```

```
// Call method by using method name and arguments
int smallerNumber = min(3, 10);
System.out.println(smallerNumber); // Prints: 3
}
}
```

Useful Methods

There are many useful methods from the **Math** class that can be implemented in our programs. For those of us taking the AP Computer Science A exam, the following methods and their descriptions will be available in the [Java Quick Reference Guide](#):

int abs(int x)

Purpose: Returns the absolute value of an int value

The absolute value states how many numbers a value is away from **0**. The absolute value of always a positive number. For example, the absolute value of **-5** is **5** because it is **5** away from **0**. In Java, we can get the absolute value of a number like this:

```
System.out.println(Math.abs(5)); // Prints: 5
System.out.println(Math.abs(-5)); // Prints: 5
```

double abs(double x)

Purpose: Returns the absolute value of a double value

This is similar to the previous method, but this method takes in and returns a **double** type value:

```
System.out.println(Math.abs(5.0)); // Prints: 5.0
System.out.println(Math.abs(-5.0)); // Prints: 5.0
```

double pow(double base, double exponent)

Purpose: Returns the value of the first parameter raised to the power of the second parameter.

The power, or exponent, describes how many times a number should be multiplied by itself. For example, **5** to the power of **3** is equivalent to **5 * 5 * 5**, or **125**. If we wanted to see this in Java, we could use **Math.pow()** like this:


```
double x = Math.pow(5, 3);  
System.out.println(x); // Prints: 125.0
```

double sqrt(double x)

Purpose: Returns the positive square root of a double value

The square root of a number represents what value can be multiplied by itself in order to equal a specified value. For example, the square root of **49** is **7** because **7 * 7** is **49**.

In java, getting the square root of a value looks like this:

```
double x = Math.sqrt(49);  
System.out.println(x); // Prints: 7.0  
double y = Math.sqrt(52);  
System.out.println(y); // Prints: 7.211102550927978
```

double random()

Purpose: Returns a double value greater than or equal to **0.0** and less than **1.0**

Randomization is a great way to add probability to our programs. There are many ways to implement **Math.random()** in Java. Its default use case is to produce a random **double** value between **0.0** and **1.0**. For example:

```
System.out.println(Math.random());  
System.out.println(Math.random());  
System.out.println(Math.random());
```

The random values can change every time we run our program. An example output of the above program is the following:

```
0.8592007008856128  
0.6120058754881421  
0.48259656765819403
```

With some manipulation, we can use **Math.random()** to create a random **int** or **double** value within a predefined range.

For example, if we wanted a random **double** value between **0** and **10**, not including **10**, we would multiply **Math.random()** by **10**

```
// Random double value between 0 and 10, not including 10
```

```
double a = Math.random() * 10;
```

If we wanted a random **int** value between **0** and **9**, we would need to implement the **(int)** casting operator in our expression like so:

```
// Random int value between 0 and 9
```

```
int b = (int)(Math.random() * 10);
```

If we wanted our range to start at **1** and end at **10**, we would have to add **1** to the end of our previous expression:

```
// Random int value between 1 and 10
```

```
int c = (int)(Math.random() * 10) + 1;
```

Using addition also gives us the ability to start the range at any number. What if we wanted an **int** value in the range of **10** up to and including **20**? We would have to implement the algorithm **(Math.random() * (maxValue - minValue + 1)) + minValue**.

For example:

```
// Random int value between 10 and 20
```

```
int d = (int)(Math.random() * 11 ) + 10;
```

We multiply **Math.random()** by **11** because **20** (our max value) minus **10** (our minimum value) plus **1** is **11**. We add **+ 10** outside the parentheses so that our smallest value is guaranteed to be **10**.

Here's another way to think about this algorithm — the value that you multiply by defines the number of possible values you can get. The number that you add defines the starting value. So, for example, **(int)(Math.random() * 3) + 5**; will give you one of three random values starting at **5**. So this could give you **5**, **6**, or **7**.

Finally, be careful of off-by-one errors when using **Math.random()**. For example, you might write some code that you think generates a number between 1 and 10, but it actually generates a number between 1 and 9. Be sure to test your code frequently to spot logical errors like these!

Conclusion

Great job reaching the end of this article. Let's recap what we learned:

- The **Math** class is part of the **java.lang** package and provides useful static methods for performing mathematical equations.
- To call these static methods, reference the class name + the dot operator + the method name. To only reference the method name, import the **Math** class into your program.
- For students taking the AP Computer Science A exam, several methods from the **Math** class will be available in a [quick reference sheet](#).

Concept Review

The public and private keywords

In Java, the keywords public and private define the access of classes, instance variables, constructors, and methods.

private restricts access to only the class that declared the structure, while public allows for access from any class.

Encapsulation

Encapsulation is a technique used to keep implementation details hidden from other classes. Its aim is to create small bundles of logic.

The private Keyword

In Java, instance variables are encapsulated by using the private keyword. This prevents other classes from directly accessing these variables.

```
public class CheckingAccount{  
    // Three private instance variables  
    private String name;  
    private int balance;  
    private String id;
```

```
}
```

Accessor Methods

In Java, accessor methods return the value of a private variable. This gives other classes access to that value stored in that variable. without having direct access to the variable itself.

Accessor methods take no parameters and have a return type that matches the type of the variable they are accessing.

```
public class CheckingAccount{  
    private int balance;  
  
    //An accessor method  
    public int getBalance(){  
        return this.balance;  
    }  
}
```

Mutator Methods

In Java, mutator methods reset the value of a private variable. This gives other classes the ability to modify the value stored in that variable without having direct access to the variable itself.

Mutator methods take one parameter whose type matches the type of the variable it is modifying. Mutator methods usually don't return anything.

```
public class CheckingAccount{  
    private int balance;  
  
    //A mutator method  
    public void setBalance(int newBalance){
```

```
    this.balance = newBalance;
}
}
```

Local Variables

In Java, local variables can only be used within the scope that they were defined in. This scope is often defined by a set of curly brackets. Variables can't be used outside of those brackets.

```
public void exampleMethod(int exampleVariable){
    // exampleVariable can only be used inside these curly brackets.
}
```

The this Keyword with Variables

In Java, the this keyword can be used to designate the difference between instance variables and local variables. Variables with this. reference an instance variable.

```
public class Dog{
    public String name;

    public void speak(String name){
        // Prints the instance variable named name
        System.out.println(this.name);

        // Prints the local variable named name
        System.out.println(name);
    }
}
```

The this Keyword with Methods

In Java, the `this` keyword can be used to call methods when writing classes.

```
public class ExampleClass{  
    public void exampleMethodOne(){  
        System.out.println("Hello");  
    }  
  
    public void exampleMethodTwo(){  
        //Calling a method using this.  
        this.exampleMethodOne();  
        System.out.println("There");  
    }  
}
```

Static Methods

Static methods are methods that can be called within a program without creating an object of the class.

```
// static method  
public static int getTotal(int a, int b) {  
    return a + b;  
}  
  
public static void main(String[] args) {  
    int x = 3;  
    int y = 2;  
    System.out.println(getTotal(x,y)); // Prints: 5  
}
```

Calling a Static Method

Static methods can be called by appending the dot operator to a class name followed by the name of the method.

```
int largerNumber = Math.max(3, 10); // Call static method
System.out.println(largerNumber); // Prints: 10
```

The Math Class

The Math class (which is part of the java.lang package) contains a variety of static methods that can be used to perform numerical calculations.

```
System.out.println(Math.abs(-7.0)); // Prints: 7
```

```
System.out.println(Math.pow(5, 3)); // Prints: 125.0
```

```
System.out.println(Math.sqrt(52)); // Prints: 7.211102550927978
```

The static Keyword

Static methods and variables are declared as static by using the static keyword upon declaration.

```
public class ATM{
    // Static variables
    public static int totalMoney = 0;
    public static int numATMs = 0;

    // A static method
    public static void averageMoney(){
        System.out.println(totalMoney / numATMs);
    }
}
```

```
}
```

Static Methods and Variables

Static methods and variables are associated with the class as a whole, not objects of the class. Both are used by using the name of the class followed by the . operator.

```
public class ATM{  
    // Static variables  
    public static int totalMoney = 0;  
    public static int numATMs = 0;  
  
    // A static method  
    public static void averageMoney(){  
        System.out.println(totalMoney / numATMs);  
    }  
  
    public static void main(String[] args){  
  
        //Accessing a static variable  
        System.out.println("Total number of ATMs: " + ATM.numATMs);  
  
        // Calling a static method  
        ATM.averageMoney();  
    }  
}
```


Static Methods with Instance Variables

Static methods cannot access or change the values of instance variables.

```
class ATM{  
    // Static variables  
    public static int totalMoney = 0;  
    public static int numATMs = 0;  
  
    public int money = 1;  
  
    // A static method  
    public static void averageMoney(){  
        // Can not use this.money here because a static method can't access instance  
        variables  
    }  
  
}
```

Methods with Static Variables

Both non-static and static methods can access or change the values of static variables.

```
class ATM{  
    // Static variables  
    public static int totalMoney = 0;  
    public static int numATMs = 0;  
    public int money = 1;
```

```
// A static method interacting with a static variable
public static void staticMethod(){
    totalMoney += 1;
}

// A non-static method interacting with a static variable
public void nonStaticMethod(){
    totalMoney += 1;
}
}
```

Static Methods and the this Keyword

Static methods do not have a this reference and are therefore unable to use the class's instance variables or call non-static methods.

```
public class DemoClass{

    public int demoVariable = 5;

    public void demoNonStaticMethod(){

    }

    public static void demoStaticMethod(){
        // Can't use "this.demoVariable" or "this.demoNonStaticMethod()"
    }
}
```

13.STATIC VARIABLES AND METHODS

13.1 Static Methods Refresher

In this lesson, we're going to dive into how to create classes with your own static methods and static variables. To begin, let's take a quick refresher on static methods.

Static methods are methods that belong to an entire class, not a specific object of the class. Static methods are called using the class name and the `.` operator. We've seen a couple of static methods already!

```
double randomNumber = Math.random();  
// Stores a random decimal between 0.0 and 1.0 in randomNumber
```

```
double number = Double.valueOf("2.5");  
// Transforms the String "2.5" into a double
```

In the first example, `random()` is a static method that belongs to the `Math` class. We didn't need to create a `Math` object (like `Math myMathObject = new Math()`) in order to use that method. We could just call it using the class name.

Similarly, `valueOf()` is a static method of the `Double` class. Given a `String` as an input, this method will turn that `String` into a `double`. Again, we don't need to create a `Double` object in order to call this method — we use the class itself to call it.

Finally, notice that our `main()` methods have been static this whole time. When Java runs your program, it calls that the `main` method of your class — `YourClassName.main()`.

13.1 Static Methods Refresher Instructions

We've set up some code with a handful of common static methods. We're using:

- [`Math.random\(\)`](#)
- [`Math.abs\(\)`](#)
- [`Integer.toString\(\)`](#)

Take a look at the documentation for these methods and try to figure out what they do.

You can also look at the [documentation for the entire Math class](#) which has many more static methods. Find a static method that looks interesting to you, and try to use it!

```
public class ExerciseOne{  
    public static void main(String[] args){  
  
        int randomNum = (int)(Math.random() * 10) + 1;  
  
        System.out.println("Your random number between one and ten is " +  
randomNum);  
  
        int negativeNum = -2;  
  
        int absNum = Math.abs(negativeNum);  
  
        System.out.println("The absolute value of "+ negativeNum + " is " + absNum);  
  
        String myNewString = Integer.toString(1);  
  
        System.out.println(myNewString);  
  
    }  
}
```

OUTPUT

Your random number between one and ten is 10

The absolute value of -2 is 2

1

13.2 Static Variables

We'll begin writing our own static methods soon, but before we do, let's take a look at static variables. Much like static methods, you can think of *static variables* as belonging to the class itself instead of belonging to a particular object of the class.

Just like with static methods, we can access static variables by using the name of the class and the `.` operator. Finally, we declare static variables by using the `static` keyword during declaration. This keyword usually comes after the variable's access modifier (`public` or `private`).

When we put this all together, we might end up with a class that looks something like this:

```
public class Dog{

    // Static variables
    public static String genus = "Canis";

    //Instance variables
    public int age;
    public String name;

    public Dog(int inputAge, String inputName){
        this.age = inputAge;
        this.name = inputName;
    }
}
```

Since all dogs share the same genus, we could use a static variable to store that information for the entire class. However, we want each dog to have its own unique name and age, so those aren't static. We could now access this static variable in a `main()` function like so:

```
public class Dog{
    //Variables, constructors and methods defined here
```

```

public static void main(String[] args){
    System.out.println(Dog.genus); // Prints Canis
}
}

```

Unlike static methods, you can still access static variables from a specific object of the class. However, no matter what object you use to access the variable, the value will always be the same. You can think of it as all objects of the class sharing the same variable

```

public static void main(String[] args){
    Dog snoopy = new Dog(3, "Snoopy");
    Dog ringo = new Dog(5, "Ringo");

    System.out.println(Dog.genus); // Prints Canis
    System.out.println(snoopy.genus); // Prints Canis
    System.out.println(ringo.genus); // Prints Canis
}

```

Finally, you might have seen a few static variables before. If you want easy access to the largest possible integer, you can get it by using `Integer.MAX_VALUE`. If you look at the [official documentation](#) you'll see that this variable is public, static, *and* final. (final means that you can't change the variable's value after creating it.) We're starting to know a lot of Java keywords!

13.2 Static Variables Instructions

1.

We're going to be building out an ATM class that has both static and non-static variables. We've started by giving you an ATM class with non-static variables, a constructor, and some methods. In the `main()` method, use the constructor to create two ATMs named `firstATM` and `secondATM`. Use the constructor so the first ATM has 1000 dollars in it and the second has 500.

If you'd like, you could print out each ATM's money variable to confirm you created them successfully.

2.

Let's create two static variables.

First, we want to create a variable to keep track of how much money is in the system across all ATMs. This should be a public static int variable and should be named `totalMoney`. This variable should begin with a value of 0.

Second, we want to know how many ATMs there are in the system. Again, this should be a public static int variable that has an initial value of 0. Name this variable `numATMs`.

3.

Let's take a look at the first of these static variables. In the `main()` method, print your `totalMoney` variable three different times. The first time, you should use `ATM.totalMoney`, the second time you should use `firstATM.totalMoney`, and the third time you should use `secondATM.totalMoney`. Do you expect these print statements to be the same or different?

Right now the value stored in `totalMoney` doesn't actually represent the total amount of money stored in all ATMs. In the next exercise, we'll look into how to change the value of static variables.

```
public class ATM{
```

```
// Step 2: Create your static variables here
```

```
// Instance variables
```

```
public int money;
```

```
public ATM(int inputMoney){
```

```
    this.money = inputMoney;
```

```
}
```

```
public void withdrawMoney(int amountToWithdraw){  
    if(amountToWithdraw <= this.money){  
        this.money -= amountToWithdraw;  
    }  
}
```

```
public static void main(String[] args){  
    // Step 1: Create your two ATMs here  
  
    // Step 3: Print your static variable in three different ways here  
}  
  
}
```

OUTPUT

1000

500

0

0

0

13.3 Modifying Static Variables

Now that we've created a couple of static variables, let's start to edit them. The good news is that editing static variables is similar to editing any other variable.

Whether you're writing code in a constructor, a non-static method, or a static method, you have access to static variables.

Before we jump into the checkpoints, let's think about times when you might want to edit static variables. Often times, you'll see static variables used to keep track of information about all objects of a class. For example, our variable `numATMs` is keeping track of the total number of ATMs in the system. Therefore, every time an ATM is created (using the constructor), we should increase that variable by 1. If we could somehow destroy an ATM, the method that destroys it should decrease `numATMs` static variable by 1.

Similarly, we have a variable named `totalMoney`. This variable is keeping track of all money across all ATMs. Whenever we remove money from an ATM using the non-static `withdrawMoney()` method, we should modify the money instance variable for that particular ATM as well as the `totalMoney` variable. In doing so, all ATMs will know how much money is in the system.

13.3 Modifying Static Variables Instructions

1.

Edit the constructor to increase `numATMs` by 1 every time an ATM is created. In the main method, we're printing out `numATMs` before and after creating some ATMs. You should expect to see that number go up as you create ATMs.

2.

Edit the constructor so when a new ATM is created, the amount of money that ATM starts with gets added to the static variable `totalMoney`.

3.

Edit the `withdrawMoney()` method so when money is taken out of a specific ATM, the static variable `totalMoney` also reflects that change.

Take a look at the `main()` method — we're printing `totalMoney` before and after withdrawing money from a couple of ATMs.

```

public class ATM{
    // Static variables
    public static int totalMoney = 0;
    public static int numATMs = 0;

    // Instance variables
    public int money;

    public ATM(int inputMoney){
        this.money = inputMoney;

        // Steps 1 and 2: Edit numATMs and total money here
    }

    public void withdrawMoney(int amountToWithdraw){
        if(amountToWithdraw <= this.money){
            this.money -= amountToWithdraw;
            // Step 3: Edit totalMoney here
        }
    }

    public static void main(String[] args){

        System.out.println("Total number of ATMs: " + ATM.numATMs);
    }
}

```

```
    ATM firstATM = new ATM(1000);  
    ATM secondATM = new ATM(500);  
    System.out.println("Total number of ATMs: " + ATM.numATMs);  
  
    System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);  
    firstATM.withdrawMoney(500);  
    secondATM.withdrawMoney(200);  
    System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);  
  
}  
  
}
```

OUTPUT

Total number of ATMs: 0

Total number of ATMs: 2

Total amount of money in all ATMs: 1500

Total amount of money in all ATMs: 800

13.4 Writing Your Own Static Methods

Nice work! Now that we've seen how static variables work, let's look into how to write our own static methods.

Let's get the syntax out of the way first — just like with variables, to create a static method, use the static keyword in the method's definition. Just like with variables, this keyword usually comes after public or private.

```
public static void myFirstStaticMethod(){  
    // Some code here  
}
```

Often times, you'll see static methods that are accessors or mutators for static variables.

```
public static int getMyStaticVariable(){  
    return myStaticVariable;  
}
```

```
public static void setMyStaticVariable(int newValue){  
    myStaticVariable = newValue;  
}
```

One important rule to note is that static methods can't interact with non-static instance variables.

To wrap your mind around this, consider why we use this when working with non-static instance variables. Let's say we have a Dog class with a non-static instance variable named age. If we have a line of code like this.age = 5;, that means we're setting the age of a specific Dog equal to 5. However, if age were static, that would mean that the variable belongs to the entire class, not a specific object.

The this keyword can't be used by a static method since static methods are associated with an entire class, not a specific object of that class. If you try to mix this with a static method, you'll see the error message non-static variable this cannot be referenced from a static context.

13.4 Writing Your Own Static Methods Instructions

1.

Let's write a static method that prints the average amount of money in all vending machines in the system.

The method name should be averageMoney and it should be void since it doesn't return anything. The method should print totalMoney divided by numATMs. Note that both of those variables are static.

After writing the method, call the method at the end of the main() method. Recall that since this is a static method, you'll use the name of the class to call the method, not an object of the class.

2.

Let's try to have averageMoney() interact with a non-static instance variable. In averageMoney(), try to print this.money. You should expect to see an error message — a static method can't interact with a non-static instance variable.

```
public class ATM{  
    // Static variables  
    public static int totalMoney = 0;  
    public static int numATMs = 0;  
  
    // Instance variables  
    public int money;  
  
    public ATM(int inputMoney){  
        this.money = inputMoney;  
        numATMs += 1;  
        totalMoney += inputMoney;  
    }  
  
    public void withdrawMoney(int amountToWithdraw){  
        if(amountToWithdraw <= this.money){  
            this.money -= amountToWithdraw;  
            totalMoney -= amountToWithdraw;  
        }  
    }  
}
```

```

    }
}

// Write your averageMoney() method here

public static void main(String[] args){

    System.out.println("Total number of ATMs: " + ATM.numATMs);
    ATM firstATM = new ATM(1000);
    ATM secondATM = new ATM(500);
    System.out.println("Total number of ATMs: " + ATM.numATMs);

    System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);
    firstATM.withdrawMoney(500);
    secondATM.withdrawMoney(200);
    System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);
    // Call averageMoney() here
}
}

```

OUTPUT

ATM.java:26: error: non-static variable this cannot be referenced from a static context

```

    System.out.println(this.money);

```

^

1 error

13.5 Review

Great work! You now have an understanding of what the static keyword does. In fact, if you've made it this far in your Java lessons, you probably have a pretty good sense of what all the keywords and jargon are doing in `public static void main(String[] args)`. Take a moment to celebrate — that line of code can be incredibly intimidating for new learners and it's a real accomplishment to learn about all of those different pieces.

To review, here are some of the main takeaways about static methods and variables:

- Static methods and variables are associated with the class as a whole, not objects of the class.
- Static methods and variables are declared as static by using the static keyword upon declaration.
- Static methods cannot interact with non-static instance variables. This is due to static methods not having a `this` reference.
- Both static methods and non-static methods can interact with static variables.

13.5 Review Instructions

We've given you the complete code for your ATM class. Feel free to experiment with these static variables and methods. In particular, you might want to create a few more ATMs in the `main()` method and see how the static variables change. You could also consider writing a `depositMoney()` method that works in the opposite way as `withdrawMoney()`.

```
public class ATM{  
    // Static variables  
  
    public static int totalMoney = 0;  
  
    public static int numATMs = 0;
```

```

// Instance variables
public int money;

public ATM(int inputMoney){
    this.money = inputMoney;
    numATMs += 1;
    totalMoney += inputMoney;
}

public void withdrawMoney(int amountToWithdraw){
    if(amountToWithdraw <= this.money){
        this.money -= amountToWithdraw;
        totalMoney -= amountToWithdraw;
    }
}

public static void averageMoney(){
    System.out.println(totalMoney / numATMs);
}

public static void main(String[] args){

    System.out.println("Total number of ATMs: " + ATM.numATMs);
}

```



```

ATM firstATM = new ATM(1000);
ATM secondATM = new ATM(500);
System.out.println("Total number of ATMs: " + ATM.numATMs);

System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);
firstATM.withdrawMoney(500);
secondATM.withdrawMoney(200);
System.out.println("Total amount of money in all ATMs: " + ATM.totalMoney);

// Call averageMoney() here
ATM.averageMoney();
}
}

```

OUTPUT

Total number of ATMs: 0

Total number of ATMs: 2

Total amount of money in all ATMs: 1500

Total amount of money in all ATMs: 800

400

13. Quiz

1. Given the code block, which line(s) of code will successfully print the value 5.

```

public class ATM{
    public static int totalMoney = 0;

```

```

public ATM(){
    totalMoney += 5;
}

public static void main(String[] args){

    System.out.println(ATM.totalMoney); //Line 1

    ATM myATM = new ATM();
    System.out.println(ATM.totalMoney); //Line 2
    System.out.println(myATM.totalMoney); //Line 3

    ATM mySecondATM = new ATM();
    System.out.println(ATM.totalMoney); //Line 4
    System.out.println(myATM.totalMoney); //Line 5
}
}

```

- a) Lines 2 and 3
- b) Lines 2, 3, 4, and 5
- c) Lines 1, 2, and 3
- d) Line 2

2. Why are static methods unable to interact with instance variables and non-static methods?

- a) Static methods are associated with an object, not the class itself. As a result, static methods have no `this` reference.
- b) Static methods *can* interact with non-static variables and methods. However, non-static methods can't interact with static variables.
- c) Static methods are associated with a class, not an object of the class. As a result, static methods have no `this` reference.

3. Fill in the code so the value 5 will be printed.

```

public class MyDemoClass{

    public static int variableA = 4;
    public int variableB;

    public MyDemoClass(){
        variableA += 1;
        this.variableB = 4;
    }

    public static void main(String[] args){

        MyDemoClass myObject = new ____ ;
        System.out.println(____.____);
    }
}

```

- MyDemoClass()
- variableB
- MyDemoClass
- this
- variableA

Click or drag and drop to fill in the blank

4. What is the mistake in the code block?

```

public class ATM{
    public static int totalMoney = 0;

    public int money;

    public ATM(int money){
        this.money = money;
    }
}

```

```
}

public static int getMoney(){
    return this.money;
}

public int getTotalMoney(){
    return totalMoney;
}

}
```

- a) The variable totalMoney can't be given the value of 0 when it is declared.
- b) The static method getMoney() is trying to access the non-static variable money.
- c) The non-static method getTotalMoney() is trying to access the static variable totalMoney.
- d) The constructor is incorrectly setting the value of an instance variable due to incorrectly using a local variable.

5. What is the difference between declaring static variables and instance variables?

- a) When declaring static variables, you use no keywords, but when declaring instance variables you use the this keyword
- b) When declaring static variables, you use keywords like private or public to make the variable static.
- c) When declaring static variables, you use the static keyword. For instance variables, you use the this keyword, or no keyword.
- d) When declaring static variables, you use the this keyword along with the variable type.

14. Inheritance & Polymorphism

14.1 Introducing Inheritance

When you hear the word “inheritance,” code may not be the first thing that springs to mind; you’re probably more likely to think of inheriting genetic traits, like eye color from your mother or a smile from your grandfather.

But [inheritance](#) is also an important feature of object-oriented programming in Java.

Suppose we are building a Shape class in Java. We might give it some points in 2D, a method for calculating the area, and another method for displaying the shape. But what happens if we want a class for a triangle that has some triangle-specific methods? Do we need to redefine all of the same methods that we created for Shape?

No! (Phew.) Lucky for us, a Java class can also *inherit* traits from another class. Because a Triangle is a Shape, we can define Triangle so that it inherits fields and methods directly from Shape. A reference of type Shape can refer to an object of Shape or an object of Triangle. The object-oriented principle of inheritance saves us the headache of redefining the same class members all over again.

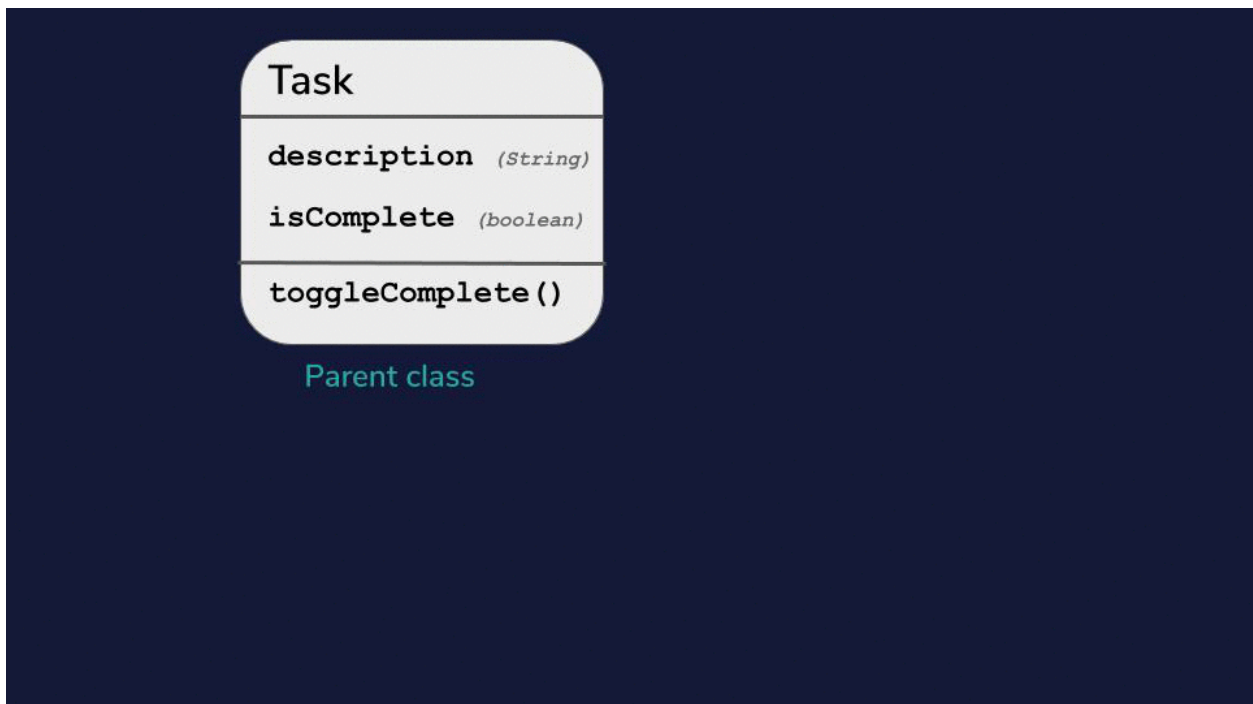
Our Triangle class will inherit all the traits of Shape, but Triangle can also contain its own unique methods and variables. For example, we could have an instance variable called hypotenuse and a method called findHypotenuse() that can only be accessed by Triangle class references. Objects of Triangle can call any method contained in Triangle or Shape. This gives us a bunch of possibilities!

There are several terms you’ll encounter frequently:

- *Parent class*, *superclass*, and *base class* refer to the class that another class inherits from (like Shape).
- *Child class*, *subclass*, and *derived class* refer to a class that inherits from another class (like Triangle).

14.1 Introducing Inheritance Instructions

Take a look at the image to see how inheritance works in object-oriented programming.



14.2 Inheritance in Practice

So how do we define a child class so that it inherits from a parent class? We use the keyword `extends` like this:

```
class Shape {  
  
    // Shape class members  
  
}  
  
class Triangle extends Shape {  
  
    // additional Triangle class members  
  
}
```

Now Triangle has inherited traits from Shape, meaning it copied over class members from Shape. When we use inheritance to extend a subclass from a superclass, we create an “is-a” relationship from the subclass to the superclass. For example, an object of Triangle *is a* member of the Shape class; however, an object of Shape is not necessarily an object of Triangle.

Until now, we’ve only been working with one class and one file. However, most Java programs utilize multiple classes, each of which requires its own file. Only one file needs a main() method — this is the file we will run.

Note: the various classes in our Java package — even though they are in different files — will have access to each other, so we can instantiate one class inside of another.

14.2 Inheritance in Practice Instructions

1.

You have two files:

- **Noodle.java**: This will house your parent class.
- **Spaghetti.java**: This is where you will build your child class.

The Noodle class has already been defined for you, including:

- The lengthInCentimeters field.
- The shape field.
- The texture field.
- A cook() method.

In **Spaghetti.java**, create a class called Spaghetti.

Then tab back over to **Noodle.java** to run your code. (We always need to run our program using the file that contains the main() method.)

2.

Tab back over to **Spaghetti.java** and make Spaghetti a child class of Noodle (if you haven't already).

3.

Tab back over to **Noodle.java**.

In main(), instantiate a new Spaghetti object called spaghettiPomodoro.

Print out its texture field to the terminal.

```
class Noodle {  
  
    double lengthInCentimeters;  
    String shape;  
    String texture = "brittle";  
  
    public void cook() {  
  
        this.texture = "cooked";  
  
    }  
  
    public static void main(String[] args) {  
  
    }  
  
}
```

OUTPUT

brittle

14.3 Inheriting the Constructor

Hang on, you might be thinking, if the child class inherits its parent's fields and methods, does it also inherit the constructor? Let's take a look at how the `super()` constructor works!

Let's say `Shape` has a `numSides` field that is set by passing an integer into the constructor. If we're instantiating a `Triangle`, we would want that number to always be 3, so we'd want to modify the constructor to automatically assign `numSides` with a value of 3.

Can we do that?

As it happens, Java has a trick up its sleeve for just this occasion: using the `super()` method which acts like the parent constructor inside the child class constructor:

```
class Triangle extends Shape {  
  
    Triangle() {  
        super(3);  
    }  
  
    // additional Triangle class members  
  
}
```

By passing 3 to `super()`, we are making it possible to instantiate a `Triangle` without passing in a value for `numSides`.

Meanwhile, `super(3)` (behaving as `Shape(3)`) will shoulder the responsibility of setting `numSides` to 3 for our `Triangle` object. It's like we called `Shape(3)`.

It is also possible to write a constructor without making a call to any `super()` constructor:

```
class Triangle extends Shape {
```

```

Triangle() {
    this.numSides = 3;
}

// additional Triangle class methods

}

```

In this situation, Java secretly calls the parent class' no-argument constructor (`super()`). So in this specific example, the `Triangle()` constructor first calls the `Shape()` constructor. That `Shape()` takes care of whatever business it needs to take care of. And then after that is complete, we go in and set `this.numSides` to 3.

If you're writing a constructor of a child class, and don't explicitly make a call to a constructor from a parent class using `super`, it's important to remember that Java will automatically (and secretly) call `super()` as the first line of your child class constructor.

14.3 Inheriting the Constructor Instructions

1.

Tab over to **Pho.java** and build a constructor for `Pho`.

Leave the constructor's parameters blank. Inside the constructor, pass the following arguments in the following order to `super()` — the constructor for `Noodle`:

- A `lengthInCentimeters` of 30.0
- A `widthInCentimeters` of 0.64
- A `shape` of "flat"
- `ingredients` of "rice flour"

2.

Tab back over to **Noodle.java**. Inside `main()`, instantiate a new `Pho` instance called `phoChay`.

Now print out phoChay's shape.

Noodle.java

```
class Noodle {  
  
    double lengthInCentimeters;  
    double widthInCentimeters;  
    String shape;  
    String ingredients;  
    String texture = "brittle";  
  
    Noodle(double lenInCent, double wthInCent, String shp, String ingr) {  
  
        this.lengthInCentimeters = lenInCent;  
        this.widthInCentimeters = wthInCent;  
        this.shape = shp;  
        this.ingredients = ingr;  
  
    }  
  
    public void cook() {  
  
        this.texture = "cooked";  
  
    }  
}
```

```

public static void main(String[] args) {

}

}

```

OUTPUT

flat

14.4 Parent Class Aspect Modifiers

You may recall that Java class members use private and public access modifiers to determine whether they can be accessed from outside the class. So does a child class inherit its parent’s private members?

Well, no. But there is another access modifier we can use to keep a parent class member accessible to its child classes and to files in the package it’s contained in — and otherwise private: protected.

Modifier	Class	Package	Child Class	Global
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
no modifier	✓	✓	✗	✗
private	✓	✗	✗	✗

Here’s what protected looks like in use:

```

class Shape {

```

```
protected double perimeter;  
  
}  
  
// any child class of Shape can access perimeter
```

In addition to access modifiers, there's another way to establish how child classes can interact with inherited parent class members: using the final keyword. If we add final after a parent class method's access modifier, we disallow any child classes from changing that method. This is helpful in limiting bugs that might occur from modifying a particular method.

Though it is not required, there is an established order when two or more field modifiers are used (eg. public final). To learn more about this read the [documentation](#).

14.4 Parent Class Aspect Modifiers Instructions

1.

Currently, Noodle has an ingredients attribute that its child class Ramen cannot access because it has a private access modifier.

Change the access modifier for ingredients so that Ramen (not just any Java file in any package) can access it.

2.

Noodle has an isTasty() method that returns true because noodles are tasty. But if you check in the Ramen class, you'll see someone overrode isTasty() to return false! That person clearly doesn't know food; all noodles are definitely tasty.

Remove the method from Ramen and add a final keyword to isTasty() in Noodle so that nobody changes this method in any other Noodle child class again.

Noodle.java

```
public class Noodle {
```

```
private double lengthInCentimeters;  
private double widthInCentimeters;  
private String shape;  
private String ingredients;  
private String texture = "brittle";
```

```
Noodle(double lenInCent, double wthInCent, String shp, String ingr) {
```

```
    this.lengthInCentimeters = lenInCent;  
    this.widthInCentimeters = wthInCent;  
    this.shape = shp;  
    this.ingredients = ingr;
```

```
}
```

```
public boolean isTasty() {
```

```
    return true;
```

```
}
```

```
public static void main(String[] args) {
```

```
Ramen yasaiRamen = new Ramen();  
//System.out.println(yasaiRamen.ingredients);  
System.out.println(yasaiRamen.isTasty());  
  
}  
  
}
```

Ramen.java

```
public class Ramen extends Noodle {  
  
    Ramen() {  
  
        super(30.0, 0.3, "flat", "wheat flour");  
  
    }  
  
    public boolean isTasty() {  
  
        return false;  
  
    }  
  
}
```

OUTPUT

wheat flour

true

14.5 Introducing Polymorphism

In Java, if Orange is a Fruit through inheritance, you can then use Orange in the same contexts as Fruit like this:

```
String makeJuice(Fruit fruit) {  
  
    return "Apple juice and " + fruit.squeeze();  
  
}  
  
// inside main()  
Orange orange = new Orange();  
System.out.println(juicer.makeJuice(orange));
```

Wait, how does that work?

This is because Java incorporates the object-oriented programming principle of *polymorphism*. Polymorphism, which derives from Greek meaning “many forms”, allows a child class to share the information and behavior of its parent class while also incorporating its own functionality.

The main advantages of polymorphic programming:

- Simplifying syntax
- Reducing cognitive overload for developers

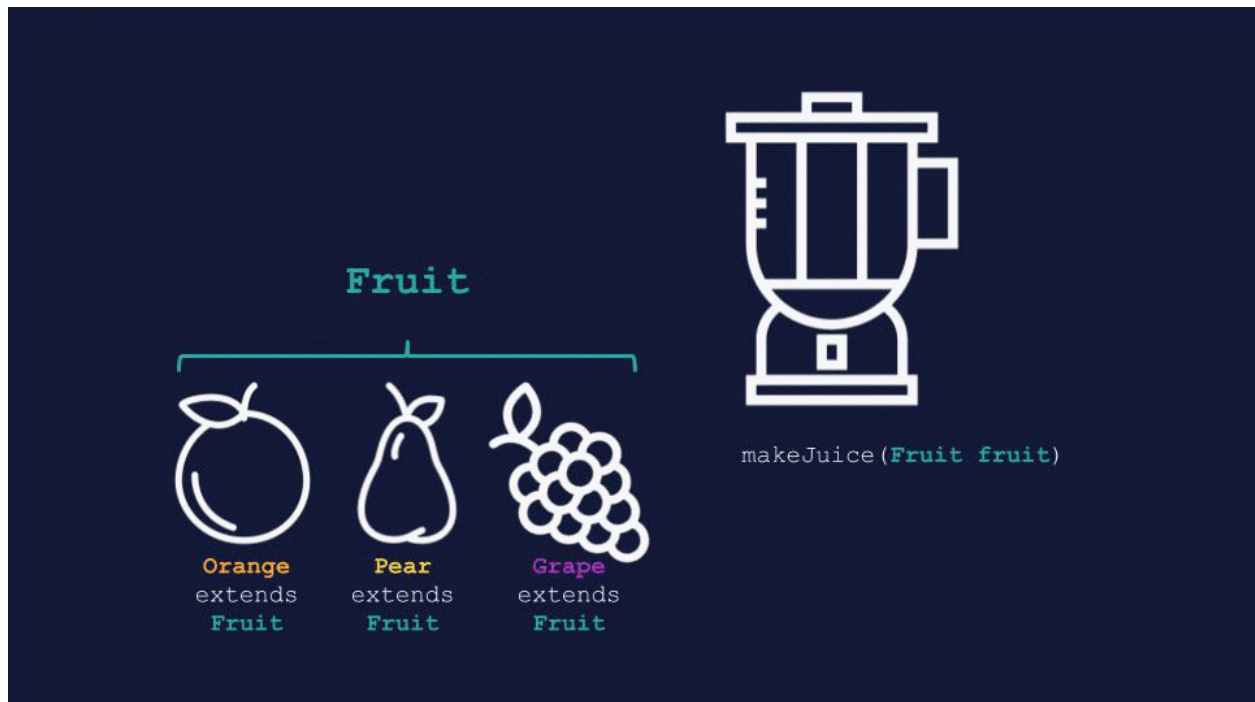
These benefits are particularly helpful when we want to develop our own Java packages for other developers to import and use.

For example, the built-in operator + can be used for both doubles and ints. To the computer, the + means something like addDouble() for one and addInt() for the other, but the creators of Java (and of other languages) didn’t want to burden us as developers with recalling each individual method.

Note that the reverse situation is not true; you cannot use a generic parent class instance where a child class instance is required. So an Orange can be used as a Fruit, but a Fruit cannot be used as an Orange.

14.5 Introducing Polymorphism Instructions

Once you feel ready, click Next to continue.



14.6 Method Overriding

One common use of polymorphism with Java classes is something we mentioned earlier — *overriding* parent class methods in a child class. Like the + operator, we can give a single method slightly different meanings for different classes. This is useful when we want our child class method to have the same name as a parent class method but behave a bit differently in some way.

Let's say we have a `BankAccount` class that allows us to print the current balance. We want to build a `CheckingAccount` class that inherits the functionality of a `BankAccount` but with a modified `printBalance()` method. We can do the following:

```
class BankAccount {  
    protected double balance;  
  
    public BankAccount(double balanceIn){  
        balance = balanceIn;  
    }  
}
```

```

    }

    public void printBalance() {
        System.out.println("Your account balance is $" + balance);
    }
}

class CheckingAccount extends BankAccount {

    public CheckingAccount(double balance) {
        super(balance);
    }

    @Override
    public void printBalance() {
        System.out.println("Your checking account balance is $" + balance);
    }
}

```

Notice that in order to properly override `printBalance()`, in `CheckingAccount` the method has the following in common with the corresponding method in `BankAccount`:

- Method name
- Return type
- Number and type of parameters

You may have also noticed the `@Override` keyword above `printBalance()` in `CheckingAccount`. This annotation informs the compiler that we want to override a method in the parent class. If the method doesn't exist in the parent class, we'll get a helpful error when we compile the program.

Keep Reading: AP Computer Science A Students

In a previous exercise, we learned that the `super` keyword can be used to call the constructor of a superclass. That's not the only use of `super`; we can also use this keyword to call the methods of a parent class. While we now have the ability to

override methods from a superclass, we may find ourselves in a unique situation where we want to use the superclass method instead of the subclass' overridden method.

If that's the case, we can call the parent class method by prepending `super` followed by the dot operator (`.`) to the method call. Note that this only works if we pass in the proper method parameters. Let's see this in action by adding a `checkBalances()` method to `CheckingAccount` that calls both versions of `printBalance()`:

```
class CheckingAccount extends BankAccount {
    public CheckingAccount(double balance) {
        super(balance);
    }

    @Override
    public void printBalance() {
        System.out.println("Your checking account balance is $" + balance);
    }

    public void checkBalances() {
        // calls method from CheckingAccount
        printBalance();
        // calls method from BankAccount
        super.printBalance();
    }

    public static void main(String[] args) {
        CheckingAccount myCheckings = new CheckingAccount(5000);
        myCheckings.checkBalances();
    }
}
```

This program will output:

```
Your checking account balance is $5000
Your account balance is $5000
```

14.6 Method Overriding Instructions

1.

Noodle has a `cook()` method that involves boiling the noodle and changing the texture to "cooked".

The German noodle [spätzle](#) involves a slightly unconventional cooking procedure: grinding or scraping the dough into the pot of boiling water.

In **Spaetzle.java**, override the inherited `cook()` method. Copy the original method body, but before "Boiling" gets printed, add a new line to print "Grinding or scraping dough."

Don't forget to add `@Override` above the method to help with debugging.

```
class Noodle {  
  
    protected double lengthInCentimeters;  
    protected double widthInCentimeters;  
    protected String shape;  
    protected String ingredients;  
    protected String texture = "brittle";  
  
    Noodle(double lenInCent, double wthInCent, String shp, String ingr) {  
  
        this.lengthInCentimeters = lenInCent;  
        this.widthInCentimeters = wthInCent;  
        this.shape = shp;  
        this.ingredients = ingr;  
  
    }  
}
```

```

public void cook() {

    System.out.println("Boiling.");

    this.texture = "cooked";

}

public static void main(String[] args) {

    Spaetzle kaesepaetzle = new Spaetzle();
    kaesepaetzle.cook();

}

}

```

Spaetzle.java

```

class Spaetzle extends Noodle {

    Spaetzle() {

        super(3.0, 1.5, "irregular", "eggs, flour, salt");
        this.texture = "lumpy and liquid";
    }
}

```

```
}
```

```
// Add the new cook() method below:
```

```
}
```

OUTPUT

Grinding or scraping dough.

Boiling.

14.7 Using a Child Class as its Parent Class

An important facet of polymorphism is the ability to use a child class object where an object of its parent class is expected.

One way to do this explicitly is to instantiate a child class object as a member of the parent class. We can instantiate a `CheckingAccount` object as a `BankAccount` like this:

```
BankAccount kaylasAccount = new CheckingAccount(600.00);
```

We can use `kaylasAccount` as if it were an instance of `BankAccount`, in any situation where a `BankAccount` object would be expected. (This would be true even if `kaylasAccount` were instantiated as a `CheckingAccount`, but using the explicit child as parent syntax is most helpful when we want to declare objects in bulk.)

It is important to note here that the compiler just considers `kaylasAccount` to be any old `BankAccount`. But because method overriding is handled at runtime, if we call `printBalance()`, we'll see something `CheckingAccount` specific:

Your checking account balance is \$600.00

This is because at runtime, `kaylasAccount` is recognized as the `CheckingAccount` it is. So, what if `CheckingAccount` has a method `transferToSavings()` that `BankAccount` does not have? Can `kaylasAccount` still use that method?

Well, no. The compiler believes that `kaylasAccount` is just a `BankAccount` that doesn't have some fancy child class `transferToSavings()` method, so it would throw an error.

14.7 Using a Child Class as its Parent Class Instructions

1.

If you take a look at the `Dinner` class, you'll see there's a `makeNoodles()` method that accepts a `Noodle` object as an argument.

In `main()`, instantiate a `BiangBiang` object as a `Noodle` called `biangBiang`.

Then call `makeNoodles()` on `noodlesDinner` with the following arguments:

- `biangBiang`
- `"soy sauce and chili oil"`

Dinner.java

```
class Dinner {  
  
    private void makeNoodles(Noodle noodle, String sauce) {  
  
        noodle.cook();  
  
        System.out.println("Mixing " + noodle.texture + " noodles made from " +  
noodle.ingredients + " with " + sauce + ".");  
  
        System.out.println("Dinner is served!");  
    }  
}
```

```
}
```

```
public static void main(String[] args) {
```

```
    Dinner noodlesDinner = new Dinner();
```

```
    // Add your code here:
```

```
}
```

```
}
```

Noodle.java

```
class Noodle {
```

```
    protected double lengthInCentimeters;
```

```
    protected double widthInCentimeters;
```

```
    protected String shape;
```

```
    protected String ingredients;
```

```
    protected String texture = "brittle";
```

```
    Noodle(double lenInCent, double wthInCent, String shp, String ingr) {
```

```
        this.lengthInCentimeters = lenInCent;
```

```
        this.widthInCentimeters = wthInCent;
```



```
        this.shape = shp;
        this.ingredients = ingr;

    }

    public void cook() {

        this.texture = "cooked";

    }

}
```

BiangBiang.java

```
class BiangBiang extends Noodle {

    BiangBiang() {

        super(50.0, 5.0, "flat", "high-gluten flour, salt, water");

    }

}
```

OUTPUT

Mixing cooked noodles made from high-gluten flour, salt, water with soy sauce and chili oil.

Dinner is served!

14.8 Child Classes in Arrays and ArrayLists

Usually, when we create an array or an ArrayList, the list items all need to be the same type. But polymorphism puts a new spin on what is considered the same type...

In fact, we can put instances of different classes that share a parent class together in an array or ArrayList! For example, let's say we have a Monster parent class with a few child classes: Vampire, Werewolf, and Zombie. We can set up an array with instances of each:

```
Monster dracula, wolfman, zombie1;
```

```
dracula = new Vampire();  
wolfman = new Werewolf();  
zombie1 = new Zombie();
```

```
Monster[] monsters = {dracula, wolfman, zombie1};
```

We can even iterate through the list of items — regardless of subclass — and perform the same action with each item:

```
for (Monster monster : monsters) {  
  
    monster.attack();  
  
}
```

In the code above, we were able to call attack() on each monster in monsters despite the fact that, in the for-each loop, monster is declared as the parent class type Monster.

14.8 Child Classes in Arrays and ArrayLists Instructions

1.

Declare and initialize an array of type Noodle called allTheNoodles with the following items:

- spaghetti
- ramen
- pho

2.

Loop through each noodle in allTheNoodles.

Call getCookPrep() on each noodle and print the result.

Noodle.java

```
class Noodle {
```

```
    protected double lengthInCentimeters;
```

```
    protected double widthInCentimeters;
```

```
    protected String shape;
```

```
    protected String ingredients;
```

```
    protected String texture = "brittle";
```

```
    Noodle(double lenInCent, double wthInCent, String shp, String ingr) {
```

```
        this.lengthInCentimeters = lenInCent;
```

```
        this.widthInCentimeters = wthInCent;
```

```
        this.shape = shp;
```

```
        this.ingredients = ingr;
```

```
    }
```

```
public String getCookPrep() {  
  
    return "Boil noodle for 7 minutes and add sauce.";  
  
}
```

```
public static void main(String[] args) {
```

```
    Noodle spaghetti, ramen, pho;
```

```
    spaghetti = new Spaghetti();
```

```
    ramen = new Ramen();
```

```
    pho = new Pho();
```

```
    // Add your code below:
```

```
}
```

```
}
```

Spaghetti.java

```
class Spaghetti extends Noodle {
```

```
Spaghetti() {
```

```
    super(30.0, 0.2, "round", "semolina flour");
```

```
}
```

```
@Override
```

```
public String getCookPrep() {
```

```
    return "Boil spaghetti for 8 - 12 minutes and add sauce, cheese, or oil and  
garlic.";
```

```
}
```

```
}
```

```
Ramen.java
```

```
class Ramen extends Noodle {
```

```
    Ramen() {
```

```
        super(30.0, 0.3, "flat", "wheat flour");
```

```
}
```

@Override

public String getCookPrep() {

 return "Boil ramen for 5 minutes in broth, then add meat, mushrooms, egg, and vegetables.";

}

}

Pho.java

class Pho extends Noodle {

 Pho() {

 super(30.0, 0.64, "flat", "rice flour");

 }

@Override

public String getCookPrep() {

 return "Soak pho for 1 hour, then boil for 1 minute in broth. Then garnish with cilantro and jalapeno.";

}

```
}
```

OUTPUT

Boil spaghetti for 8 - 12 minutes and add sauce, cheese, or oil and garlic.

Boil ramen for 5 minutes in broth, then add meat, mushrooms, egg, and vegetables.

Soak pho for 1 hour, then boil for 1 minute in broth. Then garnish with cilantro and jalapeno.

14.9 Child Classes in Method Parameters

When we call a method that contains parameters, the arguments we place in our method call must match the parameter type. Similar to the previous exercise, polymorphism gives us a little more flexibility with the arguments we can use.

If we use a superclass reference as a method parameter, we can call the method using subclass reference arguments!

For example, imagine the class `ScaryStory`, whose constructor takes in a reference to the `Monster` class:

```
class ScaryStory {
    Monster monster;
    String setting;

    public ScaryStory(Monster antagonist, String place) {
        monster = antagonist;
        setting = place;
    }

    public void tellStory(){
        System.out.println("Once upon a time, " + monster.name + " was at " + setting
+ " looking to scare some mortals.");
    }

    public static void main(String[] args) {
```

```

    Monster dracula;
    dracula = new Vampire("Dracula");
    ScaryStory countDracula = new ScaryStory(dracula, "Dracula Castle");
    countDracula.tellStory();
}
}

```

In the main() method, we used a reference of the class Vampire as our argument even though the constructor requested an object of class Monster. This is allowed because Vampire is a subclass of the Monster class.

14.9 Child Classes in Method Parameters Instructions

1.

Take a look at the code in the new file, **NoodleRestaurant.java**. Specifically, look at the order() method to see what its parameter's data type is.

Inside the main() method, call the order() method on customer1 with pho as an argument.

2.

Underneath the previous line of code, call the order() method on customer2 with ramen as an argument.

NoodleRestaurant.java

```

class NoodleRestaurant {
    String name;

    public NoodleRestaurant(String customerName){
        name = customerName;
    }
}

```



```
public void order(Noodle noodle) {  
    System.out.println(noodle.getCookPrep());  
    System.out.println("Order for " + name + " is ready.");  
}
```

```
public static void main(String[] args) {  
    Noodle ramen, pho;  
    ramen = new Ramen();  
    pho = new Pho();
```

```
    NoodleRestaurant customer1 = new NoodleRestaurant("Sagirah");  
    NoodleRestaurant customer2 = new NoodleRestaurant("Maritza");
```

```
    // Add your code here
```

```
}
```

```
}
```

Noodle.java

```
class Noodle {
```

```
    protected double lengthInCentimeters;
```

```
protected double widthInCentimeters;  
protected String shape;  
protected String ingredients;  
protected String texture = "brittle";
```

```
Noodle(double lenInCent, double wthInCent, String shp, String ingr) {
```

```
    this.lengthInCentimeters = lenInCent;  
    this.widthInCentimeters = wthInCent;  
    this.shape = shp;  
    this.ingredients = ingr;
```

```
}
```

```
public String getCookPrep() {
```

```
    return "Boil noodle for 7 minutes and add sauce.";
```

```
}
```

```
}
```

Pho.java

```
class Pho extends Noodle {
```

```
Pho() {
```

```
    super(30.0, 0.64, "flat", "rice flour");
```

```
}
```

```
@Override
```

```
public String getCookPrep() {
```

```
    return "Soak pho for 1 hour, then boil for 1 minute in broth. Then garnish with  
    cilantro and jalapeno.";
```

```
}
```

```
}
```

Ramen.java

```
class Ramen extends Noodle {
```

```
    Ramen() {
```

```
        super(30.0, 0.3, "flat", "wheat flour");
```

```
}
```

```
@Override
```

```

public String getCookPrep() {

    return "Boil ramen for 5 minutes in broth, then add meat, mushrooms, egg, and
vegetables.";

}

}

```

OUTPUT

Soak pho for 1 hour, then boil for 1 minute in broth. Then garnish with cilantro and jalapeno.

Order for Sagirah is ready.

Boil ramen for 5 minutes in broth, then add meat, mushrooms, egg, and vegetables.

Order for Maritza is ready.

14.10 Review of Inheritance and Polymorphism

Excellent work! You've learned quite a bundle about inheritance and polymorphism in Java:

- A Java class can inherit fields and methods from another class.
- Each Java class requires its own file, but only one class in a Java package needs a `main()` method.
- Child classes inherit the parent constructor by default, but it's possible to modify the constructor using `super()` or override it completely.
- You can use `protected` and `final` to control child class access to parent class members.
- Java's OOP principle of polymorphism means you can use a child class object like a member of its parent class, but also give it its own traits.

- You can override parent class methods in the child class, ideally using the `@Override` keyword.
- It's possible to use objects of different classes that share a parent class together in an array or `ArrayList`.

14. Quiz

1. Instantiate a **Stegosaurus** object as a **Dinosaur**.

```
public static void main(String[] args) {
```

```
    _____ steggy = new _____;
```

```
}
```

- `Stegosaurus()`
- `Dinosaur`
- `Stegosaurus`
- `Dinosaur()`

Click or drag and drop to fill in the blank

2. Imagine you want to build three Java classes in your package — one parent class and two child classes.

How many files will you need to create?

- a) One.
- b) Six.
- c) Two.
- d) Three.

3. Change `Dinosaur` so that only `Dinosaur`, its child classes, and the package it is in have access to the `weight` field.

```
public class Dinosaur {
```

```
    _____ double weight;
```

```

public double getWeight() {
    return this.weight;
}
}

```

- private
- protected
- final
- public

Click or drag and drop to fill in the blank

4. The parent class of Sauropoda requires that you pass a String called size into its constructor.

Modify the constructor of Sauropoda so that all Sauropoda objects will be instantiated with a size of "large".

```

Sauropoda() {
    _____;
}

```

- parent
-)
- .
- super
- String
- "large"
- (

Click or drag and drop to fill in the blank

5. Which is NOT a facet of polymorphism in Java?

- a) You can use a child class object like a member of its parent class.
- b) A Java class can inherit fields and methods from a another class.
- c) It's possible to use objects of different classes that share a parent class together in an array or ArrayList.
- d) A Java child class can override its parent's methods.

6. Set up the class so that Triceratops inherits from Dinosaur.

```
public class _____ {  
    // additional class members go here  
}
```

- Dinosaur
- Triceratops
- extends
- Mangosaurus
- inherits

Click or drag and drop to fill in the blank

Project-12

Language Families

Languages, just like people, often have families. You can even map language lineage a bit like a family tree.

Java can help us build a model to track inherited traits across families. In this case, we'll focus on something that often varies between language families: word order — where the subject, verb, and object would go in a sentence.

For example, in English, you would use subject-verb-object order:

subject	verb	Object
She	eats	Noodles

But in Japanese, you would use subject-object-verb order:

subject	object	verb
彼女は	麺を	食べる
(she)	(noodles)	(eats)

Your **Language Inheritance** package has three Java files:

- **Language.java**: for the Language parent class, which serves as the template for all languages.
- **Mayan.java**: for Mayan, a child class of Language modeled after the [Mayan language family](#).
- **SinoTibetan.java**: for SinoTibetan, a child class of Language modeled after the [Sino-Tibetan language family](#).

Build out a Java package with these classes to model real-world language families.

Tasks

A language by any other name

1.

In **Language.java**, create a Language class with a main() method and the following fields:

- name: a protected string.
- numSpeakers: a protected integer.
- regionsSpoken: a protected string.
- wordOrder: a protected string.

2.

Above the `main()` method, give `Language` a constructor that sets each field to the values passed in.

3.

Create a public method for `Language` called `getInfo()`. We'll use this method to display all of its information (using its field values).

The method should not return anything.

We want to set up the information like this:

[name] is spoken by [numSpeakers] people mainly in [regionsSpoken].

The language follows the word order: [wordOrder].

For example, if we call `spanish.getInfo();`, we'd want to see something like this:

Spanish is spoken by 555000000 people mainly in Spain, Latin America, and Equatorial Guinea.

The language follows the word order: subject-verb-object.

4.

Let's test out the code so far!

In `main()`, instantiate a new `Language` of your choice.

Then call `getInfo()` on the `Language` variable.

Run your code in the terminal to see if the information gets printed. If nothing displays, try compiling your code first.

Not just an ancient civilization

5.

Nice work! Now let's model a language family.

Tab over to **Mayan.java** and create an empty `Mayan` class that inherits from `Language`.

6.

Mayan languages share several traits in common including:

- regionsSpoken: "Central America"
- wordOrder: "verb-object-subject"

Tweak the Mayan constructor so that it isn't necessary to pass in these fields when instantiating a new Mayan language object.

Bear in mind that each language will still require its own name and numSpeakers.

7.

Mayan languages have an interesting grammatical feature: [ergativity](#).

Override the getInfo() method for Mayan so that if we called getInfo() on a Mayan language like Ki'che', we'd get the following info:

Ki'che' is spoken by 2330000 people mainly in Central America.

The language follows the word order: verb-object-subject

Fun fact: Ki'che' is an ergative language.

8.

Time to test out the tweaks you made to the Mayan class...

Tab back over to **Language.java**.

In main(), instantiate a new Mayan language of your choice (you can find one [here](#)).

Then call getInfo() on the language variable.

Run your code in the terminal to see if the information gets printed. If nothing displays, try compiling your code first.

Heading east...

9.

The Sino-Tibetan family has the second highest number of native speakers of any language family.

Tab over to **SinoTibetan.java** and build out an empty SinoTibetan class that inherits from Language.

10.

Like the Mayan language family, Sino-Tibetan languages share several traits in common. In this case:

- regionsSpoken: "Asia"
- wordOrder: "subject-object-verb"

Build a constructor for SinoTibetan that so that it isn't necessary to pass in these fields when instantiating a new SinoTibetan language object.

Remember — each language will still require its own name and numSpeakers.

11.

So that word order thing? There is actually a [split in the Sino-Tibetan family on this](#).

It turns out that at some point (a long time ago) the Chinese languages (among a few others) switched the object and verb order. So they now follow a subject-verb-object pattern. Hmm... How can we handle this?

One (imperfect) tactic is to check if the language's name field contains "Chinese". There's a [Java string method](#) to check if a string contains a substring.

In the constructor, below where you called super(), change the wordOrder to "subject-verb-object" if this.name contains "Chinese".

Wrapping up

12.

Test out the SinoTibetan class by instantiating two new Sino-Tibetan language objects of your choosing:

- One [Chinese](#) (e.g., "Mandarin Chinese")

- One [non-Chinese](#) (e.g., Burmese)

Then call `getInfo()` on each language variable.

Run your code in the terminal to see if the information gets printed. If nothing displays, try compiling your code first.

13.

Congrats on all your work with Java Inheritance and Polymorphism! You've built out some useful classes for a linguist out there.

BONUS

There are many more language families you could create and there is a lot more you can do here. Check the hint for some ideas to make your Language Inheritance project even better.

Concept Review

Inheritance in Java

Inheritance is an important feature of object-oriented programming in Java. It allows for one class (*child class*) to inherit the fields and methods of another class (*parent class*). For instance, we might want a child class `Dog` to inherit traits from a more general parent class `Animal`.

When defining a child class in Java, we use the keyword `extends` to inherit from a parent class.

// Parent Class

```
class Animal {
```

```
    // Animal class members
```

```
}
```

// Child Class

```
class Dog extends Animal {
```

```
// Dog inherits traits from Animal
```

```
// additional Dog class members
```

```
}
```

Main() method in Java

In simple Java programs, you may work with just one class and one file. However, as your programs become more complex you will work with multiple classes, each of which requires its own file. Only one of these files in the Java package requires a main() method, and this is the file that will be run in the package.

For example, say we have two files in our Java package for two different classes:

- Shape, the parent class.
- Square, the child class.

If the Java file containing our Shape class is the only one with a main() method, this is the file that will be run for our Java package.

```
// Shape.java file
```

```
class Shape {  
    public static void main(String[] args) {  
        Square sq = new Square();  
    }  
}
```

```
// Square.java file
```

```
class Square extends Shape {  
  
}
```

super() in Java

In Java, a child class inherits its parent's fields and methods, meaning it also inherits the parent's constructor. Sometimes we may want to modify the constructor, in which case we can use the `super()` method, which acts like the parent constructor inside the child class constructor.

Alternatively, we can also completely override a parent class constructor by writing a new constructor for the child class.

// Parent class

```
class Animal {  
    String sound;  
    Animal(String snd) {  
        this.sound = snd;  
    }  
}
```

// Child class

```
class Dog extends Animal {  
    // super() method can act like the parent constructor inside the child class  
    // constructor.  
    Dog() {  
        super("woof");  
    }  
    // alternatively, we can override the constructor completely by defining a new  
    // constructor.  
    Dog() {  
        this.sound = "woof";  
    }  
}
```

```
}
```

Protected and Final keywords in Java

When creating classes in Java, sometimes we may want to control child class access to parent class members. We can use the protected and final keywords to do just that.

protected keeps a parent class member accessible to its child classes, to files within its own package, and by subclasses of this class in another package.

Adding final before a parent class method's access modifier makes it so that any child classes cannot modify that method - it is immutable.

```
class Student {  
    protected double gpa;  
    // any child class of Student can access gpa  
  
    final protected boolean isStudent() {  
        return true;  
    }  
    // any child class of Student cannot modify isStudent()  
}
```

Polymorphism in Java

Java incorporates the object-oriented programming principle of *polymorphism*.

Polymorphism allows a child class to share the information and behavior of its parent class while also incorporating its own functionality. This allows for the benefits of simplified syntax and reduced cognitive overload for developers.

```
// Parent class  
class Animal {  
    public void greeting() {
```

```

        System.out.println("The animal greets you.");
    }
}

// Child class
class Cat extends Animal {
    public void greeting() {
        System.out.println("The cat meows.");
    }
}

class MainClass {
    public static void main(String[] args) {
        Animal animal1 = new Animal(); // Animal object
        Animal cat1 = new Cat(); // Cat object
        animal1.greeting(); // prints "The animal greets you."
        cat1.greeting(); // prints "The cat meows."
    }
}

```

Method Overriding in Java

In Java, we can easily *override* parent class methods in a child class. Overriding a method is useful when we want our child class method to have the same name as a parent class method but behave a bit differently.

In order to override a parent class method in a child class, we need to make sure that the child class method has the following in common with its parent class method:

- Method name
- Return type
- Number and type of parameters

Additionally, we should include the `@Override` keyword above our child class method to indicate to the compiler that we want to override a method in the parent class.

```
// Parent class
```

```
class Animal {  
    public void eating() {  
        System.out.println("The animal is eating.");  
    }  
}
```

```
// Child class
```

```
class Dog extends Animal {  
    // Dog's eating method overrides Animal's eating method  
    @Override  
    public void eating() {  
        System.out.println("The dog is eating.");  
    }  
}
```

Child Classes in Arrays and ArrayLists

In Java, polymorphism allows us to put instances of different classes that share a parent class together in an array or ArrayList.

For example, if we have an Animal parent class with child classes Cat, Dog, and Pig we can set up an array with instances of each animal and then iterate through the list of animals to perform the same action on each.

```
// Animal parent class with child classes Cat, Dog, and Pig.
```

```
Animal cat1, dog1, pig1;
```

```
cat1 = new Cat();
```

```
dog1 = new Dog();
```

```
pig1 = new Pig();
```

```
// Set up an array with instances of each animal
```

```
Animal[] animals = {cat1, dog1, pig1};
```

```
// Iterate through the list of animals and perform the same action with each
```

```
for (Animal animal : animals) {
```

```
    animal.sound();
```

```
}
```

15. DEBUGGING

15.1 Introduction to Bugs

“First actual case of bug being found.”

The story goes that on September 9th, 1947, computer scientist [Grace Hopper](#) found a moth in the Harvard Mark II computer’s log book and reported the world’s first literal computer bug. However, the term “bug”, in the sense of technical error, dates back at least to 1878 and with Thomas Edison.

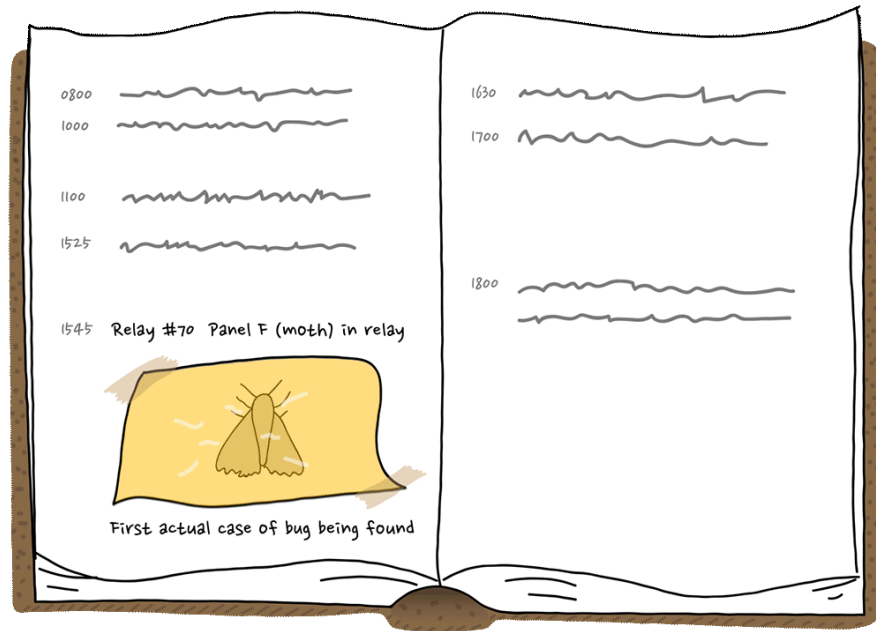
On your programming journey, you are destined to encounter innumerable red errors. Some even say that debugging is over 75% of the development time. But what makes a programmer successful isn’t avoiding errors; it’s knowing how to find the solution.

In Java, there are many different ways of classifying errors, but they can be boiled down to three categories:

- **Syntax errors:** Errors found by the compiler.
- **Run-time errors:** Errors that occur when the program is running.
- **Logic errors:** Errors found by the programmer looking for the causes of erroneous results.

Generally speaking, the errors become more difficult to find and fix as you move down the above list.

In this lesson, we will be looking at different errors and different error messages, and we’ll teach you how to think about errors in your code a little differently.



15.2 Syntax Errors

When we are writing Java programs, the compiler is our first line of defense against errors. It can catch syntax errors.

Syntax errors represent grammar errors in the use of the programming language. They are the easiest to find and correct. The compiler will tell you where it got into trouble, and its best guess as to what you did wrong.

Some common syntax errors are:

- Misspelled variable and method names
- Omitting semicolons ;
- Omitting closing parenthesis), square bracket], or curly brace }

Here's an example of a syntax error message:

Debug.java:5: error: ';' expected

```
int year = 2019
```

^

1 error

Usually the error is on the exact line indicated by the compiler, or the line just before it; however, if the problem is incorrectly nested braces, the actual error may be at the beginning of the nested block.

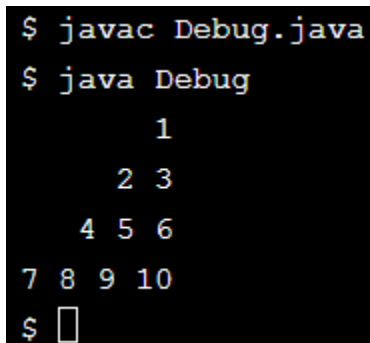
15.2 Syntax Errors Instructions

1.

Try compiling the code and find the syntax error in **Debug.java**.

```
public class Debug {  
  
    public static void main(String[] args) {  
        System.out.println("  1");  
        System.out.println("  2 3");  
        System.out.println("  4 5 6");  
        System.out.println("7 8 9 10")  
    }  
}
```

OUTPUT



```
$ javac Debug.java  
$ java Debug  
    1  
    2 3  
    4 5 6  
7 8 9 10  
$ □
```

15.3 Run-time Errors

If our program has no compile-time errors, it'll run. This is where the fun really starts.

Errors which happen during program execution (run-time) after successful compilation are called run-time errors. *Run-time errors* occur when a program with no compile-time errors asks the computer to do something that the computer is unable to reliably do.

Some common run-time errors:

- Division by zero also known as division error
- Trying to open a file that doesn't exist

There is no way for the compiler to know about these kinds of errors when the program is compiled.

Here's an example of a run-time error message:

```
Exception in thread "main"  
java.lang.ArithmeticException: / by zero  
    at Debug.main(Debug.java:8)
```

15.3 Run-time Errors Instructions

1.

There's a run-time error in **Debug.java**.

This program is supposed to find the ratio of a table's dimensions, which is supposed to be 20 x 40.

```
public class Debug {
```

```
    public static void main(String[] args) {
```

```
        int width = 0;
```

```
        int length = 40;
```

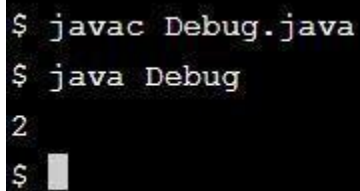
```
int ratio = length / width;
```

```
System.out.println(ratio);
```

```
}
```

```
}
```

OUTPUT

A terminal window with a black background and white text. It shows the following commands and output:

```
$ javac Debug.java
$ java Debug
2
$
```

15.4 Exceptions

In the last exercise when we were dealing with run-time errors, you might've noticed a new word in the error message: "Exception".

Java uses *exceptions* to handle errors and other exceptional events. Exceptions are the conditions that occur at runtime and may cause the termination of the program.

When an exception occurs, Java displays a message that includes the name of the exception, the line of the program where the exception occurred, and a *stack trace*. The stack trace includes:

- The method that was running
- The method that invoked it
- The method that invoked that one
- and so on...

Make sure to examine it.

Some common exceptions that you will see in the wild:

- `ArithmeticException`: Something went wrong during an arithmetic operation; for example, division by zero.
- `NullPointerException`: You tried to access an instance variable or invoke a method on an object that is currently null.
- `ArrayIndexOutOfBoundsException`: The index you are using is either negative or greater than the last index of the array (i.e., `array.length-1`).
- `FileNotFoundException`: Java didn't find the file it was looking for.

15.4 Exceptions Instructions

Run the code to revisit the `ArithmeticException` you saw in the previous exercise.

Then move on to the next exercise to see how you can use exceptions to your advantage.

```
public class Debug {  
  
    public static void main(String[] args) {  
  
        int width = 0;  
        int length = 40;  
  
        int ratio = length / width;  
  
        System.out.println(ratio);  
    }  
}
```


OUTPUT

```
Exception in thread "main"  
java.lang.ArithmeticException: / by zero  
    at Debug.main(Debug.java:8)
```

15.5 Exception Handling

Exception handling is an essential feature of Java programming that allows us to use run-time error exceptions to make our debugging process a little easier.

One way to handle exceptions is using the try/catch:

- The try statement allows you to define a block of code to be tested for errors while it is being executed.
- The catch statement allows you to define a block of code to be executed if an error occurs in the try block.

The try and catch keywords come in pairs, though you can also catch several types of exceptions in a single block:

```
try {
```

```
    // Block of code to try
```

```
} catch (NullPointerException e) {
```

```
    // Print the error message like this:
```

```
    System.err.println("NullPointerException: " + e.getMessage());
```

```
    // Or handle the error another way here
```

```
}
```

Notice how we used `System.err.println()` here instead of `System.out.println()`. `System.err.println()` will print to the standard error and the text will be in red.

You can also chain exceptions together:

```
try {  
  
    // Block of code to try  
  
} catch (NullPointerException e) {  
  
    // Code to handle a NullPointerException  
  
} catch (ArithmeticException e) {  
  
    // Code to handle an ArithmeticException  
  
}
```

You can learn more about exceptions and handling them [here](#).

15.5 Exception Handling Instructions

1.

Put the code that you think will cause an `ArithmeticException` into a try block.

Create a catch block that catches the `ArithmeticException` and prints the error message to the terminal.

```
public class Debug {  
  
    public static void main(String[] args) {  
        int width = 0;  
        int length = 40;  
        int ratio = length / width;  
    }  
}
```

OUTPUT

ArithmeticException: / by zero

15.6 Logic Errors

Once we have removed the syntax errors and run-time errors, the program runs successfully. But sometimes, the program still doesn't do what we want it to do or no output is produced. Hmmm...

These types of errors, which provide incorrect output, but appear to be error-free, are called *logic errors*. Logic errors occur when there is a design flaw in your program. These are some of the most common errors that happen to beginners and also usually the most difficult to find and eliminate.

Because logical errors solely depend on the logical thinking of the programmer, your job now is to figure out why the program didn't do what you wanted it to do.

Some common logic errors:

- Program logic is flawed
- Some "silly" mistake in an if statement or a for/while loop

Note: Logic errors don't have error messages. Sometimes, programmers use a process called [test-driven development \(TDD\)](#), a way to give logic errors error messages and save yourself a lot of headaches!

15.6 Logic Errors Instructions

1.

Run **Debug.java**. There should be no error messages.

But the program is supposed to output steps from #1 - #10. The program is somehow starting from 0...

Find the logic error.

```
public class Debug {
```

```
    public static void main(String[] args) {
```

```
int steps = 10;

for (int i = 0; i <= steps; i++) {

    System.out.println("Step #" + i);

}

}

}
```

OUTPUT

Step #1

Step #2

Step #3

Step #4

Step #5

Step #6

Step #7

Step #8

Step #9

Step #10

15.7 Debugging Techniques

If you have examined the code thoroughly, and you are sure the compiler is compiling the right source file, it is time for desperate measures:

1. Divide and conquer: Comment out or temporarily delete half the code to isolate an issue.

- If the program compiles now, you know the error is in the code you deleted. Bring back about half of what you removed and repeat.
- If the program still doesn't compile, the error must be in the code that remains. Delete about half of the remaining code and repeat.

Tip: In most code editors, one can highlight a block of code and use the keyboard shortcut command + / to comment it out.

2. Print statements for the rescue: Use `System.out.println()` to check variable/return values at various points throughout the program.

A lot of the time with logic errors, there was a flawed piece of logic, a miscalculation, a missing step, etc. By printing out the values at different stages of the execution flow, you can then hopefully pinpoint where you made a mistake.

15.7 Debugging Techniques Instructions

1.

Here's a **AreaCalculator.java** program that solves for the area of different shapes:

- Triangle:

$$area = \frac{1}{2} \cdot base \cdot height$$

- Rectangle:

$$area = length \cdot width$$

- Circle:

$$area = \pi r^2$$

There is a logical error: it is not calculating the areas right for one of the shapes.
Find the bug!

```
import java.util.*;
```

```
public class AreaCalculator {
```

```
    public static void main(String[] args) {
```

```
        Scanner keyboard = new Scanner(System.in);
```

```
        System.out.println("Shape Area Calculator");
```

```
        while(true) {
```

```
            System.out.println();
```

```
            System.out.println("-----");
```

```
            System.out.println();
```

```
            System.out.println("1) Triangle");
```

```
            System.out.println("2) Rectangle");
```

```
            System.out.println("3) Circle");
```

```
            System.out.println("4) Quit");
```

```
        System.out.println();
```

```
            System.out.print("Which shape: ");
```

```

int shape = keyboard.nextInt();

    System.out.println();

    if (shape == 1) {
System.out.print("Base: ");

        int base = keyboard.nextInt();

        System.out.print("Height: ");

        int height = keyboard.nextInt();

        area_triangle(base, height);

    } else if (shape == 2) {
System.out.print("Length: ");

        int length = keyboard.nextInt();

        System.out.print("Width: ");

        int width = keyboard.nextInt();

        area_rectangle(length, width);

    } else if (shape == 3) {
System.out.print("Radius: ");

        int radius = keyboard.nextInt();

        area_circle(radius);

    } else if (shape == 4) {

        quit();

        break;

    }

}

```

```
}
```

```
public static double area_triangle(int base, int height) {  
    System.out.println();  
    int A = (base * height) * 2;  
    System.out.println("The area is " + A + ".");  
    return A;  
}
```

```
public static int area_rectangle(int length, int width){  
    System.out.println();  
    int A = length * width;  
    System.out.println("The area is " + A + ".");  
    return A;  
}
```

```
public static double area_circle(int radius) {  
    System.out.println();  
    double A = Math.PI * radius * radius;  
    System.out.println("The area is " + A + ".");  
    return A;  
}
```

```
public static String quit() {
```



```
        System.out.println("Good Bye");  
        return null;  
    }  
}
```

OUTPUT

```
$ javac AreaCalculator.java  
$ java AreaCalculator  
Shape Area Calculator  
  
-----  
  
1) Triangle  
2) Rectangle  
3) Circle  
4) Quit  
  
Which shape: 3  
  
Radius: 54  
  
The area is 9160.884177867836.
```

15.8 Review

Finding bugs is a huge part of a programmer's life. Don't be intimidated by them... embrace them. Errors in your code mean you're trying to do something cool!

In this lesson, we have learned about the three types of Java errors:

- **Syntax errors:** Errors found by the compiler.
- **Run-time errors:** Errors found by checks in a running program.

- **Logic errors:** Errors found by the programmer looking for the causes of erroneous results.

Remember, [Google](#) and [Stack Overflow](#) are a programmer's best friends. For some more motivation, check out this blog post: [Thinking About Errors in Your Code Differently](#).

Sometimes once you've tracked down a bug, you might still be confused on how to fix it! Whenever you want to know more about how Java works and what it can do, the best place to go is documentation. You can find the Java documentation at [Oracle](#).

15. Quiz

1. Fill in the blank so that the code is handling the **FileNotFoundException** using **try/catch**

```
_____ {  
  
    // Block of code to try  
  
}  
_____(_____ e) {  
  
    // Block of code to handle exceptions  
  
}  


- try
- FileNotFoundException
- ArithmeticException
- catch

```

Click or drag and drop to fill in the blank

2. **True/False:** Errors which happen during program execution after successful compilation are called run-time errors.

- a) True.
- b) False.

3. **Find the bug:** What's the error in the code example?

```
public class Debug {  
  
    public static void main(String[] args) {  
  
        System.out.println("hello")  
        System.out.println("hi");  
  
        System.out.println("good morning");  
        System.out.println("guten tag");  
  
    }  
  
}
```

- a) The class has the wrong name.
 - b) **main()** is defined incorrectly.
 - c) There's a semicolon missing.
4. What type of error is caught by the compiler?
- a) Syntax errors.
 - b) Logic errors.
 - c) Run-time errors.

Project-13

Bug Detective

Your friend built a program to mimic a fortune cookie by randomly printing a random fortune on each run. A [fortune cookie](#) is a crisp cookie with a piece of paper inside, on which is a “fortune.”

However, your friend didn’t use incremental development and debug along the way, so she ended up with a super buggy program. Can you be a bug detective and help her debug her program **Fortune.java**?

The output should look something like:

Flattery will go far tonight.

Or:

Don't behave with cold manners.

Final tip: If the compiler reports 100 error messages, that doesn’t mean there are 100 errors in the program. Sometimes when a compiler encounters an error, it often gets thrown off track. Only the first error message is truly reliable. Always scroll to the top and try to debug the very first error first.

Tasks

Debugging:

1.

First, run the program to see the list of error messages.

2.

Start from debugging from the first error (at the very top) until the program is bug-free.

Make sure to use the line number in the error message to locate the bugs.

```
import java.lang.Math;
```

```
class Fortune {
```

```
public int generateRandom() {
```

```
    return (int)(Math.random() * 10);
```

```
}
```

```
public static void Main(String[] args) {
```

```
    Fortune fortuneCookie = new Fortune();
```

```
    int fortune = fortuneCookie.generateRandom();
```

```
    if (fortune == 0) {
```

```
        System.out.println("Flattery will go far tonight.");
```

```
    } else if (fortune == 1) {
```

```
        System.out.println("Don't behave with cold manners.")
```

```
    } else if (fortune == 2) {
```

```
        System.out.println("May you someday be carbon neutral");
```

```
    } else if (fortune == 3) {
```

```
System.out.println("You have rice in your teeth.");
```

```
} else if (fortune == 4) {
```

```
    System.out.println("A conclusion is simply the place where you got tired of  
thinking.");
```

```
} else if (fortune == 5) {
```

```
    System.out.println("No snowflake feels responsible in an avalanche.");
```

```
} else if (fortune == 6) {
```

```
    System.out.println("He who laughs last is laughing at you.");
```

```
} else if (fortune == 7) {
```

```
    System.out.println("If you look back, you'll soon be going that way.");
```

```
} else if (fortune == 8) {
```

```
    System.out.println("You will live long enough to open many fortune cookies.");
```

```
} else if (fortune == 9) {
```

```
System.out.println("The fortune you seek is in another cookie.");
```

```
}
```

```
}
```

```
}
```

OUTPUT

You will live long enough to open many fortune cookies.

16. 2D ARRAYS

16.1 Introduction to 2D Arrays

As we have learned previously, an array is a group of data consisting of the same type. This means that we can have an array of primitive data types (such as integers):

```
[1, 2, 3, 4, 5]
```

We can even have an array of Objects. For example, the following example shows an array of String Objects:

```
["hello", "world", "how", "are", "you"]
```

In Java, arrays are considered Objects; therefore, we can also have an array of arrays:

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

These are called 2D arrays since we can logically view them as a two-dimensional matrix of values containing both rows and columns.


```
[ 1 , 2 , 3 ]  
[ 4 , 5 , 6 ]  
[ 7 , 8 , 9 ]
```

Additionally, we can have 2D arrays which are not rectangular in shape. These are called jagged arrays:

```
[['a', 'b', 'c', 'd'], ['e', 'f'], ['g', 'h', 'i', 'j'], ['k']]
```

```
[ 'a' , 'b' , 'c' , 'd' ]  
[ 'e' , 'f' ]  
[ 'g' , 'h' , 'i' , 'j' ]  
[ 'k' ]
```

We won't be covering jagged arrays in this lesson, but be aware that 2D arrays don't always have to have the same number of subarrays in each array. This would cause the shape of the 2D array to not be rectangular.

Why use 2D arrays?

- It is useful to use 2D arrays for situations where you need to store and organize data by rows and columns. For example, exporting data to be used in a spreadsheet.
- You can condense multiple arrays down to a single variable using 2D arrays. For example, if you have 10 students who each have 10 different quiz grades, you can represent the overall class quiz grades as a 10x10 2D array by having each row represent a student and each column represent one of the quizzes they have taken.
- 2D arrays can be used to map out data. For example, if you want to create a game of tic-tac-toe, you can represent the game state by using a 3x3 2D array.

There are many other ways to use 2D arrays depending on the application. The only downside is that once initialized, no new rows or columns can be added or removed without copying the data to a newly initialized 2D array. This is because the length of arrays in Java are immutable (unable to be changed after creation).

16.1 Introduction to 2D Arrays Instructions

Take a look at some of the ways we can represent 2D arrays visually.

Click **next** when you are ready to continue to the next exercise!

2D Arrays

```
[ ['a','b','c'], ['d','e','f'], ['g','h','i'] ]
```

<pre>['lets', 'learn', 'about' 'two', 'dimensional', 'arrays']</pre>	<pre>[0,1,1,1,1,0,0,1]</pre>
	<pre>[0,1,1,1,1,0,0,1]</pre>
	<pre>[0,1,1,1,1,0,0,1]</pre>

16.2 Declaration, Initialization, and Assignment

When declaring 2D arrays, the format is similar to normal, one-dimensional arrays, except that you include an extra set of brackets after the data type. In this example, `int` represents the data type, the first set of brackets `[]` represent an array, and the second set of brackets `[]` represent that we are declaring an array of arrays.

```
int[][] intTwoDArray;
```

You can think of this as creating an array (`[]`) of `int` arrays (`int[]`). So we end up with `int[][]`.

Now that we've declared a 2D array, let's look at how to initialize it with starting values. When initializing arrays, we define their size. Initializing a 2D array is different because, instead of only including the number of elements in the array, you also indicate how many elements are going to be in the sub-arrays. This can also be thought of as the number of rows and columns in the 2D matrix.

```
int[][] intArray1;  
intArray1 = new int[row][column];
```

Here is an example of initializing an empty 2D array with 3 rows and 5 columns.

```
int[][] intArray2;  
intArray2 = new int[3][5];
```

This results in a matrix which looks like this:

[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]

If you already know what values are going to be in the 2D array, you can initialize it and write all of the values into it at once. We can accomplish this through initializer lists

- In Java, initializer lists are a way of initializing arrays and assigning values to them at the same time
- We can use this for 2D arrays as well by creating an initializer list of initializer lists

An example of an initializer list for a regular array would be:

```
char[] charArray = {'a', 'b', 'c', 'd'};
```

Similar to how a regular initializer list defines the size and values of the array, nested initializer lists will define the number of rows, columns, and the values for a 2D array.

There are three situations in which we can use initializer lists for 2D arrays:

1. In the case where the variable has not yet been declared, we can provide an abbreviated form since Java will infer the data type of the values in the initializer lists:

```
double[][] doubleValues = {{1.5, 2.6, 3.7}, {7.5, 6.4, 5.3}, {9.8, 8.7, 7.6}, {3.6, 5.7, 7.8}};
```

2. If the variable has already been declared, you can initialize it by creating a new 2D array object with the initializer list values:

```
String[][] stringValues;  
stringValues = new String[][] {{"working", "with"}, {"2D", "arrays"}, {"is", "fun"}};
```

3. The previous method also applies to assigning a new 2D array to an existing 2D array stored in a variable.

In the next exercise, we'll look at how to assign values of individual elements.

16.2 Declaration, Initialization, and Assignment Instructions

1.

Declare a 2D array of float values called floatTwoD.

2.

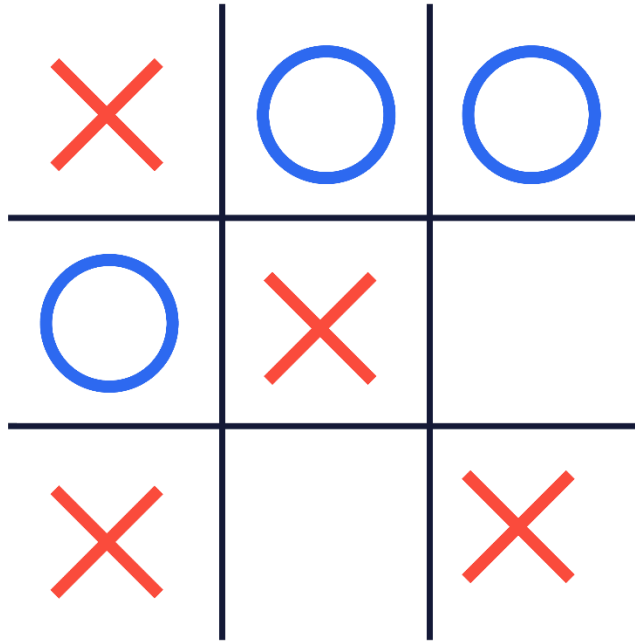
Initialize the 2D array from the last step to an empty 2D array consisting of 4 arrays with 10 elements each. Do this on a new line under the declaration.

3.

On a single line, declare and initialize an empty 2D array of ints consisting of 15 rows and 8 columns called dataChart.

4.

Create a 2D char array called ticTacToe representing this tic-tac-toe board. Use the characters 'X', 'O', and ' '.



(Note that you should use a capitalized O, not the number 0.)

5.

When no one is looking, you want to modify the game so 'O' wins. Replace the game board so that all X's are O's and all O's are X's. Do this in one line with initializer lists. Do not declare ticTacToe again.

```
public class Exercise2 {  
    public static void main(String[] args) {  
  
        // Declare a 2d array of float values called floatTwoD
```

```
// Initialize the 2d array from the last step to an empty 2d array  
consisting of 4 arrays with 10 elements each
```

```
// Declare and initialize an empty 2d array of integers consisting of 15  
rows and 8 columns called dataChart
```

```
// Create a 2D char array called ticTacToe representing the provided  
tic-tac-toe board. Use the characters 'X', 'O', and ' '.
```

```
// When no one is looking, you want to modify the game to where  
you, 'O', wins the game. Replace the game board so that all X's are O's and all O's  
are X's. Do this in one line with initializer lists.
```

```
}  
}
```

16.3 Accessing Elements in a 2D Array

Let's first review how to access elements in regular arrays.

For a normal array, all we need to provide is an index (starting at 0) which represents the position of the element we want to access. Let's look at an example!

Given an array of five Strings:

```
String[] words = {"cat", "dog", "apple", "bear", "eagle"};
```

We can access the first element using index 0, the last element using the length of the array minus one (in this case, 4), and any of the elements in between. We provide the index of the element we want to access inside a set of brackets. Let's see those examples in code:

```
// Store the first element from the String array  
String firstWord = words[0];
```

```
// Store the last element of the String array  
String lastWord = words[words.length-1];
```

```
// Store an element from a different position in the array  
String middleWord = words[2];
```

Now for 2D arrays, the syntax is slightly different. This is because instead of only providing a single index, we provide two indices. Take a look at this example:

```
// Given a 2D array of integer data  
int[][] data = {{2,4,6}, {8,10,12}, {14,16,18}};
```

```
// Access and store a desired element  
int stored = data[0][2];
```

There are two ways of thinking when accessing a specific element in a 2D array.

- The first way of thinking is that the first value represents a row and the second value represents a column in the matrix
- The second way of thinking is that the first value represents which subarray to access from the main array and the second value represents which element of the subarray is accessed

The above example of the 2D array called data can be visualized like so. The indices are labeled outside of the matrix:

Index	0	1	2
0	[2 , 4 , 6]		
1	[8 , 10 , 12]		
2	[14 , 16 , 18]		

Using this knowledge, we now know that the result of `int stored = data[0][2];` would store the integer 6. This is because the value 6 is located on the first row (index 0) and the third column (index 2). Here is a template which can be used for accessing elements in 2D arrays:

```
datatype variableName = existing2DArray[row][column];
```

Here is another way to visualize the indexing system for our example integer array seen above. We can see what row and column values are used to access the element at each position.

`[data[0][0],data[0][1],data[0][2]]`

`[data[1][0],data[1][1],data[1][2]]`

`[data[2][0],data[2][1],data[2][2]]`

When accessing these elements, if either the row or column value is out of bounds, then an `ArrayIndexOutOfBoundsException` will be thrown by the application.

16.3 Accessing Elements in a 2D Array Instructions

1.

Access the integer at the first row and fourth column of `intMatrix` and store it in a variable called `retrievedInt`.

If you'd like to check your work, print `retrievedInt` after creating it.

2.

Print the center value of `intMatrix` multiplied by 3 to the console. Make sure to access the correct element!

```
public class Exercise3 {  
  
    public static void main(String[] args) {
```

```

        // Using the provided 2D array

int[][] intMatrix = {

        {1, 1, 1, 1, 1},

        {2, 4, 6, 8, 0},

        {9, 8, 7, 6, 5}

};

        // Access the integer at the first row and fourth column of intMatrix
and store it in a variable called retrievedInt

        // Print 3 times the center value of intMatrix to the console. Make
sure to access the correct element!

}

```

}

OUTPUT

18

16.4 Modifying Elements in a 2D Array

Now let's review how to modify elements in a normal array.

For a one dimensional array, you provide the index of the element which you want to modify within a set of brackets next to the variable name and set it equal to an acceptable value:

```
storedArray[5] = 10;
```

For 2D arrays, the format is similar, but we will provide the outer array index in the first set of brackets and the subarray index in the second set of brackets. We can also think of it as providing the row in the first set of brackets and the column index in the second set of brackets if we were to visualize the 2D array as a rectangular matrix:

```
twoDArray[1][3] = 150;
```

To assign a new value to a certain element, make sure that the new value you are using is either of the same type or is castable to the type already in the 2D array.

Let's say we wanted to replace four values from a new 2D array called `intTwoD`. Look at this example code to see how to pick individual elements and assign new values to them.

```
int[][] intTwoD = new int[4][3];
```

```
intTwoD[3][2] = 16;
```

```
intTwoD[0][0] = 4;
```

```
intTwoD[2][1] = 12;
```

```
intTwoD[1][1] = 8;
```

Here is a before and after image showing when the 2D array was first initialized compared to when the four elements were accessed and modified.

Before	After
[0 , 0 , 0]	[4 , 0 , 0]
[0 , 0 , 0]	[0 , 8 , 0]
[0 , 0 , 0]	[0 , 12 , 0]
[0 , 0 , 0]	[0 , 0 , 16]

16.4 Modifying Elements in a 2D Array Instructions

1.

Replace the number 4 in `intMatrix` with the number 0.

Again, feel free to print the matrix to confirm you changed the correct number. To print a 2D array, use

```
System.out.println(Arrays.deepToString(intMatrix));
```

2.

Declare and initialize a new empty 2x2 int 2D array called subMatrix.

3.

Using 4 lines of code, multiply each of the elements in the 2x2 top left corner of intMatrix by 5 and store the results in the subMatrix you created. Notice how we have to write a similar line of code 4 times. Afterwards, uncomment the provided print statement to see the result. In the next exercise, we'll start looking into how to use loops to make this process quicker.

```
import java.util.Arrays;
```

```
public class Modifying {
```

```
    public static void main(String[] args) {
```

```
        // Using the provided 2D array
```

```
        int[][] intMatrix = {
```

```
            {1, 1, 1, 1, 1},
```

```
            {2, 4, 6, 8, 0},
```

```
            {9, 8, 7, 6, 5}
```

```
        };
```

```
        // Replace the number 4 in the 2D array with the number 0
```

```
// Declare and initialize a new empty 2x2 integer 2D array called
subMatrix

// Using 4 lines of code, multiply each of the elements in the 2x2 top
left corner of intMatrix by 5 and store the results in the subMatrix you created.
Afterwards, uncomment the provided print statement below.

//System.out.println(Arrays.deepToString(subMatrix));

}

}
```

OUTPUT

```
[[5, 5], [10, 0]]
```

16.5 Review of Nested Loops

We're about to look at how we can use loops to make our lives easier when working with 2D arrays. But before we do that, let's take a moment to refresh ourselves on how nested loops work.

Nested loops consist of two or more loops placed within each other. We will be looking at one loop nested within another for 2D traversal.

The way it works is that, for every iteration of the outer loop, the inner loop finishes all of its iterations.

Here is an example using **for** loops:

```
for(int outer = 0; outer < 3; outer++){
    System.out.println("The outer index is: " + outer);
    for(int inner = 0; inner < 4; inner++){
        System.out.println("\tThe inner index is: " + inner);
    }
}
```

The output of the above nested loops looks like so:

```
The outer index is: 0
    The inner index is: 0
    The inner index is: 1
```

The inner index is: 2
The inner index is: 3
The outer index is: 1
The inner index is: 0
The inner index is: 1
The inner index is: 2
The inner index is: 3
The outer index is: 2
The inner index is: 0
The inner index is: 1
The inner index is: 2
The inner index is: 3

From this example we can see how every time the outer loop iterates one time, the inner loop iterates fully.

This is an important concept for 2D array traversal, because for every row in a two dimensional matrix, we want to iterate through every column. We will look more at this in the next exercise.

Nested loops can consist of any type of loop and with any combination of loops. Let's take a look at a few more interesting examples.

Here is an example of nested while loops:

```
int outerCounter = 0;
int innerCounter = 0;
while(outerCounter<5){
    outerCounter++;
    innerCounter = 0;
    while(innerCounter<7){
        innerCounter++;
    }
}
```

We can even have some interesting combinations. Here is an enhanced **for** loop inside of a while loop:

```

int outerCounter = 0;
int[] innerArray = {1,2,3,4,5};

while(outerCounter<7){
    System.out.println();
    for(int number : innerArray){
        System.out.print(number * outerCounter + " ");
    }
    outerCounter++;
}

```

The output of the above example creates a multiplication table:

```

0 0 0 0 0
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
6 12 18 24 30

```

This is an interesting example, because for every iteration of the while loop, we iterate through every element of an array using an enhanced **for** loop. This is similar to the iteration pattern we use for 2D array traversal. We will be going over that in the next exercise.

Let's practice using nested loops!

You are in charge of controlling the amount of people who reserve seats for the world famous programming contest. You have two long arrays of integers which represent the contestant's IDs for two days of the competition. The index of the array represents their seat number. You need to use nested **for** loops to find if a contestant tried to register for both days. Print out the ID of the contestants who tried to register twice as well as their seat numbers for both days.

16.5 Review of Nested Loops Instructions

- 1.

Fix the outer loop header to iterate through the first array of seats.

2.

Fix the inner loop header to iterate through the second array of seats.

3.

Replace `1==2` with conditional logic to check if an element in the first array matches an element in the second array.

```
public class NestedLoops {  
    public static void main(String[] args) {  
        int[] seatsDayOne = {850007, 841141, 150017, 622393, 178505,  
952093, 492450, 790218, 515994, 926666, 476090, 709827, 908660, 718422,  
641067, 624652, 429205, 394328, 802772, 468793, 901979, 504963, 733939,  
706557, 724430, 663772, 577480, 886333, 323197, 283056, 378922, 628641,  
494605, 606387, 179993, 755472, 253608, 975198, 328457, 885712, 411958,  
418586, 254970, 299345, 632115, 915208, 661570, 328375, 538422, 321303};  
  
        int[] seatsDayTwo = {740912, 209431, 310346, 316462, 915797,  
850440, 803140, 459194, 293277, 302424, 790507, 711980, 639916, 707446,  
940339, 613076, 524157, 189604, 595934, 509691, 234133, 787575, 674602,  
944308, 710345, 889699, 622393, 151931, 964325, 944568, 357684, 933857,  
541190, 935076, 468848, 449446, 278951, 885503, 539124, 278723, 998622,  
846182, 394328, 914002, 803795, 851135, 828760, 504936, 504322, 648644};  
  
        int matchCounter = 0;  
  
        // Fix the outer loop header to iterate through the first array of seats  
        for(int i = -1; i < -1; i--) {
```

```

of seats
        // Fix the inner loop header to iterate through the second array

        for(int j = -1; j < -1; j--) {

            // Replace 1==2 with conditional logic to check if an
            element in the first array matches an element in the second array

            if(1==2) {

                matchCounter++;

                System.out.println("Contestant: " +
seatsDayOne[i] + ", Seat Day One: " + i + ", Seat Day Two: " + j);

                break;

            }

        }

    }

    System.out.println("The total number of contestants reserving seats
on both days was: " + matchCounter);

}

}

```

OUTPUT

Contestant: 622393, Seat Day One: 3, Seat Day Two: 26

Contestant: 394328, Seat Day One: 17, Seat Day Two: 42

The total number of contestants reserving seats on both days was: 2

16.6 Traversing 2D Arrays: Introduction

In the last exercise, we reviewed how to use nested loops as well as how to iterate through regular arrays using loops. In this exercise, we will apply that knowledge in order to learn how to traverse 2D arrays.

Traversing 2D arrays using loops is important because it allows us to access many elements quickly, access elements in very large 2D arrays, and even access elements in 2D arrays of unknown sizes.

Let's remember the structure of 2D arrays in Java:

```
char[][] letterBlock = {{'a','b','c'},{'d','e','f'},{'g','h','i'},{'j','k','l'}};
```

In Java, 2D arrays are like normal arrays, but each element is another array. This is shown by the initialized 2D array above. The outer array consists of four elements, where each element consists of a three element subarray.

Let's see what happens when we access elements of the outer array

```
System.out.println(Arrays.toString(letterBlock[0]) + "\n");
System.out.println(Arrays.toString(letterBlock[1]) + "\n");
System.out.println(Arrays.toString(letterBlock[2]) + "\n");
System.out.println(Arrays.toString(letterBlock[3]) + "\n");
```

Here is the output of the above code:

[a, b, c]

[d, e, f]

[g, h, i]

[j, k, l]

As you can see, we can retrieve the entire subarray from each of the outer array elements. If you look at how we are accessing these subarrays, we are just increasing the index. This means we can access each sub-array in the 2D array using a loop!

Let's take a look at an example which produces the same output, but can handle any sized 2D array.

```
for(int index = 0; index < letterBlock.length; index++){  
    System.out.println(Arrays.toString(letterBlock[index]) + "\n");  
}
```

Here is the result:

[a, b, c]

[d, e, f]

[g, h, i]

[j, k, l]

Now let's remember how to access a value from the subarray. Previously, we learned that we can use the double brackets `[][]`, where the first set of brackets contains the index of the element of the outer array and the second set of brackets contains the index of the element in the subarray. If we wanted to retrieve the letter 'f' we would use:

```
char storedLetter = letterBlock[1][2];
```

Since we know we can use a loop to retrieve each of the subarrays stored in the outer array, we can then use a nested loop to access each of the elements from the sub-array.

You might be wondering how we can figure out the number of iterations needed in order to fully traverse the 2D array.

- In order to find the number of elements in the outer array, we just need to get the length of the 2D array.
 - `int lengthOfOuterArray = letterBlock.length;`
 - When thinking about the 2D array in matrix form, this is the height of the matrix (the number of rows)
- In order to find the number of elements in the subarray, we can get the length of the subarray after it has been retrieved from the outer array.

- Remember that we retrieved the sub array earlier using this format:
 - `char[] subArray = letterBlock[0];`
- Therefore, we can use this to get the length of the first subarray in the 2D array
 - `int lengthOfSubArray = letterBlock[0].length;`
 - When thinking about the 2D array in matrix form, this is the width of the matrix (the number of columns)
- In most cases, getting the length of the first subarray in the 2D array will apply to the rest of the subarrays (if it is rectangular in shape), but there are rare occasions where the length of the subarrays could be different. This occurs if the 2D array is a jagged array. We won't be working with any jagged 2D arrays in this lesson, but it's something to keep in mind.

Let's look at an example!

```
for(int a = 0; a < letterBlock.length; a++) {
    for(int b = 0; b < letterBlock[a].length; b++) {
        System.out.print("Accessed: " + letterBlock[a][b] + "\t");
    }
    System.out.println();
}
```

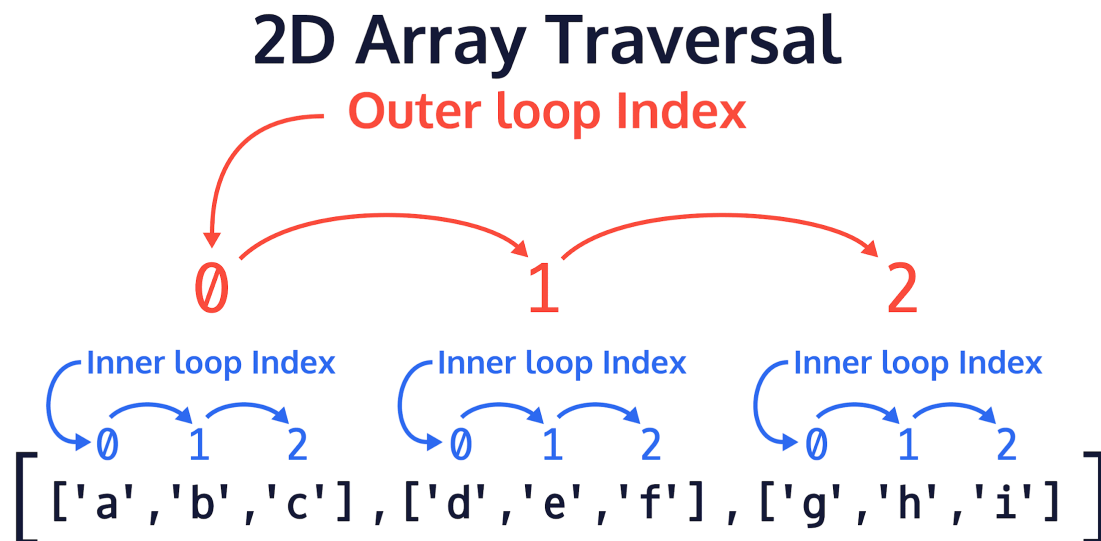
You can think of the variable `a` as being the outer loop index, and the variable `b` as being the inner loop index.

Here is the output:

```
Accessed: a  Accessed: b  Accessed: c
Accessed: d  Accessed: e  Accessed: f
Accessed: g  Accessed: h  Accessed: i
```

Within the nested **for** loop, we can see that each of the subarray elements are being accessed by using the outer loop index for the outer array, and the inner loop index for the subarray.

Here is a diagram to help visualize how the 2D array is traversed using nested loops:



We don't have to only use regular **for** loops for traversing 2D arrays. We can use enhanced **for** loops if we do not need to keep track of the indices. Since enhanced **for** loops only use the element of the arrays, it is a bit more cumbersome to keep track of which index we are at. This same idea applies to while and do-while loops as well. This is why we usually use regular **for** loops except for when we want to do something simple like printing.

We have gone over how to think about 2D array traversal in terms of arrays of arrays, but there are two main ways of thinking about traversal in terms of rows and columns. These are called row-major order and column-major order.

16.6 Traversing 2D Arrays: Introduction Instructions

1.

Store the number of subarrays of `intMatrix` into an `int` variable called `numSubArrays`.

If you want to confirm your work, print that number after creating it. You should expect to see the number of rows in the 2D array.

2.

Store the length of the subarrays using the first subarray in `intMatrix`. Store it in a variable called `subArrayLength`. This should be an `int`.

Again, feel free to use `System.out.println()` to print this value after creating the variable. What value do you expect to see?

3.

Store the number of columns in `intMatrix` into a variable called `columns` and the number of rows in `intMatrix` into a variable called `rows`.

4.

Replace the outer and inner for loop headers to iterate through the entire 2D array. Use the iterators `i` for the outer loop and `j` for the inner loop. Also, add a line of code which calculates the sum of all of the elements in the 2D array.

```
public class Introduction {  
    public static void main(String[] args) {  
        //Given the provided 2d array  
        int[][] intMatrix = {  
            { 4, 6, 8, 10, 12, 14, 16},  
            {18, 20, 22, 24, 26, 28, 30},  
            {32, 34, 36, 38, 40, 42, 44},  
            {46, 48, 50, 52, 54, 56, 58},  
            {60, 62, 64, 66, 68, 70, 79}  
        };  
        // Store the number of subarrays of intMatrix into a variable called  
        'numSubArrays'
```

```

        // Store the length of the subarrays using the first subarray in
intMatrix. Store it in a variable called subArrayLength.

        // Store the number of columns in intMatrix into a variable called
'columns'

        // Store the number of rows in intMatrix into a variable called 'rows'

        // Replace the outer and inner for loop headers to iterate through the
entire 2D array. Use the iterators `i` for the outer loop and `j` for the inner loop.
        int sum = 0;
        for(int i=-1; i<-1; i--) {
            for(int j = -1; j < -1; j--) {
                // Insert a line of code to increase the variable `sum` by
each accessed element

            }
        }
        System.out.println(sum);
    }
}

```

OUTPUT

1337

16.7 Traversing 2D Arrays: Practice with Loops

We have seen how to traverse 2D arrays using standard **for** loops, but in this exercise, we will practice traversing them using some other loop types. For example, you may want to only retrieve elements without keeping track of the indices using enhanced **for** loops, or you could continuously update the 2D array until a condition is met using **while** loops.

In enhanced **for** loops, each element is iterated through until the end of the array. When we think about the structure of 2D arrays in Java (arrays of array objects) then we know that the outer enhanced **for** loop elements are going to be arrays.

Let's take a look at an example:

Given this 2D array of character data:

```
char[][] charData = {{'a', 'b', 'c', 'd', 'e', 'f'}, {'g', 'h', 'i', 'j', 'k', 'l'}};
```

Print out every character using enhanced **for** loops:

```
for(char[] charRow : charData) {  
    for(char c : charRow) {  
        System.out.print(c + " ");  
    }  
    System.out.println();  
}
```

Remember that the syntax for enhanced **for** loops looks like so: `for( datatype elementName : arrayName){`. Since 2D arrays in Java are arrays of arrays, each element in the outer enhanced **for** loop is an entire row of the 2D array. The nested enhanced **for** loop is then used to iterate through each element in the extracted row. Here is the output of the above code:

```
a b c d e f  
g h i j k l
```

Here is an example which accomplishes the same thing, but using **while** loops:

```
int i = 0, j=0;  
while(i<charData.length) {  
    j = 0;  
    while(j<charData[i].length) {
```

```

        System.out.print(charData[i][j] + " ");
        j++;
    }
    System.out.println();
    i++;
}

```

Here is the output of the above code:

```

a b c d e f
g h i j k l

```

Notice how we can use different loop types for traversal, but still receive the same result.

Let's work some example problems using different loop types!

16.7 Traversing 2D Arrays: Practice with Loops Instructions

1.

Use nested enhanced for loops to calculate the total number of characters in the wordData 2D array and print the result to the console. (Get the string .length() of each element)

2.

Using nested while loops, iterate through all of the elements in the 2D array and print them to the console using the format: word [row][column]. The print statement has been provided (you will need to modify it if you use iterators other than i and j).

```

public class LoopPractice {

    public static void main(String[] args) {

        String[][] wordData = {"study", "consider", "examine", "learn"},
{"ponder", "read", "think", "cogitate"}};

```

//Use nested enhanced for loops to calculate the total number of characters in the wordData 2D array and print the result to the console. (Get the string .length() of each element)

```
int characterCount = 0;
```

```
System.out.println(characterCount);
```

//Using nested while loops, iterate through all of the elements in the 2D array and print them to the console using the format: word [row][column]. The formatted print statement has been provided.

```
        //System.out.println(wordData[i][j] + ": [" + i + "]" + "[" +  
j + "]);  
  
    }
```

```
}
```

OUTPUT

48

study: [0][0]

consider: [0][1]

examine: [0][2]

learn: [0][3]

ponder: [1][0]

read: [1][1]

think: [1][2]

cogitate: [1][3]

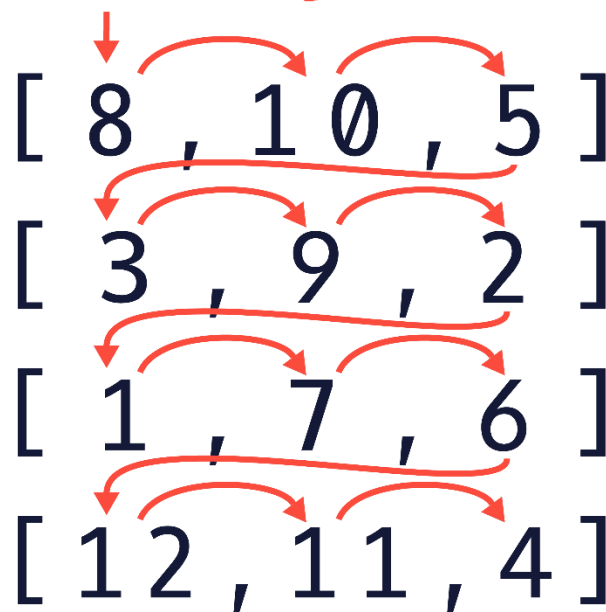
16.8 Traversing 2D Arrays: Row-Major Order

Row-major order for 2D arrays refers to a traversal path which moves horizontally through each row starting at the first row and ending with the last.

Although we have already looked at how 2D array objects are stored in Java, this ordering system conceptualizes the 2D array into a rectangular matrix and starts the traversal at the top left element and ends at the bottom right element.

Here is a diagram which shows the path through the 2D array:

Row-major order



This path is created by the way we set up our nested loops. In the previous exercise, we looked at how we can traverse the 2D array by having nested loops in a variety of formats, but if we want to control the indices, we typically use standard **for** loops.

Let's take a closer look at the structure of the nested **for** loops when traversing a 2D array:

Given this 2D array of strings describing the element positions:

```
String[][] matrix = {"[0][0]", "[0][1]", "[0][2]",  
                    "[1][0]", "[1][1]", "[1][2]",  
                    "[2][0]", "[2][1]", "[2][2]",  
                    "[3][0]", "[3][1]", "[3][2]"};
```

Lets keep track of the total number of iterations as we traverse the 2D array:

```
int stepCount = 0;  
  
for(int a = 0; a < matrix.length; a++) {  
    for(int b = 0; b < matrix[a].length; b++) {  
        System.out.print("Step: " + stepCount);  
        System.out.print(", Element: " + matrix[a][b]);  
        System.out.println();  
        stepCount++;  
    }  
}
```

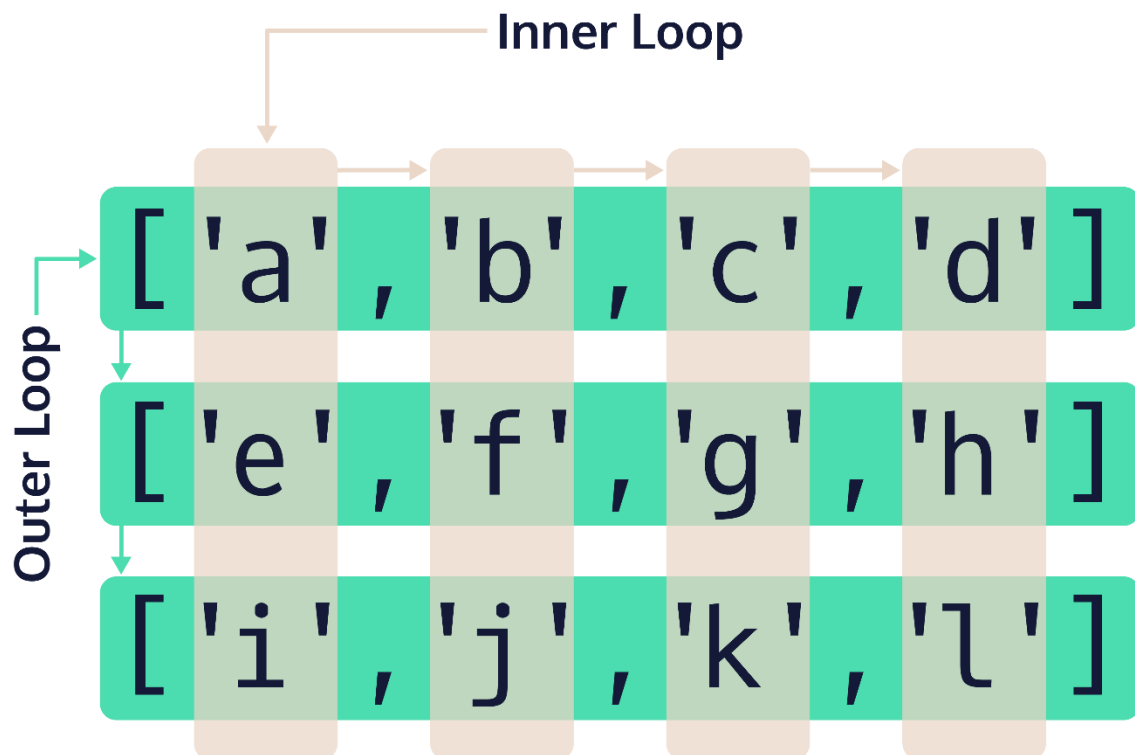
Here is the output of the above code:

```
Step: 0, Element: [0][0]  
Step: 1, Element: [0][1]  
Step: 2, Element: [0][2]  
Step: 3, Element: [1][0]  
Step: 4, Element: [1][1]  
Step: 5, Element: [1][2]  
Step: 6, Element: [2][0]  
Step: 7, Element: [2][1]  
Step: 8, Element: [2][2]  
Step: 9, Element: [3][0]  
Step: 10, Element: [3][1]  
Step: 11, Element: [3][2]
```

The step value increases with every iteration within the inner **for** loop. Because of this, we can see the order in which each element is accessed. If we follow the step value in the output shows us that the elements are accessed in the same order as the row-major diagram above. Now why is that?

This is because in our **for** loop, we are using the number of rows as the termination condition within the outer **for** loop header `a < matrix.length`; Additionally, we are using the number of columns `b < matrix[a].length` as the termination condition for our inner loop. Logically we are saying: “For every row in our matrix, iterate through every single column before moving to the next row”. This is why our above example is traversing the 2D array using row-major order.

Here is a diagram showing which loop accesses which part of the 2D array for row-major order:



Why Use Row-Major Order?

Row-major order is important when we need to process data in our 2D array by row. You can be provided data in a variety of formats and you may need to perform calculations of rows of data at a time instead of individual elements. Let's take one of our previous checkpoint exercises as an example. You were asked to calculate the sum of the entire 2D array of integers by traversing and accessing each element. Now, if we wanted to calculate the sum of each row, or take the average of each row, we can use row-major order to access the data in the order that we need. Let's look at an example!

Given a 6X3 2D array of doubles:

```
double[][] data = {{0.51,0.99,0.12},
                  {0.28,0.99,0.89},
                  {0.05,0.94,0.05},
                  {0.32,0.22,0.61},
                  {1.00,0.95,0.09},
                  {0.67,0.22,0.17}};
```

Calculate the sum of each row using row-major order:

```
double rowSum = 0.0;
for(int o = 0; o < data.length; o++) {
    rowSum = 0.0;
    for(int i = 0; i < data[o].length; i++) {
        rowSum += data[o][i];
    }
    System.out.println("Row: " + o + ", Sum: " + rowSum);
}
```

The output of the above code is:

```
Row: 0, Sum: 1.62
Row: 1, Sum: 2.16
Row: 2, Sum: 1.04
Row: 3, Sum: 1.15
Row: 4, Sum: 2.04
Row: 5, Sum: 1.06
```

An interesting thing to note is that, due to the way 2D arrays are structured in Java, enhanced **for** loops are always in row-major order. This is because an enhanced **for** loop iterates through the elements of the outer array which causes the terminating condition to be the length of the 2D array which is the number of rows.

16.8 Traversing 2D Arrays: Row-Major Order Instructions

1.

You are provided with some runner lap data. Take a look at the loops we're using to iterate through this 2D array. Replace the incorrect for loop headers to perform row-major traversal. Use the iterators outer and inner for the outer and inner loops.

2.

Enter the missing line of code within the nested for loop to sum up the values for each row in the runner data.

We've already created a variable named `runnerTime` that you can use to sum these values.

3.

We've given you a variable named `averageVal` that currently stores 0. Edit that line of code to find the average time of each runner.

```
public class RowMajor {  
    public static void main(String[] args) {  
        // Given runner lap data  
        double[][] times = {{64.791, 75.972, 68.950, 79.039, 73.006, 74.157},  
                             {67.768, 69.334, 70.450, 67.667, 75.686, 76.298}, {72.653, 77.649, 74.245, 62.121,  
                             63.379, 79.354}};
```


// Replace the incorrect for loop headers, use the iterators 'outer' and 'inner' for the outer and inner loops

```
double runnerTime = 0.0;
```

```
for(int outer = -1; outer < -1; outer++) {
```

```
    runnerTime = 0.0;
```

```
    for(int inner = -1; inner < -1; inner++) {
```

```
        System.out.println("Runner index: " + outer + ", Time index: " + inner);
```

// Enter the missing line of code to sum up the values in each row. Use the variable runnerTime

```
    }
```

// Enter the missing line of code to find the average time of each runner. Use the variable averageVal

```
double averageVal = 0;
```

```
    System.out.println("Sum of runner " + outer + " times: " +  
runnerTime);
```

```
    System.out.println("Average of runner " + outer + ": " +  
averageVal);
```

```
    }
```

```
}
```

```
}
```

OUTPUT

Runner index: 0, Time index: 0

Runner index: 0, Time index: 1

Runner index: 0, Time index: 2
Runner index: 0, Time index: 3
Runner index: 0, Time index: 4
Runner index: 0, Time index: 5
Sum of runner 0 times: 435.9149999999999
Average of runner 0: 72.65249999999999
Runner index: 1, Time index: 0
Runner index: 1, Time index: 1
Runner index: 1, Time index: 2
Runner index: 1, Time index: 3
Runner index: 1, Time index: 4
Runner index: 1, Time index: 5
Sum of runner 1 times: 427.20300000000001
Average of runner 1: 71.200500000000002
Runner index: 2, Time index: 0
Runner index: 2, Time index: 1
Runner index: 2, Time index: 2
Runner index: 2, Time index: 3
Runner index: 2, Time index: 4
Runner index: 2, Time index: 5
Sum of runner 2 times: 429.401
Average of runner 2: 71.56683333333334

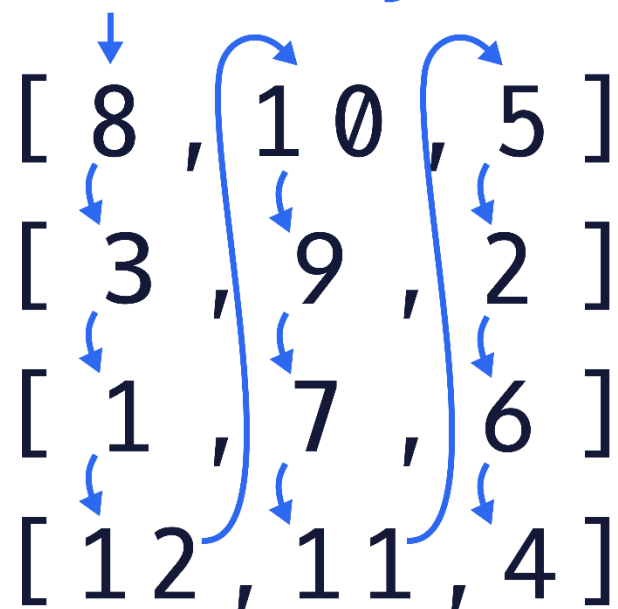
16.9 Traversing 2D Arrays: Column-Major Order

Column-major order for 2D arrays refers to a traversal path which moves vertically down each column starting at the first column and ending with the last.

This ordering system also conceptualizes the 2D array into a rectangular matrix and starts the traversal at the top left element and ends at the bottom right element. Column-major order has the same starting and finishing point as row-major order, but its traversal is completely different

Here is a diagram which shows the path through the 2D array:

Column-major order



In order to perform column-major traversal, we need to set up our nested loops in a different way. We need to change the outer loop from depending on the number of rows, to depending on the number of columns. Likewise we need the inner loop to depend on the number of rows in its termination condition.

Let's look at our example 2D array from the last exercise and see what needs to be changed.

Given this 2D array of strings describing the element positions:

```
String[][] matrix = {"[0][0]", "[0][1]", "[0][2]",  
    {"[1][0]", "[1][1]", "[1][2]"},  
    {"[2][0]", "[2][1]", "[2][2]"},  
    {"[3][0]", "[3][1]", "[3][2]"}};
```

Let's keep track of the total number of iterations as we traverse the 2D array. We also need to change the termination condition (middle section) within the outer and inner **for** loop.

```
int stepCount = 0;  
  
for(int a = 0; a < matrix[0].length; a++) {  
    for(int b = 0; b < matrix.length; b++) {  
        System.out.print("Step: " + stepCount);  
        System.out.print(", Element: " + matrix[b][a]);  
        System.out.println();  
        stepCount++;  
    }  
}
```

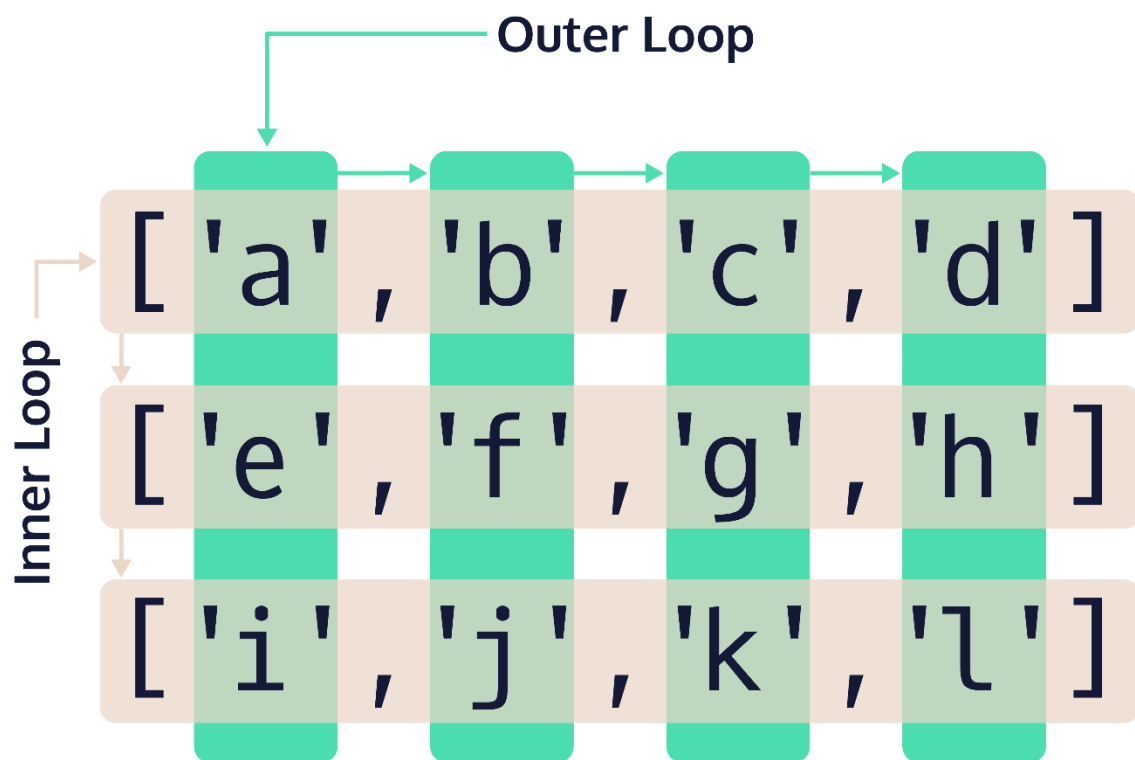
Here is the output of the above code:

```
Step: 0, Element: [0][0]  
Step: 1, Element: [1][0]  
Step: 2, Element: [2][0]  
Step: 3, Element: [3][0]  
Step: 4, Element: [0][1]  
Step: 5, Element: [1][1]  
Step: 6, Element: [2][1]  
Step: 7, Element: [3][1]  
Step: 8, Element: [0][2]  
Step: 9, Element: [1][2]  
Step: 10, Element: [2][2]  
Step: 11, Element: [3][2]
```

As you can see in the code above, the way we accessed the elements from our 2D array of strings called matrix is different from the way we accessed them when using row-major order. Let's remember that the way we get the number of

columns is by using `matrix[0].length` and the way we get the number of rows is by using `matrix.length`. Because of these changes to our **for** loops, our iterator `a` now iterates through every column while our iterator `b` iterates through every row. Since our iterators now represent the opposite values, whenever we access an element from our 2D array, we need to keep in mind what indices we are passing to our accessor. Remember the format we use for accessing the elements `matrix[row][column]`? Since `a` now iterates through our column indices, we place it in the right set of brackets, and the `b` is now placed in the left set of brackets.

Here is a diagram showing which loop accesses which part of the 2D array for column-major order:



Why Use Column-Major Order?

Column major order is important because there are a lot of cases when you need to process data vertically. Let's say that we have a chart of information which includes temperature data about each day. The top of each column is labeled with a day, and each row represents an hour. In order to find the average temperature per day, we would need to traverse the data vertically since each column represents a day. As mentioned in the last exercise, data can be provided in many different formats and shapes and you will need to know how to traverse it accordingly.

Let's look at our sum example from the last exercise, but now using column-major order.

Given a 6X3 2D array of doubles:

```
double[][] data = {{0.51,0.99,0.12},
                  {0.28,0.99,0.89},
                  {0.05,0.94,0.05},
                  {0.32,0.22,0.61},
                  {1.00,0.95,0.09},
                  {0.67,0.22,0.17}};
```

Calculate the sum of each column using column-major order:

```
double colSum = 0.0;
for(int o = 0; o < data[0].length; o++) {
    colSum = 0.0;
    for(int i = 0; i < data.length; i++) {
        colSum += data[i][o];
    }
    System.out.println("Column: " + o + ", Sum: " + colSum);
}
```

The output of the above code is:

```
Column: 0, Sum: 2.83
Column: 1, Sum: 4.31
Column: 2, Sum: 1.93
```

Let's try an example!

We will be using the same runner data from the last exercise, but this time we are going to take the average times per lap rather than per runner. This requires that we use column-major traversal.

16.9 Traversing 2D Arrays: Column-Major Order Instructions

1.

You are provided with some runner lap data. Take a look at the loops we're using to iterate through this 2D array. Replace the incorrect for loop headers to perform column-major traversal. Use the iterators outer and inner for the outer and inner loops.

2.

Enter the missing line of code within the nested for loop to sum up the values for each column in the runner data.

We've already created a variable named lapTime that you can use to sum these values.

3.

We've given you a variable named averageVal that currently stores 0. Edit that line of code to find the average time of each lap.

```
public class ColumnMajor {  
    public static void main(String[] args) {  
        // Given runner lap data  
        double[][] times = {{64.791, 75.972, 68.950, 79.039, 73.006, 74.157},  
                             {67.768, 69.334, 70.450, 67.667, 75.686, 76.298}, {72.653, 77.649, 74.245, 62.121,  
                             63.379, 79.354}};  
  
        // Replace the incorrect for loop headers, use the iterators 'outer' and  
        'inner' for the outer and inner loops
```

```

double lapTime = 0.0;
for(int outer = -1; outer < -1; outer++){
    lapTime = 0.0;
    for(int inner = -1; inner < -1; inner++){
        System.out.println("Lap index: " + outer + ", Time index:
" + inner);

        // Enter the missing line of code to sum up the values in
each column. Use the variable lapTime

    }

    // Enter the missing line of code to find the average time of
each lap. Use the variable averageVal

    double averageVal = 0;

    System.out.println("Sum of lap " + outer + " times: " +
lapTime);

    System.out.println("Average time for lap " + outer + ": " +
averageVal);
    }
}

```

OUTPUT

Lap index: 0, Time index: 0

Lap index: 0, Time index: 1

Lap index: 0, Time index: 2

Sum of lap 0 times: 205.212

Average time for lap 0: 68.404

Lap index: 1, Time index: 0

Lap index: 1, Time index: 1

Lap index: 1, Time index: 2

Sum of lap 1 times: 222.95499999999998

Average time for lap 1: 74.31833333333333

Lap index: 2, Time index: 0

Lap index: 2, Time index: 1

Lap index: 2, Time index: 2

Sum of lap 2 times: 213.645

Average time for lap 2: 71.215

Lap index: 3, Time index: 0

Lap index: 3, Time index: 1

Lap index: 3, Time index: 2

Sum of lap 3 times: 208.82700000000003

Average time for lap 3: 69.60900000000001

Lap index: 4, Time index: 0

Lap index: 4, Time index: 1

Lap index: 4, Time index: 2

Sum of lap 4 times: 212.071

Average time for lap 4: 70.69033333333333

Lap index: 5, Time index: 0

Lap index: 5, Time index: 1

Lap index: 5, Time index: 2

Sum of lap 5 times: 229.80899999999997

Average time for lap 5: 76.603

16.10 Combining Traversal and Conditional Logic

When working with 2D arrays, it is important to be able to combine traversal logic with conditional logic in order to effectively navigate and process the data. Here are a few ways in how conditional logic can affect 2D array traversal:

- Skipping or selecting certain rows and columns
- Modifying elements only if they meet certain conditions
- Complex calculations using the 2D array data
- Formatting the 2D array
- Avoiding exceptions / smart processing

Let's go over a few examples which use these ideas:

First, let's think about a situation where you have some string data inside a 2D array. We have an application which allows users to input events on a calendar. This is represented by a 5x7 2D array of strings. Due to the fact that the number of days in each month is slightly different and that there are less than 35 days in a month, we know that some of our elements are going to be empty. We want our application to do a few things:

- Detect which days of which weeks have something planned and alert us about the event.
- Count the number of events for each week
- Count the number of events for each day

Here is a visualization of what our calendar data looks like after a user has entered in some event information:

	Sun	Mon	Tues	Wed	Thurs	Fri	Sat
Wk 1	Volunteer	Delivery			Doctor		Soccer
Wk 2		Exam 1		Mechanic			Soccer
Wk 3	Volunteer	Off Work		Birthday		Concert	
Wk 4		Exam 2			Doctor		Soccer
Wk 5	Visit Family						

Here's what our calendar data looks like in our application

```
String[][] calendar = {"volunteer", "delivery", null, null, "doctor", null, "soccer"},
{"null", "exam 1", null, "mechanic", null, null, "soccer"}, {"volunteer", "off work",
null, "birthday", null, "concert", null}, {null, "exam 2", null, null, "doctor", null,
"soccer"}, {"visit family", null, null, null, null, null, null}};
```

Let's look at some code which accomplishes the requirements above. Carefully look through each line of code and read all of the comments.

There are a few things to note:

- Row-major or column-major order can be used to access the individual events
- Row-major order must be used to count the number of events per week since each row represents a week

Let's take care of the first 2 requirements in one set of nested row-major loops

```

for(int i = 0; i < calendar.length; i++) {
    numberOfEventsPerWeek = 0;
    for(int j = 0; j < calendar[i].length; j++) {
        // We need conditional logic to ensure that we do not count the empty days
        String event = calendar[i][j];
        if(event!=null && !event.equals("")) {
            // If the day does not have a null value and an empty string for an event,
            then we print it and count it
            System.out.println("Week: " + (i+1) + ", Day: " + (j+1) + ", Event: " + event);
            numberOfEventsPerWeek++;
        }
    }
    System.out.println("Total number of events for week " + (i+1) + ": "
+ numberOfEventsPerWeek + "\n");
}

```

The above code produces this output:

```

Week: 1, Day: 1, Event: volunteer
Week: 1, Day: 2, Event: delivery
Week: 1, Day: 5, Event: doctor
Week: 1, Day: 7, Event: soccer
Total number of events for week 1: 4

```

```

Week: 2, Day: 2, Event: exam 1
Week: 2, Day: 4, Event: mechanic
Week: 2, Day: 7, Event: soccer
Total number of events for week 2: 3

```

```

Week: 3, Day: 1, Event: volunteer
Week: 3, Day: 2, Event: off work
Week: 3, Day: 4, Event: birthday
Week: 3, Day: 6, Event: concert
Total number of events for week 3: 4

```

```

Week: 4, Day: 2, Event: exam 2

```

Week: 4, Day: 5, Event: doctor
Week: 4, Day: 7, Event: soccer
Total number of events for week 4: 3

Week: 5, Day: 1, Event: visit family
Total number of events for week 5: 1

Now let's complete the third requirement. Since we need to count all of the events for each of the weekdays, we will need to traverse the calendar vertically.

```
int numberOfEventsPerWeekday = 0;
// We will use this array of day strings for our output later on so we don't have
(day: 1)
String[] days = {"Sundays", "Mondays", "Tuesdays", "Wednesdays", "Thursdays",
"Fridays", "Saturdays"};
for(int i = 0; i < calendar[0].length; i++) {
    numberOfEventsPerWeekday = 0;
    for(int j = 0; j < calendar.length; j++) {
        // Don't forget to flip the iterators in the accessor since we are flipping the
direction we are navigating.
        // Remember, i now controls columns and j now controls rows
        String event = calendar[j][i];
        if(event!=null && !event.equals("")) {
            // Make sure we have an event for the day before counting it
            numberOfEventsPerWeekday++;
        }
    }
    // Use the days string array from earlier to convert the day index to a real
weekday string
    System.out.println("Number of events on " + days[i] + ": "
+ numberOfEventsPerWeekday);
}
```

The output is:

Number of events on Sundays: 3
Number of events on Mondays: 4

Number of events on Tuesdays: 0
Number of events on Wednesdays: 2
Number of events on Thursdays: 2
Number of events on Fridays: 1
Number of events on Saturdays: 3

This example uses many of the concepts we have learned before. We use row-major order, column-major order, as well as including conditional logic to ensure that we have data for the elements we are accessing.

Additionally, we can use conditional logic to skip portions of the 2D array. For example, let's say we wanted to print the events for weekdays only and skip the weekends.

We could use a conditional statement such as `if(j!=0 && j!=6)` in order to skip Sunday (0) and Saturday (6).

These modifications to our 2D array traversal are very common when processing data in applications. We need to know which cells to look at (skipping column titles for example), which cells to ignore (empty data, invalid data, outliers, etc.), and which cells to convert (converting string input from a file to numbers).

Let's try an example!

We are making a simple grayscale image editor program and we want to apply some modifications to the image. We have a 4x8 pixel image that is stored as a 2D array of integers. The integer value represents the brightness of the pixel, where the acceptable values are between 0 and 255, inclusive.

16.10 Combining Traversal and Conditional Logic Instructions

1.

First, we want to crop the image down to a 4x6 image, removing the right 2 columns. Declare and initialize a new 2D array of integers with 4 rows and 6 columns called `newImage`.

2.

Now that you have your empty image, use nested **for** loops to copy over the data from the original image (stored in `imageData`) to the new image, make sure not to include the cropped out columns (right 2 columns).

3.

You want to decrease the brightness of the new image by 50 units. The way this works is that for every integer in the new 2D array, we will subtract the value by 50. Remember that the value range for the pixel is 0-255, so if the result tries to go below 0, just set it equal to 0.

```
import java.util.Arrays;
```

```
public class Combining {
```

```
    public static void main(String[] args) {
```

```
        int[][] imageData={{100,90,255,80,70,255,60,50},
                                {255,10,5,255,10,5,255,255},
                                {255,255,255,0,255,255,255,75},
                                {255,60,30,0,30,60,255,255}};
```

```
        //First, we want to crop the image down to a 4x6 image, removing
        the right 2 columns. Declare and initialize a new 2D array of integers with 4 rows
        and 6 columns called `newImage`.
```

```
        int[][] newImage = null;
```

```
        //Now that you have your empty image, use nested **for** loops to
        copy over the data from the original image to the new image, make sure not to
        include the cropped out columns.
```

```
System.out.println(Arrays.deepToString(newImage));
```

//You want to decrease the brightness of the new image by 50 units.
The way this works is that for every integer in the new 2D array, we will subtract the value by 50. Remember that the value range for the pixel is 0-255, so if the result tries to go below 0, just set it equal to 0.

```
System.out.println(Arrays.deepToString(newImage));
```

```
}
```

```
}
```

OUTPUT

```
[[100, 90, 255, 80, 70, 255], [255, 10, 5, 255, 10, 5], [255, 255, 255, 0, 255, 255],  
[255, 60, 30, 0, 30, 60]]
```

```
[[50, 40, 205, 30, 20, 205], [205, 0, 0, 205, 0, 0], [205, 205, 205, 0, 205, 205], [205,  
10, 0, 0, 0, 10]]
```

16.11 2D Array Review

Let's review the concepts we have learned throughout this lesson.

Arrays are objects in Java, we can have arrays of objects, therefore we can also have arrays of arrays. This is the way 2D arrays are structured in Java.

We can declare and initialize 2D arrays in a few different ways depending on the situation:

```
// Declaring without initializing  
int[][] intTwoD;
```

```
// Initializing an empty 2D array which has already been declared  
intTwoD = new int[5][5];
```



```
// Declaring and initializing an empty 2D array at once
```

```
String[][] stringData = new String[3][6];
```

```
// Declaring and initializing a 2D array using initializer lists
```

```
double[][] doubleValues = {{1.5, 2.6, 3.7}, {7.5, 6.4, 5.3}, {9.8, 8.7, 7.6}, {3.6, 5.7, 7.8}};
```

```
// Initializing a 2D array using initializer lists after it has already been declared, or  
already contains data;
```

```
char[][] letters = new char[100][250];
```

```
letters = new char[][]{{'a', 'b', 'c'}, {'d', 'e', 'f'}};
```

We retrieve elements in a 2D array by providing a row and column index `char c = letters[0][1];`

- We can also think of them as the index of the outer array and the index of the subarray
- We can modify elements the same way `letters[1][2] = 'z';`

We traverse 2D arrays using nested loops.

- We can use loops of any type, but we typically use nested **for** loops to keep track of the indices
- Row-major order traverses through each row moving horizontally to the right through each row
- Column-major order traverses through each column moving vertically down through each column
- Row-major order and column-major order start and end on the same elements, but the paths are different.
- In order to convert row-major to column-major, we need to make the outer loop terminating condition depend on the number of columns, make the inner loop terminating condition depend on the number of rows, and flip the variables in our accessor within the inner loop to ensure that we don't

try to access outside of the 2D array since we flipped the direction of traversal.

Here are examples of row-major and column-major order:

```
// Row-major order
for(int o = 0; o < letters.length; o++) {
    for(int i = 0; i < letters[o].length; i++) {
        char c = letters[o][i];
    }
}
```

```
// Column-major order
for(int o = 0; o < letters[0].length; o++) {
    for(int i = 0; i < letters.length; i++) {
        char c = letters[i][o];
    }
}
```

Conditional logic in our 2D array traversal allows us to use the data in a meaningful way. We can control which rows and columns we look at, ensure that the data we are looking at is what we want, perform calculations on specific elements, avoid throwing exceptions, and more.

Here is an example of traversal with conditional logic.

Given this 2D array of Strings:

```
String[][] words = {"championship", "QUANTITY", "month"}, {"EMPLOYEE",
"queen", "understanding"}, {"method", "writer", "MOVIE"};
```

We are going to flip the capitalization of the words:

```
System.out.println("Before...");
System.out.println(Arrays.deepToString(words).replace("]", " ", "\n") + "\n");
```

```
for(int i=0; i<words.length; i++) {
    for(int j = 0; j<words[i].length; j++) {
        if(words[i][j]!=null) {
```

```

        // Check the capitalization
        boolean allCaps = true;
        for(char c : words[i][j].toCharArray())
            if(!Character.isUpperCase(c))
                allCaps = false;

        // Flip the capitalization
        if(allCaps)
            words[i][j] = words[i][j].toLowerCase();
        else
            words[i][j] = words[i][j].toUpperCase();
    }
}
}

System.out.println("After...");
System.out.println(Arrays.deepToString(words).replace("],", "],\n") + "\n");

```

Here is the output of the above code:

Before...

```

[[championship, QUANTITY, month],
 [EMPLOYEE, queen, understanding],
 [method, writer, MOVIE]]

```

After...

```

[[CHAMPIONSHIP, quantity, MONTH],
 [employee, QUEEN, UNDERSTANDING],
 [METHOD, WRITER, movie]]

```

Time to work some review problems!

After learning about 2D arrays, you have decided to become a CS professor and you are now teaching your class about 2D arrays. You are making an application which will keep track of their exam grades and show you statistics about their performance. You will be using 2D arrays to keep track of their exam grades

16.11 2D Array Review Instructions

1.

First, declare and initialize a 4x3 2D array of doubles called scores which will contain the exam data for four students. The rows will represent the student and the columns will represent the exam number. You already know the first exam scores (80.4, 96.2, 100.0, 78.9). Use initializer lists to store the first exam scores in the first column and -1 for the remaining exams. Use the provided print statement to print the result in the console.

2.

The next set of exams have occurred. Using 4 lines of code, manually enter the scores (89.7, 90.5, 93.6, 88.1) for the second exam (column 1). Use the provided print statement to print the updated 2D array as well.

3.

You have realized that you will only be keeping track of 2 exam grades instead of 3. Declare and initialize an empty 4x2 2D array of double values called newScores.

4.

Using loops, copy all of the scores for exam 1 and 2 into the new 2D array. (Do not include the -1 values)

5.

You have allowed the students to complete an extra credit activity to contribute towards their scores. For all exam grades less than 90, add 2 additional points to the grade in newScores.

```
import java.util.Arrays;
```

```
public class Review {
```

```
public static void main(String[] args) {
```

```
    //First, declare and initialize a 4x3 2D array of doubles called `scores`  
    which will contain the exam data for four students. The rows will represent the  
    student and the columns will represent the exam number. You already know the  
    first exam scores (80.4, 96.2, 100.0, 78.9). Use initializer lists to store the first  
    exam scores in the first column and -1 for the remaining exams. Use the provided  
    print statement to print the result in the console.
```

```
    double[][] scores = null;
```

```
    System.out.println(Arrays.deepToString(scores));
```

```
    //The next set of exams have occurred. Using 4 lines of code, manually  
    enter the scores (89.7, 90.5, 93.6, 88.1) for the second exam (column 1). Use the  
    provided print statement to print the updated 2D array as well.
```

```
    System.out.println(Arrays.deepToString(scores));
```

```
    //You have realized that you will only be keeping track of 2 exam grades instead  
    of 3. Declare and initialize an empty 4x2 2D array of double values called  
    newScores
```

```
    double[][] newScores = null;
```

```
//Using loops, copy all of the scores for exam 1 and 2 into the new 2D array. (do not include the -1 values)
```

```
System.out.println(Arrays.deepToString(newScores));
```

```
//You have allowed the students to complete an extra credit activity to contribute towards their scores. For all exam grades less than 90, add 2 additional points to the grade in `newScores`
```

```
System.out.println(Arrays.deepToString(newScores));
```

```
}
```

```
}
```

OUTPUT

```
[[80.4, -1.0, -1.0], [96.2, -1.0, -1.0], [100.0, -1.0, -1.0], [78.9, -1.0, -1.0]]
```

```
[[80.4, 89.7, -1.0], [96.2, 90.5, -1.0], [100.0, 93.6, -1.0], [78.9, 88.1, -1.0]]
```

```
[[80.4, 89.7], [96.2, 90.5], [100.0, 93.6], [78.9, 88.1]]
```

```
[[82.4, 91.7], [96.2, 90.5], [100.0, 93.6], [80.9, 90.1]]
```

16. Quiz

1. Which of the following is true about modifying elements in a 2D array?
 - a) When modifying an element in a 2D array, you access the element using the variable name first, the row index in brackets second, and the column index in brackets third. You then set it equal to a value.

- b) When modifying an element in a 2D array, you access the element using the row index in brackets first, column index in brackets second, and the variable name third. You then set it equal to a value.
- c) When modifying an element in a 2D array, you access the element using the row index in brackets first, the variable name second, and the column index in brackets third. You then set it equal to a value.
- d) When modifying an element in a 2D array, you access the element using the variable name first, the column index in brackets second, and the row index in brackets third. You then set it equal to a value.

2. Which answer describes how you should set up enhanced for loops to iterate through the provided 2D array of Strings?

```
public class EnhancedForLoops {
    public static void main(String[] args) {
        String[][] strTwoD = {"learn", "study", "remember"},
                             {"read", "memorize", "grasp"},
                             {"master", "know", "retain"}};
    }
}
```

- a) The outer enhanced for loop iterates through each array of Strings, and the inner enhanced for loop iterates through each array of Strings.
- b) The outer enhanced for loop iterates through each String value, while the inner enhanced for loop iterates through each array of Strings.
- c) The outer enhanced for loop iterates through each array of Strings, while the inner enhanced for loop iterates through each String value in the subarray.
- d) The outer enhanced for loop iterates through each String value, and the inner enhanced for loop iterates through each String value.

3. Choose the correct code for declaring a 2D int array in this Java application.

```
public class MyTwoDArray {
    public static void main(String[] args) {
        //Declare a 2D int array
        _____
    }
}
```

```
}  
}
```

- `int[][] myTwoD;`
- `int myTwoD[];`
- `[int] myTwoD;`
- `int[] myTwoD;`
- `int myTwoD;`

Click or drag and drop to fill in the blank

4. Which of the following is true about nested loop iteration?

- a) The outer-most loop must complete all of its iterations before the inner loop iterates.
- b) The outer-most loop and inner-most loop will always iterate at the same rate.
- c) The inner-most nested loop must complete all of its iterations before the outer loop begins.
- d) The inner-most nested loop must complete all of its iterations before the outer loop continues.

5. Fill in the code for nested loops which traverse the 2D array using column-major order.

```
public class ColumnMajor {  
    public static void main(String[] args) {  
  
        double[][] data = {{1.4, 7.5, 3.5, 8.7, 9.2},  
                            {0.8, 2.6, 6.3, 3.4, 7.5}};  
  
        for(int i = 0; i < _____ ; i++) {  
            for( int j = 0; j < _____ ; j++) {  
                System.out.println(data[j][i]);  
            }  
        }  
    }  
}
```



```
}  
}
```

- data.size
- data[j].length
- data.length
- data[0].length
- data[i].length
- data

Click or drag and drop to fill in the blank

6. Which of the following is true about Row-major order?

- a) Row-major order starts at the top left of the 2D array (when viewing it as a matrix) and traverses across each row until it ends at the bottom right corner.
- b) Row-major order starts at the bottom left of the 2D array (when viewing it as a matrix) and traverses up each column until it ends at the top right corner.
- c) Row-major order starts at the top left of the 2D array (when viewing it as a matrix) and traverses down each column until it ends at the bottom right corner.
- d) Row-major order starts at the top right of the 2D array (when viewing it as a matrix) and traverses across each row until it ends at the bottom left corner.

7. Fill in the correct code to store the char 'i' from the 2D array twoDChar.

```
public class GetChar {  
    public static void main(String[] args) {  
  
        char[][] twoDChar = {{'h', 'z', 'u', 'j'},  
                             {'o', 'x', 'm', 'p'},  
                             {'q', 'i', 'n', 'g'}};  
  
        //Store 'i' from twoDChar
```

```

        char result = _____
    }
}

```

- twoDChar[1][2];
- twoDChar[2][1];
- twoDChar[2][3];
- twoDChar[9];
- twoDChar[3][2];

Click or drag and drop to fill in the blank

8. Using initializer lists, we need the first 2D array to have a top row of {1,2,3} and a bottom row of {4,5,6}. The second 2D array needs to have a top row of {7,8,9} and a bottom row of {10,11,12}. Fill in the blanks with the correct code.

```

public class Exercise1Solution {
    public static void main(String[] args) {

        int[][] first = _____

        int[][] second;
        second = _____
    }
}

```

- {{7,8,9},{10,11,12}};
- {{1,2,3},{4,5,6}};
- new int[][] {{7,8,9},{10,11,12}};
- {7,8,9,10,11,12};
- {1,2,3,4,5,6};

Click or drag and drop to fill in the blank

Project-14

2D Arrays: Image Manipulation Project

In this project, you will be creating an application which is able to modify images as well as create new images using 2D arrays! The first section covers stretching the image horizontally, shrinking the image vertically, negating the color, applying a color filter, and inverting the image. The second section covers creating an image consisting of random pixels, placing a rectangle in the image, and using the method to randomly place many rectangles in the image.

Here is an overview about how images work in Java:

Images consist of pixels which are the individual points in the image containing some color. Each pixel has some red, green, blue, and alpha value which represents the amount of each of those colors in the pixel. The red, green, and blue values can be mixed to create all of the visible colors on your screen. The alpha value represents the transparency of the pixel (or how close the color of the pixel is to the background color of the image). A higher resolution image means that there are more pixels contained within it.

In Java, a loaded image is stored into a `BufferedImage` object. From this object, we can extract each pixel value and store it into a 2D array which we can manipulate. The pixel values are stored as ints because each pixel value in the `BufferedImage` object is represented by a hexadecimal value which contains the red, green, blue, and alpha components. The maximum value of any of the RGBA values is 255 and the minimum is 0. There are some methods provided for you in this project which handle the conversion between images and 2D arrays as well as extracting the R, G, B, and A values from a pixel. You will only need to implement methods which work with the 2D arrays.

Each of these methods is executable independently so you can select which methods you want to complete or you can complete all of them. You also may want to adjust the window sizes in the workspace to better see the code and the image. You can drag the window borders to adjust the size.

There are a lot of different parts to this project, so you may feel overwhelmed when first looking at it. Remember, the goal here is to practice using 2D arrays.

Tasks

Provided Method Details

1.

The first two sections of this project have you investigating the code that we have provided for you. You won't need to write any code, but it's important to understand the input and output of each of these methods.

There are four utility methods provided for you. The first method called `imgToTwoD()` accepts a `String` which can be a file path or image URL. It returns a 2D array of integers that contains every pixel from the image stored as int hexadecimal values containing the RGBA values for the pixel. Take a look at how the method works in the code editor. In the `main()` method, we can load image data into a 2D array of ints using the `imgToTwoD()` method. Note that you can use one of the provided images or load one from a URL.

We've provided `"/apple.jpg"`, `"/flower.jpg"`, and `"/kitten.jpg"`.

2.

The second provided method is called `twoDToImage()`. It accepts a 2D array of integers and a `String` for the file name. It converts the 2D array of int pixel data into an image and saves it.

We'll use this method to create images after we modify the 2D array of ints.

Take a look at how the method works in the code editor.

3.

The third provided method is the `getRGBAFromPixel()` method. This method accepts an int value representing the pixel hexadecimal value and returns a 4 element int array consisting of the R, G, B, and A values (between 0 and 255). Take a look at how the method works in the code editor. In Java, pixels are stored as hexadecimal values. This method is used to extract the color components from the hexadecimal value for the pixel.

4.

The fourth provided method is the `getColorIntValFromRGBA()` method. This method accepts an array of integers that represent the RGBA values and convert it into a single int value representing the pixel hexadecimal value. Take a look at how the method works in the code editor. Since pixels in Java are stored as hexadecimal values, this method converts the array of red, green, blue, and alpha values back into a hexadecimal value.

5.

In addition to these utility methods, there is a method used for extracting a 3x3 section from the top left of the image called `viewImageData`. This method is used to view the structure of the image data in both the raw pixel form and the extracted RGBA form. In the `main()` method, uncomment the call to `viewImageData` after the image is loaded (This should be line 14 if you haven't edited the code yet). Save the code and take a look at the console output.

After looking at the output, feel free to re-comment this line of code to help clean up your output for the rest of the project.

Example Method Walkthrough

6.

To start this project, we will look at an example of how to load, manipulate, and save an image step-by-step with provided code. We will look at the `trimBorders()` method. This method accepts a 2D array of int pixel data and the number of pixels to trim off of the borders of the image. It returns the modified image 2D array. Start by scrolling to where this method is defined in the project code.

7.

Try looking through the method and following along with the code. Look at how a portion of the original image is copied over to the new modified image. The input of this method is a 2D array containing the input image data and the output of this method is a new 2D array representing the modified image. A detailed description of this method is in the hint for this task.

The main takeaways are that given a 2D array of ints, we make a new 2D array, and fill that array with some values from the given array. There's some fancy math to make sure we fill the new array with the correct values, but the essential thing you should focus on is the fact that given a 2D array, we return a new 2D array.

8.

Once we have completed iterating, we return the modified image data. Now scroll back to the `main()` method in the code.

In the `main()` method, we load image data into a 2D array of ints using the `imgToTwoD()` method. Note that you can use the provided image or load one from a URL. After this, we can simply call our method we created by passing in the loaded image data and storing the returned modified image data into a new 2D array of ints. Finally, we can call the `twoDTolImage()` method using the modified image data to save the new image with a provided file name.

9.

Save the code in the editor which will run the application. Now we can view this modified image by entering the image file name which we saved into the browser on the side of the workspace. The project defaults to <http://localhost:8000/apple.jpg> so change the last part of the URL to match the image name which was saved. Hit enter and the image will appear in the browser window. You can also try changing the file name and loading the image in the browser.

Create a Negative Version of the Image

10.

Now let's fill in the `negativeColor()` method. This method will replace the color of each pixel in the image with the negative version of the pixel. This means that each color component of the pixel (R, G, and B) will be replaced with 255 minus the current value. We'll be aiming to create an image that looks like this:



Scroll down to where this method is defined.

11.

Begin by creating a new 2D array of ints which is the same size as the input image and setting up nested for loops that iterate through every pixel in the input image. It's probably easiest to iterate through the pixels in row-major order.

12.

Within the inner for loop, we need to get the R, G, B, and A values from each pixel. You can use the provided method `getRGBAFromPixel()` to accomplish this. This returns an array where each of the four elements corresponds to the RGBA values.

Elements 0 to 2 represent red, green, and blue while element 3 represents alpha. Using the method will look something like: `int[] rgba = getRGBAFromPixel(imageTwoD[i][j]);` depending on your iterator names.

13.

Now that you have the array containing the RGBA values, you need three lines of code which set the first three elements to 255 minus itself.

14.

Afterwards add another line of code which gets the int hexadecimal pixel data from the RGBA array. you can use the provided `getColorIntValFromRGBA()` method. You then need to store this value in the new image you created.

15.

Once the for loops are complete, you need to return the modified image. In the `main()` method, load an image and pass it into the `negativeColor()` method. Save the modified image and view it in the browser.

Stretch the Image Horizontally

16.

For this task, you will be implementing the method `stretchHorizontally()`. This method will double the width of the provided image data. For every pixel in the original image, you will copy it and place two duplicate pixels side-by-side into the

new modified image. You'll be looking to create an image like this:



In the code editor, scroll to this empty method.

17.

The first thing you should do in the method is to create a new 2D array of ints to hold the modified image data. The number of rows (or height) should be the same, but the number of columns (or width) should be the twice the number of columns from the original image. You can set the new 2D array to equal: `new int[imageTwoD.length][imageTwoD[0].length*2]`

18.

Next, set up nested for loops to iterate through every pixel in the input image using row-major order. To make things easier for us in the future, we can create an additional variable which will keep track of which position we are in for the modified image (since it is double the width). This variable will equal double the column index in the inner for loop. For example, if our inner loop is using `columnIndex` as the iterator. We can set our variable to equal: `it = columnIndex*2;`

19.

After we have created that additional iterator, we can now copy and duplicate each pixel from the input image. We need two lines of code. One to copy the current pixel to the `modifiedImage[rowIndex][it]` position and one to copy the current pixel to the `modifiedImage[rowIndex][it+1]` position. It is the doubled iterator from the last step and the current pixel from the input image is accessed using the outer and inner loop iterators. In the example we can assume that those are `rowIndex` and `columnIndex`, but it depends on the variable names you use.

20.

Once the for loops are complete, you need to return the modified image. Now in the `main()` method, you can load whatever image you want (either the provided image or one from a URL) as long as the resolution is not extremely high since the storage space of the workspace is limited. Pass the loaded image into the `stretchHorizontally()` method then click on the Save button in the workspace in order to run the code and save the image. Finally, enter the file name in the browser to view your modified image!

Shrink the Image Vertically

21.

In order to shrink the image vertically, you will be halving the height of the image and selecting every other pixel down each column to place in the modified image. We're trying to create an image that looks something like this (note that this ends

up looking fairly similar to the stretched image!):



This logic should be added in the `shrinkVertically()` method.

22.

Begin by creating a new 2D array of ints which has half the number of rows of the input image and the same number of columns.

23.

Next use nested for loops to iterate through each pixel in the input image using column-major order. Make the inner loop iterator increment by 2. We also need to end one pixel early in the vertical direction. Since we divided the height of the image by two and we could be working with an odd number of pixels in the vertical direction, we want to make sure that we do not try and access an out of bounds element.

24.

Inside the inner for loop, place the current pixel from the input image at the row index divided by 2 and the current column index. For example: `manipulatedImg[j/2][i] = imageTwoD[j][i];`

25.

Once the for loops are complete, you need to return the modified image. In the `main()` method, load an image and pass it into the `shrinkVertically()` method. Save the modified image and view it in the browser.

Invert the Image

26.

Next, you will invert the image. You will edit the `invertImage()` method in order to flip the image vertically and horizontally. Your final result should look something like this:



27.

Begin by creating a new 2D array which is the same size as the input image.

28.

Next, iterate through each pixel in the input image using nested for loops.

29.

Within the inner for loop, you will copy the final row position minus the current row index and the final column position minus the column index. This is similar logic to the `negativeColor()` method, but taking the negative of the pixel positions instead of the color values. It will look something like this: `invertedImg[i][j] = imageTwoD[(imageTwoD.length-1)-i][(imageTwoD[i].length-1)-j];`. See the image for an example of what takes place.

Inverting a 5x5 image

flipped horizontally and vertically

Before					After				
1	2	3	4	5	25	24	23	22	21
6	7	8	9	10	20	19	18	17	16
11	12	13	14	15	15	14	13	12	11
16	17	18	19	20	10	9	8	7	6
21	22	23	24	25	5	4	3	2	1

In this image, the pixels are numbered to help show how they change position when the image is inverted.

30.

Once the for loops are complete, you need to return the modified image. In the `main()` method, load an image and pass it into the `invertImage()` method. Save the modified image and view it in the browser.

Applying a Color Filter

31.

For this task you will implement the `colorFilter()` method. This method modifies every pixel in the image by provided R, G, and B values as input parameters. You must make sure that each color value does not leave the range of 0-255.

This is an example of a filter where the filter was -75, 30, -30 (Meaning the red values were decreased by 75, the green values were increased by 30, and the blue values were decreased by 30)



32.

Begin by creating a new 2D array which is the same size as the input image.

33.

Loop through each pixel in the input image using nested for loops.

34.

For every pixel in the input image, extract the RGBA color values using the provided method `getRGBAFromPixel()`.

35.

After retrieving the array of RGBA data, store the values of each color plus the modifier value (which can be positive or negative).

36.

For each of the new color values, test that it does not go outside of the range 0 to 255. If it is less than 0, set it equal to 0. If it is greater than 255, set it equal to 255.

37.

Set the values in the RGBA array to equal the new color values which you calculated.

38.

Convert the RGBA array to a single int containing the hexadecimal pixel data using the provided method `getColorIntValFromRGBA()` and store it in the new image.

39.

Once the for loops are complete, you need to return the modified image. In the `main()` method, load an image and pass it into the `colorFilter()` method along with modifier values for red, green, and blue. Save the modified image and view it in the browser.

Further Challenges: Painting an Image of Random Colors

40.

Good Job! You have finished the image processing application! You have seen how images can be converted to 2D arrays of pixels in order to modify them. You got to write methods that manipulate images. Feel free to keep adding to, and experimenting with, your image processing application!

In the next sections, we dive into how to use the `Random` class to paint random shapes on your images. Consider these sections to be further challenges — if

you're looking for more practice with 2D arrays, these may prove to be interesting challenges, but if you're feeling worn out, this is a good time to take a break or move on to the next lesson!

The next method you will be implementing is the `paintRandomImage()` method. It will modify the image passed in by replacing every pixel with a randomly colored pixel.

41.

Begin by creating a new object of the `Random` class. This class will be used to generate random numbers for us: `Random rand = new Random();`

42.

Next, iterate through each pixel in the provided image using nested for loops.

43.

Within the inner for loop, we need to generate a randomly colored pixel. In order to do this, we can use `rand.nextInt(256)`. This will generate a random integer between 0 and 255. Store three of these random ints by calling `rand.nextInt(256)` three times.

44.

The three randomly generated values will need to be the first, second, and third elements in a new int array. The fourth element should be 255 since that represents the alpha value (which controls transparency).

45.

After storing those values in an int array, pass it into the provided `getColorIntValFromRGBA()` method and store it into the input image. Note that this is accessing the input image instead of a newly created image like in the image manipulation methods.

46.

Once the for loops are complete, return the modified input image.

47.

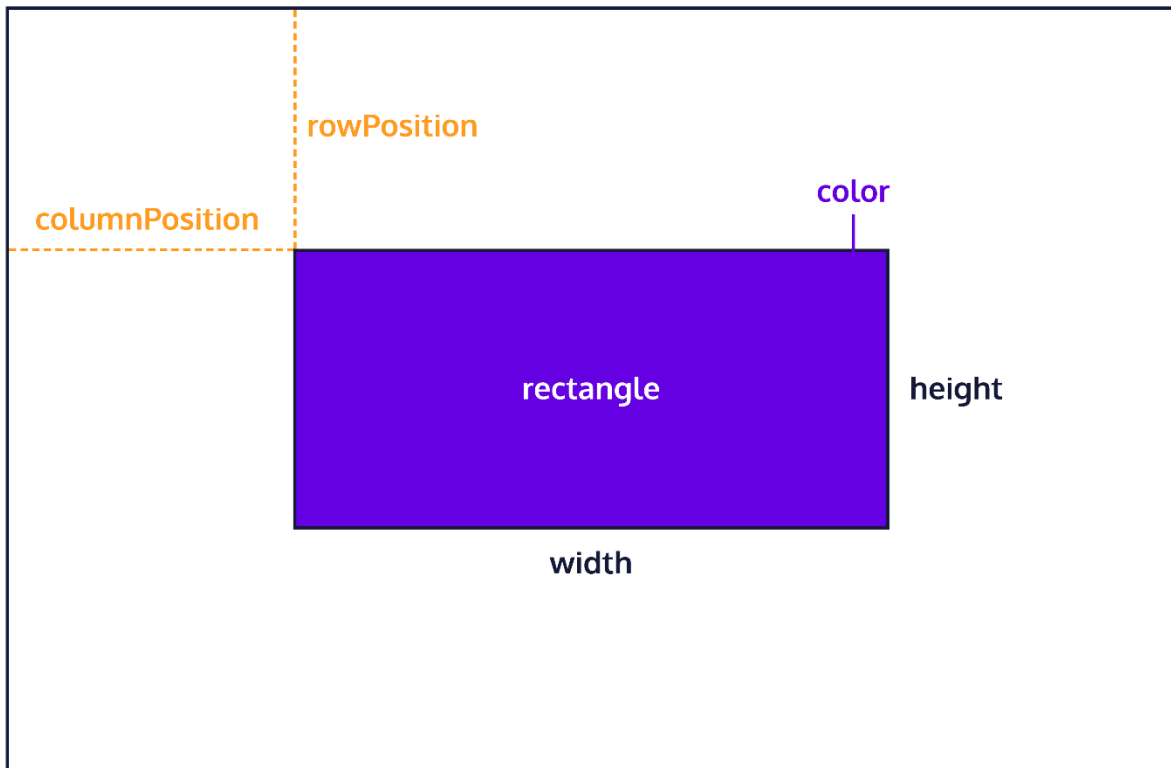
In the main() method, create a new 2D array of ints which will be our blank canvas for the painting methods. 500x500 is a good size for it. Pass the new 2D array into the paintRandomImage() method. Save the image and view it in the browser.

Drawing a Rectangle on an Image

48.

You will be implementing a method that draws a rectangle on an image using a provided width, height, rowPosition, columnPosition, and color. The image below shows what each of the input parameter does when creating the rectangle.

image



It's useful to think of these rectangles being drawn from the top left corner.

49.

Begin by iterating through every pixel in the input image using nested for loops.

50.

For every pixel, check if the current row index is greater than or equal to the provided rowPosition and also that the index is less than or equal to the rowPosition plus the width: $i \geq \text{rowPosition} \ \&\& \ i \leq \text{rowPosition} + \text{width}$.

51.

Additionally, check if the current column index is greater than or equal to the provided colPosition and also that the index is less than or equal to the colPosition plus the height: $j \geq \text{colPosition} \ \&\& \ j \leq \text{colPosition} + \text{height}$.

52.

If both of the previous tests are true, then paint the pixel the provided color since the pixel position lies within the rectangle shape.

53.

Once the iterations are complete, return the modified input image.

54.

In the main() method, pass an image into the paintRectangle() method and save the image. Enter in the desired values for your rectangle. You can create a color using an array such as: `int[] rgba = {255, 255, 0, 255};` and passing it into the `getColorIntValFromRGBA()` method. Save the image and view it in the browser.

Create Abstract Geometric Art Utilizing the paintRectangle Method

55.

Finally, you will implement the generateRectangles() method. This will use the paintRectangles() method you created in order to paint a piece of art. It will generate randomly positioned, sized, and colored rectangles based on the provided number. The input parameter, numRectangles will determine how many randomly generated rectangles will be placed in the image.

56.

Firstly, create an object of the Random class to use.

57.

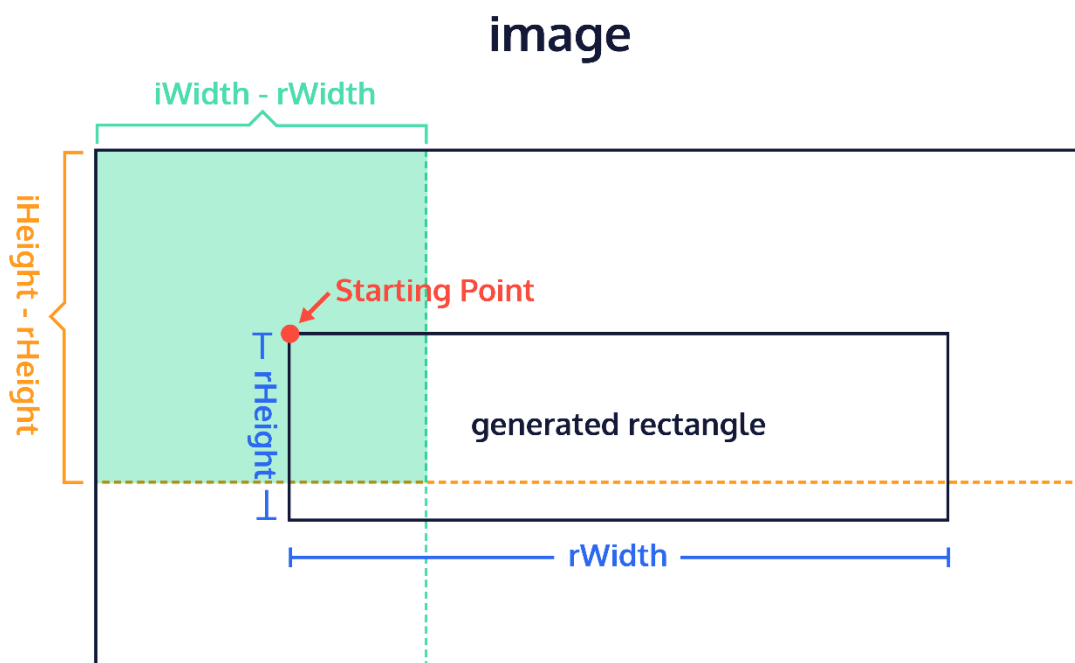
Next, iterate for the number of rectangles provided using a for loop.

58.

Generate and store two random integers for the width and height of the rectangle. For width, pass in the number of columns in the 2D array to the `rand.nextInt(num)` method. For height, pass in the number of rows into the `rand.nextInt(num)` method.

59.

Generate and store two random integers for the row position and column position. For row position, pass in the number of rows in the image minus the randomly generated height from the previous step. This will ensure that the rectangle does not get created outside of the image. For the column position generate a random integer using the number of columns in the image minus the randomly generated width from the last step. The following image shows why we perform this calculation.



In this image, the starting position of the rectangle can only be inside the shaded box. If we were to try to put the top left corner outside the shaded box, part of the rectangle would be drawn outside of the image, and we would get an error.

60.

Next, create a random color to paint the rectangle. You can do this in a similar way to the method used in `paintRandomImage()` make sure the three random color values are between 0 and 255. Store them in an array with 255 for the A value and convert it to the int value using `getColorIntValFromRGBA`.

61.

Finally, call the `paintRectangle()` method using all of the values and the input image and overwrite the input image.

62.

After the iterations are complete, return the modified input image.

63.

In the `main()` method, pass an image into the `generateRectangles()` method and a number of rectangles to generate, 1000 rectangles is a good number to use. Save the image and view the image in the browser.

Conclusion

64.

Wow, you made it through the bonus work. Pat yourself on the back, this was quite a difficult project! Reflect on all the different ways you used loops and 2D arrays to create these images.

ImageProcessing.java

```
import java.awt.Color;
```

```

import java.awt.image.BufferedImage;
import java.io.File;
import java.net.URL;
import java.util.Arrays;
import java.util.Random;
import javax.imageio.ImageIO;
public class ImageProcessing {
    public static void main(String[] args) {
        // The provided images are apple.jpg, flower.jpg, and kitten.jpg
        int[][] imageData = imgToTwoD("./apple.jpg");
        // Or load your own image using a URL!
        //int[][] imageData =
imgToTwoD("https://content.codecademy.com/projects/project_thumbnails/phas
er/bug-dodger.png");
        //viewImageData(imageData);
        int[][] trimmed = trimBorders(imageData, 60);
        twoDToImage(trimmed, "./trimmed_apple.jpg");
        // int[][] allFilters =
stretchHorizontally(shrinkVertically(colorFilter(negativeColor(trimBorders(invertIm
age(imageData), 50)), 200, 20, 40)));
        // Painting with pixels
    }
    // Image Processing Methods
    public static int[][] trimBorders(int[][] imageTwoD, int pixelCount) {
        // Example Method

```

```

        if (imageTwoD.length > pixelCount * 2 && imageTwoD[0].length >
pixelCount * 2) {
            int[][] trimmedImg = new int[imageTwoD.length - pixelCount *
2][imageTwoD[0].length - pixelCount * 2];
            for (int i = 0; i < trimmedImg.length; i++) {
                for (int j = 0; j < trimmedImg[i].length; j++) {
                    trimmedImg[i][j] = imageTwoD[i + pixelCount][j +
pixelCount];
                }
            }
            return trimmedImg;
        } else {
            System.out.println("Cannot trim that many pixels from the
given image.");
            return imageTwoD;
        }
    }

    public static int[][] negativeColor(int[][] imageTwoD) {
        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] stretchHorizontally(int[][] imageTwoD) {
        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] shrinkVertically(int[][] imageTwoD) {

```



```

        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] invertImage(int[][] imageTwoD) {
        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] colorFilter(int[][] imageTwoD, int redChangeValue, int
greenChangeValue, int blueChangeValue) {
        // TODO: Fill in the code for this method
        return null;
    }

    // Painting Methods

    public static int[][] paintRandomImage(int[][] canvas) {
        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] paintRectangle(int[][] canvas, int width, int height, int
rowPosition, int colPosition, int color) {
        // TODO: Fill in the code for this method
        return null;
    }

    public static int[][] generateRectangles(int[][] canvas, int numRectangles) {
        // TODO: Fill in the code for this method
        return null;
    }

```

```

    }

    // Utility Methods

    public static int[][] imgToTwoD(String inputFileOrLink) {
        try {
            BufferedImage image = null;

            if (inputFileOrLink.substring(0,
4).toLowerCase().equals("http")) {
                URL imageUrl = new URL(inputFileOrLink);
                image = ImageIO.read(imageUrl);
                if (image == null) {
                    System.out.println("Failed to get image from
provided URL.");
                }
            } else {
                image = ImageIO.read(new File(inputFileOrLink));
            }

            int imgRows = image.getHeight();
            int imgCols = image.getWidth();
            int[][] pixelData = new int[imgRows][imgCols];
            for (int i = 0; i < imgRows; i++) {
                for (int j = 0; j < imgCols; j++) {
                    pixelData[i][j] = image.getRGB(j, i);
                }
            }

            return pixelData;
        }
    }
}

```

```

        } catch (Exception e) {
            System.out.println("Failed to load image: " +
e.getLocalizedMessage());
            return null;
        }
    }

    public static void twoDTolImage(int[][] imgData, String fileName) {
        try {
            int imgRows = imgData.length;
            int imgCols = imgData[0].length;
            BufferedImage result = new BufferedImage(imgCols, imgRows,
BufferedImage.TYPE_INT_RGB);
            for (int i = 0; i < imgRows; i++) {
                for (int j = 0; j < imgCols; j++) {
                    result.setRGB(j, i, imgData[i][j]);
                }
            }
            File output = new File(fileName);
            ImageIO.write(result, "jpg", output);
        } catch (Exception e) {
            System.out.println("Failed to save image: " +
e.getLocalizedMessage());
        }
    }

    public static int[] getRGBAFromPixel(int pixelColorValue) {

```

```

        Color pixelColor = new Color(pixelColorValue);

        return new int[] { pixelColor.getRed(), pixelColor.getGreen(),
pixelColor.getBlue(), pixelColor.getAlpha() };
    }

    public static int getColorIntValFromRGBA(int[] colorData) {
        if (colorData.length == 4) {
            Color color = new Color(colorData[0], colorData[1],
colorData[2], colorData[3]);
            return color.getRGB();
        } else {
            System.out.println("Incorrect number of elements in RGBA
array.");
            return -1;
        }
    }

    public static void viewImageData(int[][] imageTwoD) {
        if (imageTwoD.length > 3 && imageTwoD[0].length > 3) {
            int[][] rawPixels = new int[3][3];
            for (int i = 0; i < 3; i++) {
                for (int j = 0; j < 3; j++) {
                    rawPixels[i][j] = imageTwoD[i][j];
                }
            }
            System.out.println("Raw pixel data from the top left corner.");
            System.out.print(Arrays.deepToString(rawPixels).replace("[", "
",\n") + "\n");

```

```

        int[][][] rgbPixels = new int[3][3][4];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                rgbPixels[i][j] =
getRGBAFromPixel(imageTwoD[i][j]);
            }
        }
        System.out.println();
        System.out.println("Extracted RGBA pixel data from top the left
corner.");

        for (int[][] row : rgbPixels) {
            System.out.print(Arrays.deepToString(row) +
System.lineSeparator());
        }
    } else {
        System.out.println("The image is not large enough to extract 9
pixels from the top left corner");
    }
}
}

```



Concept Review

Nested Iteration Statements

In Java, nested iteration statements are iteration statements that appear in the body of another iteration statement. When a loop is nested inside another loop, the inner loop must complete all its iterations before the outer loop can continue.

```
for(int outer = 0; outer < 3; outer++){  
    System.out.println("The outer index is: " + outer);  
    for(int inner = 0; inner < 4; inner++){  
        System.out.println("\tThe inner index is: " + inner);  
    }  
}
```

Declaring 2D Arrays

In Java, 2D arrays are stored as arrays of arrays. Therefore, the way 2D arrays are declared is similar 1D array objects. 2D arrays are declared by defining a data type followed by two sets of square brackets.

```
int[][] twoDIntArray;  
String[][] twoDStringArray;  
double[][] twoDDoubleArray;
```

Accessing 2D Array Elements

In Java, when accessing the element from a 2D array using `arr[first][second]`, the first index can be thought of as the desired row, and the second index is used for the desired column. Just like 1D arrays, 2D arrays are indexed starting at 0.

//Given a 2d array called `arr` which stores `int` values

```
int[][] arr = {{1,2,3},  
               {4,5,6}};
```

//We can get the value `4` by using

```
int retrieved = arr[1][0];
```

Initializer Lists

In Java, initializer lists can be used to quickly give initial values to 2D arrays. This can be done in two different ways.

1. If the array has not been declared yet, a new array can be declared and initialized in the same step using curly brackets.
2. If the array has already been declared, the `new` keyword along with the data type must be used in order to use an initializer list

// Method one: declaring and initializing at the same time

```
double[][] doubleValues = {{1.5, 2.6, 3.7}, {7.5, 6.4, 5.3}, {9.8, 8.7, 7.6}, {3.6, 5.7,  
7.8}};
```

// Method two: declaring and initializing separately:

```
String[][] stringValues;
```

```
stringValues = new String[][] {{"working", "with"}, {"2D", "arrays"}, {"is", "fun"}};
```

Modify 2D Array Elements

In Java, elements in a 2D array can be modified in a similar fashion to modifying elements in a 1D array. Setting `arr[i][j]` equal to a new value will modify the element in row `i` column `j` of the array `arr`.

```
double[][] doubleValues = {{1.5, 2.6, 3.7}, {7.5, 6.4, 5.3}, {9.8, 8.7, 7.6}, {3.6, 5.7, 7.8}};
```

```
doubleValues[2][2] = 100.5;
```

```
// This will change the value 7.6 to 100.5
```

Row-Major Order

“Row-major order” refers to an ordering of 2D array elements where traversal occurs across each row - from the top left corner to the bottom right. In Java, row major ordering can be implemented by having nested loops where the outer loop variable iterates through the rows and the inner loop variable iterates through the columns. Note that inside these loops, when accessing elements, the variable used in the outer loop will be used as the first index, and the inner loop variable will be used as the second index.

```
for(int i = 0; i < matrix.length; i++) {  
    for(int j = 0; j < matrix[i].length; j++) {  
        System.out.println(matrix[i][j]);  
    }  
}
```

Column-Major Order

“Column-major order” refers to an ordering of 2D array elements where traversal occurs down each column - from the top left corner to the bottom right. In Java, column major ordering can be implemented by having nested loops where the outer loop variable iterates through the columns and the inner loop variable

iterates through the rows. Note that inside these loops, when accessing elements, the variable used in the outer loop will be used as the second index, and the inner loop variable will be used as the first index.

```
for(int i = 0; i < matrix[0].length; i++) {  
    for(int j = 0; j < matrix.length; j++) {  
        System.out.println(matrix[j][i]);  
    }  
}
```

Traversing With Enhanced For Loops

In Java, enhanced for loops can be used to traverse 2D arrays. Because enhanced for loops have no index variable, they are better used in situations where you only care about the values of the 2D array - not the location of those values

```
for(String[] rowOfStrings : twoDStringArray) {  
    for(String s : rowOfStrings) {  
        System.out.println(s);  
    }  
}
```